Содержание

1 Лекция 1 (10.02) **10**

1.1 Задание для самостоятельной работы . . . 10

2 Лекция 2 (17.02) **11**

2.1 Расчётная сетка . . . 11

3 Лекция 3 (24.02) **12**

3.1 Структурированная расчётная сетка . . . 12

3.2 Метод конечных разностей. Уравнение Пуассона . . . 12

3.2.1 Постановка задачи . . . 12

3.2.2 Метод решения . . . 12

3.2.2.1 Нахождение численного решения . . . 12

3.2.2.2 Практическое определения порядка аппроксимации . . . 13

3.2.3 Программная реализация . . . 14

3.2.3.1 Функция верхнего уровня . . . 14

3.2.3.2 Детали реализации . . . 16

3.3 Задание для самостоятельной работы . . . 19

3.3.1 Одномерное уравнение Пуассона . . . 19

3.3.2 Двумерное уравнение Пуассона . . . 19

4 Лекция 4 (02.03) **21**

4.1 Форматы хранения разреженных матриц . . . 21

4.1.1 CSR-формат . . . 21

4.1.2 Массив словарей . . . 23

5 Лекция 5 (09.03) **25**

5.1 Решение СЛАУ . . . 25

5.1.1 Метод Якоби . . . 25

5.1.2 Метод Зейделя . . . 25

5.1.3 Метод последовательных верхних релаксаций (SOR) . . . 26

5.2 Задание для самостоятельной работы . . . 26

6 Лекция 6 (23.03) **30**

6.1 Метод конечных объёмов . . . 30

6.1.1 Уравнение Пуассона . . . 30

6.1.1.1 Обработка внутренних граней . . . 31

6.1.1.2 Учёт граничных условий первого рода . . . 32

6.1.2 Одномерный случай . . . 32

6.1.3 Сборка системы линейных уравнений . . . 33

6.1.3.1 Алгоритм сборки в цикле по ячейкам . . . 33

6.1.3.2 Алгоритм сборки в цикле по граням . . . 34

6.2 Задание для самостоятельной работы . . . 34

7 Лекция 7 (30.03) 36
7.1 Граничные условия второго рода . . . 36
7.2 Граничные условия третьего рода . . . 36
7.2.1 Универсальность условий третьего рода . . . 37
7.3 Задание для самостоятельной работы . . . 37

8 Лекция 8 (06.04) 38
8.1 Дополнительные точки коллокации на границах . . . 38
8.1.1 Пример . . . 39
8.2 Задание для самостоятельной работы . . . 40

9 Лекция 9 (13.04) 42
9.1 Учёт скошенности сетки в двумерной МКО-аппроксимации . . . 42
9.1.1 Уточнённая аппроксимация нормальной производной . . . 42
9.1.2 Интерполяция значения функции во вспомогательных точках . . . 43
9.1.3 Производная по границе . . . 44
9.1.4 Сборка СЛАУ для уравнения Пуассона . . . 44
9.2 Задание для самостоятельной работы . . . 45

10 Лекция 10 (20.04) 47
10.1 Учёт скошенности сетки в 3D . . . 47

11 Лекция 11 (27.04) 48
11.1 Метод взвешенных невязок . . . 48
11.2 Метод Бубнова–Галёркина . . . 48
11.2.1 Степенные базисные функции . . . 48
11.3 Метод конечных элементов . . . 48
11.3.1 Узловые базисные функции . . . 48
11.3.2 Одномерное уравнение Пуассона . . . 48
11.3.2.1 Слабая интегральная постановка задачи . . . 48
11.3.2.2 Линейный одномерный (пирамидальный) базис . . . 48

12 Лекция 12 (04.05) 49
12.1 Поэлементная сборка конечноэлементных матриц . . . 49
12.1.0.1 Элементные матрицы . . . 49

13 Лекция 13 (11.05) 50
13.1 Вычисление элементных интегралов в параметрическом пространстве . . . 50
13.2 Двумерное уравнение Пуассона . . . 50
13.2.0.1 Треугольный элемент. Линейный двумерный базис . . . 50
13.3 Разбор программной реализации МКЭ . . . 50
13.3.1 Рабочий объект . . . 51

13.3.2 Конечноэлементный сборщик . . . 52
13.3.3 Концепция конечного элемента . . . 52
13.3.3.1 Определение линейного одномерного элемента . . . 53
13.3.3.2 Геометрические свойства элемента . . . 54
13.3.3.3 Элементный базис . . . 54
13.3.3.4 Калькулятор элементных матриц . . . 55
13.4 Задание для самостоятельной работы . . . 56

14 Лекция 14 (02.11) 57
14.1 Двухслойные схемы для нестационарных уравнений . . . 57
14.1.1 Определение . . . 57
14.1.1.1 Явная схема . . . 57
14.1.1.2 Неявная схема . . . 57
14.1.1.3 Схема Кранка–Николсон . . . 58
14.1.1.4 Обобщённая двухслойная схема . . . 58
14.2 Схемы высокого порядка точности . . . 59
14.2.1 Многослойные схемы. Схемы Адамса . . . 59
14.2.1.1 Явные схемы Адамса–Башфорта . . . 59
14.2.1.2 Неявные схемы Адамса–Мультона . . . 60
14.2.2 Схемы Рунге-Кутта . . . 61
14.3 Методы исследования устойчивости расчётных схем . . . 61
14.3.1 Дискретизация по времени как итерационный процесс . . . 61
14.3.1.1 Двухслойный итерационный процесс . . . 61
14.3.1.2 Устойчивость итерационного процесса . . . 61
14.3.1.3 Источники возмущений . . . 62
14.3.2 Матричный метод . . . 63
14.3.2.1 Явная схема для нестационарного уравнения диффузии . . . 63
14.3.2.2 Неявная схема для нестационарного уравнения диффузии . . . 64
14.3.3 Метод дискретных возмущений . . . 65
14.3.3.1 Явная схема против потока для уравнения переноса . . . 65
14.3.4 Метод Неймана . . . 65
14.3.4.1 Неявная противопотоковая схема для уравнения переноса . . . 66
14.3.4.2 Противопотоковая схема Кранка-Николсон для уравнения переноса . . . 67
14.3.4.3 Явная схема для уравнения нестационарной конвекции-диффузии . . . 68
14.3.4.4 Неявная схема для уравнения нестационарной конвекции-диффузии . . . 69
14.3.5 Общие рекомендации к выбору устойчивых расчётных схем . . . 69
14.4 Программная реализация схемы для уравнения переноса . . . 70
14.4.1 Постановка задачи . . . 70
14.4.2 Функция верхнего уровня . . . 71
14.4.3 Расчётные функции . . . 72
14.4.3.1 Явная схема . . . 73
14.4.3.2 Неявная схема . . . 73

14.4.3.3 Схема Кранка-Николсон . . . 75
14.4.4 Анализ результатов работы . . . 76
14.5 Задание для самостоятельной работы . . . 77
14.5.1 Постановка задачи . . . 77
14.5.1.1 Тестовый пример 1 . . . 77
14.5.1.2 Тестовый пример 2 . . . 78
14.5.2 Расчётная схема . . . 79

15 Лекция 15 (09.11) 81
15.1 Аппроксимация уравнения переноса с ограничением потока . . . 81
15.1.1 Схемы первого и второго порядка точности . . . 81
15.1.2 Условие TVD . . . 82
15.1.3 Нелинейные TVD схемы . . . 82
15.2 TVD-схемы для неструктурированных конечнообъёмных сеток . . . 84
15.2.1 Прямая интерполяция противопоточного значения . . . 85
15.2.2 Интерполяция противопоточного значения через значение градиентов . . . 85
15.2.3 Определение градиентов в узлах коллокации. Метод наименьших квадратов . . . 86
15.2.4 Реализация для явной схемы . . . 87
15.3 Задание для самостоятельной работы . . . 88

16 Лекция 16 (16.11) 89
16.1 Алгебраический подход к построению нелинейных TVD схем . . . 89
16.2 Схемы с искусственной вязкостью . . . 89
16.2.1 Направленная искусственная вязкость . . . 89
16.2.2 SUPG . . . 89
16.3 Задание для самостоятельной работы . . . 89

17 Лекция 17 (23.11) 91
17.1 Моделирование течения вязкой несжимаемой жикости методом конечных разностей 91
17.1.1 Система уравнений Навье-Стокса . . . 91
17.1.2 Схема расчёта . . . 92
17.1.2.1 Метод SIMPLE . . . 92
17.1.3 Пространственная аппроксимация . . . 94
17.1.3.1 Разнесённая сетка . . . 94
17.1.3.2 Уравнения движения . . . 95
17.1.3.3 Уравнение для поправки давления . . . 97
17.1.3.4 Уравнение для поправки скорости . . . 99
17.1.3.5 Учёт граничных условий . . . 99
17.1.4 Оптимальные значения параметров алгоритма SIMPLE . . . 102
17.2 Программа для расчёта течения в каверне по схеме SIMPLE . . . 102
17.2.1 Постановка задачи . . . 102
17.2.2 Функция верхнего уровня . . . 104
17.2.3 Поля класса решателя . . . 105

17.2.4 Инициализация решателя . . . 106
17.2.5 Шаг итерации SIMPLE . . . 107
17.2.6 Сборка системы уравнений для поправки давления . . . 108
17.2.7 Сборка системы уравнений для пробной скорости . . . 110
17.3 Задание для самостоятельной работы . . . 111

18 Лекция 18 (30.11) 113
18.1 Нестационарное уравнение Навье-Стокса . . . 113
18.1.1 Схема расчёта по алгоритму SIMPLE . . . 113
18.2 Задача об обтекании препятствия . . . 114
18.2.1 Расчётная сетка . . . 114
18.2.2 Граничные условия . . . 115
18.2.2.1 Входное сечение . . . 115
18.2.2.2 Условия симметрии . . . 117
18.2.2.3 Условия прилипания . . . 117
18.2.2.4 Выходные граничные условия . . . 118
18.2.3 Баланс сил. Коэффициенты сил . . . 120
18.2.3.1 Сопротивление . . . 120
18.2.3.2 Подъёмная сила . . . 121
18.2.3.3 Вычисление коэффициентов сил на разнесённой сетке . . . 122
18.3 Тестовые примеры . . . 123
18.3.1 Задача о равномерном течении . . . 123
18.3.1.1 Учёт граничных условий . . . 124
18.3.1.2 Анализ результатов . . . 126
18.3.2 Течение Пуазейля . . . 126
18.3.3 Стационарное обтекание квадратного препятствия . . . 126
18.3.3.1 Функция верхнего уровня . . . 127
18.3.3.2 Учёт неактивных ячеек . . . 128
18.3.3.3 Расчёт коэффициентов сопротивления . . . 131
18.3.3.4 Результаты расчёта . . . 133
18.3.4 Нестационарное обтекание квадратного препятствия . . . 133
18.3.4.1 Функция верхнего уровня . . . 133
18.3.4.2 Учёт нестационарности . . . 135
18.4 Задание для самостоятельной работы . . . 136
18.4.1 Расчитать коэффициент подъёмной силы . . . 136
18.4.2 Расчёт течения . . . 136

19 Лекция 19 (7.12) 137
19.1 Схема PISO . . . 137
19.1.1 Итерации PISO . . . 137
19.1.2 Граничные условия . . . 139
19.1.3 Схема с внешними итарциями PIMPLE . . . 139
19.2 Конвективный теплообмен . . . 140

19.2.1 Уравнение теплопроводности . . . 140
19.2.2 Дискретизация по времени . . . 140
19.2.3 Аппроксимация на разнесённой сетке . . . 140
19.2.4 Граничные условия . . . 141
19.2.4.1 Условия первого рода . . . 141
19.2.4.2 Условия второго рода . . . 142
19.2.4.3 Условия третьего рода . . . 142
19.2.4.4 Универсальность условий третьего рода . . . 142
19.2.5 Коэффициент теплообмена . . . 143
19.3 Задание для самостоятельной работы . . . 143

20 Лекция 20 (14.12) 147
20.1 Конечнообъёмное решение нестационарной задачи об обтекании кругового цилиндра . 147
20.2 Задание для самостоятельной работы . . . 147
20.2.1 Уравнение для температуры . . . 148
20.2.2 Коэффициент теплоотдачи . . . 148
20.2.3 Порядок реализации . . . 149

21 Лекция 21 (21.12) 151

22 Лекция 22 (11.01) 152

23 Лекция 23 (25.01) 153
23.1 Вычисление матрицы Якоби через базисные функции . . . 153
23.2 Применение квадратурных формул для вычисления элементных интегралов . . . 153
23.2.1 Диагонализация матрицы масс . . . 153
23.3 Матричный алгоритм построения конечных элементов высокого порядка . . . 153
23.4 Неполные конечные элементы . . . 153
23.5 Задание для самостоятельной работы . . . 153

24 Лекция 24 (22.03) 156
24.1 Стабилизация методом характеристик . . . 156
24.1.1 Конечноэлементная процедура . . . 156
24.2 Задание для самостоятельной работы . . . 157

25 Лекция 25 (29.03) 158
25.1 CBS схема решения уравнений Навье-Стокса . . . 158
25.1.1 Постановка задачи для слабосжимаемой жидкости . . . 158
25.1.2 Конечноэлементная аппроксимация . . . 158
25.1.3 Порядок расчёта на временном слое . . . 159
25.1.4 Полунеявная схема . . . 160
25.1.5 Явная схема . . . 160
25.1.6 Стационарное течение . . . 160
25.1.7 Течения несжимаемой жидкости . . . 161

25.1.7.1 Искусственная сжимаемость . . . 161
25.1.7.2 Нестационарные задачи . . . 161
25.2 Задание для самостоятельной работы . . . 161

26 Лекция 26 (05.04) 163
26.1 Конечнообъёмная coupled схема решения уравнений Навье-Стокса . . . 163
26.1.1 Задание для самостоятельной работы . . . 163

27 Лекция 27 (12.04) 164
27.1 Моделирование граничных условий на стенке методом конечных объёмов с использованием пристеночной функции . . . 164
27.2 Задание для самостоятельной работы . . . 165
27.2.1 Рекомендации . . . 165

28 Лекция 28 (19.04) 166
28.1 Стабилизация методом наименьших квадратов . . . 166
28.2 Задание для самостоятельной работы . . . 167

29 Задание 1: $k-\epsilon$ модель турбулентности 169
29.0.1 Задание для самостоятельной работы . . . 170

30 Задание 2: $k-\omega$ модель турбулентности 171
30.0.1 Задание для самостоятельной работы . . . 171

31 Задание 3: Задача о естественной конвекции. Метод дополнений Шура. МКЭ 173

32 Задание 4: Параллельные методы решения задач гидродинамики 177
32.0.1 Распараллеливание циклов и явных схем . . . 177
32.0.2 Параллельное решение независимых подзадач . . . 178
32.0.3 Domain Decomposition Method. Метод Шварца . . . 179
32.0.3.1 Построение сетки для конечнообъёмного DDM-метода . . . 180
32.0.3.2 MPI реализация конечнообъёмного метода Шварца . . . 181
32.0.4 Задание для самостоятельной работы . . . 183
32.0.4.1 Необходимо . . . 184
32.0.4.2 Рекомендации к программированию . . . 184

A Формулы и обозначения 186
A.1 Векторы . . . 187
A.1.1 Обозначение . . . 187
A.1.2 Набла–нотация . . . 187
A.2 Интегрирование . . . 189
A.2.1 Формула Гаусса–Остроградского . . . 189
A.2.2 Интегрирование по частям . . . 189
A.2.3 Численное интегрирование в заданной области . . . 190
A.3 Интерполяционные полиномы . . . 191

A.3.1 Многочлен Лагранжа . . . 191
A.3.1.1 Узловые базисные функции . . . 191
A.3.1.2 Интерполяция в параметрическом отрезке . . . 192
A.3.1.3 Интерполяция в параметрическом треугольнике . . . 195
A.3.1.4 Интерполяция в параметрическом квадрате . . . 197
A.4 Геометрические алгоритмы . . . 200
A.4.1 Линейная интерполяция . . . 200
A.4.2 Преобразование координат . . . 200
A.4.2.1 Матрица Якоби . . . 201
A.4.2.2 Дифференцирование в параметрической плоскости . . . 202
A.4.2.3 Интегрирование в параметрической плоскости . . . 203
A.4.2.4 Двумерное линейное преобразование. Параметрический треугольник . . . 203
A.4.2.5 Двумерное билинейное преобразование. Параметрический квадрат . . . 204
A.4.2.6 Трёхмерное линейное преобразование. Параметрический тетраэдр . . . 204
A.4.3 Свойства многоугольника . . . 204
A.4.3.1 Площадь многоугольника . . . 204
A.4.3.2 Интеграл по многоугольнику . . . 206
A.4.3.3 Центр масс многоугольника . . . 206
A.4.4 Свойства многогранника . . . 207
A.4.4.1 Объём многогранника . . . 207
A.4.4.2 Интеграл по многограннику . . . 207
A.4.4.3 Центр масс многогранника . . . 207
A.4.5 Поиск многоугольника, содержащего заданную точку . . . 207
A.5 Методы решения систем уравнений . . . 208
A.5.1 Простые итерационные алгоритмы . . . 208
A.5.1.1 Метод Якоби . . . 208
A.5.1.2 Метод Зейделя . . . 208
A.5.1.3 Метод последовательных верхних релаксаций SOR . . . 208
A.5.2 Метод коррекции поправки . . . 208
A.5.3 Методы подпространств Крылова . . . 209
A.5.3.1 Итерационный процесс BiCGStab с предобуславливателем . . . 209

B Работа с инфраструктурой проекта CFDCourse **211**
B.1 Сборка и запуск . . . 212
B.1.1 Сборка проекта CFDCourse . . . 212
B.1.1.1 Подготовка . . . 212
B.1.1.2 VisualStudio . . . 212
B.1.1.3 VSCode . . . 214
B.1.2 Запуск конкретного теста . . . 215
B.1.3 Сборка релизной версии . . . 217
B.2 Git . . . 219
B.2.1 Основные команды . . . 219

B.2.2 Порядок работы с репозиторием CFDCourse 220
B.3 Paraview 222
B.3.1 Данные на одномерных сетках 222
B.3.2 Изолинии для двумерного поля 225
B.3.3 Данные на двумерных сетках в виде поверхности 226
B.3.4 Числовых значения в точках и ячейках 227
B.3.5 Векторные поля 227
B.3.6 Значение функции вдоль линии 229
B.4 Hybmesh 232
B.4.1 Работа в Windows 232
B.4.2 Работа в Linux 232

1 Лекция 1 (10.02)

1.1 Задание для самостоятельной работы

1. Клонировать репозиторий `CFDCourse25` на компьютер, собрать проект и запустить программу `cfd25_test` (см. пункт B.1)

2. В файле `cfd25_test.cpp` написать простой `TEST_CASE`, проверяющий результат простого арифмитического действия (например, $2+2$). О запуске тестов см. пункт B.1.2

2 Лекция 2 (17.02)

2.1 Расчётная сетка

TODO

3 Лекция 3 (24.02)

3.1 Структурированная расчётная сетка

TODO

3.2 Метод конечных разностей. Уравнение Пуассона

3.2.1 Постановка задачи

Рассматривается одномерное дифференциальное уравнение вида

$$-\frac{\partial^2 u}{\partial x^2} = f(x) \tag{3.1}$$

в области $x \in [a, b]$ с граничными условиями первого рода

$$\begin{cases} u(a) = u_a, \\ u(b) = u_b. \end{cases} \tag{3.2}$$

Необходимо:

- Запрограммировать расчётную схему для численного решения этого уравнения методом конечных разностей на сетке с постоянным шагом,
- С помощью вычислительных экспериментов подтвердить порядок аппроксимации расчётной схемы.

3.2.2 Метод решения

3.2.2.1 Нахождение численного решения

В области решения $[a, b]$ введём равномерную сетку из N ячеек. Шаг сетки будет равен $h = (b-a)/N$. Узлы сетки запишем в виде сеточного вектора $\{x_i\}$ длины $N+1$, где $i = \overline{0, N}$. Определим сеточный вектор $\{u_i\}$ неизвестных, элементы которого определяют значение искомого численного решения в i-ом узле сетки.

Разностная схема второго порядка для уравнения (3.1) имеет вид

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i, \qquad i = \overline{1, N-1}. \tag{3.3}$$

Здесь $\{f_i\}$ – известный сеточный вектор, определяемый через известную аналитическую функцию $f(x)$ в правой части уравнения (3.1) как

$$f_i = f(x_i). \tag{3.4}$$

Аппроксимация граничных условий (3.2) первого рода даёт дополнительные сеточные уравнения для граничных узлов

$$\begin{aligned} u_0 &= u_a, \\ u_N &= u_b \end{aligned} \tag{3.5}$$

Линейные уравнения (3.3), (3.5) составляют систему вида

$$\sum_{j=0}^{N} A_{ij}\, u_j = b_i, \qquad i = \overline{0, N}$$

с матричными коэффициентами

$$A_{ij} = \begin{cases} 1, & i = 0,\ j = 0; \\ 2/h^2, & i = \overline{1, N-1},\ j = i; \\ -1/h^2, & i = \overline{1, N-1},\ j = i-1; \\ -1/h^2, & i = \overline{1, N-1},\ j = i+1; \\ 1, & i = N,\ j = N; \\ 0, & \text{иначе.} \end{cases} \tag{3.6}$$

и правой частью

$$b_i = \begin{cases} u_a, & i = 0; \\ u_b, & i = N; \\ f_i, & i = \overline{1, N-1}. \end{cases} \tag{3.7}$$

Искомый вектор находится путём решения этой системы.

3.2.2.2 Практическое определения порядка аппроксимации

Порядок аппрокцимации показывает скорость приближения численного решения к точному с уменьшением сетки. Поэтому для подтверждения порядка необходимо

- Знать точное решение,
- Уметь вычислять функционал (норму, $||\cdot||$), характеризующий отклонение точного решения от численного,
- Сделать несколько расчётов на сетках с разной N и заполнить таблицу $||\{u_i - u^e(x_i)\}||(N)$,
- На основе этой таблицы построить график в логарифмических осях и по углу наклона кривой сделать вывод о порядке аппроксимации.

Выберем произвольную функцию u^e (достаточно сильно изменяющуюся на целевом отрезке $[a, b]$).

Далее путём прямого вычисления определим параметры задачи f, u_a, u_b такие, для которых функция u^e является точным решением задачи (3.1), (3.2).

Зададимся числом разбиений N и решим задачу для выбранным параметров. В результате определим сеточный вектор численного решения $\{u_i\}$.

В качестве нормы выберем стандартное отклонение. В интегральном виде для многомерной функции $y(\mathbf{x})$ в области $\mathbf{x} \in D$ оно имеет вид

$$||y(\mathbf{x})||_2 = \sqrt{\frac{1}{|D|} \int_D y(\mathbf{x})^2\, d\mathbf{x}}. \tag{3.8}$$

Упрощая до одномерного случая

$$||y(x)||_2 = \sqrt{\frac{1}{b-a}\int_a^b y(x)^2\,dx}.$$

Вычислим этот интеграл численно на введённой ранее равномерной сетке $\{x_i\}$:

$$||\{y_i\}||_2 = \sqrt{\frac{1}{b-a}\sum_{i=0}^{N} w_i y_i^2},$$

где $\{w_i\}$ – вес (или "площадь влияния") i-ого узла:

$$w_i = \begin{cases} h/2, & i = 0, N; \\ h, & i = \overline{1, N-1}, \end{cases}$$

такая что

$$\sum_{i=0}^{N} w_i = b - a.$$

Окончательно среднеквадратичная норма отклонения численного решения от точного запишется в виде

$$||\{u_i - u^e(x_i)\}||_2 = \sqrt{\frac{1}{b-a}\sum_{i=0}^{N} w_i\,(u_i - u_i^e)^2}. \tag{3.9}$$

3.2.3 Программная реализация

Тестовая программа для решения одномерного уравнения Пуассона реализована в файле `poisson_fdm_solve_test.cpp`.

В качестве аналитической тестовой функции используется

$$u^e = \sin(10x^2)$$

на отрезке $x \in [0, 1]$.

3.2.3.1 Функция верхнего уровня

объявлена как

```
113 TEST_CASE("Poisson 1D solver, Finite Difference Method", "[poisson1-fdm]"){
```

В программе в цикле по набору разбиений `n_cells`

```
125   for (size_t n_cells: {10, 20, 50, 100, 200, 500, 1000}){
```

создаётся решатель для тестовой задачи, использующий заданное число ячеек

```
127     TestPoisson1Worker worker(n_cells);
```

вычисляется среднеквадратичная норма отклонения численного решения от точного

```
130     double n2 = worker.solve();
```

полученное численное решение (вместе с точным) сохраняется в vtk файле `poisson1_n={10,20,...}.vtk`

```
133     worker.save_vtk("poisson1_fdm_n=" + std::to_string(n_cells) + ".vtk");
```

а полученная норма печатается в консоль напротив количества ячеек

```
136     std::cout << n_cells << " " << n2 << std::endl;
```

В результате работы программы в консоли должна отобразиться таблица вида

```
--- [poisson1] ---
10 0.179124
20 0.0407822
50 0.00634718
100 0.00158055
200 0.000394747
500 6.31421e-05
1000 1.57849e-05
```

где первый столбец – это количество ячеек, а второй – полученная для этого количества ячеек норма. Нарисовав график этой таблицы в логарифмических осях подтвердим второй порядок аппроксимации (рис. 1).

N	Norm	Log10(N)	Log10(Norm)
10	0.179124	1	-0.74684622
20	0.0407822	1.301029996	-1.38952935
50	0.00634718	1.698970004	-2.19741919
100	0.00158055	2	-2.80119176
200	0.000394747	2.301029996	-3.40368116
500	6.31E-05	2.698970004	-4.19968098
1000	1.58E-05	3	-4.80175817

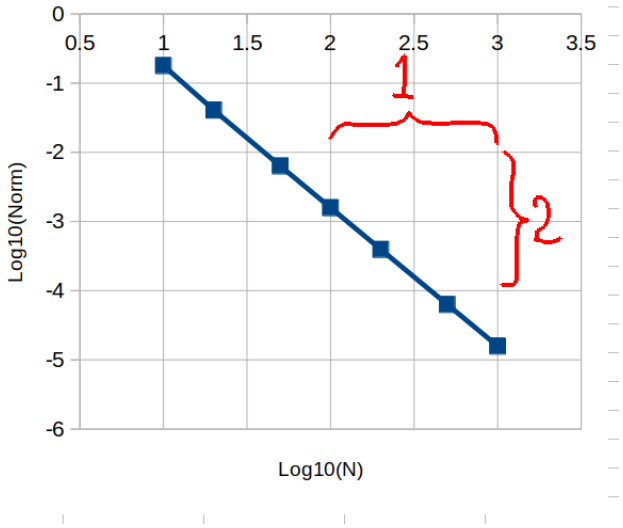


Рис. 1: Сходимость с уменьшением разбиения при решении одномерного уравнения Пуассона

Открыв один из сохранённых в процессе работы файлов vtk `poisson1_ncells=?.vtk` в paraview можно посмотреть полученные графики. В файле представлены как точное “exact”, так и численное решение “numerical” (рис. 2).

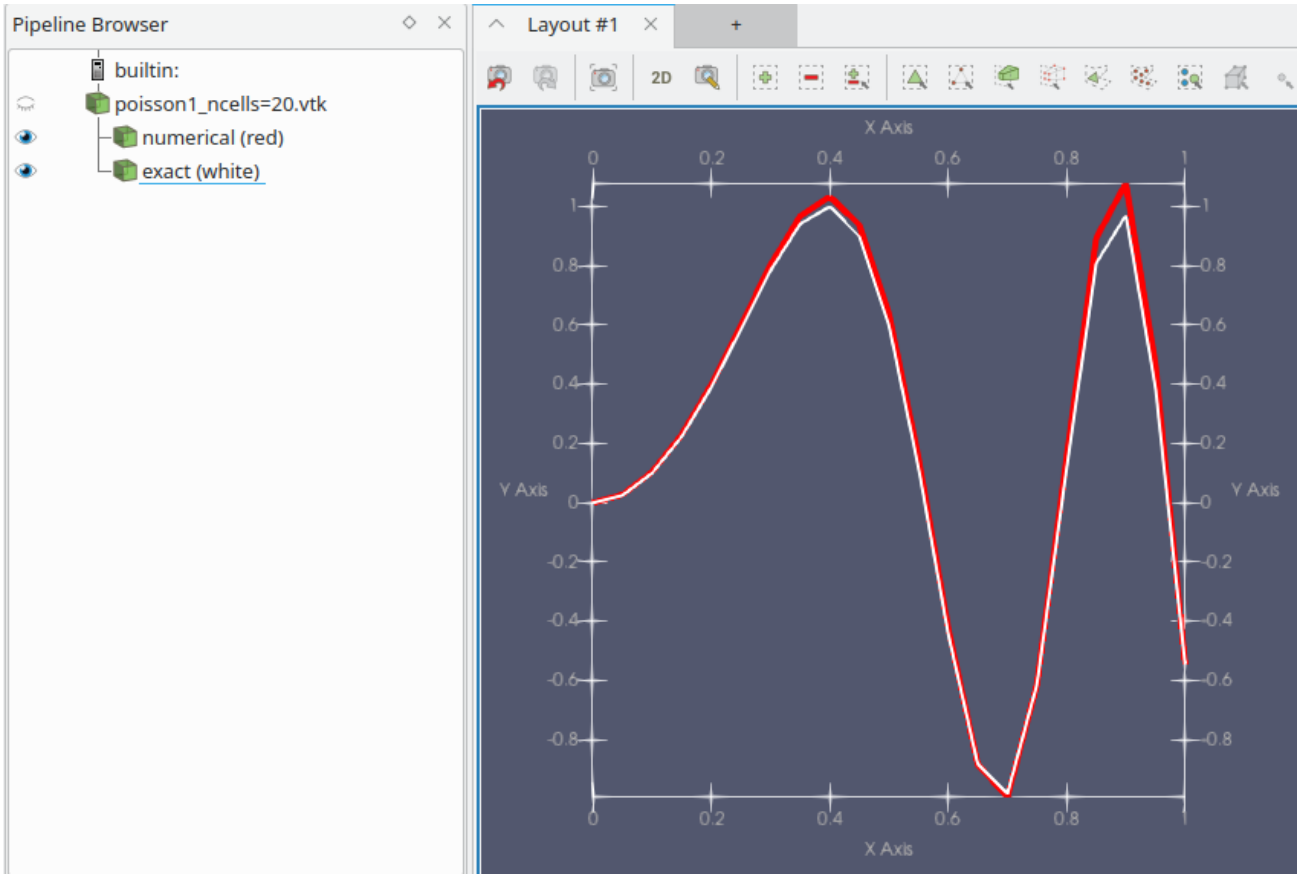


Рис. 2: Сравнение точного и численного решений уравнения Пуассона

3.2.3.2 Детали реализации

Основная работа по решению задачи проводится в классе `TestPoisson1Worker`.

В его конструкторе происходит инициализация сетки (приватного поля класса) на отрезке $[0, 1]$ с заданным разбиением `n_cells`:

```
15 class TestPoisson1Worker{
```

В методе `solve()` производится чиленное решения задачи и вычисления нормы. Для этого последовательно

1. Строится матрица левой части и вектор правой части определяющей системы уравнений. Матрицы хранятся в разреженном формате CSR, удобном для последовательного чтения.
2. Вызывается решатель СЛАУ. Решение записывается в приватное поле класса `u`.
3. Вызывается функция вычисления нормы.

```
30   double solve(){
31     // 1. build SLAE
32     CsrMatrix mat = approximate_lhs();
33     std::vector<double> rhs = approximate_rhs();
34
35     // 2. solve SLAE
36     AmgcMatrixSolver solver;
```

```
37    solver.set_matrix(mat);
38    solver.solve(rhs, u);
39
40    // 3. compute norm2
41    return compute_norm2();
42  }
```

Функции нижнего уровня (используемые в методе `solve`):

- Сборка левой части СЛАУ. Реализует формулу (3.6). Для заполнения матрицы используется формат
  `cfd::LodMatrix`, удобный для непоследовательной записи, который в конце конвертируется CSR.

```
64  CsrMatrix approximate_lhs() const{
65    // constant h = x[1] - x[0]
66    double h = grid.point(1).x() - grid.point(0).x();
67
68    // fill using 'easy-to-construct' sparse matrix format
69    LodMatrix mat(grid.n_points());
70    mat.add_value(0, 0, 1);
71    mat.add_value(grid.n_points()-1, grid.n_points()-1, 1);
72    double diag = 2.0/h/h;
73    double nondiag = -1.0/h/h;
74    for (size_t i=1; i<grid.n_points()-1; ++i){
75      mat.add_value(i, i-1, nondiag);
76      mat.add_value(i, i+1, nondiag);
77      mat.add_value(i, i, diag);
78    }
79
80    // return 'easy-to-use' sparse matrix format
81    return mat.to_csr();
82  }
```

- Сборка правой части СЛАУ. Реализует формулу (3.7).

```
84  std::vector<double> approximate_rhs() const{
85    std::vector<double> ret(grid.n_points());
86    ret[0] = exact_solution(grid.point(0).x());
87    ret[grid.n_points()-1] = exact_solution(grid.point(grid.n_points()-1).x());
88    for (size_t i=1; i<grid.n_points()-1; ++i){
89      ret[i] = exact_rhs(grid.point(i).x());
90    }
91    return ret;
92  }
```

- Вычисление нормы. Реализует формулу (3.9).

```
94  double compute_norm2() const{
95    // weights
96    double h = grid.point(1).x() - grid.point(0).x();
97    std::vector<double> w(grid.n_points(), h);
98    w[0] = w[grid.n_points()-1] = h/2;
99
100   // sum
101   double sum = 0;
102   for (size_t i=0; i<grid.n_points(); ++i){
103     double diff = u[i] - exact_solution(grid.point(i).x());
104     sum += w[i]*diff*diff;
105   }
106
107   double len = grid.point(grid.n_points()-1).x() - grid.point(0).x();
108   return std::sqrt(sum / len);
109 }
```

3.3 Задание для самостоятельной работы

3.3.1 Одномерное уравнение Пуассона

Скомпиллировать и запустить программу, описанную в п. 3.2.3. Построить график полученного численного и точного решения, аналогичный рис. 2 (инструкцию по построению одномерного графика решения в Paraview см. в п B.3.1).

Построить график, подтверждающий второй порядок точности разностной схемы (3.3).

3.3.2 Двумерное уравнение Пуассона

Написать тест, аналогичный `[poisson1]`, но для двумерной задачи на двумерной регулярной сетке

$$-\left(\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}\right)=f(x,y).$$

использовать разностную схему

$$\frac{-u_{k[i-1,j]}+2u_{k[i,j]}-u_{k[i+1,j]}}{h_x^2}+\frac{-u_{k[i,j-1]}+2u_{k[i,j]}-u_{k[i,j+1]}}{h_y^2}=f_{k[i,j]},$$

где

$$k[i,j]=i+(n_x+1)j \tag{3.10}$$

– функция, переводящая парный (i,j) индекс узла регурярной сетки (i) для оси x, j для оси y) в сквозной индекс k сеточного вектора, n_x – количество ячеек сетки в направлении x.

При вычислении весов w_k для вычисления среднеквадратичного отклонения учесть наличие граничных и угловых точек:

$$w_k=\begin{cases} h_xh_y/4, & \text{для угловых точек;}\\ h_xh_y/2, & \text{для граничных неугловых точек;}\\ h_xh_y, & \text{для внутренних точек.}\end{cases}$$

Четрые угловые точки определяются как

$$i[k],j[k]=(0,0),\ (0,n_y),\ (n_x,n_y),\ (n_x,0)$$

Граничные неугловые точки:

$$\begin{aligned} i[k],j[k]=\ & \overline{1,n_x-1},0; && \text{нижняя сторона,}\\ & n_x,\overline{1,n_y-1}; && \text{правая сторона,}\\ & \overline{1,n_x-1},n_y; && \text{верхняя сторона,}\\ & 0,\overline{1,n_y-1}; && \text{левая сторона.}\end{aligned}$$

Функции, переводящие сквозной индекс в пару i,j, имеют вид

$$\begin{aligned} i[k]&=\operatorname{mod}(k,(n_x+1)), && // \text{ остаток от деления,}\\ j[k]&=\lfloor k/(n_x+1)\rfloor, && // \text{ целая часть от деления.}\end{aligned} \tag{3.11}$$

Использовать класс `cfd::RegularGrid2D` для задания сетки. Функции перевода индексов узлов из сквозных в парные и обратно реализованы в классе двумерной регулярной сетки:

- `cfd::RegularGrid2D::to_split_point_index`
- `cfd::RegularGrid2D::to_linear_point_index`

В случае, если решатель системы линейных уравнений не решает построенную матрицу, использовать функцию `cfd::dbg::print` для отлаточной печати матрицы в консоль (размерность задачи должна быть небольшой).

Для иллюстрации двумерного решения в Paraview использовать изолинии (B.3.2) и трёхмерные поверхности (B.3.3).

Построить график сходимости для двумерного случая. Следует иметь ввиду, что на графике сходимости по оси абсцисс отложено линейное разбиение, вычисляемое как $n = 1/h$, где h – это характерный линейный размер ячейки. Для двумерных сеток этот линейный размер сетки можно вычислить через среднюю площадь ячейки A как $h = \sqrt{A}$, которую в свою очередь можно получить, разделив общую площадь на количество ячеек: $A = |D|/N$. Тогда, в случае единичного квадрата, линейное разбиение будет равно $n = \sqrt{N}$.

4 Лекция 4 (02.03)

4.1 Форматы хранения разреженных матриц

4.1.1 CSR-формат

При реализации решателей систем сеточных уравнений важно учитывать разреженный характер используемых в левой части. То есть избегать хранения и ненужных операций с нулевыми элементами матрицы.

Хотя рассмотренные ранее алогритмы конечноразностных аппроксимаций на структурированных сетках давали трёх- (для одномерных задач) или пятидиагональную (для двумерных) сеточную матрицу, здесь будем рассматривать общие форматы хранения, не привязанные к конкретному шаблону.

Любой общий формат хранения должен хранить информацию о шаблоне матрице (адресах ненулевых элементов) и значениях матричных коэффициентов в этом шаблоне.

В CSR (Compressed sparse rows) формате все ненулевые элементы хранятся в линейном массиве `vals`. А шаблон матрицы – в двух массивах

- массиве колонок `cols` – значений колонок для соответствующих ему значений из массива `vals`,
- массиве адресов `addr` – индексах массива `vals`, с которых начинается описание соответствующей строки.

В конце массива `addr` добавляется общая длина массива `vals`.

Таким образом, длины массивов `vals`, `cols` равны количеству ненулевых элементов матрицы, а длина массива `addr` равна количеству строк в матрице плюс один.

Для облегчения процедур поиска описание каждой строки должно идти последовательно с увеличением индекса колонки.

Для примера рассмотрим следующую матрицу

$$\begin{pmatrix} 2.0 & 0 & 0 & 1.0 \\ 0 & 3.0 & 5.0 & 4.0 \\ 0 & 0 & 6.0 & 0 \\ 0 & 7.0 & 0 & 8.0 \end{pmatrix} \tag{4.1}$$

Массивы, описывающие матрицу в формате CSR примут вид

	$row = 0$	$row = 1$	$row = 2$	$row = 3$	
$vals =$	$2.0, 1.0,$	$3.0, 5.0, 4.0,$	$6.0,$	$7.0, 8.0$	
$cols =$	$0, \quad 3,$	$1, \quad 2, \quad 3,$	$2,$	$1, \quad 3$	
$addr =$	$0,$	$2,$	$5,$	$6,$	8

Рассмотрим реализацию базовых алгоритмов для матриц, заданных в этом формате.

Пусть матрица задана следующими массивами:

```
std::vector<double> vals; // массив значений
std::vector<size_t> cols; // массив столбцов
std::vector<size_t> addr; // массив адресов
```

Число строк в матрице:

```
size_t nrows = addr.size()-1;
```

Число элементов в шаблоне (ненулевых элементов)

```
size_t n_nonzeros = vals.size();
```

Число ненулевых элементов в заданной строке ‘irow‘

```
size_t n_nonzeros_in_row = addr[irow+1] - addr[irow];
```

Умножение матрицы на вектор ‘v‘ (длина этого вектора должна быть равна числу строк в матрице). Здесь реализуется суммирование вида

$$r_i = \sum_{j=0}^{N-1} A_{ij} v_j,$$

при этом избегаются лишние операции с нулями

```
// число строк в матрице и длина вектора v
size_t nrows = addr.size() - 1;
// массив ответов. Инициализируем нулями
std::vector<double> r(nrows, 0);
// цикл по строкам
for (size_t irow=0; irow < nrows; ++irow){
  // цикл по ненулевым элементам строки irow
  for (size_t a = addr[irow]; a < addr[irow+1]; ++a){
    // получаем индекс колонки
    size_t icol = cols[a];
    // значение матрицы на позиции [irow, icol]
    double val = vals[a];
    // добавляем к ответу
    r[irow] += val * v[icol];
  }
}
```

Поиск значения элемента матрицы по адресу `(irow, icol)` с учётом локально сортированного вектора `cols`

```
using iter_t = std::vector<size_t>::const_iterator;
// указатели на начало и конец описания строки в массиве cols
iter_t it_start = cols.begin() + addr[irow];
iter_t it_end = cols.begin() + addr[irow+1];
```

```
// поиск значения icol в отсортированной последовательности [it_start, it_end)
iter_t fnd = std::lower_bound(it_start, it_end, icol);
if (fnd != it_end && *fnd == icol){
  // если нашли, то определяем индекс найденного элемента в массиве cols
  size_t a = fnd - cols.begin();
  // и возвращаем значение из vals по этому индексу
  return vals[a];
} else {
  // если не нашли, значит элемент [irow, icol] находится вне шаблона. Возвращаем 0
  return 0;
}
```

Формат CSR обеспечивает максимальную компактность хранения разреженной матрицы и при этом удобен для последовательной итерации по элементам матрицы (операции умножения матрицы на вектор), но его существенным недостатком является высокая сложность добавления нового элемента в шаблон.

4.1.2 Массив словарей

При реализации сеточных методов решения дифференциальных уравнений работу с матрицами можно разбить на два этапа: сборка матриц и их непосредственное использование. Сборка матрицы в свою очередь может быть разделена на этап вычисление шаблона матрицы (символьная сборка) и непосредственное вычисление коэффициентов матрицы (числовая сборка).

На этапе использования матрицы основной операцией является умножение матрицы на вектор, где наиболее эффективным является CSR-формат.

В случае использования неструктурированных сеток этап символьной сборки является нетривиальной операцией и сводится к неупорядоченному добавлению новых элементов в шаблон матрицы. Как было отмечено ранее, такая операция в случае использования CSR формата неэффективна.

Поэтому часто для этапов сборки и расчёта используют разные форматы хранения матриц, первый из которых оптимизирован для операции вставки, а второй – для операции умножения на вектор. В качестве формата, оптимизированного для вставки, можно представить формат массива словарей (List of dictionaries), где каждая строка матрицы описывается словарём, ключём которого является индекс колонки, а значением – величина соответствующего матричного коэффициента.

С использованием синтаксиса C++ такой формат может быть описан следующим образом:

```
std::vector<std::map<size_t, double>> data;
```

Марица вида (4.1) в таком формате примет вид

```
data = {
  {0: 2.0, 3: 1.0},
  {1: 3.0, 2: 5.0, 3: 4.0},
  {2: 6.0},
  {1: 7.0, 3: 8.0}
};
```

Добавление нового матричного коэффициента сведётся к вставке элемента в словарь:

```
data[i][j] = value;
```

А основной операцией для такого формата будет служить конверсия в CSR:

```
std::vector<size_t> addr{0};
std::vector<size_t> cols;
std::vector<double> vals;
for (size_t irow=0; irow < data.size(); ++irow){
  for (auto it: data[irow]){
    cols.push_back(it.first);
    vals.push_back(it.second);
  }
  addr.push_back(addr.back() + data[irow].size());
}
```

Поскольку данные в контейнере типа `std::map` итерируются в отсортированном по ключам порядке, то полученный в результе массив `cols` также является локально отсортированным.

5 Лекция 5 (09.03)

5.1 Решение СЛАУ

В рассмотренных ранее примерах использовался алгебраический многосеточный итерационный решатель, который имеет существенное время инициализации. Ниже рассмотрим некоторые более простые итерационные способы решения систем уравнений, которые, хотя и имеют значительно худшую сходимость, но не требуют дорогой инициализации.

5.1.1 Метод Якоби

Будем рассматривать систему уравнений вида

$$\sum_{j=0}^{N-1} A_{ij} u_j = r_i, \quad i = \overline{0, N-1}$$

относительно неизвестного сеточного вектора $\{u\}$.

В классическом виде алгоритм Якоби формулируется в виде

$$\hat{u}_i = \frac{1}{A_{ii}} \left(r_i - \sum_{j \neq i} A_{ij} u_j \right)$$

Произведём некоторые преобразования

$$\begin{aligned} \hat{u}_i &= \frac{1}{A_{ii}} \left(r_i - \sum_{j} A_{ij} u_j + A_{ii} u_i \right) \\ &= u_i + \frac{1}{A_{ii}} \left(r_i - \sum_{j} A_{ij} u_j \right) \end{aligned}$$

Таким образом, программировать итерацию этого алгоритма, обновляющую значения массива $\{u\}$, можно в виде

$\check{u} = u;$
for $i = \overline{0, N-1}$
$\quad u_i \mathrel{+}= \frac{1}{A_{ii}} \left(r_i - \sum_{j=0}^{N-1} A_{ij} \check{u}_j \right)$
endfor

5.1.2 Метод Зейделя

Формулируется в виде

$$\hat{u}_i = \frac{1}{A_{ii}} \left(r_i - \sum_{j<i} A_{ij} \hat{u}_j - \sum_{j>i} A_{ij} u_j \right).$$

Поскольку этот метод неявный относительно уже найденных на итерации значений, то в отличии от метода Якоби этот алгоритм не требует создания временного массива $\hat{u}$ при программировании. Псевдокод для реализации итерации этого метода можно записать как

for $i = \overline{0, N-1}$

$$u_i \mathrel{+}= \frac{1}{A_{ii}} \left(r_i - \sum_{j=0}^{N-1} A_{ij} u_j \right)$$

endfor

5.1.3 Метод последовательных верхних релаксаций (SOR)

Этот метод основан на добавлении к решению результатов итераций Зейделя с коэффициентом $\omega > 1$. То есть он измениает решение по тому же принципу, что и метод Зейделя, но искуственно увеличивает эту добавку.

Формулируется этот метод в виде

$$\hat{u}_i = (1 - \omega) u_i + \frac{\omega}{A_{ii}} \left(r_i - \sum_{j<i} A_{ij} \hat{u}_j - \sum_{j>i} A_{ij} u_j \right).$$

Для устойчивости метода необходимо $\omega < 2$. В частности, для одномерных задач, заданных на единичном отрезке, для оптимальной сходимости можно использовать соотношение $\omega \approx 2 - 5h$, где h – шаг сетки.

Итерация этого метода по аналогии с методом Зейделя может быть запрограммирована в виде

for $i = \overline{0, N-1}$

$$u_i \mathrel{+}= \frac{\omega}{A_{ii}} \left(r_i - \sum_{j=0}^{N-1} A_{ij} u_j \right)$$

endfor

5.2 Задание для самостоятельной работы

Вернутся к рассмотрению двумерного уравнения Пуассона (п. 3.3.2). Прошлая реализация этой задачи включала в себя решение СЛАУ алгебраическим многосеточным методом с помощью класса `AmgcMatrixSolver`:

```
AmgcMatrixSolver solver;
solver.set_matrix(mat);
solver.solve(rhs, u);
```

Необходимо реализовать рассмотренные ранее методы итерационного решения СЛАУ

- метод Якоби (5.1.1),

- метод Зейделя (5.1.2),
- метод SOR (5.1.3).

и использовать их вместо многосеточного решателя.

Реализовать означенные решатели нужно в виде функций вида:

```
// Single Jacobi iteration for mat*u = rhs SLAE. Writes result into u
void jacobi_step(const cfd::CsrMatrix& mat, const std::vector<double>& rhs,
↪  std::vector<double>& u){
    ...
}
```

которые делают одну итерацию соответствующего метода без проверок на сходимость. Аргумент `u` используется как начальное значение искомого сеточного вектора. Туда же пишется итоговый результат.

Все алгоритмы основаны на вычислении выражения вида

$$\frac{1}{A_{ii}}\left(r_i - \sum_{j=0}^{N-1} A_{ij}u_j\right),$$

поэтому рекомендуется выделить отдельную функцию, которая бы вычисляла это выражение и использовалась всеми тремя решателями

```
double row_diff(size_t irow, const cfd::CsrMatrix& mat, const std::vector<double>& rhs,
↪  const std::vector<double>& u){
  const std::vector<size_t>& addr = mat.addr();   // массив адресов
  const std::vector<size_t>& cols = mat.cols();   // массив колонок
  const std::vector<double>& vals = mat.vals();   // массив значений
    ...
}
```

Дополнительно понадобится реализовать функцию, которая проверяет сходимость решения путём вычисления невязки вида

$$res = \max_i \left| \sum_j A_{ij}u_j - r_i \right|$$

и сравнения с заданным малым числом $\varepsilon = 10^{-8}$.

```
bool is_converged(const cfd::CsrMatrix& mat, const std::vector<double>& rhs, const
↪  std::vector<double>& x){
  constexpr double EPS = 1e-8;
  double residual = 0;
  // ...
  return residual < EPS;
}
```

Для реализации вспомогательных функций необходимо использоватть алгоритмы работы с CSR-матрицами из п. 4.1.1

При реализации метода SOR подобрать оптимальный параметр ω, при котором метод SOR сойдётся за минимальное число итераций.

После реализации всех методов необходимо сравнить время исполнения решателей. Замеры нужно проводить в Release-версии сборки (см. п. B.1.3). Для замера времени исполнения участка кода воспользоваться функциями

- `cfd::dbg::Tic` – вызвать до начала участка кода
- `cfd::dbg::Toc` – вызвать после окончания участка кода

Код решения СЛАУ методом Якоби с вызовами профилировщика должен иметь примерно такой вид:

```
#include "dbg/tictoc.hpp"
using namespace cfd;

...

// реализация решения СЛАУ
dbg::Tic("total");  // запустить таймер total
for (size_t it=0; it < max_it; ++it){
  dbg::Tic("step");  // запустить таймер step
  jacobi_step(mat, rhs, u);
  dbg::Toc("step");  // остановить таймер step

  dbg::Tic("conv-check");  // запустить таймер conv-check
  bool is_conv = is_converged(mat, rhs, u);
  dbg::Toc("conv-check");  // остановить таймер conv-check

  if (is_conv) break;
}
dbg::Toc("total");  // остановить таймер total
```

При правильном задании функций замеров, по окончанию работы в консоль должен напечататься отчёт о времени исполнения вида:

```
      total:  6.670 sec
       step:  5.220 sec
 conv-check:  1.210 sec
```

По результатам профилировки нужно заполнить таблицу

	total, s	step, s	conv-check, s	Кол-во итераций
Amg		–	–	–
Якоби				
Зейдель				
SOR($\omega = ...$)				

Здесь Amg - исходный решатель.

6 Лекция 6 (23.03)

6.1 Метод конечных объёмов

6.1.1 Уравнение Пуассона

Пространственную аппроксимацию дифференицальных операторов методом конечных объёмов рассмотрим на примере многомерного уравнения Пуассона

$$-\nabla^2 u = f, \tag{6.1}$$

которое требуется решить в области D. Разобъём эту область на непересекающиеся подобласти E_i, $i = \overline{0, N-1}$ (рис. 3). Центры ячеек обозначим как $\mathbf{c}_i$.

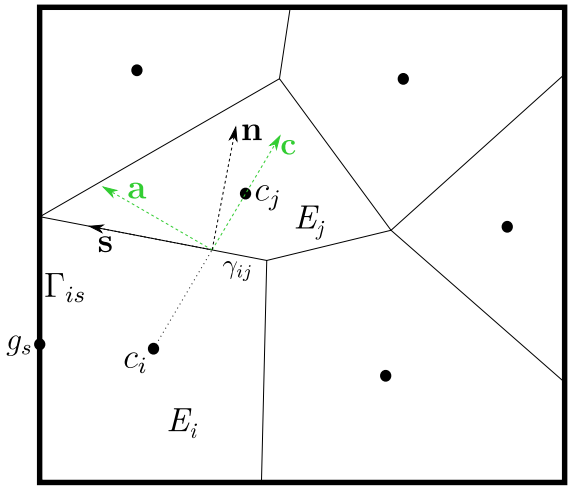


Рис. 3: Конечнообъёмная сетка

Проинтегрируем исходное уравнение по одной из подобластей E_i:

$$-\int\limits_{E_i} \nabla^2 u \, ds = \int\limits_{E_i} f \, d\mathbf{x}.$$

К интегралу в левой части применим формулу интегрирования по частям (A.13). Получим

$$-\int\limits_{\partial E_i} \frac{\partial u}{\partial n} \, ds = \int\limits_{E_i} f \, d\mathbf{x}. \tag{6.2}$$

Здесь ∂E_i – совокупность всех границ подобласти E_i, а $\mathbf{n}$ – внешняя к подобласти нормаль.

Граница ячейки E_i состоит из внутренних граней γ_{ij} (индекс j здесь соответствует индексу соседней ячейки) и граней Γ_{is}, лежащих на внешней границе расчётной области D. Тогда интеграл по общей границе ячейки распишется через сумму интегралов по плоским поверхностям

$$\int\limits_{\partial E_i} \frac{\partial u}{\partial n} \, ds = \sum_j \int\limits_{\gamma_{ij}} \frac{\partial u}{\partial n} \, ds + \sum_s \int\limits_{\Gamma_{is}} \frac{\partial u}{\partial n} \, ds.$$

Аппроксимирум производную $\partial u/\partial n$ на каждой из граней константой. Тогда её можно вынести из под интегралов и предыдущее выражение записать в виде

$$\int\limits_{\partial E_i} \frac{\partial u}{\partial n}\, ds \approx \sum_j |\gamma_{ij}| \left(\frac{\partial u}{\partial n}\right)_{\gamma_{ij}} + \sum_s |\Gamma_{is}| \left(\frac{\partial u}{\partial n}\right)_{\Gamma_{is}} \tag{6.3}$$

Аналогично, анализируя интеграл правой части (6.2), приблизим значение функции правой части f внутри элемента E_i константой f_i, которую отнесём к центру элемента. Тогда

$$\int\limits_{E_i} f\, d\mathbf{x} \approx f_i\, |E_i|\,. \tag{6.4}$$

Сеточный вектор $\{f_i\}$ – есть конечнообъёмная аппроксимация функции $f(\mathbf{x})$ на конечнообъёмную сетку. Значения f_i при аппроксимации чаще всего находятся как значения в центрах элементов

$$f_i = f(\mathbf{c}_i).$$

Хотя иногда может быть использовано и другое определение, следующее из (6.4):

$$f_i = \frac{1}{|E_i|} \int\limits_{E_i} f(\mathbf{x})\, d\mathbf{x}.$$

6.1.1.1 Обработка внутренних граней

Для начала будем рассматривать сетки, в которых вектора $\mathbf{c}$, соединяющие центры ячеек (зедёные вектора на рис. 3), коллинеарны (или почти коллинеарны) нормалям к граням $\mathbf{n}$. В этом случае производную искомой функции по нормали к грани можно записать в виде

$$\frac{\partial u}{\partial n} = \frac{\partial u}{\partial c}.$$

Далее определим значения функции u в точках c_i, c_j как u_i, u_j. Тогда значение производной $\partial u/\partial n$ на внутренней грани конечного объёма может быть приближена конечной разностью

$$\frac{\partial u}{\partial n} = \frac{\partial u}{\partial c} \approx \frac{u_j - u_i}{h_{ij}}, \quad h_{ij} = |\mathbf{c}_j - \mathbf{c}_i|. \tag{6.5}$$

Определим pebi (perpendicular-bisector) сетки как сетки, удовлетворяющие следующим свойствам

- линии, соединяющие центры двух соседних ячеек, перпендикулярны грани между этими ячейками;
- внутренние грани делят линии, соединящие центры соседних ячеек, пополам.

Очевидно, что равномерная структурированная сетка удовлетворяет этим свойствам. Для построения неструктурированных pebi-сеток используют алгоритмы построения ячеек Вороного. Для pebi-сеток разностная схема (6.5) является симметричной разностью и, поэтому, имеет второй порядок аппроксимации.

6.1.1.2 Учёт граничных условий первого рода

Для вычисления второго слагаемого в правой части (6.3) следует расписать значение нормальной к границе производной вида

$$\left(\frac{\partial u}{\partial n}\right)_{\Gamma_{is}}.$$

Это делается с помощью граничных условий.

Пусть на центре грани Γ_{is} задано значение искомой функции

$$\mathbf{x} \in \Gamma_{is}: \quad u(\mathbf{x}) = u^{\Gamma}. \tag{6.6}$$

Аппроксимацию производных будем проводить из тех же соображений, которые использовали при анализе внутренних граней. Только вместо центра соседнего элемента c_j будем использовать центр грани g_s. В первом приближении, отбрасывая касательные производные, придём к формуле аналогичной (6.5):

$$\frac{\partial u}{\partial n} \approx \frac{u^{\Gamma} - u_i}{h_{is}}, \quad h_{is} = |\mathbf{g}_s - \mathbf{c}_i|. \tag{6.7}$$

6.1.2 Одномерный случай

Рассмотрим результат конечнообъёмной аппроксимации задачи (6.1) в одномерном случае на равномерной сетке с шагом h (рис. 4).

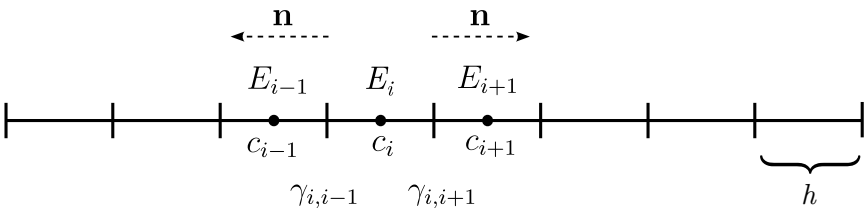


Рис. 4: Одномерная конечнообъёмная сетка

У внутренней ячейки i есть две границы: $\gamma_{i,i-1}$ и $\gamma_{i,i+1}$. Нормали по этим границам аппроксимируются по формулам **??**:

$$\gamma_{i,i-1}: \quad \frac{\partial u}{\partial n} = \frac{u_{i-1} - u_i}{h}$$

$$\gamma_{i,i+1}: \quad \frac{\partial u}{\partial n} = \frac{u_{i+1} - u_i}{h}$$

Объём ячейки в одномерном случае равен её длине h. Площадь грани следует положить единице с тем, чтобы

$$|E_i| = |\gamma| h = h.$$

Тогда, подставляя эти значения в (6.2), получим знакомую конечноразностную схему аппроксимацию уравнения Пуассона

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h} = f_i h,$$

которая имеет второй порядок точности. Разница с методом конечных разностей здесь состоит в том, что значения сеточных векторов $\{u\}$, $\{f\}$ здесь приписаны к центрам ячеек, а не к их узлам.

Это отличие проявит себя в аппроксимации граничных условий. Так, если на левой границе задано условие первого рода, то соответствующее уравнение согласно (6.7) примет вид

$$-\frac{u^\Gamma - u_0}{h/2} - \frac{u_1 - u_0}{h} = f_0 h.$$

В методе конечных разностей это условие выразилось бы в виде $u_0 = u^\Gamma$.

6.1.3 Сборка системы линейных уравнений

Подставим все полученные аппроксимации (6.5), (6.7) в уравнение (6.2):

$$-\sum_j \frac{|\gamma_{ij}|}{h_{ij}}(u_j - u_i) - \sum_{s\in \mathrm{I}} \frac{|\Gamma_{is}|}{h_{is}}\left(u^\Gamma - u_i\right) = f_i|E_i|.$$

Здесь первое слагаемое в левой части отвечает за потоки через внутренние границы, второе – граничные условия первого рода. Далее перенесём все известные значения в правую часть и окончательно получим линейное уравнение для i-го конечного объёма:

$$\sum_j \frac{|\gamma_{ij}|}{h_{ij}}(u_i - u_j) + \sum_{s\in \mathrm{I}} \frac{|\Gamma_{is}|}{h_{is}} u_i = f_i|E_i| + \sum_{s\in \mathrm{I}} \frac{|\Gamma_{is}|}{h_{is}} u^\Gamma \tag{6.8}$$

Таким образом мы получили систему из N (по количеству подобластей) линейных уравнений относительно неизвестного сеточного вектора $\{u_i\}$

$$Au = b.$$

6.1.3.1 Алгоритм сборки в цикле по ячейкам

Матрицу A и правую часть b системы (6.8) можно собирать в цикле по ячейкам: строчка за строчкой. Такой алгоритм выглядел бы следующим образом

for $i = \overline{0, N-1}$ – цикл по строкам СЛАУ
  $b_i = |E_i| f_i$
  for $j \in$ nei(i) – цикл по ячейкам, соседним с ячейкой i
    $v = |\gamma_{ij}|/h_{ij}$
    $A_{ii} \mathrel{+}= v$
    $A_{ij} \mathrel{-}= v$
  endfor
  for $s \in$ bnd1(i) – цикл по граням ячейки i с условиями первого рода
    $v = |\Gamma_{is}|/h_{is}$
    $A_{ii} \mathrel{+}= v$
    $b_i \mathrel{+}= u^\Gamma v$
  endfor
endfor

Первым недостатком такого алгоритма является наличие вложенных циклов. Во-вторых, коэффициент, отвечающий за поток через внутреннюю грань γ_{ij}, равный $|\gamma_{ij}|/h_{ij}$ в таком алгоритме будет учитываться дважды: в строке i и в строке j.

6.1.3.2 Алгоритм сборки в цикле по граням

Вместо общего цикла по ячейкам, будем использовать цикл по граням. В таком цикле коэффициенты потоков будут вычисляться один раз и вставляться сразу в две строки матрицы, соответствующие соседним с гранью ячейкам. Вложенных циклов в такой постановке удаётся избежать, потому что у грани есть только две соседние ячейки (в то время как у ячейки может быть произвольное количество соседних граней).

Разделим все грани на исходной сетки на внутренние и граничные (отдельный набор для каждого вида граничных условий). Тогда для внутренних граней можно записать

$$\begin{aligned}
&\textbf{for } s \in \text{internal} && \text{– цикл по внутренним граням}\\
&\quad i, j = \text{nei_cells(s)} && \text{– две ячейки, соседние с текущей гранью}\\
&\quad v = |\gamma_{ij}|/h_{ij}\\
&\quad A_{ii}{+}{=}v; \quad A_{jj}{+}{=}v && \text{– диагональные коэффициенты матрицы}\\
&\quad A_{ij}{-}{=}v; \quad A_{ji}{-}{=}v && \text{– внедиагональные коэффициенты матрицы}\\
&\textbf{endfor}
\end{aligned} \tag{6.9}$$

Граничные условия учитываются в отдельных циклах. Здесь будем учитывать, что у грани, принадлежащей границе области, есть только одна соседняя ячейка. Условия первого рода:

$$\begin{aligned}
&\textbf{for } s \in \text{bnd1} && \text{– грани с условиями первого рода}\\
&\quad i = \text{nei_cells(s)} && \text{– соседняя с граничной гранью ячейка}\\
&\quad v = |\Gamma_{is}|/h_{is}\\
&\quad A_{ii}{+}{=}v\\
&\quad b_i{+}{=}u^{\Gamma}v\\
&\textbf{endfor}
\end{aligned} \tag{6.10}$$

Первое слагаемое в правой части (6.8) учтём отдельным циклом:

$$\begin{aligned}
&\textbf{for } i = \overline{0, N-1} && \text{– цикл по ячейкам}\\
&\quad b_i = |E_i| f_i\\
&\textbf{endfor}
\end{aligned} \tag{6.11}$$

6.2 Задание для самостоятельной работы

В тесте `poisson1-fvm` из файла `poisson_fvm_solve_test.cpp` реализовано решение одномерного уравнения Пуассона с граничными условиями первого рода. Проводится расчёт на сгущающихся сетках с количеством ячеек от 10 до 1000 и расчитываются среднеквадратичные нормы отклонения полученного численного решения от точного. Решения сохраняются в vtk-файлы `poisson1_fvm_n={}.vtk`.

Отталкиваясь от этой реализации необходимо:

1. написать аналогичный тест для двумерного уравнения и случая неструктурированных сеток,
2. провести серию расчётов на сгущающихся сетках разных типов (структурированных, pebi и скошенных)
3. визуализировать решение, полученное на этих сетках
4. построить графики сходимости решения и определить порядок аппроксимации метода

Построение неструктурированных сеток В папке `test_data` корневой директории репозитория лежат скрипты построения сеток в программе `HybMesh`:

- `pebigrid.py` – pebi–сетка,
- `tetragrid.py` – сетка, состоящая из произвольных (скошенных) трех- и четырехугольников.

Инструкции по запуску этих скриптов смотри п. B.4. Эти скрипты строят равномерную неструктурированную сетку в единичном квадрате и записывают её в файл vtk, который впоследствии можно загрузить в расчётную программу. В каждом из скриптов есть параметр `N`, означающий примерное количество ячеек в итоговой сетке. Меняя его значение можно строить сетки разного разрешения.

Для загрузки построенной сетки в решатель необходимо файл с сеткой поместить в каталог `test_data` и далее загрузить её в класс `UnstructuredGrid2D`. Нижеследующий код прочитает файл `test_data/pebigrid.vtk` и создаст рабочий класс с использованием прочитанной сетки

```
std::string fn = test_directory_file("pebigrid.vtk");
UnstructuredGrid2D grid = UnstructuredGrid2D::vtk_read(fn);
```

Рекомендации к программированию При написании новых тестов следует переиспользовать уже написанный код, избегая копирования. Для этого необходимо пользоваться механизмами наследования классов. В частности, следует обратить внимание, что все сетки наследуются от единого интерфейса `IGrid`. А уже написанный “одномерный” код в своей алгоритмической части использует только функции этого интерфейса.

7 Лекция 7 (30.03)

7.1 Граничные условия второго рода

Учёт условий второго рода тривиален. Если на центре грани Γ_{is} задано значение нормальной производной

$$\mathbf{x} \in \Gamma_{is}: \quad \frac{\partial u}{\partial n} = q(\mathbf{x}), \tag{7.1}$$

то это значение просто подставляется вместо соответствующей производной в (6.3).

По аналогии с (6.10) учёт граничных условий второго рода при сборке по граням будет иметь следующий вид

$$\begin{array}{ll} \textbf{for } s \in \text{bnd2} & \text{– грани с условиями второго рода} \\ \quad i = \text{nei_cells(s)} & \text{– соседняя с граничной гранью ячейка} \\ \quad b_i \mathrel{+}= |\Gamma_{is}| q & \\ \textbf{endfor} & \end{array} \tag{7.2}$$

7.2 Граничные условия третьего рода

Теперь рассмотрим условия третьего рода

$$\mathbf{x} \in \Gamma_{is}: \quad \frac{\partial u}{\partial n} = \alpha(\mathbf{x}) u + \beta(\mathbf{x}). \tag{7.3}$$

Распишем производную в форме (6.7):

$$\frac{u^\Gamma - u_i}{h_{is}} = \alpha u^\Gamma + \beta,$$

откуда выразим u^Γ:

$$u^\Gamma = \frac{u_i + \beta h_{is}}{1 - \alpha h_{is}}.$$

Подставляя это выражение в исходное граничное условие (7.3) получим

$$\frac{\partial u}{\partial n} = \frac{\alpha}{1 - \alpha h_{is}} u_i + \frac{\beta}{1 - \alpha h_{is}}. \tag{7.4}$$

Учёт граничных условий третьего рода при сборке по граням будет иметь вид

$$\begin{array}{ll} \textbf{for } s \in \text{bnd3} & \text{– грани с условиями третьего рода} \\ \quad i = \text{nei_cells(s)} & \text{– соседняя с граничной гранью ячейка} \\ \quad v = |\Gamma_{is}| / (1 + \alpha h_{is}) & \\ \quad A_{ii} \mathrel{-}= \alpha v & \\ \quad b_i \mathrel{+}= \beta v & \\ \textbf{endfor} & \end{array} \tag{7.5}$$

7.2.1 Универсальность условий третьего рода

Условие третьего рода (7.3) можно использовать для моделирования условий первого и второго рода. Так, условия второго рода (7.1) получаются, если положить $\alpha = 0$, $\beta = q$. А условия первого (6.6), – если

$$\alpha = \varepsilon^{-1}, \quad \beta = -\varepsilon^{-1} u^{\Gamma}, \tag{7.6}$$

где ε – малое положительное число.

Если подставить эти выражения в формулу (7.4), то можно убедится, что они дадут выражения (6.7) и (7.1) (в пределе при $\varepsilon \to 0$) соответственно.

7.3 Задание для самостоятельной работы

Сделать задачу, аналогичную п. 6.2, но использовать граничные условия 3-его рода. Необходимо имитировать условия первого рода через подход (7.6).

По формуле

$$n = \sqrt{\frac{\sum_i (u_i - u'_i)^2 V_i}{\sum_i V_i}}$$

подсчитать норму отклонения численного решения u_i задачи с использованием истинных граничных условий первого рода (6.7) от численного решения u' задачи, расчитанной с имитицией граничных условий первого рода через граничные условия третьего рода (7.4), (7.6).

Нарисовать график $n(\varepsilon)$ для расчётов на структурированной и скошенной сетках (в логарифмических координатах).

8 Лекция 8 (06.04)

8.1 Дополнительные точки коллокации на границах

До сих пор мы соотносили элементы сеточных векторов, которые получаются при аппроксимации функции на конечнообъёмную сетку, с центрами конечных объёмов. То есть точками коллокации служили центры объёмов, а длина сеточных векторов (количество точек коллокации) равнялась количеству ячеек сетки. Бывает удобно расширить набор точек коллокаций за счёт постановки точек на центры граничных граней.

Такой подход позволяет универсализировать подходы к аппроксимации перетоков черех граничные грани. То есть для каждой граничной грани вместо использования одного из алгоритмов (6.10), (7.2), (7.5) в зависимости от типа граничного условия, нужно использовать универсальный алгоритм, основанный на внутреннем приближении типа (6.5):

$$\frac{\partial u}{\partial n} \approx \frac{u_j - u_i}{h_{ij}},$$

где i - индекс ячейки, соседней с граничной гранью, j - индекс точки коллокации, соответствующей граничной грани, h_{ij} – расстояние между двумя точками коллокации. С учётом этого соотношения обработка граничных граней для сборки строк матрицы, соответствующих центрам ячеек, примет вид

for $s \in$ bnd – граничные грани
  $i =$ nei_cells(s) – соседняя с граничной гранью ячейка
  $j =$ bnd_col(s) – индекс точки коллокации, соответствующей грани
  $v = |\Gamma_{is}|/h_{is}$
  $A_{ii}{+}{=}v$
  $A_{ij}{-}{=}v$
endfor (8.1)

Строки матрицы, соответствующие граничным точкам коллокации, будут содержеть аппроксимированные граничные условия. Так, для граней с условиями первого рода будет аппроксимироваться непосредственно выражение (6.6). Алгоритмическом виде это примет вид

for $s \in$ bnd1 – грани с условиями первого рода
  $j =$ bnd_col(s) – индекс точки коллокации, соответствующей грани
  $A_{jj} = 1$
  $b_j = u^{\Gamma}$
endfor (8.2)

Для условий третьего рода (7.3) примем аппроксимацию

$$\frac{u_j - u_i}{h_{ij}} \approx \alpha u_j + \beta.$$

По прежнему будем считать, что i – точка коллокации, отнесённая к центру приграничной ячейки, j – точка коллокации отнесённая к центру граничной грани с условиями третьего рода. При сборки

СЛАУ просто запишем эту аппроксимацию в строки матриц, соответствующие граням с условиями третьего рода:

$$
\begin{aligned}
&\textbf{for } s \in \text{bnd3} && \text{– грани с условиями третьего рода}\\
&\quad i = \text{nei_cells(s)} && \text{– соседняя с граничной гранью ячейка}\\
&\quad j = \text{bnd_col(s)} && \text{– индекс точки коллокации, соответствующей грани}\\
&\quad A_{jj} = 1/h_{is} - \alpha\\
&\quad A_{ji} = -1/h_{is}\\
&\quad b_j = \beta\\
&\textbf{endfor}
\end{aligned}
\tag{8.3}
$$

Условия второго рода получаются из условий третьего упрощением $\alpha = 0$.

Преимуществами такого подхода является:

- Более очевидный учёт граничных условий в отдельной строке СЛАУ,
- Наличие явно выраженного граничного значения функции в сеточном векторе.

8.1.1 Пример

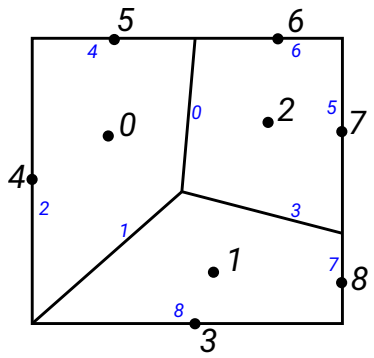


Рис. 5: Расширенный набор точек коллокации

На рис. 5. представлена конечнообъёмная сетка, содержащая три ячейки и девять граней. Индексация граней обозначена синими цифрами. Всего три внутренних грани и шесть граничных. Согласно стандартной методике конечных объёмов сеточная функция будет представлена массивом из трёх элементов. В расширенном наборе будет девять точек коллокации (обозначены чёрными кругами и проиндексированы чёрными цифрами): три соответствуют центрам ячеек и ещё шесть – центрам граничных граней.

Пусть в области с рис. 5 нужно решить уравнение Пуассона (6.1). Пусть на ниженей грани задано условие первого рода: $u = C$, а на правой – условие третьего рода: $\partial u/\partial n = \alpha u + \beta$.

Старый подход Согласно ранее рассмотренному методу конечных объёмов аппроксимация задачи в ячейке с индексом 1 будет иметь следующий вид

$$\frac{u_1 - u_0}{h_{10}}|\gamma_1| + \frac{u_1 - u_2}{h_{12}}|\gamma_3| + \frac{u_1 - C}{h_{13}}|\gamma_8| + \frac{\alpha u_1 + \beta}{1 - \alpha h_{18}}|\gamma_7| = V_1 f_1.$$

Общая размерность матрицы СЛАУ при таком подходе будет равна 3×3, а её элементы в 1-ой строке равны

$$a_{10} = -\frac{|\gamma_1|}{h_{10}}, \quad a_{12} = -\frac{|\gamma_3|}{h_{12}}, \quad a_{11} = \frac{|\gamma_1|}{h_{10}} + \frac{|\gamma_3|}{h_{12}} + \frac{|\gamma_8|}{h_{13}} + \frac{\alpha|\gamma_7|}{1 - \alpha h_{18}}.$$

Справа в 1-ой строке будет стоять

$$b_1 = V_1 f_1 + \frac{C|\gamma_8|}{h_{13}} - \frac{\beta|\gamma_7|}{1 - \alpha h_{18}}.$$

Новый подход В расширенным набором точек коллокаций матрица правой части будет иметь размерность 9×9. Из них первые три будут собираться согласно классической процедуре метода конечных объёмов, но учитывая наличие дополнитиельных точек коллокации в центрах граничных граней. Так, 1-ое уравнение итоговой СЛАУ примет вид

$$\frac{u_1 - u_0}{h_{10}}|\gamma_1| + \frac{u_1 - u_2}{h_{12}}|\gamma_3| + \frac{u_1 - u_3}{h_{13}}|\gamma_8| + \frac{u_1 - u_8}{h_{18}}|\gamma_7| = V_1 f_1.$$

Остальные шесть уравнений будут представлять из себя аппроксимацию граничных условий для соответствующих граней. Так, 3-е уравнение будет соответствовать условию первого рода на грани γ_8:

$$u_3 = C,$$

а уравнение 8 – условию третьего рода на грани γ_7:

$$\frac{u_8 - u_1}{h_{18}} = \alpha u_8 + \beta$$

Переводя рассмотренные уравнения в матричные коэффициенты, получим следующие ненулевые коээфиициенты итоговой матрицы $\{a_{ij}\}$ и вектора правой части $\{b_i\}$. Для 1-ой строки

$$a_{10} = -\frac{|\gamma_1|}{h_{10}}, \quad a_{12} = -\frac{|\gamma_3|}{h_{12}}, \quad a_{13} = -\frac{|\gamma_8|}{h_{13}}, \quad a_{18} = -\frac{|\gamma_7|}{h_{18}}, \quad a_{11} = -(a_{10} + a_{12} + a_{13} + a_{18}), \quad b_1 = V_1 f_1,$$

для 3-ей строки

$$a_{33} = 1, \quad b_3 = C,$$

для 8-ой строки

$$a_{81} = -\frac{1}{h_{18}}, \quad a_{88} = -a_{81} - \alpha, \quad b_8 = \beta$$

8.2 Задание для самостоятельной работы

1. Решить задачу из п. 6.2, с истинными граничными условиями первого рода, и с граничными условиями первого рода, поставленными через условия третьего рода.

2. Убедится, что полученный ответ с точностью то ошибки решения СЛАУ (10^{-8}) совпадает с результатами, полученным в двух предыдущих заданиях.

3. Для постановки с условиями третьего рода построить график максимального отклонения граничного значения полученного сеточной функции $\{u\}$ от точного решения в зависимости от

выбранного ε^{-1} из (7.6) от точного решения:

$$n_b = \max_{j>N_c} |u_j - u^e(\mathbf{x}_j)|,$$

N_c – количество ячеек сетки, j – индекс граничных точек коллокации, x_j – координата точки коллокации.

Рекомендации к программированию

- В структуру `DirichletFaces` необходимо добавить поле `icol` – индекс точки коллокации для текущей грани. Нумерацию граничных точек коллокации нужно вести начиная от `grid.n_cells()` согласно порядку вектора `_dirichlet_faces`.
- Следует учитывать, что вектор неизвестных `_u` будет иметь длину, большую чем количество ячеек. В частности, это может привести к ошибке сохранения в vtk. Чтобы ограничить количество сохраняемых элементов вектора, вызов сохранения необходимо осуществять с дополнительным параметром:

```
VtkUtils::add_cell_data(_u, "numerical", filename, _grid.n_cells());
```

9 Лекция 9 (13.04)

9.1 Учёт скошенности сетки в двумерной МКО-аппроксимации

9.1.1 Уточнённая аппроксимация нормальной производной

Ключевым моментом для аппроксимации оператора Лапласа методом конечных объёмов является выражение нормально производной по грани конечного объёма. До сих пор мы пользовались соотношением (6.5), которая на скошенных сетках приводила к большим численным погрешностям и не давала желаемый второй порядок аппроксимации (см. задачу из п. 6.2). Для устранения этого недостатка распишем нормальную производную по грани более точно.

Для двумерного случая распишем градиент u в системе координат, образованной еодиничными векторами нормали $\mathbf{n}$ и касательной $\mathbf{s} = (-n_y, n_x)$ к грани γ_{ij} (см. рисунок рис. 3):

$$\nabla u = \frac{\partial u}{\partial n}\mathbf{n} + \frac{\partial u}{\partial s}\mathbf{s}.$$

С помощью значений функции в точках коллокации i и j мы можем аппроксимировать производную

$$\left.\frac{\partial u}{\partial c}\right|_{\gamma_{ij}} = \frac{u_j - u_i}{h_{ij}} + O(h^2)$$

в центре грани γ_{ij} вторым порядком точности как симметричную разность. Эта производная есть проекция градиента функции u на вектор $\mathbf{c}$. Тогда можно записать:

$$\frac{\partial u}{\partial c} = \nabla u \cdot \mathbf{c} = \frac{\partial u}{\partial n}\mathbf{n}\cdot\mathbf{c} + \frac{\partial u}{\partial s}\mathbf{s}\cdot\mathbf{c} = \frac{\partial u}{\partial n}\cos(\widehat{\mathbf{n},\mathbf{c}}) + \frac{\partial u}{\partial s}\sin(\widehat{\mathbf{n},\mathbf{c}}).$$

Отсюда выразим искомую производную

$$\frac{\partial u}{\partial n} = \frac{\partial u}{\partial c}\frac{1}{\cos(\widehat{\mathbf{n},\mathbf{c}})} - \frac{\partial u}{\partial s}\tan(\widehat{\mathbf{n},\mathbf{c}}). \tag{9.1}$$

Для записи аппроксимации второго порядка нормальной производной необходимо с заданной точностью аппроксимировать производную по касательной к грани. В двумерном случае введём вспомогательные точки $\mathbf{c}^+$ и $\mathbf{c}^-$ как показано на рисунке (6). Тогда в центре грани запишем искомую симметричную разность

$$\frac{\partial u}{\partial s} = \frac{u^+ - u^-}{|\mathbf{c}^+ - \mathbf{c}^-|} + O(h^2).$$

Тогда получим аппроксимацию второго порядка искомой производной:

$$\frac{\partial u}{\partial n} \approx \frac{u_j - u_i}{|\mathbf{c}_j - \mathbf{c}_i|}\frac{1}{\cos(\widehat{\mathbf{n},\mathbf{c}})} - \frac{u^+ - u^-}{|\mathbf{c}^+ - \mathbf{c}^-|}\tan(\widehat{\mathbf{n},\mathbf{c}}). \tag{9.2}$$

Точки $c^\pm$ не являются точками коллокации, поэтому значения $u^\pm$ явно присутствовать в аппроксимационной схеме не могут. Однако, их можно проинтерполировать через значения u в точках коллокации.

9.1.2 Интерполяция значения функции во вспомогательных точках

Для простой линейной интерполяции функции, заданной на двумерной плоскости, необходимо три узловые точки, не лежащие на одной прямой. Пусть двумя из этих трёх точек будут уже обозначенные $\mathbf{c}_i$ и $\mathbf{c}_j$. Третью точку $\mathbf{c}_k$ будем выбирать из коллокации, соседних с временной точкой: центров ячеек и центров граничных граней, которые содержат точку $\mathbf{c}^-$.

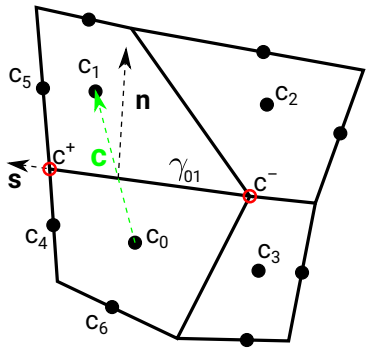


Рис. 6: Пример расположения вспомогательных точек (показаны красными кругами) для аппроксимации производной по грани

На примере с рис. 6 для точки $\mathbf{c}^-$ соседними будут точки c_0, c_1, c_2, c_3. Точки $\mathbf{c}_0$ и $\mathbf{c}_1$ – это первые две точки интерполяции (узлы i и j в предыдущих обозначениях). Третью точку выберем как ближайшую к $\mathbf{c}^-$ из остальных, а именно c_3. Для точки $\mathbf{c}^+$ третьей точкой будет c_4, выбранная аналогичным образом из набора c_0, c_1, c_4, c_5.

Отметим, что для написания надёжного кода необходимо удостовериться, что выбранная третья точка не лежит на одной прямой с двумя первыми. И если лежит, то необходимо выбрать следующего кандидата. А если кандидатов больше нет, то поиск необходимо продолжить среди несоседних точек коллокации.

После того, как три узловые точки интерполяции выбраны можно выразить вспомогательные значения

$$u^- = C_i^- u_i + C_j^- u_j + C_k^- u_k,$$
$$u^+ = C_i^+ u_i + C_j^+ u_j + C_s^+ u_s,$$

где коэффициенты интерполяции C вычисляются следуя формуле (A.27). Например,

$$C_i^- = \frac{|\triangle_{jk-}|}{\triangle_{ijk}|} = \frac{(\mathbf{c}_k - \mathbf{c}_j) \times (\mathbf{c}^- - \mathbf{c}_j)}{(\mathbf{c}_j - \mathbf{c}_i) \times (\mathbf{c}_k - \mathbf{c}_i)}$$

Подставив полученные интерполяционные соотношения в (9.2), получим аппроксимацию нормальной производной в виде линейную формы

$$\frac{\partial u}{\partial n} \approx C_i u_i + C_j u_j + C_k u_k + C_s u_s. \tag{9.3}$$

Коэффициенты которой равны

$$
\begin{aligned}
C_i &= -w_1 - (C_i^+ - C_i^-)w_2, \\
C_j &= w_1 - (C_j^+ - C_j^-)w_2, \\
C_k &= C_k^- w_2, \\
C_s &= -C_s^+ w_2, \\
w_1 &= \frac{1}{|\mathbf{c}_j - \mathbf{c}_i| \cos(\widehat{\mathbf{n}, \mathbf{c}})}, \\
w_2 &= \frac{\tan(\widehat{\mathbf{n}, \mathbf{c}})}{|\mathbf{c}^+ - \mathbf{c}^-|}
\end{aligned}
$$

9.1.3 Производная по границе

Из-за использования расширенного набора точек коллокации, процедура аппроксимации нормальной производной по граничной грани ничем не отличается от аналогичной процедуры для внутренней грани. Так для точки коллокации $\mathbf{c}_4$ линейная форма, аппроксимирующая нормальную производную, будет содержать значения u_4, u_0, u_6, u_1, последние два из которых используются для интерполяции вспомогательных точек.

Рассмотрим условие третьего рода (7.3) на стенке, соответсвующей индексу коллокации j, соседней с ячейкой i. С учётом (9.3) получим:

$$
\frac{\partial u}{\partial n} = C_i u_i + C_j u_j + C_k u_k + C_s u_s = \alpha u_j + \beta,
$$

тогда j-ое уравнение СЛАУ будет иметь вид

$$
C_i u_i + (C_j - \alpha) u_j + C_k u_k + C_s u_s = \beta. \tag{9.4}
$$

9.1.4 Сборка СЛАУ для уравнения Пуассона

Рассмотрим процедуру сборки системы линейных уравнений для решения уравнения Пуассона. Будем использовать цикл по граням по аналогии с (6.9). Ключевым моментом алгоритма будет являтся вычисление линейной формы вида (9.3). Отметим, что цикл по внутренним (6.9) и граничным (8.1) граням можно объединить в один цикл по всем граням. При этом следует иметь ввиду, что этот цикл собирает уравнение внутри конечного объема (6.2), и поэтому вносит изменения только в стро-

ки, соответствующие центрам ячеек.

for $s \in$ faces — цикл по всем граням
  $i, j, k, s =$ dn_cells(s) — индексы точек коллокации для аппроксимации $\partial u/\partial n$
  $v_i, v_j, v_k, v_s =$ dn_coefs(s) — коэф-ты линейной формы для аппроксимации $\partial u/\partial n$
  $a =$ face_area(s) — площадь грани
  if i is cell center — если i – коллокация для центра ячейки
    $A_{ii} -= a\, v_i;$ — i-ая строка матрицы (против нормали к грани)
    $A_{ij} -= a\, v_j;$
    $A_{ik} -= a\, v_k;$
    $A_{is} -= a\, v_s;$ (9.5)
  endif
  if j is cell center — если j – коллокация для центра ячейки
    $A_{ji} += a\, v_i;$ — j-ая строка матрицы (по нормали к грани)
    $A_{jj} += a\, v_j;$
    $A_{jk} += a\, v_k;$
    $A_{js} += a\, v_s;$
  endif
endfor

В случае, если используются граничные условия второго или третьего рода, та же процедура должна быть использована для выражения на границах (9.4) по аналогии с (8.3).

for $s \in$ bnd3 — грани с условиями третьего рода
  $i, j, k, s =$ dn_cells(s) — индексы точек коллокации для аппроксимации $\partial u/\partial n$
  $v_i, v_j, v_k, v_s =$ dn_coefs(s) — коэф-ты линейной формы для аппроксимации $\partial u/\partial n$
  if j is cell center
    swap(i, j) — переворачиваем нормаль. Теперь j – коллокация по грани
    $v_i = -v_i, \quad v_j = -v_j$
    $v_k = -v_k, \quad v_s = -v_s$
  endif
  $A_{ji} = C_i, \qquad A_{jj} = C_j - \alpha$
  $A_{jk} = C_k, \qquad A_{js} = C_s$
  $b_j = \beta$
endfor

9.2 Задание для самостоятельной работы

Решить двумерную задачу Пуассона с граничными условиями первого рода, аналогичную 6.2, но использовать поправку на скошенность. Провести расчёт на сгущающихся сетках и проиллюстрировать порядок аппроксимации для структурированной, pebi и скошенной сетки. Сравить графики сходимости с аналогичными, полученными в п. 6.2.

Рекомендации к программированию Для вычисления линейной формы (9.3) использовать класс `FvmLinformFacesDn` из файла `fvm/fvm_assembler.hpp`.

Объект этого класс необходимо собрать на этапе инициализации задачи. Далее линейные формы для заданной грани вычисляются вызовом метода `FvmLinformFacesDn::linear_combination(size_t iface)`. Этот метод возвращает упорядоченную линейную комбинацию из четырех (для двумерной задачи) слагаемых. Получение индексов и коэффициентов линейной формы согласно алгоритму (9.5) осуществляется следующим образом

```
FvmLinformFacesDn faces_dn(grid);
for (size_t iface=0; iface < grid.n_faces(); ++iface){
  auto linform = faces_dn.linear_combination(iface);
  size_t i = linform[0].first;
  size_t j = linform[1].first;
  size_t k = linform[2].first;
  size_t s = linform[3].first;
  double vi = linform[0].second;
  double vj = linform[1].second;
  double vk = linform[2].second;
  double vs = linform[3].second;
}
```

10 Лекция 10 (20.04)

10.1 Учёт скошенности сетки в 3D

11 Лекция 11 (27.04)

11.1 Метод взвешенных невязок

11.2 Метод Бубнова–Галёркина

11.2.1 Степенные базисные функции

11.3 Метод конечных элементов

11.3.1 Узловые базисные функции

11.3.2 Одномерное уравнение Пуассона

11.3.2.1 Слабая интегральная постановка задачи

11.3.2.2 Линейный одномерный (пирамидальный) базис

12 Лекция 12 (04.05)

12.1 Поэлементная сборка конечноэлементных матриц

12.1.0.1 Элементные матрицы

Матрица масс

$$m_{ij}^k = \int\limits_{E^k} \phi_i(\mathbf{x})\phi_j(\mathbf{x})\, d\mathbf{x} \tag{12.1}$$

Вектор нагрузок

$$l_i^k = \int\limits_{E^k} \phi_i(\mathbf{x})\, d\mathbf{x} \tag{12.2}$$

Матрица жёсткости

$$s_{ij}^k = \int\limits_{E^k} \nabla\phi_i \cdot \nabla\phi_j\, d\mathbf{x} \tag{12.3}$$

13 Лекция 13 (11.05)

13.1 Вычисление элементных интегралов в параметрическом пространстве

Будем вычислять элементные интегралы (12.1) – (12.3) в параметрическом пространстве $\boldsymbol{\xi}$. Для этого введём преобразование координат $\mathbf{x} \to \boldsymbol{\xi}$ согласно п.A.4.2. Интеграл для определения локальной матрицы масс (12.1) в параметрическом пространстве распишется согласно формуле (A.34)

$$m_{ij}^k = \int\limits_{\tilde{E}^k} \tilde{\phi}_i^{(k)}(\boldsymbol{\xi})\tilde{\phi}_j^{(k)}(\boldsymbol{\xi})|J^{(k)}(\boldsymbol{\xi})|\,d\boldsymbol{\xi} \tag{13.1}$$

Здесь $\tilde{E}^k$ – параметрический образ конечного элемента E^k, $J^{(k)}$ - Якобиан преобразования для k-ого элемента, $\tilde{\phi}_i^{(k)}$ – часть базисной функции $\phi_{c_i^k}$, определённая в конечном элементе E^k и заданная в параметрических координатах, а c_i^k – глобальный индекс базисной функции, которая в k-ом элементе имеет локальный индекс i. Таким образом справедливо

$$\tilde{\phi}_i^{(k)}(\boldsymbol{\xi}) = \phi_{c_i^k}(\mathbf{x}(\boldsymbol{\xi}))$$

Локальный вектор нагрузок

$$l_i^k = \int\limits_{\tilde{E}^k} \tilde{\phi}_i^{(k)}(\boldsymbol{\xi})|J^{(k)}(\boldsymbol{\xi})|\,d\boldsymbol{\xi} \tag{13.2}$$

Локальная матрица жёсткости

$$s_{ij}^k = \int\limits_{\tilde{E}^k} \nabla_{\mathbf{x}}\tilde{\phi}_i^{(k)} \cdot \nabla_{\mathbf{x}}\tilde{\phi}_j^{(k)}|J^{(k)}(\boldsymbol{\xi})|\,d\boldsymbol{\xi} \tag{13.3}$$

Здесь $\nabla_{\mathbf{x}}\tilde{\phi}_i^k$ – градиент локального базиса (заданного в параметрическом пространстве) по физическим координатам. Для его вычисления следует воспользоваться формулами (A.33)

13.2 Двумерное уравнение Пуассона

13.2.0.1 Треугольный элемент. Линейный двумерный базис

Матрица масс (из (13.1)):

$$M_{ij}^E = \int\limits_0^1 \int\limits_0^{1-\xi} \phi_i(\xi,\eta)\phi_j(\xi,\eta)|J|\,d\eta d\xi = \frac{|J|}{24}\begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} \tag{13.4}$$

13.3 Разбор программной реализации МКЭ

Численное решение уравнения Пуассона с граничными условиями первого рода реализовано в файле `poisson_fem_solve_test.cpp`. Будем рассматривать решение одномерной задачи с использованием

пирамидальных базисов (тест `[poisson1-fem-lintri]`). В этом тесте определяется двумерная аналитическая функция

$$f(x) = \sin(10x^2),$$

и формулируется уравнение Пуассона с граничными условиями первого рода, для которого эта функция является точным решением. Далее уравнение Пуассона решается численно и полученный численный результат сравнивается с точным ответом. Норма полученной ошибки печатается в консоль.

В функции верхнего уровня происходит построение одномерной сетки, создание рабочего объекта, вызов решения с возвращением нормы полученной ошибки и вывод данныех (сохранение решения в vtk-файл и печать нормы в консоль):

```
200  Grid1D grid(0, 1, 10);
201  TestPoissonLinearSegmentWorker worker(grid);
202  double nrm = worker.solve();
203  worker.save_vtk("poisson1_fem.vtk");
204  std::cout << grid.n_cells() << " " << nrm << std::endl;
```

Основная работа происходит в классе `TestPoissonLinearSegmentWorker`.

13.3.1 Рабочий объект

Класс `TestPoissonLinearSegmentWorker` наследуется от `ITestPoisson1FemWorker`. В этом классе сформулированы аналитические функции, служащие правой частью, точным решением и условиями первого рода уравнения Пуассона. А этот класс в свою очередь наследуется от `ITestPoissonFemWorker`, в котором и происходит решение уравнения.

```
53 double ITestPoissonFemWorker::solve(){
54   // 1. build SLAE
55   CsrMatrix mat = approximate_lhs();
56   std::vector<double> rhs = approximate_rhs();
57   // 2. Dirichlet bc
58   for (size_t ibas: dirichlet_bases()){
59     mat.set_unit_row(ibas);
60     Point p = _fem.reference_point(ibas);
61     rhs[ibas] = exact_solution(p);
62   }
63   // 3. solve SLAE
64   AmgcMatrixSolver solver({ {"precond.relax.type", "gauss_seidel"} });
65   solver.set_matrix(mat);
66   solver.solve(rhs, _u);
67   // 4. compute norm2
68   return compute_norm2();
69 }
```

Для получения решения сначала собирается левая и правая часть системы линеных уравнений, потом происходит учёт граничных условий первого рода , вызывается решатель системы уравнений и вычислитель нормы ошибки.

Функция сборки матрицы левой части реализует сборку глобальной матрицы жёсткости через набор локальных матриц

```
84 CsrMatrix ITestPoissonFemWorker::approximate_lhs() const{
85   CsrMatrix ret(_fem.stencil());
86   for (size_t ielem=0; ielem < _fem.n_elements(); ++ielem){
87     const FemElement& elem = _fem.element(ielem);
88     std::vector<double> local_stiff = elem.integrals->stiff_matrix();
89     _fem.add_to_global_matrix(ielem, local_stiff, ret.vals());
90   }
91   return ret;
92 }
```

Основой для сборки служит специальный объёкт `_fem` класса `FemAssembler` – сборщик. Этот объёкт сначала используется для задания шаблона итоговой матрицы, потом в цикле по элементам вычисляются локальные матрицы и с помощью метода этого класса `FemAssembler::add_to_global_matrix` локальные матрицы добавляются в глобальную.

По аналогичной процедуре работает и сборка правой части `approximate_rhs`.

13.3.2 Конечноэлементный сборщик

Конечноэлементный сборщик `FemAssembler` – основной класс, хранящий всю информацию о текущей конечноэлементной аппроксимации: массив конечных элементов и их связность. Эта информация подаётся ему при конструировании (реализация в файле `cfd/fem/fem_assembler.hpp`).

```
12   FemAssembler(size_t n_bases,
13                const std::vector<FemElement>& elements,
14                const std::vector<std::vector<size_t>>& tab_elem_basis);
```

Связность `tab_elem_basis` имеет формат “элемент-глобальный базис” и определяет глобальный индекс для каждого локального базисного индекса. В рассматренных нами узловых конечных элементах базис связан с узлом сетки. То есть эта таблица – это связность локальной и глобальной нумерации узлов сетки для каждой ячейки сетки.

Конечноэлементный сборщик создаётся в методе `TestPoissonLinearSegmentWorker::build_fem` итогового рабочего класса (то есть сборщик специфичен для конкретной сетки и конкретного выбора типов элементов). Далее он пробрасывается в конструктор базового рабочего класса.

13.3.3 Концепция конечного элемента

Класс конечного элемента `FemElement` определён в файле `fem/fem_element.hpp` как

```
224 struct FemElement{
225   std::shared_ptr<const IElementGeometry> geometry;
226   std::shared_ptr<const IElementBasis> basis;
227   std::shared_ptr<const IElementIntegrals> integrals;
228 };
```

Главная задача объекта этого класса – вычисление элементных матриц, которые впоследствии используются сборщиком для создания глобальных матриц. Для расчёта элементных матриц в свою очередь требуется

- Геометрия элемента, включающая в себя правило отображения элемента из физической в параметрическую область,
- Набор локальных базисных функций, заданных в параметрическом пространстве на указанной геометрии,
- Непосредственно правило интегрирования в параметрической области.

Каждый из этих трёх алгоритмов определён через интерфейсы

- `IElementGeometry`
- `IElementBasis`
- `IElementIntegrals`

Определение конечного элемента заключается в задании конкретных реализаций этих интерфейсов.

13.3.3.1 Определение линейного одномерного элемента

. Так, в рассматриваемом нами тесте `"[poisson1-fem-linsegm]"`, используются только линейные одномерные элементы. Используется следующее определение элемента:

```
182     auto geom = std::make_shared<SegmentLinearGeometry>(p0, p1);
183     auto basis = std::make_shared<SegmentLinearBasis>();
184     auto integrals = std::make_shared<SegmentLinearIntegrals>(geom->jacobi({}));
185     FemElement elem{geom, basis, integrals};
```

Здесь последовательно определяются:

- геометрия отрезка `geom` – путём задания двух точек в физичекой плоскости `p0, p1`,
- линейный одномерный базис `basis`,
- правила интегрирования
  `integrals` по параметрическому отрезку $x \in [-1, 1]$ с использованием точных формул. Эти формулы зависят только от матрицы Якоби `jac` (размерности 1×1), которая вычисляется с использованием геометрических свойств элемента (в данном случае матрица Якоби постоянная, поэтому её можно вычислять в любой точке параметрической плоскости)

Этих трёх алгоритмов достаточно для полного определения конечного элемента `elem`.

13.3.3.2 Геометрические свойства элемента

Интерфейс `IElementGeometry`, заданный в файле `cfd/fem/fem_element.hpp`, определяет геометрические свойства элемента:

```
class IElementGeometry{
public:
  virtual ~IElementGeometry() = default;

  virtual JacobiMatrix jacobi(Point xi) const = 0;
  virtual Point to_physical(Point xi) const  { _THROW_NOT_IMP_; }
  virtual Point to_parametric(Point p) const { _THROW_NOT_IMP_; }
  virtual Point parametric_center() const {_THROW_NOT_IMP_; }
};
```

Для вычиселния элементных матриц главным геометрическим свойством элемента является функция для вычисления матрицы Якоби (`jacobi`). В простейших реализациях этого интерфейса для сиплексных геометрий матрица Якоби постоянна для любой точки, то есть функция `jacobi` возвращает один и тот же ответ вне зависимости от переданного аргумента.

Кроме того, этот интерфейс предоставляет функции преобразования координат из физического простравнства в параметрическое и обратно: `to_parametric`, `to_physical`. А также задает центральную точку в параметрическом пространстве `parametric_center`.

13.3.3.3 Элементный базис

Интерфейс для определения локального элементного базиса имеет вид

```
class IElementBasis{
public:
  virtual ~IElementBasis() = default;

  virtual size_t size() const = 0;
  virtual std::vector<Point> parametric_reference_points() const = 0;
  virtual std::vector<BasisType> basis_types() const = 0;
  virtual std::vector<double> value(Point xi) const = 0;
  virtual std::vector<Vector> grad(Point xi) const = 0;
  virtual std::vector<std::array<double, 6>> upper_hessian(Point xi) const {
   ↪  _THROW_NOT_IMP_; }
};
```

Этот интерфейс работает только с параметрическим пространстсвом и определяет следующие методы:

- `size` – количество базисных функций;

- `parametric_reference_points` – вектор из параметрических координат точек, приписанных к соответствующим базисам;
- `value` – значение базисных функций в заданной точке;
- `grad` – градиент (в параметрическом пространстве) базисных функций по заданным точкам.
- `basis_type` – тип базисной функции. До сих пор мы имели дело только с узловыми (`BasisType::Nodal`) функциями.
- `upper_hessian` – верхняя часть матрицы Гессе (вторые производные базисных функций в заданной точке)

Конкретная реализация для линейного треугольного элемента `TriangleLinearBasis` (в файле `cfd/fem/elem2d/triangle_linear.cpp`) включает в себя линейный Лагранжев базис в двумерном пространстве согласно (A.23):

```
39 size_t TriangleLinearBasis::size() const {
40   return 3;
41 }

43 std::vector<Point> TriangleLinearBasis::parametric_reference_points() const {
44   return {Point(0, 0), Point(1, 0), Point(0, 1)};
45 }

51 std::vector<double> TriangleLinearBasis::value(Point xi_) const {
52   double xi = xi_.x();
53   double eta = xi_.y();
54   return { 1 - xi - eta, xi, eta };
55 }

57 std::vector<Vector> TriangleLinearBasis::grad(Point xi) const {
58   return { Vector(-1, -1), Vector(1, 0), Vector(0, 1) };
59 }
```

13.3.3.4 Калькулятор элементных матриц

Интерфейс `IElementIntegrals` предоставляет методы для вычисления элементных матриц. До сих пор были рассмотрены две элементные матрицы: матрица масс (13.1) и матрица жёсткости (13.3). Для вычисления этих матриц используются функции
`mass_matrix`, `stiff_matrix`. Возвращают эти функции локальные квадратные матрицы с числом строк, равным количеству базисов в элементе. Выходные матрицы развёрнуты в линейный массив. Так, для треугольного элемента с тремя базисными функциями на выходе будет массив из девяти элементов:

$$m_{00}, m_{01}, m_{02}, m_{10}, m_{11}, m_{12}, m_{20}, m_{21}, m_{22}.$$

Два подхода к вычислению элементных интегралов: точное и численное интегрирование, отражены в разных реализациях этого интерфейса.

До сих пор мы рассматривали только точное вычисление. Точные формулы интегрирования зависят от вида элемента. Так, для линейного одномерного элемента аналитическое интегрирование реализовано в классе `SegmentLinearIntegrals` в файле `cfd/fem/elem1d/segment_linear.hpp`, а для линейного треугольного элемента – `TriangleLinearIntegrals` в файле `cfd/fem/elem2d/triangle_linear.hpp` Все интегралы для этих симлексных элементов будут зависеть только от матрицы Якоби, которая и передётся этому классу в конструктор. Например, вычисление матрицы масс (по (13.4)) запрограммировано в виде

```
94 std::vector<double> TriangleLinearIntegrals::mass_matrix() const {
95   double s0 = _jac.modj/12.0;
96   double s1 = _jac.modj/24.0;
97   return {s0, s1, s1,
98     s1, s0, s1,
99     s1, s1, s0};
100 }
```

13.4 Задание для самостоятельной работы

- Показать второй порядок аппроксимации решения одномерного уравнения Пуассона на линейных конечных элементах;
- Решить двумерное уравнение Пуассона с граничными условиями первого рода в квадратной области на треугольной сетке. Определить порядок аппроксимации двумерного уравнения Пуассона на треугольных элементах. Для построения треугольных сеток различного разрешения использовать скрипт `trigrid.py`.
- Показать второй порядок аппроксимации решения двумерного уравнения Пуассона на линейных треугольных конечных элементах;
- Сравнить сходимость на сгущающейся сетке конечноэлементного и конечнообъёмного решения двумерного уравнения Пуассона на одних и тех же треугольных сетках. Конечнообъёмное решение получать с использованием поправки на скошенность.

Рекомендации к программированию Для реализации двумерного решения следует по аналогии с одномерным написать класс `ITestPoisson2FemWorker`, в котором реализовать двумерные функции точного решения и правой части, и наследуемый от него рабочий класс `TestPoissonLinearSegmentWorker`, в котором реализовать статическую функцию `build_fem`. При реализации последней использовать алгоритмы для треугольников.

```
auto geom = std::make_shared<TriangleLinearGeometry>(p0, p1, p2);
auto basis = std::make_shared<TriangleLinearBasis>();
auto integrals = std::make_shared<TriangleLinearIntegrals>(geom->jacobi({}));
```

14 Лекция 14 (02.11)

14.1 Двухслойные схемы для нестационарных уравнений

14.1.1 Определение

Рассмотрим дифференциальное уравнение вида

$$\frac{\partial u}{\partial t} = f, \quad f(x,t) = g(x,t) - Lu(x,y) \tag{14.1}$$

где L – произвольный пространственный дифференциальный оператор. При использовании двухслойной схемы аппроксимации производная по времени записывается в виде конечной разности с шагом τ, которая может приближать производную в одном из трёх моментов времени:

$$\frac{u(t+\tau)-u(t)}{\tau} = \begin{array}{lll} \left.\frac{\partial u}{\partial t}\right|_{t} & +o(\tau) & \text{– разность вперёд;} \\ \left.\frac{\partial u}{\partial t}\right|_{t+\tau} & +o(\tau) & \text{– разность назад;} \\ \left.\frac{\partial u}{\partial t}\right|_{t+\frac{\tau}{2}} & +o(\tau^2) & \text{– симметричная разность.} \end{array} \tag{14.2}$$

Момент времени t будем называть текущим временным слоем, момент $t+\tau$ – следующим, а момент $t+\tau/2$ – промежуточным. Считается, что значение функции на текущий момент времени $u(t)$ известно, а значение на следующий момент $u(t+\tau)$ подлежит определению.

14.1.1.1 Явная схема

При использовании разности назад уравнение (14.1) в полудискретизованном (то есть дискретизованном только по времени, но не по пространству) виде запишется как

$$\frac{u(x,t+\tau)-u(x)}{\tau} + Lu(x,t) = g(x,t)$$

или, после переноса всех известных слагаемых вправо

$$u(x,t+\tau) = (E-\tau L)\,u(x,t) + \tau g(x,t). \tag{14.3}$$

Здесь E – единичный оператор. Схема (14.3) называется явной схемой и имеет первый порядок точности.

14.1.1.2 Неявная схема

Выбрав разность назад из выражения (14.2) полудискретизованная схема для уравнения (14.1) примет вид

$$\frac{u(x,t+\tau)-u(x)}{\tau} + Lu(x,t+\tau) = g(x,t+\tau).$$

В результате преобразования получим неявную схему первого порядка точности

$$(E+\tau L)\,u(x,t+\tau) = u(x,t) + \tau g(x,t+\tau). \tag{14.4}$$

14.1.1.3 Схема Кранка–Николсон

Подставим симметричную разность из (14.2) в уравнение (14.1). Формально получим

$$\frac{u(x,t+\tau)-u(x)}{\tau}+Lu(x,t+\frac{\tau}{2})=g(x,t+\frac{\tau}{2}).$$

Для определения выражения функций на промежуточном временном слое распишем значение u на текущем и следующем слоях в ряд Тейлора относительно значения на момент $t+\tau/2$:

$$u\left(t\right)=u\left(t+\frac{\tau}{2}\right)-\frac{\tau}{2}\frac{\partial u}{\partial t}\bigg|_{t+\frac{\tau}{2}}+o(\tau^2)$$
$$u\left(t+\tau\right)=u\left(t+\frac{\tau}{2}\right)+\frac{\tau}{2}\frac{\partial u}{\partial t}\bigg|_{t+\frac{\tau}{2}}+o(\tau^2)$$

Взяв полусумму этих выражений получим аппроксимацию функции на промежуточном слое:

$$u\left(x,t+\frac{\tau}{2}\right)=\frac{1}{2}u\left(x,t\right)+\frac{1}{2}u\left(x,t+\tau\right)+o(\tau^2) \tag{14.5}$$

Аналогичная запись справедлива и для свободного члена g. Если оператор L – нестационарный или нелининый, то аппроксимацию (14.5) следует записывать для всего выражения Lu:

$$\left(Lu\right)_{t+\frac{\tau}{2}}=\frac{1}{2}\left(Lu\right)_{t}+\frac{1}{2}\left(Lu\right)_{t+\tau}+o(\tau^2)$$

С учётом (14.5) симметричная разностная схема запишется как

$$\frac{u(x,t+\tau)-u(x)}{\tau}+\frac{1}{2}Lu(x,t)+\frac{1}{2}Lu(x,t+\tau)=\frac{1}{2}g(x,t)+\frac{1}{2}g(x,t+\tau)$$

или

$$\left(E+\frac{\tau}{2}L\right)u(x,t+\tau)=\left(E-\frac{\tau}{2}L\right)u(x,t)+\frac{\tau}{2}\left(g(x,t)+g(x,t+\tau)\right). \tag{14.6}$$

Такая схема называется схемой Кранка–Николсон и имеет второй порядок аппроксимации по времени.

В случае, если оператор L зависит от времени, то в левой части схемы (14.6) его нужно брать на следующем временном слое, а в правой – на текущем.

14.1.1.4 Обобщённая двухслойная схема

Выражения (14.3), (14.4), (14.6) можно записать в обобщённой форме

$$\left(E+\theta\tau L\right)u(x,t+\tau)=\left(E+\left(\theta-1\right)\tau L\right)u(x,t)+\left(1-\theta\right)g(x,t)+\theta g(x,t+\tau). \tag{14.7}$$

Коэффициент θ – степень неявности схемы:

- $\theta=0$ – явная схема (14.3),
- $\theta=1$ – полностью неявная схема (14.4),
- $\theta=1/2$ – схема Кранка–Николсон (14.6).

Отметим, что только при $\theta = 1/2$ схема (14.7) имеет второй порядок точности по времени. Для других значений (в том числе промежуточных) схема будет иметь ошибку первого порядка $o(\tau)$.

14.2 Схемы высокого порядка точности

Существуют два подхода к построению схем дискретизации по времени произвольного высокого порядка: первый из них предполагает использование нескольких временных слоев: $t+\tau, t, t-\tau, t-2\tau, ...$, второй – использование большого количества промежуточных точек на единственном временном отрезке: $t + c_i\tau, c_i \in [0, 1]$. Первый подход используется для построения схем Адамса, второй – схем Рунге–Кутта.

14.2.1 Многослойные схемы. Схемы Адамса

В общем виде многослойную расчётную схему можно записать в виде

$$\sum_{i=1}^{s} a_i u_i = \tau \sum_{i=1}^{s} b_i f_i. \tag{14.8}$$

Здесь нижние индексы функций использованы для указания временного слоя:

$$u_i = u(x, t + (i + 1 - s)\tau),$$
$$f_i = g(x, t + (i + 1 - s)\tau) - Lu_i,$$

а s – это общее количество используемых временных слоёв. Значение всех функций на слоях $i < s$ считаются известными, а значение u_s на последнем (текущем) слое подлежит определению. Коэффициенты a_i. b_i определяют конкретную схему. Для однозначности принято задавать коэфициет перед значением u на текущем слое равным единице: $a_s = 1$. Так, для двухслойной ($s = 2$) схемы Кранка–Николсон (14.6) эти коэффициенты примут вид:

$$a_1 = -1, \quad a_2 = 1,$$
$$b_1 = 1/2, \quad b_2 = 1/2.$$

Для того чтобы схема, записанная в общем виде (14.8), была явной, необходимо, чтобы выполнялось условие $b_s = 0$.

14.2.1.1 Явные схемы Адамса–Башфорта

Запишем интерполяционный полином для правой части уравнения (14.1) по значениям на предыдущих временных слоях $f_i, i < s$:

$$p(t) = \sum_{i=1}^{s-1} \left(f_i \prod_{\substack{m=1 \\ m \neq i}}^{s-1} \frac{t - t_i}{t_m - t_i} \right).$$

Порядок аппроксимации этого полинома на единицу больше его степени. С учётом $t_{i+1} - t_i = \tau$ можно записать

$$f(t) = p(t) + o(\tau^{s-1}).$$

Далее проинтегрируем уравнение по текущему временному отрезку: $[t_{s-1}, t_s]$:

$$u_s - u_{s-1} = \sum_{i=1}^{s-1} \left(f_i \int_{t_{s-1}}^{t_s} \prod_{\substack{m=0 \\ m \neq i}}^{s-1} \frac{t - t_i}{t_m - t_i} \, dt \right).$$

Отсюда коэффициенты в общей форме (14.8) примут вид

$$a_i = \begin{cases} 1, & i = s \\ -1, & i = s-1 \\ 0, & i < s-1 \end{cases} \qquad b_i = \begin{cases} 0, & i = s \\ \dfrac{1}{\tau} \displaystyle\int_{t_{s-1}}^{t_s} \prod_{\substack{m=0 \\ m \neq i}}^{s-1} \frac{t - t_i}{t_m - t_i} \, dt, & i < s. \end{cases}$$

Примеры трёх-, четырех- и пятислойной явных схем приведены ниже:

$$\begin{aligned} \frac{u_3 - u_2}{\tau} &= \frac{3f_2 - f_1}{2} + o(\tau^2), \\ \frac{u_4 - u_3}{\tau} &= \frac{23f_3 - 16f_2 + 5f_1}{12} + o(\tau^3), \\ \frac{u_5 - u_4}{\tau} &= \frac{55f_4 - 59f_3 + 37f_2 - 9f_1}{24} + o(\tau^4). \end{aligned}$$

14.2.1.2 Неявные схемы Адамса–Мультона

Теперь снимем требование явности и запишем интерполяционный полином для правой части исходного уравнения (14.1) с использованием s точек:

$$p(t) = \sum_{i=1}^{s} \left(f_i \prod_{\substack{m=1 \\ m \neq i}}^{s} \frac{t - t_i}{t_m - t_i} \right).$$

Тогда порядок аппроксимации этого полинома будем на единицу больше, чем для полинома явного метода. Далее, проинтегрируем исходное уравенение по отрезку t_{s-1}, t_s и получим коэффициенты общей формы (14.8):

$$a_i = \begin{cases} 1, & i = s \\ -1, & i = s-1 \\ 0, & i < s-1 \end{cases} \qquad b_i = \frac{1}{\tau} \int_{t_{s-1}}^{t_s} \prod_{\substack{m=0 \\ m \neq i}}^{s} \frac{t - t_i}{t_m - t_i} \, dt.$$

Двуслойная схема Адама–Мультона будет аналогична неявной двухслойной схеме (14.4), трёх-

слойная – схеме Кранка–Николсон (14.6). Примеры схемы высоких порядков представлены ниже:

$$\begin{aligned}
\frac{u_3 - u_2}{\tau} &= \frac{5f_3 + 8f_2 - f_1}{12} + o(\tau^3), \\
\frac{u_4 - u_3}{\tau} &= \frac{9f_4 + 19f_3 - 5f_2 + f_1}{24} + o(\tau^4), \\
\frac{u_5 - u_4}{\tau} &= \frac{251f_5 + 646f_4 - 264f_3 + 106f_2 - 19f_1}{720} + o(\tau^5).
\end{aligned}$$

14.2.2 Схемы Рунге-Кутта

TODO

14.3 Методы исследования устойчивости расчётных схем

14.3.1 Дискретизация по времени как итерационный процесс

14.3.1.1 Двухслойный итерационный процесс

Простой двухслойный итерационный процесс определяется как

$$u^{n+1} = Au^n + b, \tag{14.9}$$

где n – индекс итерационного слоя, A – оператор преобразования, b – свободный член.

Определение значения функции на следующий момент времени $u(t + \tau)$ по двухслойной схеме (14.7) можно представить как простой итерационный процесс (14.9), где

$$A = (E + \theta\tau L)^{-1} (E + (\theta - 1)\tau L),$$

$$b = (E + \theta\tau L)^{-1} (\theta g(x, t + \tau) + (1 - \theta) g(x, t)).$$

Итерационный процесс называется сходящимся, если

$$\lim_{n=\infty} \left\| u^{n+1} - u^n \right\| = 0.$$

14.3.1.2 Устойчивость итерационного процесса

Рассмотрим два простых итерационных процесса, имеющих на нулевом слое значение $u^0 = 1$:

$$\begin{aligned}
\text{(I)}: &\quad u^{n+1} = 2u^n - 1, \\
\text{(II)}: &\quad u^{n+1} = 0.5u^n + 0.5.
\end{aligned}$$

Оба этих процесса при выбранном начальном приближении, очевидно, сходятся. На каждой итерации справделиво $u^n = 1$. Возмутим начальное условие: пусть

$$u^0 = 1 + \varepsilon,$$

и проведём итерации.

	(I)	(II)
u^1	$1+2\varepsilon$	$1+\frac{\varepsilon}{2}$
u^2	$1+4\varepsilon$	$1+\frac{\varepsilon}{4}$
u^3	$1+8\varepsilon$	$1+\frac{\varepsilon}{8}$
...		
u^∞	∞	1

Видно, что процесс (I) теряет сходимость и стремится к бесконечности, в то время, как процесс (II) сохраняет свои свойства.

Свойство итерационных процессов уменьшать малые возмущения называется устойчивостью. В примере выше процесс (I) является неустойчивым, а процесс (II) – устойчивым.

Нетрудно видеть, что для рассматриваемого скалярного итерационного процесса, условие устойчивости запишется в виде $|A| \leq 1$.

14.3.1.3 Источники возмущений

На практике возникновение возмущений в решениях неизбежно: они могут быть следствием ошибок дискретизации функций и операторов, погрешностей решения СЛАУ, ошибок при проведении арифметических операций на числах с плавающей точкой и т.д. Поэтому любой итерационный процесс, используемый для решений математических задач, должен быть устойчив.

Возникновение непреднамеренных ошибок вследствии компьютерного округления можно проиллюстрировать на примере программы, в которой рассматривается сходящийся для любого начального условия, но неустойчивый итерационный процесс

$$u^{n+1} = 10u^n - 9u^0.$$

```
double u0 = 0.625;
double u = u0;
for (int i=0; i<1000; ++i){
    u = 10*u - 9*u0;
}
std::cout << u << std::endl;
```

Если начальное значение может быть точно представлено в числах с плавающей точкой (путём конечной суммы степеней двойки), то арифметическая ошибка не возникает. Так, представленный выше код на выходе печатает ожидаемое $u = 0.625$. Потому что начальное приближение может быть разложено как $u^0 = 2^{-1} + 2^{-3}$.

Однако, если заменить начальное приближение на любое число, которое не может быть записано точно во floating-point формате, то процесс быстро уходит в бесконечность. Например, для $u^0 = 0.626$

бесконечные (непредставимые в машинном формате) значения появляются на 324-ой итерации, а при переключении на работу в числах одинарной точности ‘float‘ – уже на 46-ой.

14.3.2 Матричный метод

Итерационные процессы, возникающие при численном решении дифференциальных уравнений сеточными методами, имеют матричную природу. То есть оператор преобразования A в выражении (14.9) – это матрица, а функции u и b – векторы-столбцы.

Как было показано выше, условием устойчивости скалярного итерационного процесса является неравенство $|A| \leq 1$. Аналогом этого условия для матричного процесса является ограничение на спектральный радиус $S(A)$:

$$S(A) = \max_j |\lambda_j| \leq 1, \tag{14.10}$$

где λ_j – собственные числа матрицы A.

Для некоторых видов матриц, возникающих при аппроксимации простейших дифференциальных уравнений, собственные числа известны.

14.3.2.1 Явная схема для нестационарного уравнения диффузии

Например, рассмотрим одномерное нестационарное уравнение диффузии с граничными условиями первого рода

$$\frac{\partial u}{\partial t} = k\frac{\partial^2 u}{\partial x^2},$$
$$u(x,0) = u_0(x),$$
$$u(x_a,t) = u_a,$$
$$u(x_b,t) = u_b.$$

Используем явную дискретизацию по времени и аппроксимацию второго порядка по пространству. Тогда разностная схема запишется в виде:

$$\hat{u}_i = u_i + \gamma(u_{i-1} - 2u_i + u_{i+1}), \quad i = \overline{1, N-1}, \tag{14.11}$$

где введено обозначение для значения функции на следующем временном слое $\hat{u} = u(t+\tau)$ и $\gamma = \tau k/h^2$. В матричном виде схема имеет вид

$$\hat{u} = Au, \quad A = \begin{pmatrix} 1 & 0 & & & & \\ \gamma & 1-2\gamma & \gamma & & & \\ & \gamma & 1-2\gamma & \gamma & & \\ & & \ddots & \ddots & \ddots & \\ & & & \gamma & 1-2\gamma & \gamma \\ & & & & 0 & 1 \end{pmatrix}.$$

Первая и последняя строки этой матрицы – следствие учёта граничных условий первого рода.

Собственные числа для полученной трёхдиагональной матрицы преобразования в правой части имеют вид

$$\lambda_j = 1 - 4\gamma \sin^2\left(\frac{j\pi}{2N}\right), \quad j = \overline{1, N-1}$$

Тогда, исходя из выражения (14.10), запишем условие устойчивости для явной схемы (14.11)

$$\gamma \leq \frac{1}{2}$$

14.3.2.2 Неявная схема для нестационарного уравнения диффузии

Аналогично, рассмотрим неявную схему

$$\hat{u}_i - \gamma(\hat{u}_{i-1} - 2\hat{u}_i + \hat{u}_{i+1}) = u_i, \quad i = \overline{1, N-1}, \tag{14.12}$$

В матричном виде

$$\hat{u} = A^{-1}u, \quad A = \begin{pmatrix} 1 & 0 & & & & \\ -\gamma & 1+2\gamma & -\gamma & & & \\ & -\gamma & 1+2\gamma & -\gamma & & \\ & & \ddots & \ddots & \ddots & \\ & & & -\gamma & 1+2\gamma & -\gamma \\ & & & & 0 & 1 \end{pmatrix}.$$

Собственные числа такой матрицы имеют вид

$$\lambda_j = 1 + 4\gamma \sin^2\left(\frac{j\pi}{2N}\right), \quad j = \overline{1, N-1}$$

Поскольку в правой части итерационного процесса используется матрица, обратная к A, а собственные числа обратных матриц равны $1/\lambda_j$, то условие устойчивости примет вид

$$\lambda_j \geq 1.$$

Очевидно, что оно выполняется всегда. Поэтому неявная схема (14.12) безусловно устойчива.

14.3.3 Метод дискретных возмущений

Метод дискретных возмущений заключается в использовании в качестве начального приближения нулевого вектора, с возмущением ε в одном из узлов:

$$u^0 = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \varepsilon \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

и дальнейшем анализом распространения этого возмущения с прохождением по временным слоям. Во многом этот метод аналогичен тому алгоритму, по которому мы иллюстрировали устойчивость простейшего скалярного итерационного процесса (14.3.1.2).

14.3.3.1 Явная схема против потока для уравнения переноса

Для иллюстрации рассмотрим одномерное уравнение переноса

$$\frac{\partial u}{\partial t} + V\frac{\partial u}{\partial x} = 0 \tag{14.13}$$

и явную противопоточную схему для него (при условии $V > 0$)

$$\hat{u}_i = u_i - C\left(u_i - u_{i-1}\right), \tag{14.14}$$

где число Куранта определено как $C = \tau V/h$.

Пусть $u_i = \varepsilon$. Тогда

$$\begin{aligned} \hat{u}_i &= (1-C)\varepsilon && \Rightarrow \quad 0 \le C \le 2, \\ \hat{u}_{i+1} &= C\varepsilon && \Rightarrow \quad -1 \le C \le 1. \end{aligned}$$

Поскольку C по определению больше нуля, то условием устойчивости для схемы (14.14) будет выражение

$$C \le 1.$$

14.3.4 Метод Неймана

Запишем обратное преобразование Фурье для функции u(x):

$$u(x) = \int v(\kappa) e^{\mathbf{i}\kappa x}\, d\kappa,$$

κ – волновое число, $\mathbf{i}$ – мнимая единица, $v(\kappa)$ – Фурье образ исходной функции.

Зададим такое начальное возмущение, которое имеет единичную амплитуду на одной частоте, соответствующей волновому числу κ_0:

$$v(\kappa) = \delta(\kappa - \kappa_0),$$

$\delta(x)$ – функция Дирака. Кроме того, учтём, что $x_i = ih$. Тогда выбранное начальное возмущение на одной выбранной частоте, взятое в i-ом узле, примет вид

$$u_i = e^{\mathbf{i}i\theta}, \quad \theta = \kappa_0 h \tag{14.15}$$

На следующем временном шаге это возмущение примет вид:

$$\hat{u}_i = Ge^{\mathbf{i}i\theta}. \tag{14.16}$$

G – коэффициент усиления. Он показывает во сколько раз увеличилась амплитуда выбранного возмущения за один шаг по времени. Для того, чтобы все возмущения затухали, необходимо

$$|G| \leq 1, \quad \forall\theta$$

14.3.4.1 Неявная противопотоковая схема для уравнения переноса

Для примера анализа устойчивости методом Неймана опять рассмотрим задачу (14.13), но на этот раз рассмотрим чисто неявную аппроксимацию

$$\hat{u}_i + C\left(\hat{u}_i - \hat{u}_{i-1}\right) = u_i. \tag{14.17}$$

Подставим (14.15), (14.16)

$$Ge(i) + C\left(Ge(i) - Ge(i-1)\right) = e(i).$$

где для краткости введено обозначение

$$e(i) = e^{\mathbf{i}\theta i}.$$

Поделим на $e(i)$ с использованием свойств этой степенной функции. Тогда

$$G + CG\left(1 - e(-1)\right) = 1 \qquad \Rightarrow$$

$$G = \left(1 + C(1 - e(-1))\right)^{-1}.$$

По определению комплексной экспоненты имеем

$$e(-1) = \cos\theta - \mathbf{i}\sin\theta.$$

Требуется показать, что $|G| \leq 1$. Отсюда

$$|1 + C(1 - \cos\theta + \mathbf{i}\sin\theta)| \geq 1 \qquad \Rightarrow$$

$$|1 + C(1 - \cos\theta) + C\mathbf{i}\sin\theta)|^2 \geq 1 \qquad \Rightarrow$$

$$1 + C^2(1 - \cos\theta)^2 + 2C(1 - \cos\theta) + C^2\sin^2\theta \geq 1 \qquad \Rightarrow$$

$$C^2(1 - \cos\theta) + 2C + C^2(1 + \cos\theta) \geq 0 \qquad \Rightarrow$$

$$C^2 + 2C \geq 0.$$

По определению число Куранта больше 0, поэтому последнее выражение выполняется всегда. Отсюда следует вывод, что неявная разностная схема вида (14.17) безусловно устойчива.

14.3.4.2 Противопотоковая схема Кранка-Николсон для уравнения переноса

Для того же самого уравнения (14.13) рассмотрим схему Кранка-Николсон (14.5):

$$\hat{u}_i + \frac{C}{2}(\hat{u}_i - \hat{u}_{i-1}) = u_i - \frac{C}{2}(u_i - u_{i-1}). \tag{14.18}$$

Так же подставим (14.15), (14.16) и поделим на $e(i)$:

$$G + \frac{CG}{2}(1 - e(-1)) = 1 - \frac{C}{2}(1 - e(-1)) \qquad \Rightarrow$$

$$G = \frac{1-p}{1+p}, \quad p = \frac{C}{2}(1 - e(-1)).$$

Для выполнения условия устойчивости $|G| \leq 1$, необходимо

$$|1 - p|^2 \leq |1 + p|^2 \qquad \Rightarrow$$

$$(1 - \Re(p))^2 + \Im(p)^2 \leq (1 + \Re(p))^2 + \Im(p)^2 \qquad \Rightarrow$$

$$\Re(p) \geq 0$$

Здесь $\Re(p)$, $\Im(p)$ – дейсвительная и мнимая часть комплексного числа.

Поскольку число Куранта больше нуля, то и действительная часть выражения p неотрицательная для любого θ.

$$\Re(p) = \frac{C}{2}(1 - \cos\theta) \geq 0.$$

Получаем, что схема видеа (14.18) безусловно устойчива.

14.3.4.3 Явная схема для уравнения нестационарной конвекции-диффузии

Рассмотрим уравнение конвекции-диффузии

$$\frac{\partial u}{\partial t} + V\frac{\partial u}{\partial x} = k\frac{\partial^2 u}{\partial x^2} \tag{14.19}$$

Сначала напишем чисто явную схему второго порядка по пространству:

$$\frac{\hat{u}_i - u_i}{\tau} + V\frac{u_{i+1} - u_{i-1}}{2h} = k\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} \tag{14.20}$$

Введем число Куранта $C = V\tau/h$ и параметр $\gamma = k\tau/h^2$. Тогда

$$\hat{u}_i = \left(\gamma - \frac{C}{2}\right) u_{i+1} + \left(\gamma + \frac{C}{2}\right) u_{i-1} + (1 - 2\gamma)u_i.$$

Далее подставим (14.15), (14.16)

$$Ge(i) = \left(\gamma - \frac{C}{2}\right) e(i+1) + \left(\gamma + \frac{C}{2}\right) e(i-1) + (1 - 2\gamma)e(i).$$

Поделим на $e(i)$ с использованием свойств этой степенной функции. Тогда

$$G = \gamma(e(1) + e(-1)) - \frac{C}{2}(e(1) - e(-1)) + (1 - 2\gamma)$$

По определению комплексной экспоненты имеем

$$e(1) = \cos\theta + \mathbf{i}\sin\theta,$$
$$e(-1) = \cos\theta - \mathbf{i}\sin\theta,$$

Отсюда

$$G = 2\gamma\cos\theta - \mathbf{i}C\sin\theta + (1 - 2\gamma)$$

Запишем квадрат модуля комплексного числа G:

$$|G|^2 = (1 - 2\gamma(1 - \cos\theta))^2 + C^2\sin^2\theta =$$
$$1 + 4\gamma^2(1 - \cos\theta)^2 - 4\gamma(1 - \cos\theta) + C^2(1 - \cos^2\theta).$$

Требование $|G| \le 1$ эквивалентно $|G|^2 \le 1$, или

$$1 + 4\gamma^2(1 - \cos\theta)^2 - 4\gamma(1 - \cos\theta) + C^2(1 - \cos^2\theta) \le 1 \qquad \Rightarrow$$
$$4\gamma^2(1 - \cos\theta)^2 - 4\gamma(1 - \cos\theta) + C^2(1 - \cos^2\theta) \le 0 \qquad \Rightarrow$$
$$4\gamma^2(1 - \cos\theta) - 4\gamma + C^2(1 + \cos\theta) \le 0 \qquad \Rightarrow$$
$$(C^2 - 4\gamma^2)\cos\theta + 4\gamma^2 - 4\gamma + C^2 \le 0$$

Поскольку неравенство должно выполняться для всех θ, а полученное выражение линейно за-

висит от $\cos\theta$, то будет достаточно рассмотреть два экстремальных значения косинуса, из которых окончательно запишем два условия устойчивости для явной дискретизации уравнения конвекции-диффузии вида (14.20):

$$\begin{aligned} \cos\theta = 1 \quad &\Rightarrow \quad C \le \sqrt{2\gamma}, \\ \cos\theta = -1 \quad &\Rightarrow \quad \gamma \le 1/2. \end{aligned} \tag{14.21}$$

Обычно вместо первого из условий (14.21) применяют более жёсткое (в случае $2\gamma < 1$) условие

$$C \le 2\gamma,$$

которое с учётом определений сводится к условию на шаг по пространству, формулируемому в терминах сеточного числа Рейнольдса Re_c:

$$\frac{Vh}{k} \equiv \mathrm{Re}_c \le 2.$$

14.3.4.4 Неявная схема для уравнения нестационарной конвекции-диффузии

Аналогичным образом рассмотрим неявную диффузии схему для уравнения (14.19) вида

$$\frac{\hat{u}_i - u_i}{\tau} + V\frac{u_{i+1} - u_{i-1}}{2h} = k\frac{\hat{u}_{i+1} - 2\hat{u}_i + \hat{u}_{i-1}}{h^2} \tag{14.22}$$

Подставляя представление для возмущения с волновым числом θ, получим

$$G = \frac{1 - \mathbf{i}C\sin\theta}{1 - 2\gamma(\cos\theta - 1)}$$

Для устойчивости необходимо

$$\begin{aligned} &|1 - \mathbf{i}C\sin\theta|^2 \le |1 - 2\gamma(\cos\theta - 1)|^2 \qquad \Rightarrow \\ &1 + C^2\sin^2\theta \le 1 + 4\gamma^2(1 - \cos\theta)^2 + 4\gamma(1 - \cos\theta) \qquad \Rightarrow \\ &C^2(1 + \cos\theta) \le 4\gamma^2(1 - \cos\theta) + 4\gamma \qquad \Rightarrow \\ &\cos\theta(C^2 + 4\gamma^2) + C^2 - 4\gamma - 4\gamma^2 \le 0 \end{aligned}$$

Наибольшего значения выражение слева достигает при $\theta = 0$. Тогда единственное условие устойчивости примет вид

$$C \le \sqrt{2\gamma}.$$

14.3.5 Общие рекомендации к выбору устойчивых расчётных схем

Теоретический анализ условий устойчивости возможен лишь для простейших уравнений с постоянными шагами дискретизации. В практических приложениях, имеющих дело, как правило, с неструктурированными сетками и сложными нелинейными системами уравнений, параметры устойчивого счёта приходится определять эмпирически. Однако, такой теоретический анализ позволяет выделить принципы, которыми следует руководствоваться для построения устойчивых схем.

Неявные схемы более устойчивы, чем явные

- Это можно видеть, сравнив результаты анализа для безусловно устойчивой неявной схемы (14.3.2.2) для уравнения диффузного переноса и для условно устойчивой явной схемы (14.3.2.1).
- Для уравнения переноса с разностью против потока явная (14.3.3.1) схема условно устойчива, в то время как неявная (14.3.4.1) – устойчива безусловно.
- Даже если только часть схемы неявная, это повышает устойчивость. Так, явная (14.3.4.3) схема для уравнения конвекции-диффузии имеет два условия устойчивости, в то время как схема, неявная по диффузии (14.3.4.4) – только одно.
- Аналогично, явная (14.3.3.1) схема против потока для уравнения переноса условно устойчива, а схема Кранка-Николсон (14.3.4.2) для того же уравнения устойчива при любых параметрах.

Конвективное слагаемое провоцирует неусточивость, а диффузионное – напротив, добавляет устойчивость

- Так, схемы с центральными разностями для уравнения конвекции-диффузии (и явная (14.3.4.3), и полунеявная (14.3.4.4)), условно устойчивы. Явная схема с центральными разностями для чистого уравнения переноса всегда неустойчива. В последнем можно убедится, подставив $k = 0$ в условия устойчивости для уравнений конвекции-диффузии.

14.4 Программная реализация схемы для уравнения переноса

14.4.1 Постановка задачи

Рассматриваются три схемы по времени для противопотоковой аппроксимации уравнения переноса (14.13):

- явная схема (14.14) (тест называется `[transport1-explicit]`),
- неявная схема (14.17) (`[transport1-implicit]`),
- схема Кранка–Николсон (14.18) (`[tranport1-cn]`).

Уравнение решается на отрезке $x \in [0, 1]$ с единичной скоростью $V = 1$ на сетке из 1000 ячеек. Временные итерации продолжаются до момента времени $t = 0.5$.

Начальным условием является функция вида

$$u(x, 0) = e^{-x^2/\sigma^2}, \quad \sigma = 0.1$$

Точное решение уравнения, с которого будут сниматься граничные условия и производится сравнения полученного численного решения, запишется как

$$u(x, t) = u(x - t, 0) = e^{-(x-t)^2/\sigma^2}.$$

На каждом шаге по времени функция сохраняется в vtk-формате. В конце выводится значение отклонения от точного решения на конечный момент времени.

В качестве цели решения обозначим построение решения и визуальное сравнение решений при числе $C = 0.9$ по трём разным схемам. А также построение графика сходимости отклонения точного решения от численного при изменении числа Куранта и фиксированном шаге по пространству (то есть сходимость при уменьшении шага по времени).

Программы реализованы в файле `transport_solve_test.cpp`.

14.4.2 Функция верхнего уровня

Для всех трёх программ функция верхнего уровня имеет один и тот же вид. Рассмотрим на примере первой из них:

```
95 TEST_CASE("Transport 1D solver, explicit", "[transport1-explicit]"){
```

В начале происходит установка параметров численной схемы:

- конечного момента времени,
- скорости переноса,
- длины расчётной области,
- разбиения по пространству,
- числа Куранта

```
98  const double tend = 0.5;
99  const double V = 1.0;
100 const double L = 1.0;
101 size_t n_cells = 100;
102 double Cu = 0.9;
```

Далее вычисляются используемые шаги:

- шаг по пространству (из длины области и разбиения),
- шаг по времени (из шага по пространству и числа Куранта)

```
103 double h = L/n_cells;
104 double tau = Cu * h / V;
```

Потом устанавливается рабочий класс, в котором будет производится решение

```
107 TestTransport1WorkerExplicit worker(n_cells);
```

Конструируется класс, используемый для связного сохранения полей на разные моменты времени. Этот класс создаёт `transport1-explicit.vtk.series` со списком всех сохранённых полей и отнесёнными к ним моментами времени, который можно впоследствии открыть в Paraview и использовать функции анимации для воспроизведения поведения решения во времени.

```
110 VtkUtils::TimeSeriesWriter writer("transport1-explicit");
```

Далее нужно в этот класс сохранить решение на начальный момент времени. Для этого туда сначала добавляется запись о нулевом моменте времени

```
111  std::string out_filename = writer.add(0);
```

В переменную
`out_filename` записывается конкретное имя vtk-файла, куда следует сохранить решение. Уже это имя используется для сохранения решения на текущий (начальный) момент времени.

```
112  worker.save_vtk(out_filename);
```

Далее начинается цикл по времени, продолжающийся до тех пор, пока внутреннее время решателя не достигнет конечного

```
115  while (worker.current_time() < tend - 1e-6) {
```

Внутри вызывается функция решения, которая продвигает внутреннее время решателя на τ, обновляет актуальное состояние вектора решения и возвращает текущую норму.

```
117    norm = worker.step(tau);
```

Потом повторяется процедура сохранения текущего состояния решателя

```
119    out_filename = writer.add(worker.current_time());
120    worker.save_vtk(out_filename);
```

После завершения цикла в консоль печатается установленное разбиение по времени и полученное отклонение от точного решения на конечный момент времени

```
122  std::cout << 1.0/tau << " " << norm << std::endl;
```

14.4.3 Расчётные функции

Три класса-решателя для трёх заявленных задач:
`TestTransport1WorkerExplicit`, `TestTransport1WorkerImplicit`,
`TestTransport1WorkerCN` наследуются от одного абстрактного класса
`ATestTransport1Worker`. В этом абстрактном классе реализованы все общие для всех решателей функции: создание сетки, сохранение в vtk, расчёт нормы, продвижение по времени.

Этот класс также хранит в себе параметры, полностью определяющие текущее состояние решения:

- расчётную сетку,
- шаг по времени (это параметр, производный от сетки, он сохранён в отдельное поле для удобства расчётов),
- вектор решения на текущий момент,
- текущее время.

Эти поля хранятся в ‘protected‘ секции, таким образом все производные классы имеют к этим полям полный доступ.

```
protected:
  Grid1D _grid;
  double _h;
  std::vector<double> _u;
  double _time = 0;
```

Функция решения, также реализована в абстрактном классе. Она продвигает текущее время и вызывает виртуальный метод `impl_step`, который изменяет значение вектора решения, а в конце вызывает функцию вычисления ошибки.

```
  double step(double tau){
    _time += tau;
    impl_step(tau);
    return compute_norm2();
  }
```

Функция `impl_step` уже зависит от конкретной схемы и реализована в производных классах

14.4.3.1 Явная схема

Её решатель реализован в классе `TestTransport1WorkerExplicit`. Рабочая функция по порядку:

- копирует текущий вектор значений во вспомогательный вектор `u_old`. Этот шаг добавлен сюда для ясности. Вообще говоря, его можно было избежать.
- устанавливает граничное условие в левой точке
- далее в цикле по точкам реализует расчётную схему (14.14).

```
  void impl_step(double tau) override {
    std::vector<double> u_old(_u);
    _u[0] = exact_solution(_grid.point(0).x());
    for (size_t i=1; i<_grid.n_points(); ++i){
      _u[i] = u_old[i] - tau/_h*(u_old[i] - u_old[i-1]);
    }
  }
```

14.4.3.2 Неявная схема

Её решатель реализован в классе `TestTransport1WorkerImplicit`. Поскольку здесь для нахождения решения требуется решить СЛАУ, то порядок действий включает в себя:

- формирование класса-решателя.
- формирование столбца свободных членов

- вызова функции решения СЛАУ для найденного столбца правой части. Ответ записывается во внутреннее поле класса `_u`

```
135  void impl_step(double tau) override {
136    AmgcMatrixSolver& slv = build_solver(tau);
137    std::vector<double> rhs = build_rhs(tau);
138    slv.solve(rhs, _u);
139  }
```

Для построения и инициализации решателя необходимо собрать матрицу правой части системы уравнений (14.17). Матрица зависит от шага по времени (через число Куранта), при этом шаг по времени является аргументом функции `build_solver`, которая приходит от пользователя решателя через аргумент функции `step()`‘.

Таким образом, в логике работы приложения, нам придётся пересобирать матрицу на каждой временной итерации. При этом, почти всегда шаги по времени постоянны для временных слоёв. То есть одну и ту же операцию (сборку матрицы) при одним и тех же аргументах (шаге по времени) придётся повторять.

Поскольку сборка матрицы – дорогая операция, то результат работы функции `build_solver` мы кэшируем (сохраняем во внутреннее поле класса `_solver`). С тем чтобы на следующем временном слое в случае, если шаг по времени не изменился (`_last_used_tau == tau`), просто вернуть ответ, посчитанный ранее.

```
144  AmgcMatrixSolver& build_solver(double tau){
145    if (_last_used_tau != tau){
146      CsrMatrix mat = build_lhs(tau);
147      _solver.set_matrix(mat);
148      _last_used_tau = tau;
149    }
150    return _solver;
151  }
```

Сама сборка двухдиагональной матрицы происходит в функции `build_lhs`. В первой и последней строке учитываются граничные условия, а строки, соответствующие внутренним узлам, заполняются согласно схеме (14.17)

```
153  virtual CsrMatrix build_lhs(double tau){
154    LodMatrix mat(_u.size());
155    mat.set_value(0, 0, 1.0);
156    mat.set_value(_u.size()-1, _u.size()-1, 1.0);
157    double diag = 1.0 + tau/_h;
158    double nondiag = -tau/_h;
159    for (size_t i=1; i<_u.size()-1; ++i){
160      mat.set_value(i, i, diag);
161      mat.set_value(i, i-1, nondiag);
```

```
    }
    return mat.to_csr();
  }
```

Сборка правой части СЛАУ происходит в функции `build_rhs`. Согласно схеме (14.17) правый столбец равен значению функции на предыдущем временном слое. В коде мы создаём столбец `rhs` как копию вектора `_u`. А далее переписываем первый и последний элемент с тем, чтобы учесть граничные условия.

```
  virtual std::vector<double> build_rhs(double tau){
    std::vector<double> rhs(_u);
    rhs[0] = exact_solution(_grid.point(0).x());
    rhs.back() = exact_solution(_grid.point(_grid.n_points()-1).x());
    return rhs;
  }
```

14.4.3.3 Схема Кранка-Николсон

Её решатель реализован в классе `TestTransport1WorkerCN`.

По аналогии с предыдущей программой, здесь требуется решить СЛАУ, возникающую из схемы (14.18). Таким образом, вся логика работы этого класса (включая кэширование решателя) повторяет логику работы рассмотренного ранее класса для чисто неявной схемы `TestTransport1WorkerImplicit`. Отличаются эти классы только реализацией функций построения матрицы и правой части. Поэтому настоящий класс наследуется от `TestTransport1WorkerImplicit`

```
class TestTransport1WorkerCN: public TestTransport1WorkerImplicit{
```

и переопределяет только функции сборки левой части (14.18) с учётом граничных условий

```
  CsrMatrix build_lhs(double tau) override{
    LodMatrix mat(_u.size());
    mat.set_value(0, 0, 1.0);
    mat.set_value(_u.size()-1, _u.size()-1, 1.0);
    double diag = 1.0 + 0.5*tau/_h;
    double nondiag = -0.5*tau/_h;
    for (size_t i=1; i<_u.size()-1; ++i){
      mat.set_value(i, i, diag);
      mat.set_value(i, i-1, nondiag);
    }
    return mat.to_csr();
  }
```

и правой части (14.18) с учётом граничных условий

```
  std::vector<double> build_rhs(double tau) override{
    std::vector<double> rhs(_u);
```

```
219     rhs[0] = exact_solution(_grid.point(0).x());
220     rhs.back() = exact_solution(_grid.point(_grid.n_points()-1).x());
221     for (size_t i=1; i<rhs.size()-1; ++i){
222       rhs[i] -= 0.5 * tau / _h * (_u[i] - _u[i-1]);
223     }
224     return rhs;
225   }
```

14.4.4 Анализ результатов работы

Сравнение полученных ответов (по явной и неявной схемам) с точным решением представлено на рис. 7.

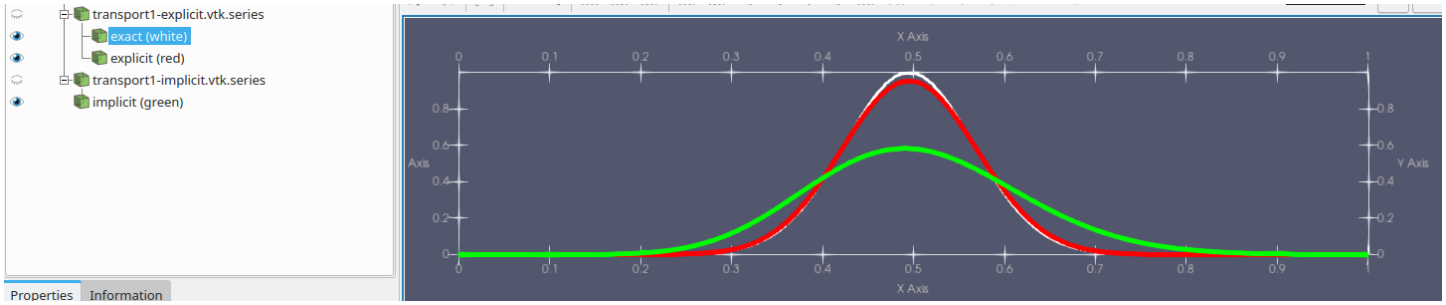


Рис. 7: Сравнение точного (белая линия) решения и численных решений по явной (красная) и неявной (зелёная) схемам

Чтобы получить такую картинку необходимо открыть в Paraview сгенерированные в результате работы программ выходные файлы `transport1_explicit.vtk.series` и `transport1_implicit.vtk.series` И далее проделать преобразования, описанные в пункте B.3.1.

Для построения графиков сходимости, необходимо преобразовать написанные программы, запустив цикл по различным значениям числа Куранта

```
for (double Cu: { ... }){
    // solution
    ...

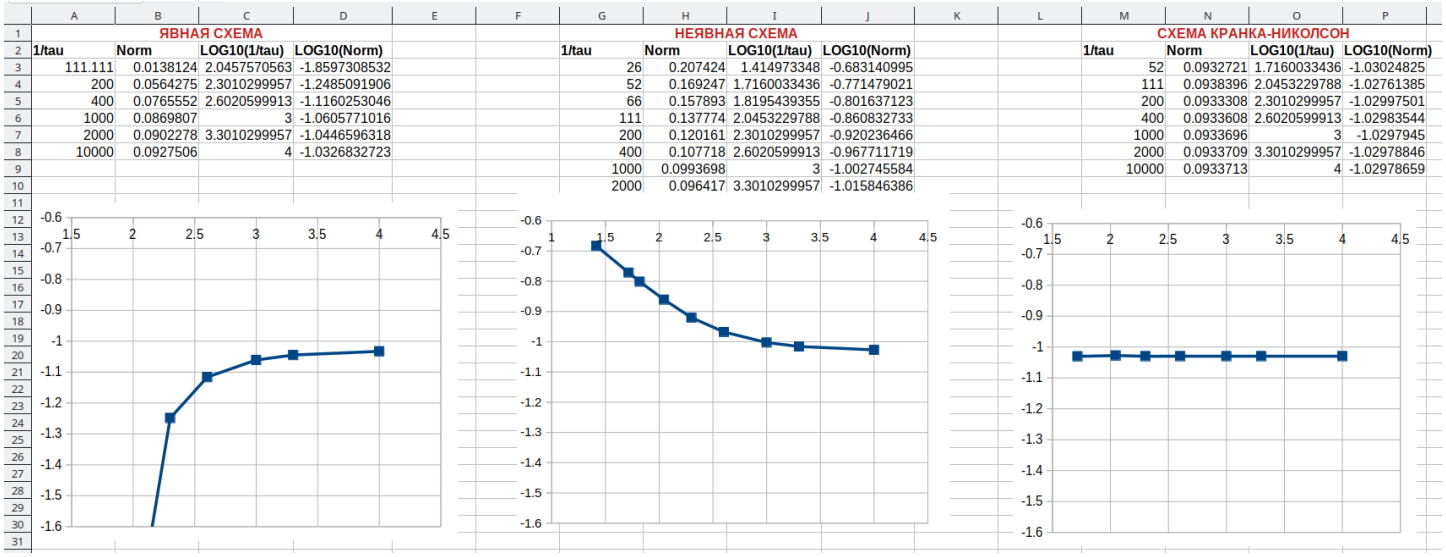
    std::cout << 1.0/tau << " " << norm << std::endl;
}
```

и построить график полученной таблицы в логарифмических осях. При задании диапазона изменений C следует учитывать, что явная схема устойчива только при $C \leq 1$, в то время как две другие схемы безусловно устойчивы.

Графики сходимости с уменьшением шага по времени представлены на рис. 8.

Видно, что для явной схемы с уменьшением шага по времени ответ отдаляется от точного, а для неявной – наоборот, приближается.

Это объясняется тем, что в случае явной схемы ошибки по времени и по пространству имеют разный знак и (в случае их равенства) компенсируют друг друга. А для неявной эти ошибки имеют одинаковый знак.



ЯВНАЯ СХЕМА			
1/tau	Norm	LOG10(1/tau)	LOG10(Norm)
111.111	0.0138124	2.0457570563	-1.8597308532
200	0.0564275	2.3010299957	-1.2485091906
400	0.0765552	2.6020599913	-1.1160253046
1000	0.0869807	3	-1.0605771016
2000	0.0902278	3.3010299957	-1.0446596318
10000	0.0927506	4	-1.0326832723

НЕЯВНАЯ СХЕМА			
1/tau	Norm	LOG10(1/tau)	LOG10(Norm)
26	0.207424	1.414973348	-0.683140995
52	0.169247	1.7160033436	-0.771479021
66	0.157893	1.8195439355	-0.801637123
111	0.137774	2.0453229788	-0.860832733
200	0.120161	2.3010299957	-0.920236466
400	0.107718	2.6020599913	-0.967711719
1000	0.0993698	3	-1.002745584
2000	0.096417	3.3010299957	-1.015846386

СХЕМА КРАНКА-НИКОЛСОН			
1/tau	Norm	LOG10(1/tau)	LOG10(Norm)
52	0.0932721	1.7160033436	-1.03024825
111	0.0938396	2.0453229788	-1.02761385
200	0.0933308	2.3010299957	-1.02997501
400	0.0933608	2.6020599913	-1.02983544
1000	0.0933696	3	-1.0297945
2000	0.0933709	3.3010299957	-1.02978846
10000	0.0933713	4	-1.02978659

Рис. 8: Сходимость решения уравнения переноса с уменьшением τ

В пределе (с минимальным шагом по времени) все три схемы сходятся к одной и той же ошибке (ошибке схемы по пространству).

14.5 Задание для самостоятельной работы

14.5.1 Постановка задачи

Написать двумерный решатель для уравнения переноса

$$\frac{\partial u}{\partial t} + U\frac{\partial u}{\partial x} + V\frac{\partial u}{\partial y} = 0.$$

Решение проводить в квадрате $x, y \in [-1, 1]$.
Требуется

- расчитать и нарисовать в Paraview нестационарное решение (см. B.3.3);
- построить график, иллюстрирующий увеличение нормы ошибки с продвижением по времени;
- исследовать устойчивость схемы. Эмпирическим путём выяснить, какое максимально возможный шаг по времени можно брать при фиксированном разбиении по пространству;
- построить график, иллюстрирующий сходимость нормы ошибки при уменьшении шага по времени при фиксированном разбиении по пространству.

14.5.1.1 Тестовый пример 1

На этапе первичного тестирования использовать значения скорости

$$U = 1, \quad V = 0.$$

А в качестве начального решения брать простой "столбик"(рис. 9)

$$u(x,y,0) = u_0(x,y) = \begin{cases} 1, & -1 \le x \le -0.8,\ -0.1 \le y \le 0.1, \\ 0, & \text{иначе.} \end{cases}$$

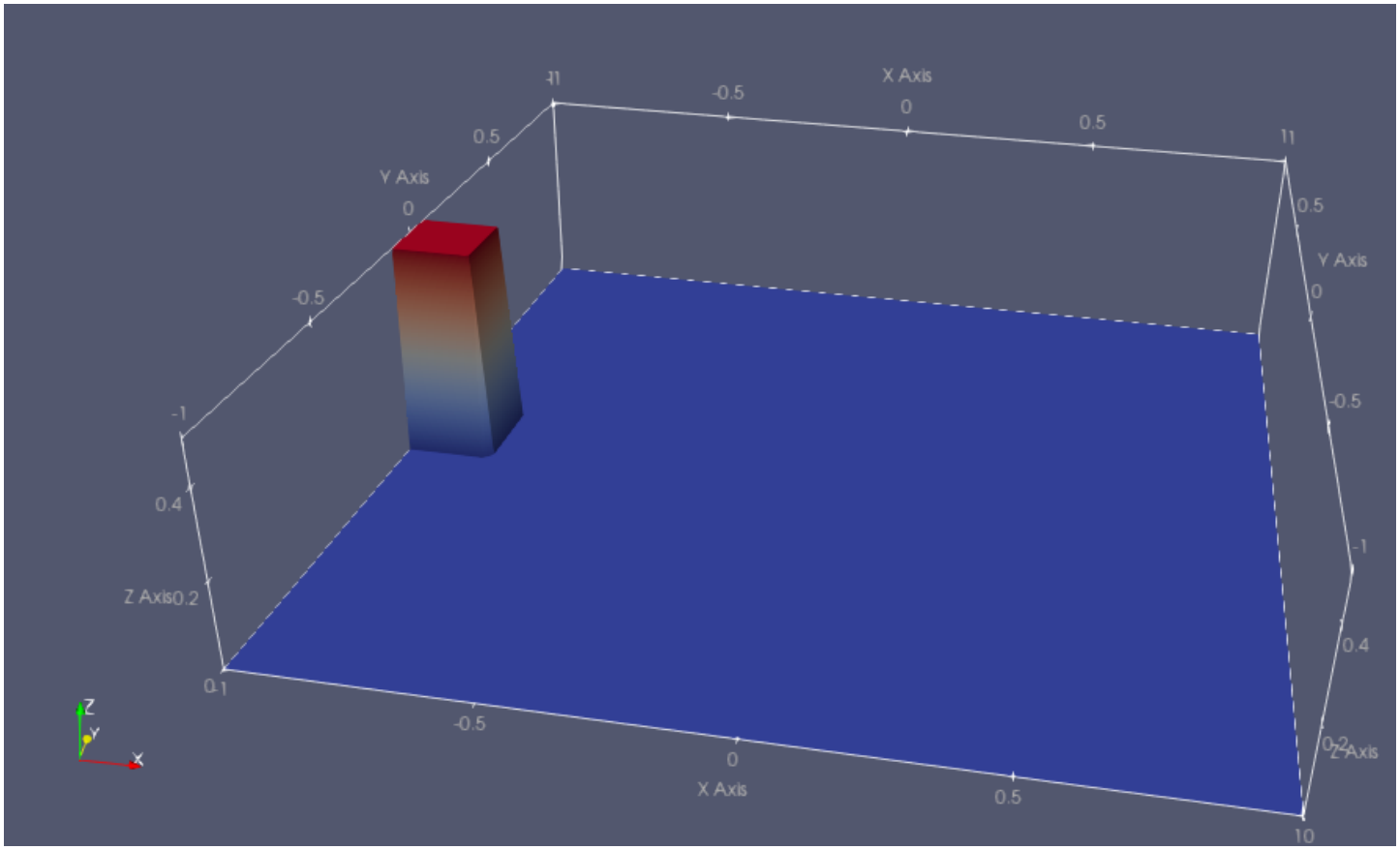


Рис. 9: Начальные условия для первого тестового примера

Точным решением будет функция

$$u^e(x,y,t) = u_0(x-t,y)$$

То есть этот столбик будет двигаться вправо с единичной скоростью и за время 2 полностью покинет расчётную область.

14.5.1.2 Тестовый пример 2

После того, как этот тест будет пройден, использовать постановку с непостоянной по пространству скоростью

$$U(x,y) = -y, \quad V(x,y) = x.$$

и начальным решением вида (рис. 10)

$$r_0(x,y) = \sqrt{(x-0.5)^2 + y^2};\ \sigma = 0.05;$$
$$u(x,y,0) = u_0(x,y) = e^{-r_0^2(x,y)/\sigma^2}$$

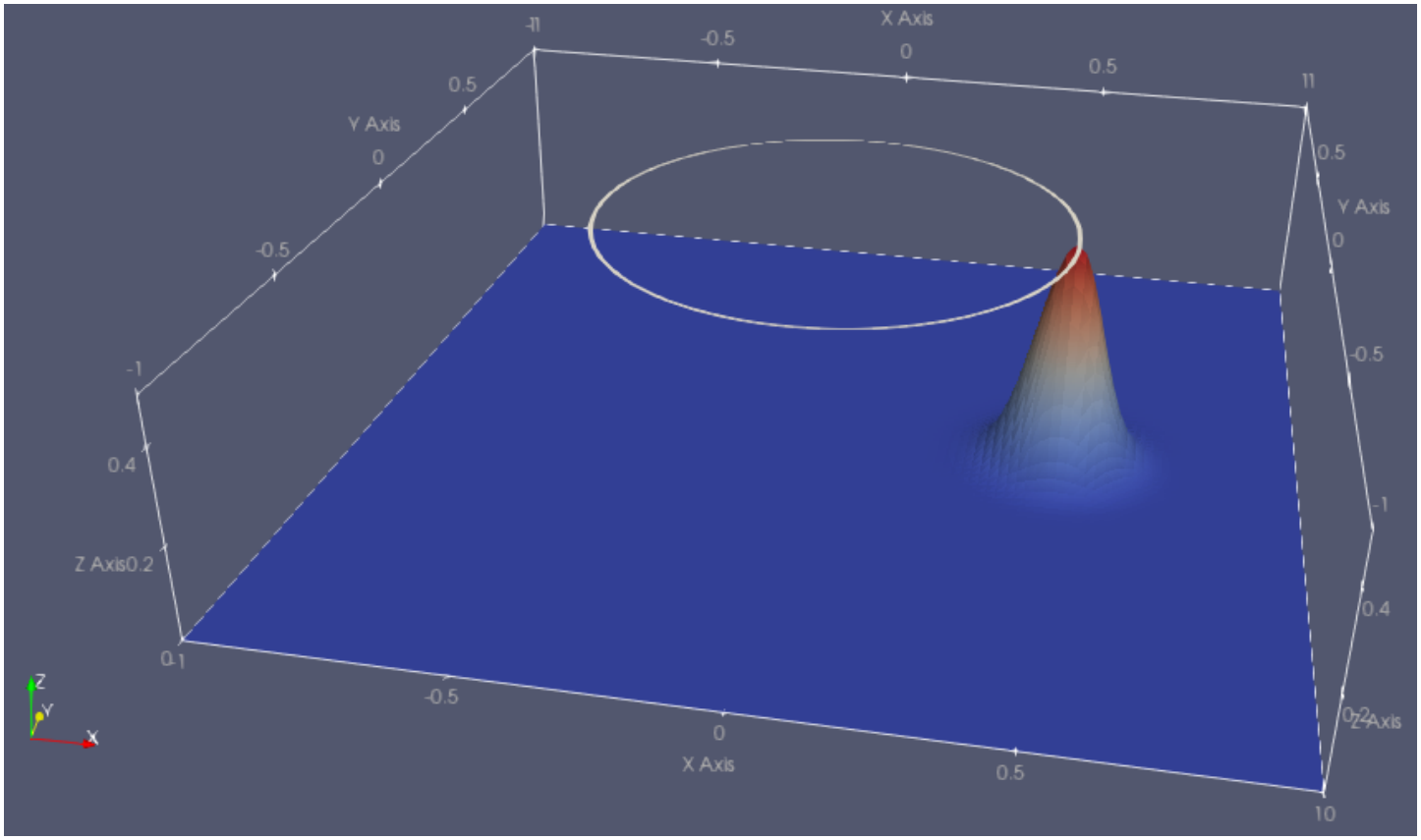


Рис. 10: Начальные условия для второго тестового примера

В процессе решения этот “холмик” будет двигаться по окружности с единичной скоростью, описывая полный оборот за время $t = \pi$.

Точное решение на момент времени t будет иметь вид

$$
\begin{aligned}
&x_c(t) = 0.5\cos(2t);\\
&y_c(t) = 0.5\sin(2t);\\
&r(x,y,t) = \sqrt{(x - x_c(t))^2 + (y - y_c(t))^2};\\
&u^e(x,y,t) = e^{-r^2(x,y,t)/\sigma^2}
\end{aligned}
$$

14.5.2 Расчётная схема

Использовать противопотоковую явную схему:

$$\frac{\hat{u}_k - u_k}{\tau} + |U_k|\frac{u_k - u_{\mathrm{upx}[k]}}{h_x} + |V_k|\frac{u_k - u_{\mathrm{upy}[k]}}{h_y} = 0$$

Здесь $\mathrm{upx}\,[k]$, $\mathrm{upy}\,[k]$ – значения индексов, расположенных против потока относительно узла k в направлениях x и y соответственно.

Поскольку скорость в настоящей постановке непостоянная и зависит от точки пространства, то вычислять индекс узла, расположенного против потока приходится в зависимости от значения скорости. С использованием ранее введёных алгоритмов перехода от парных (i, j) индексов к сквозному

индексу k (3.10) и обратно (3.11) запишем

$$
\begin{aligned}
&i = i[k];\\
&j = j[k];\\
&\mathrm{upx}\,[k] = \begin{cases} k[i-1,j], & U_k \geq 0,\\ k[i+1,j], & U_k < 0, \end{cases}\\
&\mathrm{upy}\,[k] = \begin{cases} k[i,j-1], & V_k \geq 0,\\ k[i,j+1], & V_k < 0. \end{cases}
\end{aligned}
$$

В схеме скорости переноса взяты по абсолютному значению. Это связано с зависимостью направления конечной разности от знака скорости. Так если $U_k > 0$, то для дискретизации производной по x используется разность назад:

$$U\frac{\partial u}{\partial x} \approx U_k\frac{u_{k[i,j]} - u_{k[i-1,j]}}{h_x} = |U_k|\frac{u_{k[i,j]} - u_{k[i-1,j]}}{h_x} = |U_k|\frac{u_{k[i,j]} - u_{\mathrm{upx}[k]}}{h_x}$$

Если же $U_k < 0$, то используется разность вперёд

$$U\frac{\partial u}{\partial x} \approx U_k\frac{u_{k[i+1,j]} - u_{k[i,j]}}{h_x} = -U_k\frac{u_{k[i,j]} - u_{k[i+1,j]}}{h_x} = |U_k|\frac{u_{k[i,j]} - u_{k[i+1,j]}}{h_x} = |U_k|\frac{u_{k[i,j]} - u_{\mathrm{upx}[k]}}{h_x}$$

На границах использовать условия первого рода. Можно просто нули, поскольку они соответствуют постановке.

15 Лекция 15 (09.11)

15.1 Аппроксимация уравнения переноса с ограничением потока

15.1.1 Схемы первого и второго порядка точности

Рассмотрим уравнение переноса в одномерной постановке

$$\frac{\partial u}{\partial t} + U\frac{\partial u}{\partial x} = 0$$

Все дальнейшие выкладки будем приводить исходя из условия положительности скорости переноса $U > 0$.

Рассмотрим два вида пространственной аппроксимации конвективного слагаемого на равномерной сетке: схемой против потока и симметричной схемой

$$\frac{\partial u_i}{\partial t} + U\frac{u_i - u_{i-1}}{h} = 0, \tag{15.1}$$

$$\frac{\partial u_i}{\partial t} + U\frac{u_{i+1} - u_{i-1}}{2h} = 0. \tag{15.2}$$

Первая схема является (условно) устойчивой но при этом обладает первым порядком аппроксимации. Вторая неустойчива, но имеет порядок $o(h^2)$. Идея методов аппроксимации с ограничением потока состоит в том, чтобы на основе комбинации первой и второй схем построить устойчивое решение, имеющее “почти везде” второй порядок аппроксимации.

Запишем эти аппроксимации в общем виде:

$$\frac{\partial u_i}{\partial t} = \frac{f_{i-1/2} - f_{i+1/2}}{h}. \tag{15.3}$$

Здесь $f_{i+1/2}$ – численный поток, который в зависимости от выбранной схемы будет равен

$$\begin{aligned} f^L_{i+1/2} &= Uu_i && \text{– схема против потока} \\ f^H_{i+1/2} &= U\frac{u_i + u_{i+1}}{2} && \text{– симметричная схема.} \end{aligned} \tag{15.4}$$

Здесь f^L, f^H означают потоки низкого (Low) и высокого (High) порядка аппроксимации.

Аппроксимацию с ограничением потока запишем в виде

$$f_{i+1/2} = f^L_{i+1/2} + \Phi_{i+1/2}\left(f^H_{i+1/2} - f^L_{i+1/2}\right). \tag{15.5}$$

Φ в этой записи называется ограничителем, который служит переключателем: при $\Phi = 0$ мы получаем схему первого порядка, при $\Phi = 1$ – схему второго порядка.

Далее будем выбирать Φ таким образом, чтобы не допустить возникновения осцилляций в численном решении.

15.1.2 Условие TVD

В качестве критерия, характеризующего возникновение и развитие осцилляций, выберем полную вариацию:

$$\begin{aligned} TV(u) &= \int |\nabla u| \, dx = \\ &= \int \left| \frac{\partial u}{\partial x} \right| dx = \\ &= \sum_i |u_i - u_{i-1}| \, . \end{aligned} \qquad (15.6)$$

в одномерном случае

для сеточной функции

Условие уменьшения осцилляций в решении на следующем временном слое примет вид

$$TV(\hat{u}) \leq TV(u).$$

Численные схемы, удовлетворяющие этому условию, называются TVD (Total variation diminishing) схемами.

Запишем численную схему в общем виде

$$\frac{\partial u_i}{\partial t} = c_{i-1/2}(u_{i-1} - u_i) + c_{i+1/2}(u_{i+1} - u_i). \qquad (15.7)$$

Согласно теореме Хартена такая схема удовлетворяет свойству TVD, если $c_{i\pm 1/2} \geq 0$. Схема против потока (15.1) является TVD-схемой:

$$c_{i-1/2} = \frac{U}{h}, \quad c_{i+1/2} = 0,$$

а симметричная схема (15.2) – нет:

$$c_{i-1/2} = \frac{U}{2h}, \quad c_{i+1/2} = -\frac{U}{2h}.$$

Подставляя уравнение (15.5) в (15.3) и приводя к форме (15.7) получим

$$\frac{\partial u_i}{\partial t} = \frac{U}{2h}\left(2 - \Phi_{i-1/2}\right)(u_{i-1} - u_i) + \frac{U}{2h}\left(-\Phi_{i+1/2}\right)(u_{i+1} - u_i) \qquad (15.8)$$

То есть для удовлетворения свойства TVD необходимо, чтобы $\Phi \leq 0$. Для второго порядка точности требуется $\Phi = 1$. То есть линейные схемы TVD не могут иметь высокий порядок точности.

15.1.3 Нелинейные TVD схемы

Для того, чтобы преодолеть это ограничение, будем строить нелинейные схемы. Общая идея построения таких схем состоит в том, чтобы выбрать такую Φ, при которой второе слагаемое равенства (15.8) можно было отнести к первому. То есть можно бы было записать

$$\Phi_{i+1/2}(u_{i+1} - u_i) = -\Phi'_{i+1/2}(u_{i-1} - u_i). \qquad (15.9)$$

Тогда условием TVD станет выражение

$$2 - \Phi_{i-1/2} + \Phi'_{i+1/2} \geq 0. \tag{15.10}$$

Для характеристики поведения функции выберем соотношение наклонов (slope ratio), который в одномерном виде запишется в виде:

$$r_i = \frac{u_i - u_{i-1}}{u_{i+1} - u_i} \tag{15.11}$$

Нелинейность схемы будет выражаться в зависимости

$$\Phi_{i+1/2} = \Phi(r_i).$$

Из (15.9) следует

$$\Phi'_{i+1/2} = \frac{\Phi(r_i)}{r_i}$$

а неравенство перепишется в виде

$$2 - \Phi(r_i) + \frac{\Phi(r_i)}{r_i} \geq 0.$$

Чтобы из этого условия получить ограничение для Φ, явно не зависящее от r_i, потребуем

$$\Phi\left(\frac{1}{r_i}\right) = \frac{\Phi(r_i)}{r_i}. \tag{15.12}$$

Тогда неравентсво (15.10) примет вид

$$2 - \Phi(r_i) + \Phi\left(\frac{1}{r_i}\right) \geq 0.$$

Отсюда получим условие для Φ:

$$0 \leq \Phi(r_i) \leq 2. \tag{15.13}$$

Дополнительно потребуем, чтобы в точках с гладким поведением функции использовать схему второго порядка точности:

$$\Phi(1) = 1 \tag{15.14}$$

а в точках локального экстремума (которые особенно подвержены появлению осцилляций) гарантировать переключение на схему первого порядка:

$$\Phi(r \leq 0) = 0. \tag{15.15}$$

Таким образом, для построения TVD схемы, функция ограничитель должна удовлетворять условиям (15.12) – (15.15). Ниже представлены некоторые часто используемые ограничители, удовлетво-

ряющие этим свойствам:

$$\Phi(r) = \begin{cases} \max(0, \min(r, 1)) & \text{– minmod;} \\ \dfrac{r + |r|}{1 + |r|} & \text{– Van Leer;} \\ \max(0, \min(2r, \dfrac{1+r}{2}, 2) & \text{– monotonized central (MC);} \\ \max(0, \min(2, r), \min(1, 2r)) & \text{– superbee.} \end{cases} \tag{15.16}$$

15.2 TVD-схемы для неструктурированных конечнообъёмных сеток

Рассмотрим многомерное уравнение переноса

$$\frac{\partial u}{\partial t} + \mathbf{U} \cdot \nabla u = 0.$$

Применим конечнообъёмную процедуру для получения слабой интегральной постановки задачи. Для этого проинтегрируем это уравнение по конечному объёму E_i и применим формулу интегрирования по частям. Получим

$$|V_i| \frac{\partial u}{\partial t} + \sum_{j \in \text{nei}(i)} f_{ij} \, |\gamma_{ij}| = 0, \quad f_{ij} = u_{ij} U_{ij}. \tag{15.17}$$

Здесь $|V_i|$ – объём конечного элемента, $\text{nei}(i)$ – совокупность всех точек коллокации, инцидентных ячейке i (центров соседних ячеек и соседних граничных граней), f_{ij} – поток из точки коллокации i в точку коллокации j, $|\gamma_{ij}|$ – площадь грани конечного объёма i, через которую этот объём соединяется с точкой коллокации j, u_{ij} – значение функции u, отнесённое к этой грани, U_{ij} – скорость потока в направлении внешней по отношению к ячейке i нормали.

Для потока справедливо

$$f_{ij} = -f_{ji}. \tag{15.18}$$

То есть для вычисления потока на грани достаточно найти значение для одного направления. Выберем это направление $\overrightarrow{ij}$ таким образом, чтобы $U_{ij} > 0$ (рис. 11).

Будем считать, что скорость переноса U – известная функция. Тогда запишем значения потоков высокого и низкого порядка согласно (15.4):

$$\begin{aligned} f_{ij}^L &= U_{ij} u_i && \text{– схема против потока} \\ f_{ij}^H &= U_{ij} \frac{u_i + u_j}{2} && \text{– симметричная схема.} \end{aligned} \tag{15.19}$$

Поток при этом запишется по аналогии с (15.5):

$$f_{ij} = f_{ij}^L + \Phi(r_{ij}) \left(f_{ij}^H - f_{ij}^L \right). \tag{15.20}$$

В одномерном случае для записи соотношения наклонов r_i (15.11) использовались три точки: текущий узел i, узел против потока $i-1$, и узел по потоку $i+1$. Для случае неструктурированной сетки лишь две из этих трёх точек являются узлами коллокации: текущий узел i и узел по потоку

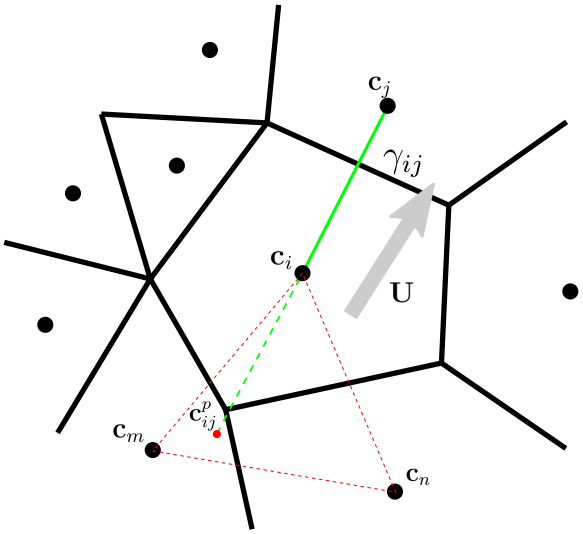


Рис. 11: Вспомогательный узел $\mathbf{c}_{ij}^p$ на конечнообъёмной сетке

j. Определим точку против потока симметричным отражением: $\mathbf{c}_{ij}^p = 2\mathbf{c}_i - \mathbf{c}_j$ (см. рис. 11). Значение функции в этой точке обозначим как u_{ij}^p. Тогда соотношение наклонов запишется в виде

$$r_i = \frac{u_i - u_{ij}^p}{u_j - u_i} \tag{15.21}$$

Точка $\mathbf{c}^p$ (в отличии от x_{i-1} из одномерного случая) не является точкой коллокации. То есть значение u^p нельзя достать из вектора столбца сеточной функции u. Однако, это значение можно интерполировать по значениям в ближайших точках коллокации.

15.2.1 Прямая интерполяция противопоточного значения

Так, в двумерном случае для определения u_{ij}^p необходимо найти три точки коллокации, ближайшие к точке $\mathbf{c}_{ij}^p$ и не лежащие на одной прямой (или две точки коллокации, помимо c_i). На рис. 11 они помечены индексами $\mathbf{c}_m$, $\mathbf{c}_n$). И далее в треуольнике, образованном этими тремя точками ($\triangle_{imn}$) провести интерполяцию по формуле (A.27). Специально отметим, что точка $\mathbf{c}_{ij}^p$ не обязана содержаться внутри треугольника $\triangle_{imn}$.

15.2.2 Интерполяция противопоточного значения через значение градиентов

. Другой подход к определению u_{ij}^p основан на записи симметричной конечной разности по направлению $\mathbf{c}_{ij}$:

$$\left.\frac{\partial u}{\partial c_{ij}}\right|_i = \frac{u_j - u_{ij}^p}{2|\mathbf{c}_{ij}|} \qquad \Rightarrow \quad u_{ij}^p = u_j - 2|\mathbf{c}_{ij}| \left.\frac{\partial u}{\partial c_{ij}}\right|_i .$$

Производная по направлению c_{ij} находится как проекция градиента:

$$|\mathbf{c}_{ij}| \left.\frac{\partial u}{\partial c_{ij}}\right|_i = \mathbf{c}_{ij} \cdot (\nabla u)_i .$$

Таким образом, задача интерполяции сводится к задаче определения градиента функции u в узлах коллокации.

15.2.3 Определение градиентов в узлах коллокации. Метод наименьших квадратов

Будем рассматривать узел i, имеющий N_i соседних узлов j. Для каждого j можно записать линейное приближение

$$u_j = u_i + |\mathbf{c}_{ij}|\frac{\partial u}{\partial c_{ij}} = u_i + \mathbf{c}_{ij} \cdot \nabla u, \qquad j = \overline{0, N_i - 1}.$$

Для двумерного случая можно записать:

$$(\mathbf{c}_{ij})_x \frac{\partial u}{\partial x} + (\mathbf{c}_{ij})_y \frac{\partial u}{\partial y} = u_j - u_i, \qquad j = \overline{0, N_i - 1}.$$

Это выражение – есть система линейных уравнений с двумя неизвестными $\partial u/\partial x$, $\partial u/\partial y$ и N_i строками. Запишем её в матричном виде:

$$Ay = f, \quad \text{где} \quad \begin{array}{ll} A_{j0} = (\mathbf{c}_{ij})_x & A_{j1} = (\mathbf{c}_{ij})_y, \\ y_0 = \partial u/\partial x & y_1 = \partial u/\partial y\ , \\ f_j = u_j - u_i & . \end{array}$$

В двумерном случае размерность матрицы A есть $[N_i, 2]$ (для трёхмерной задачи следуя аналогичным рассуждениям получим матрицу с размерностью $[N_i, 3]$).

При этом в двумерном случае у конечного элемента будет минимум три грани (или четыре в трёхмерном случае). То есть $N_i \geq 3$ и полученная система имеет неизвестных больше, чем количество уравнений. Эта система в общем случае не имеет точного решения, но можно найти такие y, при котором невязка будет минимальной. Определим невязку как

$$r_i = \sum_{j=0}^{N_i} (A_{ij} y_j) - f_i, \qquad i = 0, 1.$$

и будем минимизировать её квадрат

$$F = \sum_i r_i^2 \to \min$$

Запишем условие экстремума как

$$\frac{\partial F}{\partial y_i} = 2\sum_j r_j \frac{\partial r_j}{\partial y_i} = 2\sum_j \left(\sum_k (A_{jk} y_k) - f_j \right) A_{ji} = 0, \qquad i = 0, 1.$$

Отсюда получим систему уравнений

$$\sum_j \left(A_{ji} \sum_k (A_{jk} y_k) \right) = \sum_j A_{ji} f_j = 0, \qquad i = 0, 1.$$

Или, возвращаясь к матричной записи,

$$A^T A y = A^T f.$$

Полученная система имеет размерность 2×2 (или 3×3 в трёхмерном случае). Значение компонент

градиента в точке коллокации запишется как её прямое решение:

$$y = \left(A^T A\right)^{-1} A^T f.$$

Отметим, что матрица A зависит только от геометрии сетки. Поэтому в программной реализации матричное выражение $\left(A^T A\right)^{-1} A^T$ может быть расчитано один раз для каждого узла коллокации на этапе инициализации. Тогда определение градиента в центрах ячеек на этапе решения задачи сведётся к сборке вектора f и умножении его на это выражение.

15.2.4 Реализация для явной схемы

Для примера рассмотрим написание TVD-схемы рассмотрим чисто явную схему для полудискретизованного уравнения (15.17):

$$|V_i| \frac{\hat{u} - u}{\tau} + \sum_{j \in \mathrm{nei}(i)} f_{ij} \, |\gamma_{ij}| = 0. \tag{15.22}$$

Для того, чтобы избежать повторного вычисления потоков f_{ij} и f_{ji}, будем собирать эту схему в цикле по граням. Пусть через границу притока нет (то есть для граничные граней $f_{ij} = 0$. Тогда останется только цикл по внутренним граням:

$\hat{u} = u$	– инициализируем следующий шаг
for $s \in$ internal	– цикл по внутренним граням
$i, j =$ nei_cells(s)	– две ячейки, соседние с текущей гранью
$\mathbf{U}_{ij}$	– вектор скорости в центре грани
$\mathbf{n}_{ij}$	– вектор нормали к грани от ячейки i к j
$U_{ij} = \mathbf{U}_{ij} \cdot \mathbf{n}_{ij}$	– проекция скорости на нормаль
if $U_{ij} \le 0$	– схема против потока
$\mathrm{swap}(i, j); U_{ij} = -U_{ij}$	– гарантируем, что жидкость течет от i к j
endif	
$f^L_{ij} = U_{ij} u_i$	– поток 1-го порядка (15.19)
$f^H_{ij} = U_{ij}(u_i + u_j)/2$	– поток 2-го порядка (15.19)
$\mathbf{c}_i, \mathbf{c}_j$	– центры ячеек
$\mathbf{c}^p = 2\mathbf{c}_i - \mathbf{c}_j$	– вспомогательная точка
$u^p = \mathrm{interpolate}(i, j, u, \mathbf{c}^p)$	– интерполируем u в точке $\mathbf{c}^p$
$r = (u_i - u^p)/(u_j - u_i)$	– отношение наклонов (15.21)
$F = \mathrm{limiter}(r)$	– ограничитель (15.16)
$f_{ij} = f^L_{ij} + F(f^H_{ij} - f^L_{ij})$	– вычисление потока (15.20)
$\hat{u}_i$-=$\tau/\lvert V_i\rvert \, f_{ij} \, \lvert\gamma_{ij}\rvert$	– добавление в противопотоковую ячейку
$\hat{u}_j$+=$\tau/\lvert V_j\rvert \, f_{ij} \, \lvert\gamma_{ij}\rvert$	– добавление в попотоковую ячейку
endfor	

(15.23)

Отметим, что использование противоположенного знака при добавлении в правую от грани ячейку j связано с тожедством (15.18). То есть на самом деле в ячейку j должен бы добавляться поток f_{ji}, но поскольку отдельной обработки этого направления не предусмотрено, мы добавляем f_{ij} с обратным знаком. При реализации функции interpolate должен использоваться один из методов, изложенных

в пп. 15.2.1, 15.2.2.

15.3 Задание для самостоятельной работы

В тесте `[transport2-fvm-upwind-explicit]` из файла `transport_fvm_solve_test.cpp` реализовано решение двумерного уравнения переноса по явной противопотоковой схеме. Реализация алгоритма в целом соответствует циклу (15.23) с упрощениями, следующими из отсутствия необходимости вычислять r в схеме против потока (где всегда $F = 0$).

Отталкиваясь от этого теста нужно решить двумерное уравнение переноса с помощью МКО аппроксимации на неструктурированной сетке. Использовать постановку из п. 14.5.1.2 с $\sigma = 0.1$. Рассмотреть схему против потока и MC TVD-схему пространственной аппроксимации и явную схему для дискретизации по времени. Для интерполяции противопотокового значения u^p использовать алгоритм 15.2.2.

Неструктурированную сетку строить с помощью скрипта `test_data/hexagrid.py`. Количество элементов сетки подобрать так, чтобы видеть эффект от применения схемы высокого порядка.

1. Проиллюстрировать динамику численного решения на неструктурированной конечнообъёмной сетке. Сравнить решение MC и Upwind.

2. Для выбранной сетки опытным путём определить максимально допустимый шаг по времени, при котором решение не разваливается;

3. Посчитать нормы отклоения полученного численного решения от известного точного (по максимуму и среднеквадратичную):

$$n_{max} = 1 - \max_i(u_i);$$
$$n_2 = \left(\frac{\sum_i \left(u_i - u^e(c_i)\right)^2 |V_i|}{\sum_i |V_i|} \right)^{1/2}$$

   Нарисовать графики этих норм от времени $n(t)$ для двух рассмотренных схем.

Рекомендации к программированию Для вычисления градиентов в центрах ячеек использовать класс `FvmCellGradient` из файла `fvm/fvm_assembler.hpp`, экземпляр которого нужно создать на этапе инициализации задачи. Тогда для вычисления градиентов от известной сеточной функции `u` достаточно вызывать метод `FvmCellGradient::compute`.

16 Лекция 16 (16.11)

16.1 Алгебраический подход к построению нелинейных TVD схем

TODO

16.2 Схемы с искусственной вязкостью

TODO

16.2.1 Направленная искусственная вязкость

TODO

16.2.2 SUPG

TODO

16.3 Задание для самостоятельной работы

В тестовом примере `[convdiff-fem-supg]` из файла `convdiff_fem_test.cpp` производится численного решение одномерного нестационарного уравнения конвекции-диффузии

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} - \varepsilon\frac{\partial^2 u}{\partial x^2} = 0$$

в области $x \in [0, 4]$ с точным решением вида

$$u^e(x,t) = \frac{1}{\sqrt{4\pi\varepsilon(t+t_0)}} \exp\left(-\frac{(x-vt)^2}{4\varepsilon(t+t_0)}\right)$$

Точное решение используется для формулировки начальных ($t = 0$) и граничных ($x = 0, 4$) условий первого рода. Результат расчёта сохраняется в файл `convdiff-supg.vtk.series`.

Задача полудискретизуется по схеме Кранка-Николсон с шагом по времени, вычисленным через число Куранта $\mathrm{C} = 0.5$.

После дискретизации задача сводится к СЛАУ относительно неизвестного сеточного вектора u

$$\begin{aligned}
&\mathrm{A}u = \mathrm{B}\check{u}.\\
&\mathrm{A} = \mathrm{M} + \tau\theta\mathrm{K} + \varepsilon\tau\theta\mathrm{S},\\
&\mathrm{B} = \mathrm{M} - \tau(1-\theta)\mathrm{K} - \varepsilon\tau(1-\theta)\mathrm{S}.
\end{aligned}$$

Здесь матрица масс M – результат коненчноэлементной аппроксимации единичного оператора, матрица переноса K – конвективного оператора, а матрица жёсткости S – оператора диффузии, τ – шаг по времени, и для схемы Кранка–Николсон $\theta = 0.5$.

Стабилизированные матрицы вычислялись по следующим формулам:

$$\mathrm{M} = \int\limits_{\Omega} \phi_j(\phi_i + s\mathbf{v}\cdot\nabla\phi_i)\,d\mathbf{x}$$

$$\mathrm{K} = \int\limits_{\Omega} \mathbf{v}\cdot\nabla\phi_j(\phi_i + s\mathbf{v}\cdot\nabla\phi_i)\,d\mathbf{x}$$

$$\begin{aligned}
\mathrm{S} &= -\int\limits_{\Omega} \nabla^2\phi_j(\phi_i + s\mathbf{v}\cdot\nabla\phi_i)\,d\mathbf{x} \\
&\quad \text{по формуле (A.12)} \\
&= \int\limits_{\Omega} \nabla\phi_j\cdot\nabla(\phi_i + s\mathbf{v}\cdot\nabla\phi_i)\,d\mathbf{x} \\
&\quad \text{если } \phi_i \text{ линейны, то } \nabla^2\phi_i = 0 \\
&= \int\limits_{\Omega} \nabla\phi_j\cdot\nabla\phi_i\,d\mathbf{x}
\end{aligned}$$

где $s = \mu h/|v|^2$, h – характерный линейный размер элемента, а μ – параметр SUPG-стабилизации.

Сборки необходимых матриц осуществляется в процедуре `asseble_solver`. Локальные матрицы для операторов M, K вычисляются с помощью численного интегрирования в процедуре `custom_matrix`, которой в качестве аргумента передаётся функция подинтегрального выражения.

Необходимо:

1. В одномерном тесте `[convdiff-supg]` с помощью анимированных графиков продемонстрировать наличие осцилляций при выбранных параметрах решения при отсутствии стабилизации `mu_supg = 0`, а так же эффект от SUPG-слагаемого (`mu_supg > 0`);

2. Нарисовать в сравнении результаты расчёта на конечный промежуток времени для различных μ;

3. Подобрать оптимальную величину параметра μ, минимизирующую норму отклонения численного решения от точного;

4. Написать аналогичный тест для двумерного случая (решение в единичном квадрате) с точным решением
$$u^e(x,t) = \frac{1}{4\pi\varepsilon(t+t_0)}\exp\left(-\frac{(x-v_x t)^2 + (y-v_y t)^2}{4\varepsilon(t+t_0)}\right)$$
Использовать $\mathbf{v} = (1,1)$. Взять треугольную сетку, построенную процедурой `trigrid.py`. Проанализировать результаты, полученные на грубой и подробной сетке.

В целом имеющийся решатель не зависит от геометрической размерности задачи. Для двумерного решателя нужно

- изменить значение точного решения и скорости (функции `velocity`, `nonstat_solution`);
- внести изменения в процедуру постановки граничных условий в функциях `assemble_solver`, `assemble_rhs` (сместо двух точек начала и конца одномерной области необходимо пройтись по всем граничным узлам двумерной сетки);

17 Лекция 17 (23.11)

17.1 Моделирование течения вязкой несжимаемой жикости методом конечных разностей

17.1.1 Система уравнений Навье-Стокса

Будем рассматривать стационарную двумерную систему уравнений Навье-Стокса для вязкой несжимаемой жидкости. В безразмерном консервативном виде в декартовой системе координат она имеет вид

$$\frac{\partial u^2}{\partial x} + \frac{\partial uv}{\partial y} = -\frac{\partial p}{\partial x} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{17.1}$$

$$\frac{\partial uv}{\partial x} + \frac{\partial v^2}{\partial y} = -\frac{\partial p}{\partial y} + \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right), \tag{17.2}$$

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \tag{17.3}$$

Неизвестными являются поля скорости: u – в направлении оси x, v – в направлении оси y, и давления p.

Число Рейнольдса определено через характерную скорость U, [м/с] и характерный линейный размер L, [м] как

$$\mathrm{Re} = \frac{UL\rho}{\mu},$$

где ρ, [кг/м^3] – постоянная (вследствии несжимаемости) плотность жидкости, а μ, [Па·с] – динамическая вязкость жидокости.

Характерое значение для давление выпишется в виде: $p^0 = \rho U^2$, [Па].

Для решения этой системы будем использовать метод конечных разностей с аппроксимацией по пространству второго порядка и последовательное (раздельное) решение входящих в неё уравнений.

Глядя на вид уранений (17.1) – (17.3) можно выделить несколько проблем, которые необходимо решить при построении расчётной схемы:

- нелинейность конвективного оператора в (17.1), (17.2),
- отсутствие явного уравнения для определения давления,
- аппркосимация первых производных для давления и скорости со вторым порядком точности.

Для решения первой проблемы будем использовать итерационный процесс с линеаризацией – то есть записывать уравнение на итерационном слое используя значения неизвестных полей с прошлого слоя. Вторую проблему будем решать с помощью алгоритма SIMPLE связывания давления и скорости (Pressure-Velocity Coupling). Решать третью проблему будем с помощью пространственной аппроксимации на разнесённой сетке (Staggered Grid).

17.1.2 Схема расчёта

Стационарную задачу (17.1)-(17.3) будем решать методом установления. Для этого в первые два уравнения введём фиктивную производную по времени, которую распишем по неявной двухслойной схеме с шагом τ. Тогда задача на одном итерационном слое примет вид

$$\frac{\hat{u}-u}{\tau}+\frac{\partial u\hat{u}}{\partial x}+\frac{\partial v\hat{u}}{\partial y}=-\frac{\partial \hat{p}}{\partial x}+\frac{1}{\mathrm{Re}}\left(\frac{\partial^2\hat{u}}{\partial x^2}+\frac{\partial^2\hat{u}}{\partial y^2}\right), \tag{17.4}$$

$$\frac{\hat{v}-v}{\tau}+\frac{\partial u\hat{v}}{\partial x}+\frac{\partial v\hat{v}}{\partial y}=-\frac{\partial \hat{p}}{\partial y}+\frac{1}{\mathrm{Re}}\left(\frac{\partial^2\hat{v}}{\partial x^2}+\frac{\partial^2\hat{v}}{\partial y^2}\right), \tag{17.5}$$

$$\frac{\partial \hat{u}}{\partial x}+\frac{\partial \hat{v}}{\partial y}=0. \tag{17.6}$$

При записи была произведена линеаризация конвективного слагаемого: один из множителей в производной был отнесён на предыдущий временной слой. В остальном схема неявная.

На временном слое значения u, v, p известны, а $\hat{u}, \hat{v}, \hat{p}$ подлежат определению.

Критерием выхода из итерационного процесса является пороговое условие на невязку, вычисленную с использованием найденных на слое значений неизвестных:

$$r_u=\frac{\partial \hat{u}\hat{u}}{\partial x}+\frac{\partial \hat{u}\hat{v}}{\partial y}+\frac{\partial \hat{p}}{\partial x}-\frac{1}{\mathrm{Re}}\left(\frac{\partial^2\hat{u}}{\partial x^2}+\frac{\partial^2\hat{u}}{\partial y^2}\right),$$

$$r_v=\frac{\partial \hat{u}\hat{v}}{\partial x}+\frac{\partial \hat{v}\hat{v}}{\partial y}+\frac{\partial \hat{p}}{\partial y}-\frac{1}{\mathrm{Re}}\left(\frac{\partial^2\hat{v}}{\partial x^2}+\frac{\partial^2\hat{v}}{\partial y^2}\right),$$

$$\max(\|r_u\|,\|r_v\|)<\varepsilon. \tag{17.7}$$

17.1.2.1 Метод SIMPLE

Приведём алгоритм для явного выражения уравнения для давления из уравнения неразрывности (17.6).

Распишем искомые перенные в виде суммы

$$\begin{aligned}\hat{u}&=u^*+u',\\ \hat{v}&=v^*+v',\\ \hat{p}&=p+p'.\end{aligned} \tag{17.8}$$

Пусть введённые выше поля u^*, v^* удовлетворяют уравнениям

$$u^*+\tau\frac{\partial uu^*}{\partial x}+\tau\frac{\partial vu^*}{\partial y}-\frac{\tau}{\mathrm{Re}}\left(\frac{\partial^2u^*}{\partial x^2}+\frac{\partial^2u^*}{\partial y^2}\right)=-\tau\frac{\partial p}{\partial x}+u, \tag{17.9}$$

$$v^*+\tau\frac{\partial uv^*}{\partial x}+\tau\frac{\partial vv^*}{\partial y}-\frac{\tau}{\mathrm{Re}}\left(\frac{\partial^2v^*}{\partial x^2}+\frac{\partial^2v^*}{\partial y^2}\right)=-\tau\frac{\partial p}{\partial y}+v. \tag{17.10}$$

Тогда уравнение для поправки u' запишем вычтя последнее выражение из уравнения (17.4), умноженного на τ:

$$u' + \tau\frac{\partial uu'}{\partial x} + \tau\frac{\partial vu'}{\partial y} - \frac{\tau}{\mathrm{Re}}\left(\frac{\partial^2 u'}{\partial x^2} + \frac{\partial^2 \hat{u}'}{\partial y^2}\right) = -\tau\frac{\partial p'}{\partial x}. \tag{17.11}$$

Основная идея алгоритма SIMPLE заключается в приближённом представлении выражения (17.11) в явном виде относительно поправки. Для этого все дифференциальные операторы, включающие в себя поправку скорости, из выражения убираются, а для компенсации в правую часть добавляется множитель d^u:

$$u' \approx -\tau d^u(x,y)\frac{\partial p'}{\partial x}. \tag{17.12}$$

Аналогичные рассуждения в отношении поправки поперечной скорости v' приводят к выражению

$$v' \approx -\tau d^v(x,y)\frac{\partial p'}{\partial y}. \tag{17.13}$$

К точному определению значения полей d^u, d^v вернёмся позднее, когда будем расписывать эти выражения на матричном уровне.

Далее используем уравнение неразрывности (17.6). Подставим в него разложения (17.8) и используем (17.12), (17.13). Тогда получим уравнение Пуассона с непостоянным по пространству векторным коэффициентом диффузии (d^u, d^v) относительно поправки давления p':

$$-\left[\frac{\partial}{\partial x}\left(d^u\frac{\partial p'}{\partial x}\right) + \frac{\partial}{\partial y}\left(d^v\frac{\partial p'}{\partial y}\right)\right] = -\frac{1}{\tau}\left(\frac{\partial u^*}{\partial x} + \frac{\partial v^*}{\partial y}\right). \tag{17.14}$$

Определим порядок вычислений на итерационном слое. Напомним, что значения u, v, p с предыдущего слоя нам известно и задача состоит в нахождении значений $\hat{u}, \hat{v}, \hat{p}$ на текущем слое.

1. Из уравнений (17.9), (17.10) вычисляются значения u^*, v^*;
2. Они используются для вычисления правой части уравнения (17.14), в результате решения которого находится поправка давления p';
3. Дифференцируя найденную поправку давления найдём поправки скорости u', v' из выражений (17.12), (17.13);
4. Окончательно выразим значения переменных для текущего слоя из (17.8). Для улучшения стабильности алгоритма значение давления вычисляют с некоторым коэффициентом релаксации α_p:
$$\hat{p} = p + \alpha_p p';$$
5. Далее проводится вычисление невязки с ипользованием найденных значений $\hat{u}, \hat{v}, \hat{p}$ из выражения (17.7). Если она недостаточно мала, то выполняется присваивание $u = \hat{u},\ v = \hat{v},\ p = \hat{p}$ и возвращение на шаг 1.

Полученные на каждом шаге итерационного процесса компоненты скорости $\hat{u}, \hat{v}$ точно удовлетворяют уравнению неразрывности (17.6) в “чёрных” узлах сетки, но уравнения движения (17.4), (17.5) выполняются лишь приближённо.

Всего в алгоритме SIMPLE есть два параметра: коэффициент релаксации давления α_p и фиктивный шаг по времени τ (который можно трактовать как коэффициент релаксации скорости).

17.1.3 Пространственная аппроксимация

Для численной реализации алгоритма решения необходимо провести пространственную аппроксимацию полудискретизованных выражений (17.9), (17.10), (17.12), (17.13), (17.14).

17.1.3.1 Разнесённая сетка

Будем использовать структурированную четырёхугольную сетку с постоянным шагом по пространству. При этом неизвестные параметры будем задавать по схеме, представленной на рис. 12.

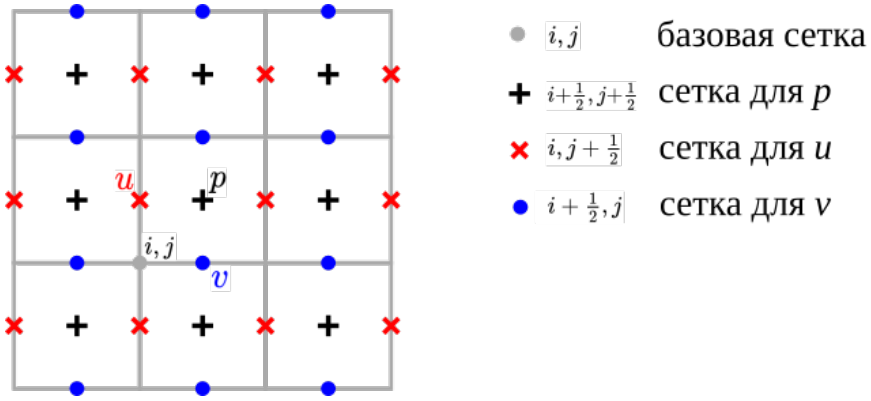


Рис. 12: Разнесённая сетка

Введём разбиение сетки: n_x – количество ячеек в направлении x, n_y – количество ячеек в направлении y.

Очевидно, что при использовании такого разнесённого шаблона, количество точек, в которых заданы значения, будет различным для разных параметров. Так количество узловых значений давления будет равно $n_x \times n_y$, продольной скорости u – $(n_x + 1) \times n_y$, а поперечной v – $n_x \times (n_y + 1)$.

Использование такого расположения узловых точек даёт преимущество при аппроксимации первых производных. Так, конечная разность

$$\left.\frac{\partial p}{\partial x}\right|_{i,j+\frac{1}{2}} = \frac{p_{i+\frac{1}{2},j+\frac{1}{2}} - p_{i-\frac{1}{2},j+\frac{1}{2}}}{h_x} + o(h_x^2)$$

будем симметричной в узле $i, j + \frac{1}{2}$, где задана компонента скорости u, и поэтому будет иметь там второй порядок точности.

Выражения (17.9), (17.12) аппроксимируются на сетке для u, выражения (17.10), (17.13) – на сетке для v, а (17.14) – на сетке для p.

Введём сквозную линейную нумерацию узлов сетки: нулевой узел разположим в левом нижнем углу, далее будем индексировать слева направа и потом снизу вверх. Для основной сетки перевод двумерного индекса i, j в сквозной индекс будет проводится по формуле

$$k(i, j) = j(n_x + 1) + i. \tag{17.15}$$

Для сеток, на которых заданы сеточные параметры, такой перевод примет вид

$$k(i+\tfrac{1}{2}, j+\tfrac{1}{2}) = jn_x + i, \quad - \text{сетка для давления } p \tag{17.16}$$

$$k(i, j+\tfrac{1}{2}) = j(n_x+1) + i, \quad - \text{сетка для продольной скорости } u \tag{17.17}$$

$$k(i+\tfrac{1}{2}, j) = jn_x + i, \quad - \text{сетка для поперечной скорости } v \tag{17.18}$$

17.1.3.2 Уравнения движения

Запишем конечноразностную аппроксимацию уравнения (17.9) для пробной скорости u^* в "красных" узлах сетки $\left(i, j+\frac{1}{2}\right)$:

$$\begin{aligned} & u^*_{i,j+\frac{1}{2}} + \frac{\tau}{h_x}\left((uu^*)_{i+\frac{1}{2},j+\frac{1}{2}} - (uu^*)_{i-\frac{1}{2},j+\frac{1}{2}}\right) \\ & + \frac{\tau}{h_y}\left((vu^*)_{i,j+1} - (vu^*)_{i,j}\right) \\ & - \frac{1}{\mathrm{Re}}\frac{\tau}{h_x^2}\left(u^*_{i-1,j+\frac{1}{2}} - 2u^*_{i,j+\frac{1}{2}} + u^*_{i+1,j+\frac{1}{2}}\right) \\ & - \frac{1}{\mathrm{Re}}\frac{\tau}{h_y^2}\left(u^*_{i,j-\frac{1}{2}} - 2u^*_{i,j+\frac{1}{2}} + u^*_{i,j+\frac{3}{2}}\right) \\ & = u_{i,j+\frac{1}{2}} - \frac{\tau}{h_x}\left(p_{i+\frac{1}{2},j+\frac{1}{2}} - p_{i-\frac{1}{2},j+\frac{1}{2}}\right). \end{aligned} \tag{17.19}$$

В приведённом выражении за исключением конвективных слагаемых вида uu все остальные сеточные вектора используются на своих сетках. Конвективные слагаемые распишем через полусуммы вида:

$$u_{i+\frac{1}{2}} = \frac{u_i + u_{i+1}}{2} + o(h^2)$$

Тогда

$$\begin{aligned} (uu^*)_{i+\frac{1}{2},j+\frac{1}{2}} &= \left(u_{i+\frac{1}{2},j+\frac{1}{2}}\right)\left(u^*_{i+\frac{1}{2},j+\frac{1}{2}}\right) = \frac{1}{4}\left(u_{i,j+\frac{1}{2}} + u_{i+1,j+\frac{1}{2}}\right)\left(u^*_{i,j+\frac{1}{2}} + u^*_{i+1,j+\frac{1}{2}}\right), \\ (uu^*)_{i-\frac{1}{2},j+\frac{1}{2}} &= \frac{1}{4}\left(u_{i,j+\frac{1}{2}} + u_{i-1,j+\frac{1}{2}}\right)\left(u^*_{i,j+\frac{1}{2}} + u^*_{i-1,j+\frac{1}{2}}\right), \\ (vu^*)_{i,j+1} &= \frac{1}{4}\left(v_{i+\frac{1}{2},j+1} + v_{i-\frac{1}{2},j+1}\right)\left(u^*_{i,j+\frac{3}{2}} + u^*_{i,j+\frac{1}{2}}\right), \\ (vu^*)_{i,j} &= \frac{1}{4}\left(v_{i+\frac{1}{2},j} + v_{i-\frac{1}{2},j}\right)\left(u^*_{i,j+\frac{1}{2}} + u^*_{i,j-\frac{1}{2}}\right). \end{aligned}$$

Схему (17.19) можно записать в виде системы линейных уравнений вида

$$S^u u^* = b^u. \tag{17.20}$$

Сеточная матрица A^u будет иметь $(n_x+1)n_y$ строк. Для строки, соответствующей $\left(i, j+\frac{1}{2}\right)$ узлу ненулевыми будут столбцы, соответствующие узлам:

- $\left(i, j+\frac{1}{2}\right)$,
- $\left(i+1, j+\frac{1}{2}\right)$,
- $\left(i-1, j+\frac{1}{2}\right)$,
- $\left(i, j+\frac{3}{2}\right)$,
- $\left(i, j-\frac{1}{2}\right)$.

В случае использования стандартной нумерации узлов структурированной сетки, когда нулевой индекс соответствуют левому нижнему узлу и далее нумерация идёт с быстрым индексом i, то матрица будет пятидиагональной.

Подставим полученные выражения в конвективную часть выражения (17.19). Множитель при диагональном элементе $u^*_{i,j+\frac{1}{2}}$ будет равен:

$$\frac{\tau}{4}\left(\underbrace{\frac{u_{i,j+\frac{1}{2}}-u_{i-1,j+\frac{1}{2}}}{h_x}}_{\left.\frac{\partial u}{\partial x}\right|_{i-\frac{1}{2},j+\frac{1}{2}}}+\underbrace{\frac{u_{i+1,j+\frac{1}{2}}-u_{i,j+\frac{1}{2}}}{h_x}}_{\left.\frac{\partial u}{\partial x}\right|_{i+\frac{1}{2},j+\frac{1}{2}}}+\underbrace{\frac{v_{i+\frac{1}{2},j+1}-v_{i+\frac{1}{2},j}}{h_y}}_{\left.\frac{\partial v}{\partial y}\right|_{i+\frac{1}{2},j+\frac{1}{2}}}+\underbrace{\frac{v_{i-\frac{1}{2},j+1}-v_{i-\frac{1}{2},j}}{h_y}}_{\left.\frac{\partial v}{\partial y}\right|_{i-\frac{1}{2},j+\frac{1}{2}}}\right)$$

Сумма первого и четвёртого слагаемых представляет собой разностный аналог уравнения неразрывности (17.6), записанной для “чёрного” узла сетки $i-\frac{1}{2}, j+\frac{1}{2}$ относительно компонент скорости с предыдущей итерации. Как было сказано ранее, в настоящем алгоритме уравнение неразрывности для итоговых по результатам итерации скорости в этих узлах выполняется точно. Поэтому эта сумма в точности будет равна нулю. Аналогичный результат получится и для суммы второго и третьего слагаемых. Отсюда следует вывод, что конвективное слагаемое не даёт вклад в диагональ итоговой матрицы (как и следовало ожидать от симметричной аппроксимации).

Окончательно запишем все пять ненулевых вхождений в строку матрицы:

$$S^u\left[k\left(i,j+\tfrac{1}{2}\right),k\left(i,j+\tfrac{1}{2}\right)\right]=1+\frac{2\tau}{\mathrm{Re}}\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right) \quad - \text{ основная диагональ,} \tag{17.21}$$

$$S^u\left[k\left(i,j+\tfrac{1}{2}\right),k\left(i+1,j+\tfrac{1}{2}\right)\right]=-\frac{\tau}{\mathrm{Re}}\frac{1}{h_x^2}+\frac{\tau}{4h_x}\left(u_{i,j+\frac{1}{2}}+u_{i+1,j+\frac{1}{2}}\right) \quad - \text{ первая верхняя диагональ,}$$

$$S^u\left[k\left(i,j+\tfrac{1}{2}\right),k\left(i-1,j+\tfrac{1}{2}\right)\right]=-\frac{\tau}{\mathrm{Re}}\frac{1}{h_x^2}-\frac{\tau}{4h_x}\left(u_{i,j+\frac{1}{2}}+u_{i-1,j+\frac{1}{2}}\right) \quad - \text{ первая нижняя диагональ,}$$

$$S^u\left[k\left(i,j+\tfrac{1}{2}\right),k\left(i,j+\tfrac{3}{2}\right)\right]=-\frac{\tau}{\mathrm{Re}}\frac{1}{h_y^2}+\frac{\tau}{4h_y}\left(v_{i+\frac{1}{2},j+1}+v_{i-\frac{1}{2},j+1}\right) \quad - \text{ вторая верхняя диагональ,}$$

$$S^u\left[k\left(i,j+\tfrac{1}{2}\right),k\left(i,j-\tfrac{1}{2}\right)\right]=-\frac{\tau}{\mathrm{Re}}\frac{1}{h_y^2}-\frac{\tau}{4h_y}\left(v_{i+\frac{1}{2},j}+v_{i-\frac{1}{2},j}\right) \quad - \text{ вторая нижняя диагональ.}$$

Здесь $k(i,j)$ – функция перевода двумерного индекса в сквозной (17.17).

Аналогичные выкладки для второго из уравнений движения (17.10) дают систему уравнений

$$S^v v^* = b^v, \tag{17.22}$$

элементы пятидиагональной матрицы которой имеют вид

$$S^v\left[k\left(i+\tfrac{1}{2},j\right),k\left(i+\tfrac{1}{2},j\right)\right] = 1+\frac{2\tau}{\mathrm{Re}}\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right) \quad - \text{ основная диагональ,} \tag{17.23}$$

$$S^v\left[k\left(i+\tfrac{1}{2},j\right),k\left(i+\tfrac{3}{2},j\right)\right] = -\frac{\tau}{\mathrm{Re}}\frac{1}{h_x^2}+\frac{\tau}{4h_x}\left(u_{i+1,j+\frac{1}{2}}+u_{i+1,j-\frac{1}{2}}\right) \quad - \text{ первая верхняя диагональ,}$$

$$S^v\left[k\left(i+\tfrac{1}{2},j\right),k\left(i-\tfrac{1}{2},j\right)\right] = -\frac{\tau}{\mathrm{Re}}\frac{1}{h_x^2}-\frac{\tau}{4h_x}\left(u_{i,j+\frac{1}{2}}+u_{i,j-\frac{1}{2}}\right) \quad - \text{ первая нижняя диагональ,}$$

$$S^v\left[k\left(i+\tfrac{1}{2},j\right),k\left(i+\tfrac{1}{2},j+1\right)\right] = -\frac{\tau}{\mathrm{Re}}\frac{1}{h_y^2}+\frac{\tau}{4h_y}\left(v_{i+\frac{1}{2},j}+v_{i+\frac{1}{2},j+1}\right) \quad - \text{ вторая верхняя диагональ,}$$

$$S^v\left[k\left(i+\tfrac{1}{2},j\right),k\left(i+\tfrac{1}{2},j-1\right)\right] = -\frac{\tau}{\mathrm{Re}}\frac{1}{h_y^2}-\frac{\tau}{4h_y}\left(v_{i+\frac{1}{2},j}+v_{i+\frac{1}{2},j-1}\right) \quad - \text{ вторая нижняя диагональ.}$$

Правая часть аппроксимируется в виде

$$b^{v^*}[k(i+\tfrac{1}{2},j)] = 1-\frac{\tau}{h_y}\left(p_{i+\frac{1}{2},j+1}-p_{i+\frac{1}{2},j}\right).$$

Используется функция перевода двумерного индекса в сквозной из (17.18).

17.1.3.3 Уравнение для поправки давления

Распишем уравнение (17.14) на "чёрной" сетке методом конечных разностей. Для первого слагаемого получим

$$\begin{aligned}\left.\frac{\partial}{\partial x}\left(d^u\frac{\partial p'}{\partial x}\right)\right|_{i+\frac{1}{2},j+\frac{1}{2}} &\approx \frac{1}{h_x}\left(d^u_{i+1,j+\frac{1}{2}}\left.\frac{\partial p'}{\partial x}\right|_{i+1,j+\frac{1}{2}}-d^u_{i,j+\frac{1}{2}}\left.\frac{\partial p'}{\partial x}\right|_{i,j+\frac{1}{2}}\right)\\ &= \frac{1}{h_x}\left(d^u_{i+1,j+\frac{1}{2}}\frac{p'_{i+\frac{3}{2},j+\frac{1}{2}}-p'_{i+\frac{1}{2},j+\frac{1}{2}}}{h_x}-d^u_{i,j+\frac{1}{2}}\frac{p'_{i+\frac{1}{2},j+\frac{1}{2}}-p'_{i-\frac{1}{2},j+\frac{1}{2}}}{h_x}\right).\end{aligned} \tag{17.24}$$

Аналогично расписываются остальные слагаемые. В результате получим систему линейных уравнений вида

$$A^p p' = b^p, \tag{17.25}$$

где ненулевые коэффициенты пятидиагональной матрицы примут вид

$$A^p[k(i+\tfrac{1}{2},j+\tfrac{1}{2}),k(i+\tfrac{1}{2},j+\tfrac{1}{2})] = \frac{1}{h_x^2}\left(d^u_{i+1,j+\frac{1}{2}} + d^u_{i,j+\frac{1}{2}}\right) + \frac{1}{h_y^2}\left(d^v_{i+\frac{1}{2},j} + d^v_{i+\frac{1}{2},j+1}\right), \tag{17.26}$$

$$\begin{aligned}
A^p[k(i+\tfrac{1}{2},j+\tfrac{1}{2}),k(i+\tfrac{3}{2},j+\tfrac{1}{2})] &= -\frac{1}{h_x^2} d^u_{i+1,j+\frac{1}{2}},\\
A^p[k(i+\tfrac{1}{2},j+\tfrac{1}{2}),k(i-\tfrac{1}{2},j+\tfrac{1}{2})] &= -\frac{1}{h_x^2} d^u_{i,j+\frac{1}{2}},\\
A^p[k(i+\tfrac{1}{2},j+\tfrac{1}{2}),k(i+\tfrac{1}{2},j+\tfrac{3}{2})] &= -\frac{1}{h_y^2} d^v_{i+\frac{1}{2},j+1},\\
A^p[k(i+\tfrac{1}{2},j+\tfrac{1}{2}),k(i+\tfrac{1}{2},j-\tfrac{1}{2})] &= -\frac{1}{h_y^2} d^v_{i+\frac{1}{2},j}.
\end{aligned} \tag{17.27}$$

Столбец свободных членов аппроксимируется в виде

$$b^p[k(i+\tfrac{1}{2},j+\tfrac{1}{2})] = -\frac{1}{\tau}\left(\frac{u^*_{i+1,j+\frac{1}{2}} - u^*_{i,j+\frac{1}{2}}}{h_x} + \frac{v^*_{i+\frac{1}{2},j+1} - v^*_{i+\frac{1}{2},j}}{h_y}\right). \tag{17.28}$$

Здесь используется функция перевода двумерного индекса в сквозной из (17.16).

Далее определим значения d^u, d^v. Согласно идее алгоритма SIMPLE d^u должна быть такой функцией, которая максимально приближает выражение (17.11) к (17.12).

Пространственная аппроксимация выражения (17.11) приводит к системе уравнений

$$S^u u' = -\tau \frac{\partial p'}{\partial x}$$

где матрица S^u – та же самая матрица, которая использовалась при аппроксимации уравнения движения (17.19).

Сравнивая предыдущее выражение с (17.12) сделаем вывод, что d^u должна быть такой, чтобы

$$d^u \frac{\partial p'}{\partial x} \approx (S^u)^{-1} \frac{\partial p'}{\partial x}.$$

То есть поэлементное умножение сеточного вектора d^u на другой вектор должно действовать похоже на умножение обратной к S^u матрицы на этот же самый вектор.

Исходя из свойств матрицы S^u (17.21) можно положить

$$d^u = (\operatorname{diag}(S^u))^{-1} = \left(1 + \frac{2\tau}{\mathrm{Re}}\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right)\right)^{-1} \tag{17.29}$$

и аналогично из (17.23)

$$d^v = (\operatorname{diag}(S^v))^{-1} = \left(1 + \frac{2\tau}{\mathrm{Re}}\left(\frac{1}{h_x^2} + \frac{1}{h_y^2}\right)\right)^{-1}. \tag{17.30}$$

Равентство коэффициентов $d^u = d^v$ – следствие использования симметричной аппроксимации конвективного слагаемого в уравнениях движения.

Таким образом мы получили выражения для коэффициентов уравнения для поправки давления, которые зависят только от разбиения сетки. В случае, если разбиение равномерное ($h_x = \text{const}, h_y = \text{const}$), то все значения коэффициентов одинаковы. Однако, для неравномерных разбиений, они будут зависеть от пространства и задаваться на "красной" (для d^u) и "синей" (для d^v) сетках.

В результате использования (17.29), (17.30) левая часть системы уравнений (17.25) будет постоянна на всех итерациях, что удобно для инициализации алгебраических решателей этой системы (можно провести инициализацию один раз до начала счёта).

Это отличает эту систему от двух других систем, возникающих из аппроксимации уравнений движения (17.20), (17.22), левые части которых зависят от значений с предыдущих итерационных слоёв. Этот момент обуславливает выбор решателей для этих систем, которые в эффективных гидродинамических кодах обычно отличаются, от решателя для системы (17.25).

17.1.3.4 Уравнение для поправки скорости

И наконец рассмотрим аппроксимацию выражений (17.12), (17.13), которые примут явный вид

$$u'_{i,j+\frac{1}{2}} = -\tau d^u_{i,j+\frac{1}{2}} \frac{p'_{i+\frac{1}{2},j+\frac{1}{2}} - p'_{i-\frac{1}{2},j+\frac{1}{2}}}{h_x}, \tag{17.31}$$

$$v'_{i+\frac{1}{2},j} = -\tau d^v_{i+\frac{1}{2},j} \frac{p'_{i+\frac{1}{2},j+\frac{1}{2}} - p'_{i+\frac{1}{2},j-\frac{1}{2}}}{h_x}. \tag{17.32}$$

17.1.3.5 Учёт граничных условий

Для уравнений Навье-Стокса на каждой границе расчётной области требуется столько условий, сколько есть уравнений движения. Для двумерной задачи (17.1) – (17.3) нужно задать два граничных условия.

При использовании разнесённой сетки граница области проходит по граням основной сетки. На нижней и верхней границах расчётной области присутствуют узлы для v, но отсутствуют узлы для u. На правой и левой границах, наоборот, есть узлы с заданными компонентами u, но нет узлов с компонентами v. Узловые значения для давления p никогда не бывают граничными.

Для простоты пока будем рассматривать только случай с заданными значениями двух компонент скорости на каждой из границ задачи:

$$u(x,y)|_{x,y\in\Gamma} = u^\Gamma(x,y),$$
$$v(x,y)|_{x,y\in\Gamma} = v^\Gamma(x,y).$$

В схеме SIMPLE частные граничные условия для скорости учитываются при решении задачи для пробных скоростей u^*, v^*. Тогда для поправки скорости u', v' на границах будут справедливы соответствующие однородные граничные условия (нулевые значения в нашем случае):

$$
\begin{aligned}
u^*(x,y)|_{x,y\in\Gamma} &= u^\Gamma(x,y),\\
v^*(x,y)|_{x,y\in\Gamma} &= v^\Gamma(x,y),\\
u'(x,y)|_{x,y\in\Gamma} &= 0,\\
v'(x,y)|_{x,y\in\Gamma} &= 0.
\end{aligned}
\tag{17.33}
$$

Для учёта граничных условий по скорости требуется модифицировать системы линейных уравнений (17.20), (17.22).

Рассмотрим нижнюю границу $j = 0$.

На нижней границе явно присутствуют узлы “синей” сетки. Значит можно явно установить значения для скорости v путём постановки нулей с единицой на диагонали в строке матрицы и отнесением необходимого граничного значение в правый вектор столбец системы (17.22):

$$
\begin{aligned}
&A^v[k(i+\tfrac{1}{2},0),s] = \delta_{ks}, \quad \forall i,\ \forall s\\
&b^v[k(i+\tfrac{1}{2},0)] = v^\Gamma.
\end{aligned}
\tag{17.34}
$$

Такая модификация просто заменяет $k(i+\frac{1}{2},0)$ -ое уравнение системы (17.22) на выражение

$$
v^*_{i+\frac{1}{2},0} = v^\Gamma.
$$

Узлов для компонет u на нижней границе нет. Рассмотрим первый ряд точек “красной” сетки: $(i,\frac{1}{2})$. Если бы мы захотели заполнить коэффициенты системы линейных уравнений (17.20) по выведенным выше формулам (17.21) для узла, расположенного в этом ряду, мы бы столкнулись с необходимостью установки значения в фиктивную колонку: последнее из уравнений (17.21) предписывает нам установить значение по адресу $[k(i,\frac{1}{2}),k(i,-\frac{1}{2})]$, который, очевидно, не присутствует в матрице.

Действительно, $k(i,\frac{1}{2})$-ая строка системы уравнений (17.21) имеет вид

$$
Du^*_{i,\frac{1}{2}} + U^1u^*_{i+1,\frac{1}{2}} + L^1u^*_{i-1,\frac{1}{2}} + U^2u^*_{i,\frac{3}{2}} + L^2u^*_{i,-\frac{1}{2}} = b^u_{i,\frac{1}{2}},
\tag{17.35}
$$

где D – коэффициент с основной диагонали, $U^{1,2}$, $L^{1,2}$ – коэффициенты с двух верхних и двух нижних диагоналей, вычисляемые по формулам (17.21). Вторая нижняя диагональ у этой строки матрицы отсутствует. Она соответствует вкладу от узла $(i,-\frac{1}{2})$, который лежит вне области расчёта, на полшага ниже нижней границе.

Тем не менее, такой фиктивный узел мы можем использовать для записи аппроксимации

$$
u^*_{i,0} = u^\Gamma = \frac{u^*_{i,\frac{1}{2}} + u^*_{i,-\frac{1}{2}}}{2} + o(h_x^2).
$$

или

$$
u^*_{i,-\frac{1}{2}} \approx 2u^\Gamma - u^*_{i,\frac{1}{2}}.
$$

Подставляя это выражение в строку (17.35) получим

$$(D - L^2)u^*_{i,\frac{1}{2}} + U^1 u^*_{i+1,\frac{1}{2}} + L^1 u^*_{i-1,\frac{1}{2}} + U^2 u^*_{i,\frac{3}{2}} = b^u_{i,\frac{1}{2}} + 2u^\Gamma.$$

Таким образом, добавление коэффициента в фиктивную колонку строки матрицы при наличие условия первого рода на границе равносильно вычитанию этого коэффициента из диагонального элемента этой строки и вычитанием удвоенного граничного значения из правой части. В случае нижней границы получим

$$A^u[k(i,\tfrac{1}{2}), k(i,\tfrac{1}{2})] \mathrel{-}= A^u[k(i,-\tfrac{1}{2})], \tag{17.36}$$

$$b^u[k(i,\tfrac{1}{2})] \mathrel{-}= 2u^\Gamma.$$

Приёмы (17.34), (17.36) используются и на остальных границах для постановки граничных условий для скорости.

При сборке системы линейных уравнений для поправки давления (17.25) так же возникает проблема с обращением к фиктивным узлам. Например, при рассмотрении левой стенки ($i = 0$ третье из уравнений (17.26) описывает несуществующий столбец $k(-\frac{1}{2}, j + \frac{1}{2})$. Если обратиться к выражению (17.24), то будет видно, что это слагаемое пришло в результате расписывания граничной производной p', которая, исходя из выражения (17.12) пропорциональна граничному значению u', то есть, вспоминая (17.33), равна нулю:

$$\left.\frac{\partial p'}{\partial x}\right|_{0,j+\frac{1}{2}} = -\frac{1}{\tau d^u} u'_{0,j+\frac{1}{2}} = 0.$$

То есть добавлять слагаемые, соответствующие фиктивным узлам, в матрицу A^p не нужно. Не нарушая общности выведённых ранее выражений (17.26), просто модифицируем значения коэффициентов d^u, d^v:

$$d^u_{0,j+\frac{1}{2}} = d^u_{n_x+1,j+\frac{1}{2}} = 0, \tag{17.37}$$

$$d^v_{i+\frac{1}{2},0} = d^u_{i+\frac{1}{2},n_y+1} = 0.$$

В исходных уравнениях (17.1)-(17.3) давление присутствует только в виде своих производных. Если в задаче нигде не задано явное граничное условие для давления, то решение для давления будет определено только с точностью до константы. Чтобы убрать эту неопределённость рекомендуется явно положить давление нулю в любом узле. Например, в случае нулевого узла, по аналогии с (17.34) запишем:

$$A^p[k(\tfrac{1}{2},\tfrac{1}{2}), s] = \delta_{ks}, \tag{17.38}$$

$$b^p[k(\tfrac{1}{2},\tfrac{1}{2})] = 0.$$

17.1.4 Оптимальные значения параметров алгоритма SIMPLE

Введем обозначение

$$E = \frac{4\tau}{\mathrm{Re}\,\bar{h}^2}$$

где $\bar{h}^2$ – среднее гармоническое значение шага, определяемое как

$$\bar{h}^2 = \frac{2\,h_x^2\,h_y^2}{h_x^2 + h_y^2}.$$

Значение E – есть диагональное компонента матрицы диффузии в аппроксимированных уравнениях движения (второе слагаемое в правой части первой формулы (17.21)).

При независимом задании релаксаций по скорости и давлению, оптимальной сходимости соответствуют значения

$$E = 1 \quad \Rightarrow \quad \tau = \frac{\mathrm{Re}\,\bar{h}^2}{4},$$
$$\alpha_p = 0.8.$$

Ещё более эффективной сходимости соответствуют параметры

$$E \approx 4, \quad \alpha_p = \frac{1}{1+E}. \tag{17.39}$$

Это выражение соответствует алгоритму SIMPLEC (согласованный алгоритм, SIMPLE Consistent).

17.2 Программа для расчёта течения в каверне по схеме SIMPLE

17.2.1 Постановка задачи

Для иллюстрации работы алгоритма рассмотрим задачу о течении в каверне. Постановку задачи представлена на рис. 13.

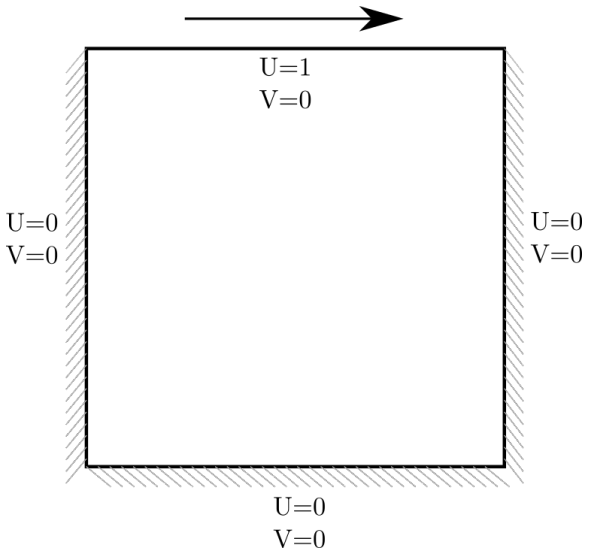


Рис. 13: Область расчёта задачи о каверне

Задача реализована в тесте `[cavity2-simple]` в файле `cavity_simple_test.cpp`.

Программа проводит итерации стартуя от начального нулевого состояния $u = v = p = 0$ до тех пор, пока невязка не достигнет заданного порога. На каждой итерации поле давления и векторное поле скорости сохраняются на основной сетке в файл `cavity2.vtk.series`.

Итоговый результат (для $\varepsilon = 10^{-2}$) представлен на рис. 14.

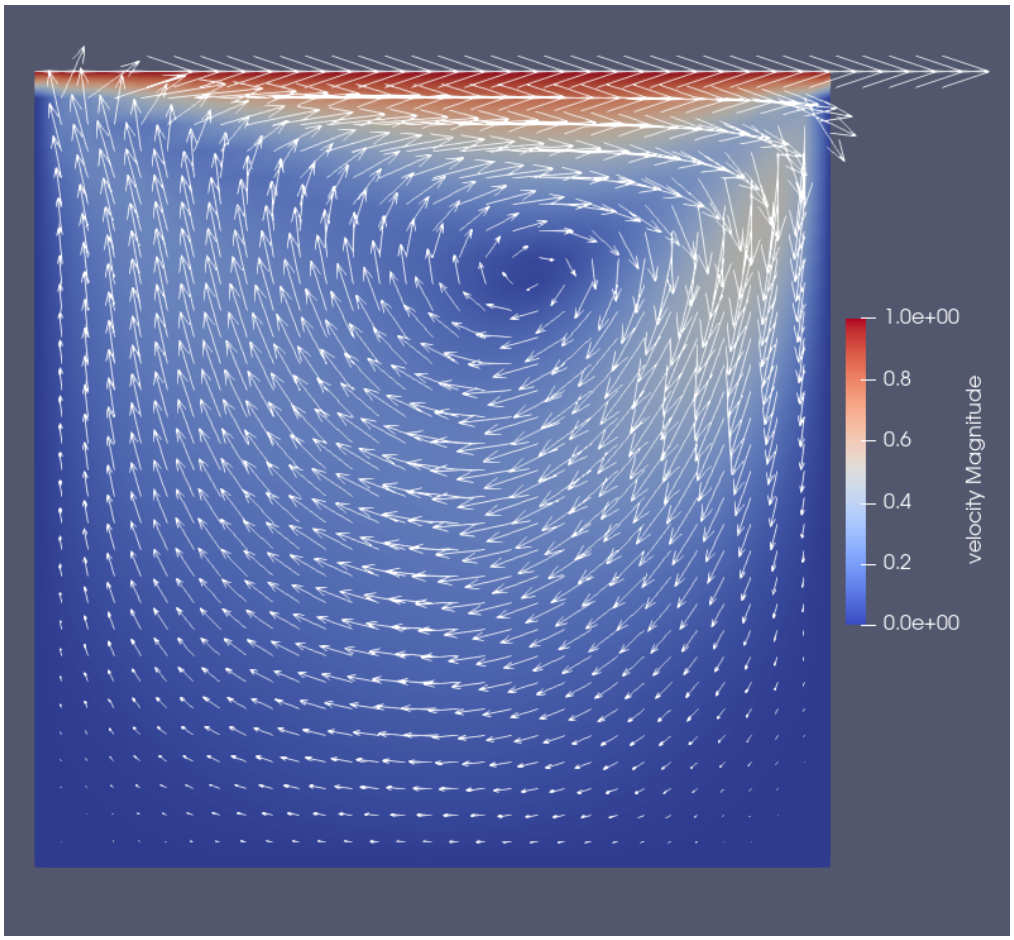


Рис. 14: Область расчёта задачи о каверне

Для отображения вектора поля скорости в Paraview см. справку в B.3.5.

Для работы с разнесённой сеткой в классе `cfd::RegularGrid2D` представлены функции

- `cfd::RegularGrid2D::cell_centered_grid()` – построить сетку по центрам ячеек (“чёрную” сетку для p),
- `cfd::RegularGrid2D::xface_centered_grid()` – построить сетку по центрам x-граней (“синюю” сетку для v),
- `cfd::RegularGrid2D::yface_centered_grid()` – построить сетку по центрам y-граней (“красную” сетку для u),

и функции перевода индексов

- `cfd::RegularGrid2D::cell_centered_grid_index_ip_jp` – посчитать линейный индекс “чёрной” сетки (17.16),
- `cfd::RegularGrid2D::xface_grid_index_ip_j` – посчитать линейный индекс “синей” сетки (17.18),

- `cfd::RegularGrid2D::yface_grid_index_i_jp` – посчитать линейный индекс “красной” сетки (17.17).

17.2.2 Функция верхнего уровня

```
453 TEST_CASE("Cavity 2D, SIMPLE algorithm", "[cavity2-simple]"){
```

Сначала устанавливаются параметры задачи: число Рейнолдса,

```
457   double Re = 100;
```

параметры алгоритма SIMPLE,

```
458   double tau = 0.03;
459   double alpha = 0.8;
```

разбиение сетки,

```
460   size_t n_cells = 30;
```

максимальное количество итераций

```
461   size_t max_it = 10000;
```

и значение невязки, при котором итерации прекращаются

```
462   double eps = 1e-0;
```

Затем происходит инициализация решателя, который определён в классе `Cavity2DSimpleWorker`

```
465   Cavity2DSimpleWorker worker(Re, n_cells, tau, alpha);
```

и параметров сохранения. Здесь первым параметром является флаг сохранения точных сеточных значений, который установлен в `false`, а также имя файла с итоговым результатом. Таким образом сохраняться будет только решение, интерполированное на основную сетку. Для целей отладки программы (для просмотра действительных, не интерполированных полей решения) следует первый флаг установить в `true`. Тогда помимо `cavity2.vtk.series`, будут создаваться также файлы `cavity2-u`, `cavity2-v`, `cavity2-p`.

```
466   worker.initialize_saver(false, "cavity2");
```

Потом происходит установка начальных значений искомых сеточных векторов: $u = v = p = 0$

```
469   std::vector<double> u_init(worker.u_size(), 0.0);
470   std::vector<double> v_init(worker.v_size(), 0.0);
471   std::vector<double> p_init(worker.p_size(), 0.0);
472   worker.set_uvp(u_init, v_init, p_init);
```

и начинается итерационный процесс.

```
477   for (it=1; it < max_it; ++it){
```

Внутри цикла выполняется шаг итерационного процесса, который возвращает значение итоговой невязки в переменную `nrm`.

```
478     double nrm = worker.step();
```

На печать выводится индекс итерации, значение невязки и значение давления в правом верхнем узле (для контроля сходимости)

```
481     std::cout << it << " " << nrm << " " << worker.pressure().back() << std::endl;
```

Сохраняется состояние решателся на пройденную итерацию

```
484     worker.save_current_fields(it);
```

и производится проверка на сходимость

```
487     if (nrm < eps){
488       break;
489     }
```

В конце производится проверка: при установленных параметрых решение должно сойтись за 9 итераций:

```
491   CHECK(it == 9);
```

17.2.3 Поля класса решателя

Класс `Cavity2DSimpleWorker` хранит в себе набор полей, характеризующих состояние итерационного процесса. Некоторые из этих полей (параметры решателя) постоянны (`const`) и определяются непосредственно перед вызовом конструктора в инициализаторе. Другие меняются с продвижением по итерациям.

Среди постоянных полей заданы 4 сетки: основная `_grid`, “чёрная” сетка `_cc_grid` (cell-centered) для давления, “красная” сетка `_yf_grid` (y-face) для u, “синяя” сетка `_xf_grid` (x-face) для v (рис. 12).

```
35   const RegularGrid2D _grid;
36   const RegularGrid2D _cc_grid;
37   const RegularGrid2D _xf_grid;
38   const RegularGrid2D _yf_grid;
```

Далее заданы скалярные параметры: число Рейнольдса, шаги сетки и параметры алгоритма SIMPLE

```
39   const double _hx;
40   const double _hy;
41   const double _Re;
42   const double _tau;
43   const double _alpha_p;
```

Далее следуют сеточные вектора, характеризующие текущее состояние решателя: найденные на последней итерации давление и скорости.

```
45  std::vector<double> _p;
46  std::vector<double> _u;
47  std::vector<double> _v;
```

Также определяется данные для решения системы уравнений для нахождения p' (17.25): значения d^u, d^u, а так же инициализированный решатель системы уравнений. Поскольку используется постоянные шаги по времени, d^u, d^v являются скалярами.

```
49  double _du;
50  double _dv;
51  AmgcMatrixSolver _p_prime_solver;
```

Хранятся левая и правая части систем уравнений (17.20), (17.22) для определения пробных значений скорости и расчета невязки.

```
53  CsrMatrix _mat_u;
54  CsrMatrix _mat_v;
55  std::vector<double> _rhs_u;
56  std::vector<double> _rhs_v;
```

Указатели на классы, помогающие сохранять найденные вектора в vtk - формат. Эти классы инициализируются только в случае, если пользователь указал на необходимость сохранения.

```
58  std::shared_ptr<VtkUtils::TimeSeriesWriter> _writer_u;
59  std::shared_ptr<VtkUtils::TimeSeriesWriter> _writer_v;
60  std::shared_ptr<VtkUtils::TimeSeriesWriter> _writer_p;
61  std::shared_ptr<VtkUtils::TimeSeriesWriter> _writer_all;
```

17.2.4 Инициализация решателя

В секции инициализации конструктора созаются сетки в единичном квадрате и переписываются параметры решения. Далее в теле конструктора вычисляются значения d^u, d^v по формулам (17.29), (17.30) и собирается решатель для p'. Как было указано ранее, матрица системы A^p не меняется с продвижением по итерациям, поэтому этот решатель можно собрать один раз до начала счёта.

```
80 Cavity2DSimpleWorker::Cavity2DSimpleWorker(double Re, size_t n_cells, double tau,
   ↪  double alpha_p):
81  _grid(0, 1, 0, 1, n_cells, n_cells),
82  _cc_grid(_grid.cell_centered_grid()),
83  _xf_grid(_grid.xface_centered_grid()),
84  _yf_grid(_grid.yface_centered_grid()),
85  _hx(1.0/n_cells),
86  _hy(1.0/n_cells),
87  _Re(Re),
88  _tau(tau),
89  _alpha_p(alpha_p)
```

```
{
  _du = 1.0 / (1 + 2.0*_tau/_Re * (1.0/_hx/_hx + 1.0/_hy/_hy));
  _dv = 1.0 / (1 + 2.0*_tau/_Re * (1.0/_hx/_hx + 1.0/_hy/_hy));
  assemble_p_prime_solver();
}
```

Начальные значения устанавливаются через вызов функции `set_uvp`. Эти начальные значения будут использоваться в качестве значений с предыдущего итерационного слоя на первой итерации.

В функции происходит переписывание переданных векторов в приватные поля класса.

```
double Cavity2DSimpleWorker::set_uvp(const std::vector<double>& u, const
↪  std::vector<double>& v, const std::vector<double>& p){
  _u = u;
  _v = v;
  _p = p;
```

После этого данных в классе-решателе достаточно, для сборки матриц A^u, A^v и правых частей b^u, b^v для системы уравнений (17.20), (17.22).

```
  assemble_u_slae();
  assemble_v_slae();
```

Если посмотреть на выражение для невязки (17.7) убрав в нём крышки над переменными, то можно убедится, что оно аппроксимируется в виде

$$r_u = \frac{1}{\tau}\left(A^u u - b^u\right).$$

Поэтому после сборки систем уравнений движения, можно вычислить невязку, характеризующую отклонение установленного в этой процедуре решения от желаемого:

```
  // residuals
  auto r_u = compute_residual_vec(_mat_u, _rhs_u, _u);
  auto r_v = compute_residual_vec(_mat_v, _rhs_v, _v);
  double nrm_u = (*std::max_element(r_u.begin(), r_u.end()))/_tau;
  double nrm_v = (*std::max_element(r_v.begin(), r_v.end()))/_tau;

  return std::max(nrm_u, nrm_v);
};
```

17.2.5 Шаг итерации SIMPLE

Осуществляется в процедуре

```
double Cavity2DSimpleWorker::step(){
  // Predictor step: U-star
  std::vector<double> u_star = compute_u_star();
```

```
  std::vector<double> v_star = compute_v_star();
  // Pressure correction
  std::vector<double> p_prime = compute_p_prime(u_star, v_star);
  // Velocity correction
  std::vector<double> u_prime = compute_u_prime(p_prime);
  std::vector<double> v_prime = compute_v_prime(p_prime);
  // Set final values
  std::vector<double> u_new = vector_sum(u_star, 1.0, u_prime);
  std::vector<double> v_new = vector_sum(v_star, 1.0, v_prime);
  std::vector<double> p_new = vector_sum(_p, _alpha_p, p_prime);

  return set_uvp(u_new, v_new, p_new);
}
```

и представляет собой буквальное пошаговое следование алгоритму SIMPLE (17.1.2.1). В конце опять вызывается функция `set_uvp` для сборки матриц для следующей итерации и подсчёта невязки на текущей итерации.

17.2.6 Сборка системы уравнений для поправки давления

Сборка системы уравнений (17.25) осуществляется в процедуре

```
void Cavity2DSimpleWorker::assemble_p_prime_solver(){
```

Сборка происходит с использованием матрицы формата `cfd::LodMatrix`, удобного для непоследовательной записи.

```
  LodMatrix mat(p_size());
```

Заполнение происходит в цикле по раздвоенным индексам ij “чёрной” сетки для давления:

```
  for (size_t j = 0; j < _cc_grid.ny()+1; ++j)
  for (size_t i = 0; i < _cc_grid.nx()+1; ++i){
```

Внутри цикла устанавливаются флаги, характеризующие граничный статус текущего узла

```
    bool is_left = (i==0);
    bool is_right = (i==_cc_grid.nx());
    bool is_bottom = (j==0);
    bool is_top = (j==_cc_grid.ny());
```

Вычисляется значение сквозного индекса по формуле (17.16)

```
    size_t ind0 = _grid.cell_centered_grid_index_ip_jp(i, j);
```

и значения коэффициентов в формулах (17.26). Поскольку сетка равномерная, эти значения не меняются для разных узлов

```
176     double coef_x = _du/_hx/_hx;
177     double coef_y = _dv/_hy/_hy;
```

Далее формулы (17.26) применяются для заполнения матриц с учётом аппроксимированного граничного условия (17.37). Так, запись

```
178     // x
179     if (!is_right){
180       size_t ind1 = _grid.cell_centered_grid_index_ip_jp(i+1, j);
181       mat.add_value(ind0, ind0, coef_x);
182       mat.add_value(ind0, ind1, -coef_x);
183     }
```

для всех неправых узлов с линейным индексом `ind0` вычисляет индекс узла, расположенного правее него с линейным индексом `ind1`, добавляет слагаемое в диагональный (первое из уравнений (17.26)) и вычитает из недиагонального (четвёртое из уравнений (17.26)) элемента строки `ind0`. Для правых узлов работает граничное условие (17.26) и выполнять эту процедуру не нужно.

После заполнения в матрицу вводится граничное условие (17.38)

```
201   mat.set_unit_row(0);
```

И матрица передаётся в решатель СЛАУ предварительно сконверованная в формат `cfd::CsrMatrix`

```
202   _p_prime_solver.set_matrix(mat.to_csr());
```

Правая часть собирается заново на каждой итерации по формуле (17.28). Её реализация представлена в функции

```
351 std::vector<double> Cavity2DSimpleWorker::compute_p_prime(const std::vector<double>&
    ↪  u_star, const std::vector<double>& v_star){
```

Сначала собирается правая часть системы (17.25) по формуле (17.28):

```
353   for (size_t i = 0; i < _grid.nx(); ++i)
354   for (size_t j = 0; j < _grid.ny(); ++j){
355     size_t ind0 = _grid.cell_centered_grid_index_ip_jp(i, j);
356     size_t ind_left = _grid.yface_grid_index_i_jp(i, j);
357     size_t ind_right = _grid.yface_grid_index_i_jp(i+1, j);
358     size_t ind_bot = _grid.xface_grid_index_ip_j(i, j);
359     size_t ind_top = _grid.xface_grid_index_ip_j(i, j+1);
360     rhs[ind0] = -(u_star[ind_right] - u_star[ind_left])/_tau/_hx - (v_star[ind_top] -
      ↪  v_star[ind_bot])/_tau/_hy;
361   }
```

потом осуществляется установка граничного условия (17.38)

```
362   rhs[0] = 0;
```

и вызывается решатель СЛАУ

```
363   std::vector<double> p_prime;
364   _p_prime_solver.solve(rhs, p_prime);
365   return p_prime;
366 }
```

17.2.7 Сборка системы уравнений для пробной скорости

Сборка системы (17.20) (как правой, так и левой частей) реализована в функции

```
205 void Cavity2DSimpleWorker::assemble_u_slae(){
```

Основной цикл идёт по негрничным узлам “красной” сетки, в котором реализуются формулы (17.21)

```
239   for (size_t j=0; j < _grid.ny(); ++j)
240   for (size_t i=1; i < _grid.nx(); ++i){
241     size_t row_index = _grid.yface_grid_index_i_jp(i, j); //[i, j+1/2]
242 
243     double u0_plus   = u_ip_jp(i, j);   //_u[i+1/2, j+1/2]
244     double u0_minus  = u_ip_jp(i-1, j); //_u[i-1/2, j+1/2]
245     double v0_plus   = v_i_j(i, j+1);   //_v[i,j+1]
246     double v0_minus  = v_i_j(i, j);     //_v[i,j]
247 
248     // u_(i,j+1/2)
249     add_to_mat(row_index, {i, j}, 1.0);
250     //      + tau * d(u0*u)/ dx
251     add_to_mat(row_index, {i+1,j}, _tau/2.0/_hx*u0_plus);
252     add_to_mat(row_index, {i-1,j}, -_tau/2.0/_hx*u0_minus);
253     //      + tau * d(v0*u)/dy
254     add_to_mat(row_index, {i, j+1}, _tau/2.0/_hy*v0_plus);
255     add_to_mat(row_index, {i, j-1}, -_tau/2.0/_hy*v0_minus);
256     //      - tau / Re * d^2u/dx^2
257     add_to_mat(row_index, {i, j}, 2.0*_tau/_Re/_hx/_hx);
258     add_to_mat(row_index, {i+1, j}, -_tau/_Re/_hx/_hx);
259     add_to_mat(row_index, {i-1, j}, -_tau/_Re/_hx/_hx);
260     //      - tau / Re * d^2u/dy^2
261     add_to_mat(row_index, {i, j}, 2.0*_tau/_Re/_hy/_hy);
262     add_to_mat(row_index, {i, j+1}, -_tau/_Re/_hy/_hy);
263     add_to_mat(row_index, {i, j-1}, -_tau/_Re/_hy/_hy);
264     // = u0_(i,j+1/2)
265     _rhs_u[row_index] += _u[row_index];
266     //       - tau * dp/dx
267     _rhs_u[row_index] -= _tau/_hx*(p_ip_jp(i, j) - p_ip_jp(i-1, j));
268   }
```

Как было отмечено в пункте 17.1.3.5, граничные условия первого рода в этом уравнении учитываются двумя разными способами: узлы расположенные непосредственно на границе (нижней и верхней) учитываются по схеме (17.34), которая реализована в цикле

```
228  for (size_t j=0; j< _grid.ny(); ++j){
229    size_t index_left = _grid.yface_grid_index_i_jp(0, j);
230    add_to_mat(index_left, {0, j}, 1.0);
231    _rhs_u[index_left] = 0.0;
232
233    size_t index_right = _grid.yface_grid_index_i_jp(_grid.nx(), j);
234    add_to_mat(index_right, {_grid.nx(), j}, 1.0);
235    _rhs_u[index_right] = 0.0;
236  }
```

А фиктивные узлы, возникающие при обработке узлов расположенных в полушаге от границ (левой и правой), обрабатываются по схеме (17.36). Эта схема реализована в виде препроцессинга алгоритма добавления элемента в матрицу в лямбда-функции

```
210  auto add_to_mat = [&](size_t row_index, std::array<size_t, 2> ij_col, double value){
211    if (ij_col[1] == _grid.ny()){
212      // ghost index => top boundary condition: u = u_top
213      size_t ind1 = _grid.yface_grid_index_i_jp(ij_col[0], ij_col[1]-1);
214      mat.add_value(row_index, ind1, -value);
215      _rhs_u[row_index] -= 2.0*value * top_velocity()[0];
216    } else if (ij_col[1] == (size_t)-1){
217      // ghost index => bottom boundary condition: u = u_bot
218      size_t ind1 = _grid.yface_grid_index_i_jp(ij_col[0], ij_col[1]+1);
219      mat.add_value(row_index, ind1, -value);
220      _rhs_u[row_index] -= 2.0*value * bottom_velocity()[0];
221    } else {
222      size_t ind1 = _grid.yface_grid_index_i_jp(ij_col[0], ij_col[1]);
223      mat.add_value(row_index, ind1, value);
224    }
225  };
```

Эта лямбда вызывается везде, где нужно добавить в строку `row_index` и колонку, соответствующую узлу `ij_col`, значение `value`. Она перехватывает ситуации с “фиктивным” узлом ($j = -1, j = n_y$) и применяет алгоритм (17.36).

17.3 Задание для самостоятельной работы

1. Подобрать оптимальные параметры алгоритма SIMPLE τ, α_p для задачи в каверне, при которых сходимость происходит за наименьшее число итераций. Для этого лучше понизить пороговый $\varepsilon = 0.01$. Сравнить полученные вами эмпирически значения с рекомендованными. Увеличить разбиение и отметить, как величина шага по пространству влияет на количество требуемых

итераций. Для ускорения параметрических расчётов лучше собирать программу в “релизной” (B.1.3) версии и убрать сохранение в vtk внутри каждой итерации.

2. Нарисовать поле невязок r_u, r_v в динамике по каждой итерации. Отметить в каком из уравнений и в каких местах области расчёта наблюдаются наибольшие проблемы со сходимостью. Обратить внимание, что невязка r_u задана на “красной” сетке. При этом сохранение на этой сетке делается через объект
`_writer_u`. Невязка r_v задается на “синей” сетке с объектом сохранения `_writer_v`.

3. Решить аналогичную задачу, в которой скорость не только на верхней, но и на нижней стенке равна $U = 1$. Для этого завести новый тест `[cavity2-simple-sym]`.

18 Лекция 18 (30.11)

18.1 Нестационарное уравнение Навье-Стокса

Запишем безразмерную систему (17.1) – (17.3) в нестационарной постановке

$$\begin{aligned}
&\frac{\partial u}{\partial t}+\frac{\partial u^2}{\partial x}+\frac{\partial uv}{\partial y}=-\frac{\partial p}{\partial x}+\frac{1}{\mathrm{Re}}\left(\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}\right),\\
&\frac{\partial v}{\partial t}+\frac{\partial uv}{\partial x}+\frac{\partial v^2}{\partial y}=-\frac{\partial p}{\partial y}+\frac{1}{\mathrm{Re}}\left(\frac{\partial^2 v}{\partial x^2}+\frac{\partial^2 v}{\partial y^2}\right),\\
&\frac{\partial u}{\partial x}+\frac{\partial v}{\partial y}=0.
\end{aligned} \tag{18.1}$$

Характерное время, на которое было произведено обезразмеривание, равно $t^0 = L/U$.

18.1.1 Схема расчёта по алгоритму SIMPLE

Производную по времени будет аппроксимировать по двухслойной неявной схеме.

$$\frac{\partial u}{\partial t}=\frac{\hat{u}-\check{u}}{\triangle t}+o(\triangle t),$$

где символом $\check{\cdot}$ обозначены значения с предыдущего временно́го слоя.

Внутри каждого временно́го слоя будем исполнять итерационный процесс по типу (17.4) – (17.6) с добавлением дискретизованной производной по времени:

$$\begin{aligned}
&\frac{\hat{u}-\check{u}}{\triangle t}+\frac{\hat{u}-u}{\tau}+\frac{\partial u\hat{u}}{\partial x}+\frac{\partial v\hat{u}}{\partial y}=-\frac{\partial \hat{p}}{\partial x}+\frac{1}{\mathrm{Re}}\left(\frac{\partial^2 \hat{u}}{\partial x^2}+\frac{\partial^2 \hat{u}}{\partial y^2}\right),\\
&\frac{\hat{v}-\check{v}}{\triangle t}+\frac{\hat{v}-v}{\tau}+\frac{\partial u\hat{v}}{\partial x}+\frac{\partial v\hat{v}}{\partial y}=-\frac{\partial \hat{p}}{\partial y}+\frac{1}{\mathrm{Re}}\left(\frac{\partial^2 \hat{v}}{\partial x^2}+\frac{\partial^2 \hat{v}}{\partial y^2}\right),\\
&\frac{\partial \hat{u}}{\partial x}+\frac{\partial \hat{v}}{\partial y}=0.
\end{aligned} \tag{18.2}$$

Далее на основе этих уравнений проведём рассуждения, аналогичные приведённым в п. 17.1.2.1. Уравнения для пробной скорости типа (17.9), (17.10) примут вид

$$\begin{aligned}
&\left(1+\frac{\tau}{\triangle t}\right)u^*+\tau\frac{\partial uu^*}{\partial x}+\tau\frac{\partial vu^*}{\partial y}-\frac{\tau}{\mathrm{Re}}\left(\frac{\partial^2 u^*}{\partial x^2}+\frac{\partial^2 u^*}{\partial y^2}\right)=-\tau\frac{\partial p}{\partial x}+u+\frac{\tau}{\triangle t}\check{u},\\
&\left(1+\frac{\tau}{\triangle t}\right)v^*+\tau\frac{\partial uv^*}{\partial x}+\tau\frac{\partial vv^*}{\partial y}-\frac{\tau}{\mathrm{Re}}\left(\frac{\partial^2 v^*}{\partial x^2}+\frac{\partial^2 v^*}{\partial y^2}\right)=-\tau\frac{\partial p}{\partial y}+v+\frac{\tau}{\triangle t}\check{v}.
\end{aligned} \tag{18.3}$$

Уравнения для поправок скорости (17.12), (17.13) и давления (17.14) можно оставить в неизменном виде если модифицировать входящие в них множители d^u, d^v. По аналогии с (17.29), (17.30) запишем

$$\begin{aligned}
d^u&=\left(\mathrm{diag}\left(S^u\right)\right)^{-1}=\left(1+\frac{\tau}{\triangle t}+\frac{2\tau}{\mathrm{Re}}\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right)\right)^{-1}\\
d^v&=\left(\mathrm{diag}\left(S^v\right)\right)^{-1}=\left(1+\frac{\tau}{\triangle t}+\frac{2\tau}{\mathrm{Re}}\left(\frac{1}{h_x^2}+\frac{1}{h_y^2}\right)\right)^{-1}.
\end{aligned} \tag{18.4}$$

Здесь S^u, S^v – матрицы левых частей выражений (18.3).

Схема расчёта на временно́м слое остаётся аналогичной стационарному случаю, с той разницей, что первая итерация использует значение расчётных полей с предыдущего шага по времени. Порядок действий на временном слое:

1. Присвоить $u = \check{u}$, $v = \check{v}$, $p = \check{p}$;
2. Из уравнений (18.3) вычислить значения u^*, v^*;
3. Определить поправку давления p' из уравнения (17.14) с использованием (18.4);
4. Найти поправки скорости u', v' из выражений (17.12), (17.13) с использованием (18.4);
5. Выразить значения переменных для текущего слоя из (17.8); Для определения давления использовать сглаживание с коэффициентом α_p;
6. Найти невязку с ипользованием найденных значений $\hat{u}, \hat{v}, \hat{p}$ из выражения (17.7). Если она недостаточно мала, то выполняется присваивание $u = \hat{u},\ v = \hat{v},\ p = \hat{p}$ и возвращение на шаг 2. Если сходимость достигнута, то перейти на следующий шаг по времени. Для этого выполнить $\check{u} = \hat{u},\ \check{v} = \hat{v},\ \check{p} = \hat{p}$ и перейти на шаг 1.

18.2 Задача об обтекании препятствия

18.2.1 Расчётная сетка

Рассмотрим постановку граничных условий и особенности пространственной аппроксимации для задачи о внешнем обтекании. Поскольку рассматриваемые нами методы пока ограничены аппроксимациями на структурированной прямоугольной сетке, то будем рассматривать такую область расчёта, которую легко можно отобразить на такой сетке. Пусть внешняя область расчёта и обтекаемое препятствие представляет из себя прямоугольники.

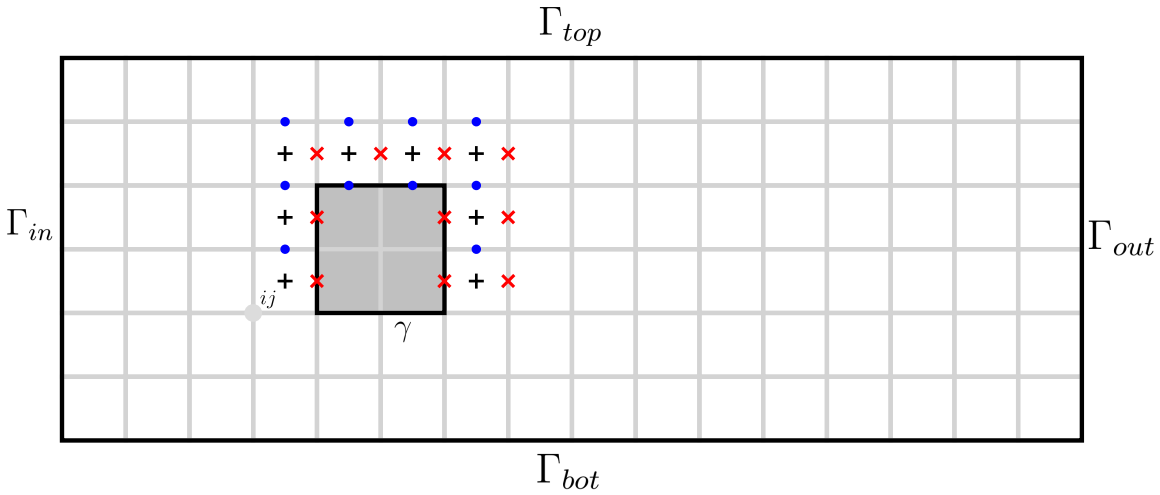


Рис. 15: Область расчёта и разнесённая сетка для задачи обтекания

По аналогии с рис. 12 введём в этой области прямоугольную сетку (рис. 15).

В случае сохранения естестественной нумерации узлов и ячеек сетки часть их этих пронумерованных ячеек выпадает из области расчёта (попадает внутрь препятствия). Для таких случаев существует два способа работы с нумерацией:

- Можно сохранить естественную нумерацию, при этом часть ячеек пометить как неактивные (например, введя специальный массив признаков `actnum`, i-ый элемент которого равен единице для активной ячейки и нулю для неактивной). Количество элементов в сеточных векторах тогда будет равно общему количеству всех ячеек (и активных и неактивных). Но значения этих векторов в неактивных ячейках будут фиктивными (нулями).

- Можно нумеровать лишь активные ячейки (и узлы), тем самым нарушив естественную нумерацию.

Оба этих подхода имеют свои очевидные плюсы и минусы. Первый подход сохраняет простые зависимости для перевода двумерного индекса в сквозной и диагональную структуру сеточных матриц. Второй подход более экономичен в хранении данных.

18.2.2 Граничные условия

Рассмотрим постановку со следующими граничными условиями:

- во входном сечении зададим равномерный профиль скорости

$$(x, y) \in \Gamma_{in} : u = 1,\ v = 0; \tag{18.5}$$

- на нижней и верхней границах – условие симметрии (идеального скольжения). Это условие моделирует зеркальное отражение расчётной области относительно соответствующих границ $\Gamma_{top}, \Gamma_{bot}$.

$$(x, y) \in \Gamma_{top}, \Gamma_{bot} : \frac{\partial u}{\partial n} = 0,\ v = 0; \tag{18.6}$$

- на самом обтекаемом деле – условия прилипания

$$(x, y) \in \gamma : u = 0,\ v = 0; \tag{18.7}$$

- в выходном сечении – условия выхода потока. Их точную формулировку определим позднее.

На каждом шаге алогоритма SIMPLE требуется решить три дифференциальных уравнения (17.9), (17.10), (17.14). относительно неизвестных u^*, v^*, p'. Значит из представленных выше граничных условий требуется выразить граничные значения для этих трёх неизвестных сеточных векторов и расписать способ их учёта при сборке соответствующих систем линейных уравнений.

18.2.2.1 Входное сечение

При разложении скорости на пробное значение и поправку (17.8) условия для скорости (18.5) раскладываются следующим образом:

$$(x, y) \in \Gamma_{in} : u^* = 1,\ v^* = 0,\ u' = v' = 0 \tag{18.8}$$

Условия первого рода для пробной скорости учитываются при решении уравнений (17.9), (17.10). Для сеточного вектора u^*, узлы которого лежат непосредственно на границе, учёт этого условия сводится к модификации соответствующей строки матрицы A^u и правой части b^u. В строке $k = k\left[0, j + \frac{1}{2}\right]$:

$$A^u_{km} = \delta_{km}, \quad b^u_k = 1. \tag{18.9}$$

Для сеточного вектора v^* учёт производится с помощью выражения для значения в фиктивном узле $k_1 = k\left[-\frac{1}{2}, j\right]$ через значение в настоящем узле $k_0 = k\left[\frac{1}{2}, j\right]$:

$$\frac{v^*_{k_0} + v^*_{k_1}}{2} = 0 \quad \Rightarrow \quad v^*_{k_1} = -v^*_{k_0}.$$

Поэтому при сборке матрицы A^v по формулам (17.23) при необходимости добавить значение a в колонку, соответствующую фиктивному узлу, требуется добавить это значение в диагональ с обратным знаком:

$$A^v_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^v_{k_0,k_0} \mathrel{-}= a \tag{18.10}$$

Из условий на поправку скорости $u' = v' = 0$ и уравнений (17.12), (17.13) следует граничное условие для поправки давления

$$x, y \in \Gamma_{in} : \frac{\partial p'}{\partial x} = 0 \tag{18.11}$$

Из этого условия получаем соотношение для давления в фиктивном узле $k_1 = k\left[-\frac{1}{2}, j + \frac{1}{2}\right]$ через значение в реальном узле $k_0 = k\left[\frac{1}{2}, j + \frac{1}{2}\right]$:

$$p'_{k_1} = p'_{k_0}$$

Тогда добавление значения a в фиктивную колонку k_1 эквивалентно

$$A^p_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^p_{k_0,k_0} \mathrel{+}= a \tag{18.12}$$

Следует понимать, что выражение (17.12) является приближением, используемым в расчётной схеме SIMPLE. В действительности, использование условия (18.11) (в случае нулевого начального приближения давления) приводит к нулевой производной для всего давления (а не только поправки)

$$x, y \in \Gamma_{in} : \frac{\partial p}{\partial x} = 0.$$

Это выражение никак не следует из постановки задачи. Действительно, если расписать уравнение (17.1) с учётом условий (18.5) и уравнения неразрывности (17.3), то получим соотношение

$$x, y \in \Gamma_{in} : \frac{\partial p}{\partial x} = \frac{1}{\mathrm{Re}} \frac{\partial^2 u}{\partial x^2} = -\frac{1}{\mathrm{Re}} \frac{\partial}{\partial x} \left(\frac{\partial v}{\partial y} \right). \tag{18.13}$$

(при выводы учтено, что $\partial v / \partial y = - \partial u / \partial x = 0$). Однако, практика показывает, что в большинстве случаев, условий типа (18.11) оказывается достаточно. Выражение (18.13) равно нулю, если поперечная компонента скорости не появляется сразу за входным сечением. То есть течение остается прямолинейным на начальном участке расчётной области. Чтобы это исполнялось, входное сечение

необходимо размещать на таком растоянии от препятствия, на котором поток еще не чувствует его присутствия (не начинает разворачиваться).

18.2.2.2 Условия симметрии

Однородные условие для скорости (18.6) расписываются как

$$(x,y) \in \Gamma_{in} : \frac{\partial u^*}{\partial y} = \frac{\partial u'}{\partial y} = 0,\ v^* = v' = 0.$$

Из условия на u^* запишем соотношение для фиктивного узла около нижней границы, которое будем использовать при сборке матрицы A^u:

$$\begin{aligned} &k_0 = k\left[i+\tfrac{1}{2},\tfrac{1}{2}\right], \quad k_1 = k\left[i+\tfrac{1}{2},-\tfrac{1}{2}\right], \\ &u^*_{k_1} = u^*_{k_0}, \\ &A^u_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^u_{k_0,k_0} \mathrel{+}= a. \end{aligned}$$

Условие на v^* можно использовать явно:

$$\begin{aligned} &k = k\left[i+\tfrac{1}{2},\tfrac{1}{2}\right], \\ &A^u_{k,s} = \delta_{ks}, \quad b^u_k = 0. \end{aligned}$$

Граничное условие для поправки давления можно получить из уравнения движения (17.2) (в неконсервативном виде) с учётом уравнения неразрывности. Используя

$$\begin{aligned} (x,y) \in \Gamma_{top,bot} : v = 0 \quad &\Rightarrow \quad \frac{\partial v}{\partial x} = 0 \quad \Rightarrow \quad \frac{\partial^2 v}{\partial x^2} = 0, \\ : \frac{\partial u}{\partial y} = 0 \quad &\Rightarrow \quad \frac{\partial}{\partial x}\frac{\partial u}{\partial y} = 0 \quad \Rightarrow \quad \frac{\partial^2 v}{\partial y^2} = 0, \end{aligned}$$

получим

$$(x,y) \in \Gamma_{top,bot} : \frac{\partial p}{\partial y} = 0.$$

При использовании нулевого начального приближения давления, для поправки давления так же справедливо

$$(x,y) \in \Gamma_{top,bot} : \frac{\partial p'}{\partial y} = 0.$$

Учёт этого условия на матричном уровне аналогичен процедуре (18.12).

18.2.2.3 Условия прилипания

Учёт условий прилипания на границе обтекаемого тела (18.7) в целом аналогичен алгоритму учёта входной границы. Для компонент скорости, узлы которых лежат на границе работает процедура (18.9) (с нулём в правой части). В случае если узлы не лежат на границе, то используется процедура (18.10).

Для поправки давления так же используется однородное условие второго рода (18.11) И все комментарии к этому условию, указанные в пункте 18.2.2.1, остаются справедливыми.

18.2.2.4 Выходные граничные условия

На выходной границе отсутствует возможность указать какие-либо физичные условия для искомых переменных. При этом, как правило, поведение течения в этой области большого интереса не представляет. Поэтому здесь требуется написать такие выражения, учёт которых не оказывал бы влияния на течение в основной области расчёта. Отсюда возникает проблема формулировки неотражающих граничных условий. Цель состоит в том, чтобы жидкость выходила из области расчёта естественным для себя образом, не подстраиваясь под выходную границу.

Простейшим решением этой проблемы является использование уравнения переноса на выходной границе:

$$\begin{gathered}(x,y) \in \Gamma_{out} : \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0, \\ \frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} = 0.\end{gathered} \tag{18.14}$$

В стационарном случае ($\partial u, v/\partial t = 0$) из этих условий следует, что поперечная скорость равна нулю:

$$\frac{\partial u}{\partial x} = 0 \quad \Rightarrow \quad \frac{\partial v}{\partial y} = 0 \quad \Rightarrow \quad v = v|_{\Gamma_{bot}} = 0.$$

По аналогии с уравнениями движения (17.4) в уравнение на выходной границе так же добавим фиктивную производную по времени. Тогда условия для компонент скорости на итерации SIMPLE примут вид (для стационарного случая)

$$\begin{gathered}(x,y) \in \Gamma_{out} : \frac{\hat{u} - u}{\tau} + u\frac{\partial \hat{u}}{\partial x} = 0, \\ \hat{v} = 0.\end{gathered} \tag{18.15}$$

Подставим разложение (17.8) и запишем условия для уравнений пробной скорости

$$(x,y) \in \Gamma_{out} : u^* + \tau u\frac{\partial u^*}{\partial x} = u, \tag{18.16}$$

$$v^* = 0. \tag{18.17}$$

Пробная скорость в алгоритме SIMPLE не удовлетворяет уравнению неразрывности. Поэтому нет гарантий, что найденная u^* сохраняет баланс масса в расчётной области. На практике это означает, что количество жидкости, которое втекает через Γ_{in} не равно количеству жидкости, которое вытекает через Γ_{out}.

Но финальная по итогам SIMPLE итерации скорость должна сохранять баланс массы. То есть

$$\triangle Q = \int_{\Gamma_{in}} \hat{u}\, ds - \int_{\Gamma_{out}} \hat{u}\, ds = 0.$$

Раскладывая это выражение через пробную скорость и поправку с учётом нулевого значения u' на входной границе (18.8), получим

$$\int_{\Gamma_{out}} u'\,ds = \int_{\Gamma_{in}} u^*\,ds - \int_{\Gamma_{out}} u^*\,ds$$

Положим, что u' на выходной границе постоянна. Это предположение не влияет на итоговый результат SIMPLE итераций, так как при его сходимости поправки скорости обнуляются. Тогда запишем значение поправки скорости на выходной границе

$$(x,y) \in \Gamma_{out}:\ u' = \left(\int_{\Gamma_{in}} u^*\,ds - \int_{\Gamma_{out}} u^*\,ds\right) / |\Gamma_{out}| \tag{18.18}$$

Подставляя это выражение в (17.12), получим граничные условия на поправку давления

$$(x,y) \in \Gamma_{out}:\ d^u \frac{\partial p'}{\partial x} = -\frac{u'}{\tau} \tag{18.19}$$

Таким образом, мы вывели граничные условия для всех трёх дифференциальных уравнений: (18.16), (18.17), (18.19).

Отметим, что если выходных границ несколько, то выражение (18.18) следует записывать для каждой из границ. При этом следует дополнительно задавать долю расхода C_i, вытекающую через каждую из границ. Пусть $\Gamma_{out} = \Gamma_{o1} \cap \Gamma_{o2}$. Тогда

$$(x,y) \in \Gamma_{o1}:\ u' = \left(C_1 \int_{\Gamma_{in}} u^*\,ds - \int_{\Gamma_{o2}} u^*\,ds\right) / |\Gamma_{o1}|$$
$$(x,y) \in \Gamma_{o2}:\ u' = \left(C_2 \int_{\Gamma_{in}} u^*\,ds - \int_{\Gamma_{o1}} u^*\,ds\right) / |\Gamma_{o2}|$$
$$C_1 + C_2 = 1.$$

Учёт условия для u^* (18.16) Просто перепишем уравнение в строках СЛАУ, соответствующих выходным узлам $k_0 = k\left[n_x, j + \frac{1}{2}\right]$. Для этого аппроксимируем конвективную производную по схеме против потока (с противопоточным узлом $k_1 = k\left[n_x - 1, j + \frac{1}{2}\right]$).

$$u^*_{k_0} + \tau U_{k_0} \frac{u^*_{k_0} - u^*_{k_1}}{h_x} = u_{k_0},$$

где U - скорость переноса в k_0-ом узле. Она должна быть всегда больше нуля (иначе схема перестаёт быть противопотоковой). Можно просто положить её равной среднерасходной (единице в нашем случае). А можно взять из предыдущей итерации с проверкой на положительность:

$$U_{k_0} = \max\left(0, u_{k_0}\right).$$

На матричном уровне получим:

$$A^u_{k0,s} = \begin{cases} 1 + \dfrac{\tau U_{k_0}}{h_x}, & s = k_0 \\ -\dfrac{\tau U_{k_0}}{h_x}, & s = k_1 \\ 0, & \text{иначе,} \end{cases}, \qquad b^u_{k_0} = u_{k_0} \tag{18.20}$$

Учёт условия для v^* (18.17) будем осуществлять за счёт введения фиктивного узла:

$$k_0 = k\left[n_x - \tfrac{1}{2}, j\right],\ k_1 = k\left[n_x + \tfrac{1}{2}, j\right],$$
$$\frac{v^*_{k_0} + v^*_{k_1}}{2} = v^*_{\Gamma_{out}} = 0 \quad \Rightarrow \quad v^*_{k_1} = -v^*_{k_0}.$$

Отсюда добавление элемента a в фиктивную колонку будет осуществляться в виде

$$A^v_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^v_{k_0,k_0} \mathrel{-}= a.$$

Учёт условия для p' (18.19) Также введём фиктивный узел k_1 и расположенные левее от него реальный узел k_0:

$$k_0 = k\left[n_x - \tfrac{1}{2}, j + \tfrac{1}{2}\right],\ k_1 = k\left[n_x + \tfrac{1}{2}, j + \tfrac{1}{2}\right],$$

Из (18.19)

$$d^u \frac{p'_{k_1} - p'_{k_0}}{h_x} = -\frac{u'}{\tau} \quad \Rightarrow \quad p'_{k_1} = p'_{k_0} - \frac{h_x}{\tau d^u} u'$$

На матричном уровне добавление фиктивной колонки даёт

$$A^v_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^v_{k_0,k_0} \mathrel{+}= a,\ b^v_{k_0} \mathrel{+}= a\frac{h_x u'}{\tau d^u}. \tag{18.21}$$

18.2.3 Баланс сил. Коэффициенты сил

18.2.3.1 Сопротивление

Проинтегрируем уравнение движения (17.1) по области расчёта D:

$$\int\limits_D \frac{\partial u^2}{\partial x}\, d\mathbf{x} + \int\limits_D \frac{\partial uv}{\partial y}\, d\mathbf{x} = -\int\limits_D \frac{\partial p}{\partial x}\, d\mathbf{x} + \frac{1}{\mathrm{Re}} \int\limits_D \nabla^2 u\, d\mathbf{x}.$$

Интегрирование по частям даёт:

$$\int\limits_D \frac{\partial f}{\partial x}\, d\mathbf{x} = \int\limits_{\Gamma_{out}} f\, ds - \int\limits_{\Gamma_{in}} f\, ds + \int\limits_{\gamma} f n_x\, ds$$

$$\int\limits_D \frac{\partial f}{\partial y}\, d\mathbf{x} = \int\limits_{\Gamma_{top}} f\, ds - \int\limits_{\Gamma_{bot}} f\, ds + \int\limits_{\gamma} f n_y\, ds,$$

$$\int\limits_D \nabla^2 f\, d\mathbf{x} = \int\limits_{\Gamma} \frac{\partial f}{\partial n}\, ds + \int\limits_{\gamma} \frac{\partial f}{\partial n}\, ds, \quad \Gamma = \Gamma_{in} \cap \Gamma_{out} \cap \Gamma_{bot} \cap \Gamma_{top}$$

Учтём, что на обтекаемом теле скорости равны нулю, а верхняя и нижняя границы непротекаемы. Тогда

$$\int\limits_{\Gamma_{in}} \left(u^2 + p\right)\, ds - \int\limits_{\Gamma_{out}} \left(u^2 + p\right)\, ds = \int\limits_{\gamma} p\, n_x\, ds - \frac{1}{\mathrm{Re}} \int\limits_{\gamma} \frac{\partial u}{\partial n}\, ds$$

Полученное выражение есть баланс сил в x направлении. Слева стоит сила, обусловленная перепадом динамического давления (если считать профили скорости на входе и на выходе примерно одинаковыми, то останется только перепад статического давления). А справа - силы сопротивления потоку вследствии наличия препятствия. И эти силы уравнавешивают друг друга. Первое слагаемое в правой части – есть сопротивление формы, второе – сопротивление трения из-за эффектов вязкости.

Коэффициенты этих сил имеют следующее выражение:

$$\begin{aligned} C_x^p &= 2 \int\limits_{\gamma} p\, n_x\, ds && \text{– коэффициент сопротивления формы} \\ C_x^f &= -\frac{2}{\mathrm{Re}} \int\limits_{\gamma} \frac{\partial u}{\partial n}\, ds && \text{– коэффициент сопротивления трения} \\ C_x &= C_x^p + C_x^f && \text{– коэффициент сопротивления} \end{aligned} \tag{18.22}$$

Чтобы из этих безразмерных коэффициентов получить реальные силы, измеряемые в Ньютонах, нужно умножить их на $\frac{1}{2}\rho U^2 L^2$.

18.2.3.2 Подъёмная сила

Аналогично проинтегрируем уравнение движение в направлении y (17.2). С учётом граничных условий получим выражение для баланса сил в поперечном направлении

$$\int\limits_{\Gamma_{bot}} p\, ds - \int\limits_{\Gamma_{top}} p\, ds = \int\limits_{\gamma} p\, n_y\, ds - \frac{1}{\mathrm{Re}} \int\limits_{\gamma} \frac{\partial v}{\partial n}\, ds$$

и соответствующие коэффициенты

$$\begin{aligned} C_y^p &= 2\int_\gamma p n_y \, ds \\ C_y^f &= -\frac{2}{\mathrm{Re}}\int_\gamma \frac{\partial v}{\partial n}\, ds \\ C_y &= C_y^p + C_y^f \qquad \text{– коэффициент подъёмной силы} \end{aligned} \tag{18.23}$$

18.2.3.3 Вычисление коэффициентов сил на разнесённой сетке

Вычисления коэффициентов C_x, C_y по формулам (18.22), (18.23) сводятся к интегрированию давления и производных скорости по поверхности обтекаемого тела. Само вычисление интеграла происходит простым суммированием:

$$\int_\gamma f\, ds \approx \sum_i f_i |\gamma_i|, \tag{18.24}$$

где γ_i – отрезок границы поверхности γ, а f_i – значение функции в центре этого отрезка. В случае равномерной сетки $|\gamma_i| = h_x, h_y$ для горизонтальных и вертикальных отрезков соответственно. Задача сводится к определению значению функций $p,\ \partial u, v/\partial n$ в центрах отрезков.

Горизонтальная граница Зафиксируем узел $i+\frac{1}{2}, j$ на такой границе. На этой границе n_x равна нулю, и вклад в C_x^p он не даёт. Для вычисления вклада в C_x^f нужно вычислить $\partial u/\partial y$. Для верхней границе запишем:

$$\begin{aligned} u_{i+\frac{1}{2},j} &= 0 \quad \text{из граничных условий прилипания,} \\ u_{i+\frac{1}{2},j+\frac{1}{2}} &= \frac{u_{i,j+\frac{1}{2}} + u_{i+1,j+\frac{1}{2}}}{2} \end{aligned}$$

отсюда

$$\frac{\partial u}{\partial n} = -\frac{\partial u}{\partial y} = \frac{u_{i+\frac{1}{2},j} - u_{i+\frac{1}{2},j+\frac{1}{2}}}{h_y/2} = -\frac{u_{i,j+\frac{1}{2}} + u_{i+1,j+\frac{1}{2}}}{h_y} \tag{18.25}$$

Аналогично для нижней границы

$$\frac{\partial u}{\partial n} = \frac{\partial u}{\partial y} = \frac{u_{i+\frac{1}{2},j} - u_{i+\frac{1}{2},j-\frac{1}{2}}}{h_y/2} = -\frac{u_{i,j-\frac{1}{2}} + u_{i+1,j-\frac{1}{2}}}{h_y} \tag{18.26}$$

Для вычисления вклада в C_y^p необходимо определить давление в $i+\frac{1}{2}, j$. На границе γ мы использовали условие $\partial p/\partial n = 0$ (см п. 18.2.2.3). Отсюда на верхней границе:

$$\frac{\partial p}{\partial n} = \frac{p_{i+\frac{1}{2},j} - p_{i+\frac{1}{2},j+\frac{1}{2}}}{h_y} = 0 \quad \Rightarrow \quad p_{i+\frac{1}{2},j} = p_{i+\frac{1}{2},j+\frac{1}{2}}.$$

Тогда подинтегральное выражение равно

$$p\, n_y = -p_{i+\frac{1}{2},j+\frac{1}{2}} \tag{18.27}$$

Аналогично на нижей границе получим:

$$p\,n_y = p_{i+\frac{1}{2},j-\frac{1}{2}} \tag{18.28}$$

Вклад горизонтальной границы в коэффициент C_y^f вычисляется через значение $\partial v/\partial y$, которое равно нулю из-за условий прилипания.

Вертикальная граница Теперь зафиксируем узел $i, j+\frac{1}{2}$ на вертикальной границе. Вклад этой границы в коэффициенты C_y^p, C_x^f равны нулю: первого из-за значения n_y, а второго из-за $\partial u/\partial x = -\partial v/\partial y = 0$.

Для коэффициента C_x^p напишем:

$$\begin{aligned} p\,n_x &= p_{i-\frac{1}{2},j+\frac{1}{2}} \quad \text{– левая граница,} \\ p\,n_x &= -p_{i+\frac{1}{2},j+\frac{1}{2}} \quad \text{– правая граница.} \end{aligned} \tag{18.29}$$

Для C_y^f:

$$\begin{aligned} \frac{\partial v}{\partial n} &= -\frac{v_{i-\frac{1}{2},j} + v_{i-\frac{1}{2},j+1}}{h_x} \quad \text{– левая граница,} \\ \frac{\partial v}{\partial n} &= -\frac{v_{i+\frac{1}{2},j} + v_{i+\frac{1}{2},j+1}}{h_x} \quad \text{– правая граница.} \end{aligned} \tag{18.30}$$

18.3 Тестовые примеры

18.3.1 Задача о равномерном течении

Рассмотрим задачу о стационарном прямолинейном течении с граничными условиями

$$\begin{aligned} (x,y) \in \Gamma_{in}: &\quad u = 1, v = 0, \\ (x,y) \in \Gamma_{top,bot}: &\quad \frac{\partial u}{\partial n} = 0, v = 0, \\ (x,y) \in \Gamma_{out}: &\quad u\frac{\partial u}{\partial x} = 0, v = 0. \end{aligned}$$

Очевидно, что точным решением этой задачи будут $u = 1, v = 0, p = 0$. В случае использования алгоритма инициализации (п. **??**) мы бы сразу получили этот ответ. Но здесь в качестве теста будем начинать итерации из состояния покоя $u = v = p = 0$.

Задача решается в области $[0,2] \times [-\frac{1}{2}, \frac{1}{2}]$ с использованием алгоритма SIMPLEC с $E = 4$ и разбиением на единицу длины $n_{un} = 20$.

Программа реализована в тесте `linear2-simple` в файле `linear_simple_test.cpp`.

Программа по расчёту этой задачи отличается от рассмотренной ранее задачи в каверне (п. **??**) только наличием условий входного и выходного сечений.

Шаг алгоритма SIMPLE В функции `step()`, описывающей основной шаг алгоритма SIMPLE, добавлено вычисление граничных значений

поправки скорости u' из (18.18) необходимых для соблюдения баланса массы (`compute_u_prime_outflow`).

```
116 double Linear2DSimpleWorker::step(){
117   // Predictor step: U-star
118   std::vector<double> u_star = compute_u_star();
119   std::vector<double> v_star = compute_v_star();
120   std::vector<double> u_prime_outflow = compute_u_prime_outflow(u_star);
121   // Pressure correction
122   std::vector<double> p_prime = compute_p_prime(u_star, v_star, u_prime_outflow);
123   // Velocity correction
124   std::vector<double> u_prime = compute_u_prime(p_prime, u_prime_outflow);
125   std::vector<double> v_prime = compute_v_prime(p_prime);
126   // Set final values
127   std::vector<double> u_new = vector_sum(u_star, 1.0, u_prime);
128   std::vector<double> v_new = vector_sum(v_star, 1.0, v_prime);
129   std::vector<double> p_new = vector_sum(_p, _alpha_p, p_prime);
130 
131   return set_uvp(u_new, v_new, p_new);
132 }
```

В дальнейшем эти условия используются для расчёта поправки давления и для расчёта самой поправки скорости.

18.3.1.1 Учёт граничных условий

Вычисление поправки скорости на выходной границе Функция `compute_u_prime_outflow`, реализующая вычисление формулы (18.18), имеет вид

```
378 std::vector<double> Linear2DSimpleWorker::compute_u_prime_outflow(const
    ↪  std::vector<double>& u_star) const{
379   double qin = 0;
380   double qout = 0;
381   for (size_t j=0; j<_grid.ny(); ++j){
382     size_t ind_left = _grid.yface_grid_index_i_jp(0, j);
383     size_t ind_right = _grid.yface_grid_index_i_jp(_grid.nx(), j);
384     qin += u_star[ind_left]*_hy;
385     qout += u_star[ind_right]*_hy;
386   }
387   double Lout = _grid.Ly();
388   double diff_u = (qin - qout)/Lout;
389   std::vector<double> ret(_grid.ny(), diff_u);
390   return ret;
391 }
```

. Здесь в цикле по вертикальным граням вычисляются расходы по входному и выходному сечениям (`qin`, `qout`), далее находится поправка скорости (`diff_u`), постоянная для всех выходных отрезков, и возвращается вектор, содержащий эту поправку для всех выходных отрезков.

Учёт граниных условий для u^* Для учёта граничных условий входа и выхода при сборке уравнения для u^* в функции `assemble_u_slae` используется цикл

```
222  for (size_t j=0; j< _grid.ny(); ++j){
223    // left boundary: u = 1
224    {
225      size_t index_left = _grid.yface_grid_index_i_jp(0, j);
226      mat.set_value(index_left, index_left, 1.0);
227      _rhs_u[index_left] = 1.0;
228    }
229    // right boundary: u*du/dx = 0
230    {
231      size_t index_right = _grid.yface_grid_index_i_jp(_grid.nx(), j);
232      size_t index_right_minus = _grid.yface_grid_index_i_jp(_grid.nx()-1, j);
233      double u0 = std::max(0.0, _u[index_right]);
234      double coef = _tau*u0/_hx;
235      mat.set_value(index_right, index_right, 1.0 + coef);
236      mat.set_value(index_right, index_right_minus, -coef);
237      _rhs_u[index_right] = u0;
238    }
239  }
```

Здесь для левой границы согласно (18.9) жёстко устанавливается единичное значение. А для правой границы используются соотношения (18.20).

Учёт граничных условий для p' Найденные поправки скорости на выходной границе должны быть учтены при решении задачи для p' согласно (18.21) Сборка матрицы левой части при этом останется неизменной. Действительно, распишем производную на правой (выходной) границе

$$d^u \frac{\partial p'}{\partial x}_{n_x, j+\frac{1}{2}} \approx d^u \frac{p'_{k_1} - p'_{k_0}}{h_x}$$

входящую в выражение (17.24). Где $k_0 = k[n_x - \frac{1}{2}, j + \frac{1}{2}]$ – реальный, а $k_1 = k[n_x + \frac{1}{2}, j + \frac{1}{2}]$ – находящийся правее него фиктивный узлы сетки. Наличие этой производной требует добавления выражения d^u/h_x^2 в реальный (диагональный) столбец k_0 и выражения $-d^u/h_x^2$ в фиктивный столбец k_1 матрицы A^p в строке k_0. Следуя алгоритму (18.21) добавление значения в фиктивный столбец равносильно добавлению этого же значения в диагональный столбец. То есть два этих значения взаимоуничтожаться. Останется только модифицировать столбец правых членов. Альтернативно можно просто подставить значение производной (18.19) в дисретизованное выражение (17.24), и, поскольку оно не содержит в себе p', унести его в правую часть с обратным знаком и делением на h_x.

Учёт граничных условий в правой части осуществляется в функции `assemble_p_prime_solver` за счёт модификации правой части для узлов, расположенных около выходной границы:

```
367     // outflow compensation
368     if (i == _grid.nx()-1){
369       rhs[ind0] -= (u_prime_outflow[j]) / _tau / _hx;
370     }
```

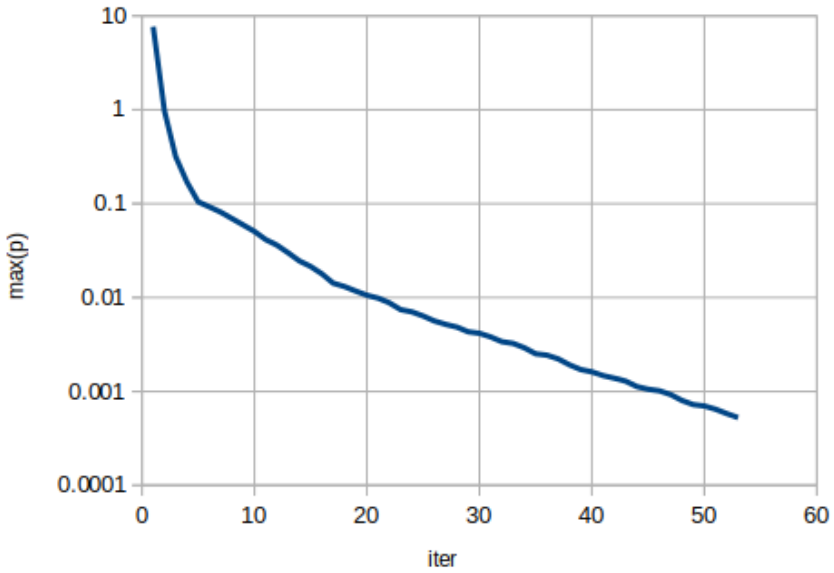
Учёт граничных условий для u' Явным образом предварительно найденные граничные значения для поправки скорости присваиваются в фукнции `"compute_u_prime"`:

```
402   // outflow
403   for (size_t j=0; j<_grid.ny(); ++j){
404     size_t ind0 = _grid.yface_grid_index_i_jp(_grid.nx(), j);
405     u_prime[ind0] = u_prime_outflow[j];
406   }
```

18.3.1.2 Анализ результатов

До невязки $\varepsilon = 10^{-2}$ задача сходится за 53 итерации. При этом для скорости точный ответ получается уже на первой итерации, а всё остальное время происходит подстройка давления.

Максимальное значение давление в зависимости от итерации приведено на графике ниже



18.3.2 Течение Пуазейля

TODO

18.3.3 Стационарное обтекание квадратного препятствия

В тесте `obstacle2-simple` из файла `obstacle_simple_test.cpp` рассматривается задача о стационарном

обтекании квадратного препятствия (см. постановку в п. 18.2). По окончании расчёта в консоль печатаются коэффициенты сопротивления и подъёмной силы.

Используется естественная нумерация узлов с неактивными ячейками. Класс `RegularGrid2D` предлагает следующие методы, связанные с неактивными ячейками:

- `void RegularGrid2D::deactivate_cells(Point bot_left, Point top_right)` – установить область неактивных ячеек;
- `bool RegularGrid2D::is_active_cell(size_t icell)` – проверить, является ли ячейка активной.

Кроме того, в задаче появились внутренние границы. То есть для постаноки граничных условий уже не достаточно использовать крайние значение индексов i, j, а нужен механизм для получения граничных отрезков сетки. Для этого введены следующие функции

- `RegularGrid2d::boundary_yfaces()` – получить список всех вертикальных граничных фасок (возвращает парные индексы в соответствии `yface_centered_grid`.
- `RegularGrid2d::boundary_xfaces()` – получить список всех горизонтальных граничных фасок (возвращает парные индексы в соответствии `xface_centered_grid`.
- `RegularGrid2d::yface_type(size_t yface_index)` – узнать тип вертикальной грани по её глобальному индексу. Возвращает перечисление

```
enum struct FaceType{
  Internal,     // внутренняя
  Boundary,     // граничная
  Deactivated   // неактивная (находится внутри неактивной области)
};
```

- `RegularGrid2d::xface_type(size_t xface_index)` – узнать тип вертикальной грани по её глобальному индексу.

18.3.3.1 Функция верхнего уровня

Здесь сначала происходит установка параметров расчёта: числа Рейнольдса, параметра E, количества итераций, порога сходимости и разбиения единичного интервала.

```
622  double Re = 20;
623  double E = 4.0;
624  size_t max_it = 10000;
625  double eps = 1e-1;
626  size_t n_unit = 10;  // partition per unit length
```

Далее строится сетка

```
629  RegularGrid2D grid(0, 12, -2, 2, 12*n_unit, 4*n_unit);
```

В этом примере сетка строится в четырёхугольнике $[0, 12] \times [-2, 2]$. Потом для описания квадратного препятствия происходит деактивация ячеек, находящихся в единичном квадрате с нижней левой координатой $(2, -0.5)$ и верхней правой координатой $(3, 0.5)$.

```
630 grid.deactivate_cells({2, -0.5}, {3, 0.5});
```

Потом создаётся решатель, инициализируются функции сохранения и вызывается алгоритм потенциальной инициализации расчётных полей.

```
631 Obstacle2DSimpleWorker worker(Re, grid, E);
632 worker.initialize_saver(false, "obstacle2");
633
634 // initial condition
635 worker.initialize();
```

Затем идёт стандартный цикл по SIMPLE-итерациям

```
638 size_t it = 0;
639 for (it=1; it < max_it; ++it){
640   double nrm = worker.step();
641
642   // print norm
643   std::cout << it << " " << nrm << std::endl;
644
645   // break if residual is low enough
646   if (nrm < eps){
647     break;
648   }
649 }
```

По окочании цикла вызывается функция сохранения решения в vtk

```
651 worker.save_current_fields(it);
```

В конце происходит расчёт коэффициентов сил и их печать в консоль

```
653 Obstacle2DSimpleWorker::Coefficients coefs = worker.coefficients();
654 std::cout << "=== Drag" << std::endl;
655 std::cout << "Cpx = " << coefs.cpx << std::endl;
656 std::cout << "Cfx = " << coefs.cfx << std::endl;
657 std::cout << "Cx  = " << coefs.cx  << std::endl;
```

Результирующее поле течения сохраняется в файл `obstacle2.vtk.series`.

18.3.3.2 Учёт неактивных ячеек

Неактивные ячейки учитываются во всех алгоритмах сборки систем линейных уравнений. Рассмотрим на примере сборки матрицы для пробной скорости, реализованной в функции `assemble_u_slae`.

```
230 void Obstacle2DSimpleWorker::assemble_u_slae(){
```

Рассмотрим цикл сборки внутренних узлов “красной” сетки для u (или, что тоже самое, цикл по всем вертикальным граням основной сетки)

```
275  for (size_t j=0; j < _grid.ny(); ++j)
276  for (size_t i=1; i < _grid.nx(); ++i){
```

Сначала вычисляется индекс строки (сквозной индекс текущей грани):

```
277     size_t row_index = _grid.yface_grid_index_i_jp(i, j);    //[i, j+1/2]
```

Эта грань может быть либо внутренней, либо граничной (принадлежать внутренней вертикальной границе), либо неактивной. Выполняется проверка, является ли эта грань внутренней

```
278     if (_grid.yface_type(row_index) == RegularGrid2D::FaceType::Internal){
```

Если да, то выполняется обычная процедура сборки

```
279       double u0_plus   = u_ip_jp(i, j);   // _u[i+1/2, j+1/2]
280       double u0_minus  = u_ip_jp(i-1, j); // _u[i-1/2, j+1/2]
281       double v0_plus   = v_i_j(i, j+1);   // _v[i,j+1]
282       double v0_minus  = v_i_j(i, j);     // _v[i,j]
283
284       // u_(i,j+1/2)
285       add_to_mat(row_index, {i, j}, 1.0);
286       //     + tau * d(u0*u)/ dx
287       add_to_mat(row_index, {i+1,j}, _tau/2.0/_hx*u0_plus);
288       add_to_mat(row_index, {i-1,j}, -_tau/2.0/_hx*u0_minus);
289       //     + tau * d(v0*u)/dy
290       add_to_mat(row_index, {i, j+1}, _tau/2.0/_hy*v0_plus);
291       add_to_mat(row_index, {i, j-1}, -_tau/2.0/_hy*v0_minus);
292       //     - tau / Re * d^2u/dx^2
293       add_to_mat(row_index, {i, j}, 2.0*_tau/_Re/_hx/_hx);
294       add_to_mat(row_index, {i+1, j}, -_tau/_Re/_hx/_hx);
295       add_to_mat(row_index, {i-1, j}, -_tau/_Re/_hy/_hy);
296       //     - tau / Re * d^2u/dy^2
297       add_to_mat(row_index, {i, j}, 2.0*_tau/_Re/_hy/_hy);
298       add_to_mat(row_index, {i, j+1}, -_tau/_Re/_hy/_hy);
299       add_to_mat(row_index, {i, j-1}, -_tau/_Re/_hy/_hy);
300       // = u0_(i,j+1/2)
301       _rhs_u[row_index] += _u[row_index];
302       //      - tau * dp/dx
303       _rhs_u[row_index] -= _tau/_hx*(p_ip_jp(i, j) - p_ip_jp(i-1, j));
```

Если нет (то есть грань либо неактивная, либо принадлежит внутренней границе), то в диагональ ставится единица, в правую часть 0.

```
304     } else {
305       mat.set_value(row_index, row_index, 1.0);
306       _rhs_u[row_index] = 0;
307     }
```

Это отражает тот факт, что на внутренних границах $u = 0$ из-за условий прилипания, а для неактивных мы пишем тривиальное уравнение, просто чтобы матрица не была вырождена.

Учёт условий прилипания на внутренних горизонтальных границах осуществляется через фиктивный узел в лямбда-функции `"add_to_mat"`, которая перехватывает все ситуации, когда алгоритм требует добавить что-либо в фиктивную колонку матрицы.

```
235   auto add_to_mat = [&](size_t row_index, std::array<size_t, 2> ij_col, double value){
```

Такие ситуации могут произойти либо при сборке около вертикальной грани, находящейся рядом с верхней границей:

```
236     if (ij_col[1] == _grid.ny()){
237       // ghost index => top boundary condition: du/dn = 0
238       size_t ind1 = _grid.yface_grid_index_i_jp(ij_col[0], ij_col[1]-1);
239       mat.add_value(row_index, ind1, value);
```

либо около вертикальной грани, находящейся рядом с нижней границей границей:

```
240     } else if (ij_col[1] == (size_t)-1){
241       // ghost index => bottom boundary condition: du/dn = 0
242       size_t ind1 = _grid.yface_grid_index_i_jp(ij_col[0], ij_col[1]+1);
243       mat.add_value(row_index, ind1, value);
```

либо около вертикальнй грани, находящейся непосредственно над или под препятствием. В этом случае индекс фиктивной колонки, в которую трубуется поставить будет соответствовать неакотвной вертикальной грани. Мы вычисляем этот индекс

```
244     } else {
245       size_t ind1 = _grid.yface_grid_index_i_jp(ij_col[0], ij_col[1]);
```

Если он неактивный, то следуем по процедуре добавления фиктивного узла около границы с нулевым значением.

```
246       if (_grid.yface_type(ind1) == RegularGrid2D::FaceType::Deactivated){
247         // ghost index => obstacle boundary u = 0
248         mat.add_value(row_index, row_index, -value);
249       } else {
```

Иначе – это нормальная колонка и мы добавляем туда значение по стандартной процедуре

```
250         mat.add_value(row_index, ind1, value);
```

18.3.3.3 Расчёт коэффициентов сопротивления

Расчёт коэффициентов сил по формулам (18.22) осуществляется в процедуре `coefficients()`. Она возвращает структуру, куда входят все шесть искомых значений

```
33  struct Coefficients{
34    double cpx;
35    double cfx;
36    double cx;
37  };
```

Процедура, объявленная как

```
565 Obstacle2DSimpleWorker::Coefficients Obstacle2DSimpleWorker::coefficients() const{
```

производит вычисления четырёх интегралов по простой квадратуре (18.24). Результаты аггрегируются в переменные

```
566   double sum_cpx = 0;
567   double sum_cfx = 0;
```

Операции проводятся в циклах по внутренним граничным отрезкам. Сначала рассматриваются вертикальные границы:

```
570   for (const RegularGrid2D::split_index_t& yface: _grid.boundary_yfaces()){
```

Здесь в переменную
`yface` попадают все парные индексы вертикальных граней, лежащих на границах. Сначала нужно отфильтровать границы, лежащие во входном и выходном сечениях

```
571     if (yface[0] == 0){
572       // input => ignore
573     } else if (yface[0] == _grid.nx()){
```

На вертикальных границах актуально вычисление коэффициентов C_x^p (из пункта 18.2.3.3). Дли их определения на каждой сеточной грани мы должны определить $p\,n_x$ по формулам (18.29).

```
576       double pnx;
```

Для использования этих формул нужно определить, является ли это левой или правой границей обтекаемого тела. Мы вычисляем индексы ячеек, лежащих слева и справа. Если левая ячейка активна, значит это левая граница, если правая активна, значит это правая граница.

```
577       size_t left_cell = _grid.cell_centered_grid_index_ip_jp(yface[0]-1, yface[1]);
578       size_t right_cell = _grid.cell_centered_grid_index_ip_jp(yface[0], yface[1]);
```

Далее левой границы

```
579       if (_grid.is_active_cell(left_cell)){
580         pnx = _p[left_cell];
```

для правой

```
581       } else if (_grid.is_active_cell(right_cell)){
582         pnx = -_p[right_cell];
```

иначе (если это и не правая и не левая граница) бросается исключение, потому что так быть не должно: у любой внутренней границы должна быть хоть одна соседняя активная ячейка

```
583       } else {
584         _THROW_UNREACHABLE_;
585       }
```

После вычисления $p\,n_x$ они добавляются в искомые интегралы согласно (18.24):

```
586       sum_cpx += pnx * _hy;
```

Далее аналогичная процедура проводится для горизонтальных граней, где актуально вычисление коэффициентов C_x^f. Дли их определения на каждой сеточной грани мы должны определить $\partial u/\partial n$ по формулам (18.25), (18.26).

```
591   for (const RegularGrid2D::split_index_t& xface: _grid.boundary_xfaces()){
592     if (xface[1] == 0){
593       // bottom => ignore
594     } else if (xface[1] == _grid.ny()){
595       // top => ignore
596     } else {
597       double dudn;
598       size_t bot_cell = _grid.cell_centered_grid_index_ip_jp(xface[0], xface[1]-1);
599       size_t top_cell = _grid.cell_centered_grid_index_ip_jp(xface[0], xface[1]);
600       if (_grid.is_active_cell(bot_cell)){
601         dudn = -u_ip_jp(xface[0], xface[1]-1)/(_hy/2.0);
602       } else if (_grid.is_active_cell(top_cell)){
603         dudn = -u_ip_jp(xface[0], xface[1])/(_hy/2.0);
604       } else {
605         _THROW_UNREACHABLE_;
606       }
607       sum_cfx += dudn * _hx;
608     }
609   }
```

В конце функции искомые коэффициенты вычисляются через уже найденные интегралы согласно (18.22):

```
611   Coefficients coefs;
612   coefs.cpx = 2.0*sum_cpx;
613   coefs.cfx = -2.0/_Re*sum_cfx;
614   coefs.cx = coefs.cpx + coefs.cfx;
615   return coefs;
```

18.3.3.4 Результаты расчёта

Картина течения, полученная для сетки
`n_part = 10` при Re = 20, представлена на рис. 16 Полученные коэффициенты сопротивления:

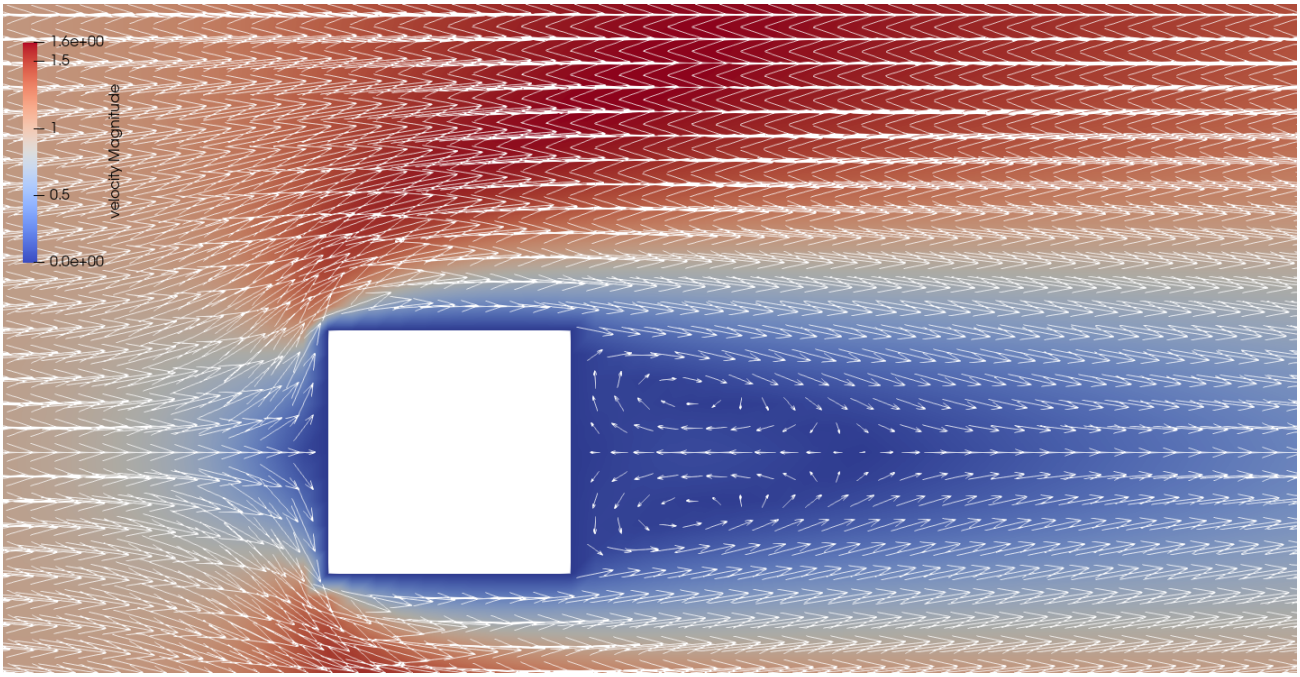


Рис. 16: Обтекание квадратного препятствия в стационарном режиме

```
=== Drag
Cpx = 2.97224
Cfx = 1.08639
Cx  = 4.05863
```

Для $\varepsilon = 10^{-1}$ решение сошлось за 29 итераций.

18.3.4 Нестационарное обтекание квадратного препятствия

Эта задача реализована в файле `obstacle_nonstat_simple_test.cpp` в тесте
`[obstacle2-nonstat-simple]`.

Программа решает задачу в той же области, которая рассматривалась в предыдущем пункте, но в нестационарной постановке (18.1)

Поля течения для разных моментов времени пишутся в файл
`obstacle-nonstat.vtk.series`. Кроме того, в файл
`c.txt` пишутся вычисленные на разные моменты времени коэффициенты сопротивления.

18.3.4.1 Функция верхнего уровня

В начале обозначим параметры задачи: числа Рейнольдса, разбиение единичного отрезка, шаг по времени $\triangle t$ и конечное время, параметр внутреннего итерационного процесса E, максимальное количество итераций во внутреннем итерационном процессе и порог по невязке:

```
676  double Re = 100;
677  size_t n_unit = 10;  // partition per unit length
678  double time_step = 0.25;
679  double end_time = 5;
680  double E = 4.0;
681  size_t max_it = 10000;
682  double eps = 1e-0;
```

Далее проводится создание сетки (так же, как и в предыдущем примере) и начальная инициализация решателя

```
685  RegularGrid2D grid(0, 12, -2, 2, 12*n_unit, 4*n_unit);
686  grid.deactivate_cells({2, -0.5}, {3, 0.5});
687  ObstacleNonstat2DSimpleWorker worker(Re, grid, E, time_step);
688  worker.initialize_saver(false, "obstacle2-nonstat");
689
690  // initial condition
691  worker.initialize();
692  worker.save_current_fields(0);
```

После всех инициализаций начинается цикл по времени

```
695  for (double time=time_step; time<end_time+1e-6; time+=time_step){
```

Отметим, что поскольку значению $t = 0$ соответствует начальное состояние решения, то цикл начинается сразу с первого шага $t = \triangle t$.

Внутри цикла по времени производится цикл внутренних итераций SIMPLE

```
696    size_t it = 0;
697    for (it=1; it < max_it; ++it){
698      double nrm = worker.step();
699
700      // break inner iterations if residual is low enough
701      if (nrm < eps){
702        break;
703      } else if (it == max_it -1) {
704        std::cout << "WARNING: internal SIMPLE interations not converged with nrm = "
705                  << nrm << std::endl;
706      }
707    }
```

Далее, если текущее время кратно 1.0, производится сохранение решения в файл vtk и запись коэффициентов сил в файл:

```
709    if (std::abs(time - round(time)) < 1e-6){
710      worker.save_current_fields(time);
711    }
```

Печатается информация о сходимости текущей итерации

```
712   std::cout << convergence_report(time, it);
```

и производится переход на следующий шаг по времени:

```
715   worker.to_next_time_step();
```

18.3.4.2 Учёт нестационарности

Согласно пункту 18.1 наличие производной по времени учитывается:

- При вычислении коэффициентов d^u, d^v (18.4):

```
107   _du = 1.0 / (1 + _tau/_time_step + 2.0*_tau/_Re * (1.0/_hx/_hx + 1.0/_hy/_hy));
108   _dv = 1.0 / (1 + _tau/_time_step + 2.0*_tau/_Re * (1.0/_hx/_hx + 1.0/_hy/_hy));
```

- При сборке систем уравнений для u^*, u^* (18.3) как прибавка к диагонали

```
316       add_to_mat(row_index, {i, j}, 1.0 + _tau/_time_step);
```

и правой части

```
334       _rhs_u[row_index] += (_tau/_time_step)*_u_old[row_index];
```

- А так же в граничных условиях на выходе. В этом случае условия (18.14) для u по аналогии с (18.15) аппроксимируются к виду

$$(x,y) \in \Gamma_{out}: \quad \frac{\hat{u}-\check{u}}{\triangle t} + \frac{\hat{u}-u}{\tau} + u\frac{\partial \hat{u}}{\partial x} = 0.$$

Для упрощения по прежнему будем использовать “стационарное” условие для поперечной скорости $v = 0$. Тогда для u^* можно записать

$$(x,y) \in \Gamma_{out}: \quad \left(1 + \frac{\tau}{\triangle t}\right) u^*_{n_x,j+\frac{1}{2}} + \tau U_{n_x,j+\frac{1}{2}} \frac{u^*_{n_x,j+\frac{1}{2}} - u^*_{n_x-1,j+\frac{1}{2}}}{h_x} = u_{n_x,j+\frac{1}{2}} + \frac{\tau}{\triangle t}\check{u}_{n_x,j+\frac{1}{2}}$$

Это выражение и добавляется в соответствующие строки матрицы и правой части

```
293   // right boundary: du/dt + u*du/dx = 0
294   {
295     size_t index_right = _grid.yface_grid_index_i_jp(_grid.nx(), j);
296     size_t index_right_prev = _grid.yface_grid_index_i_jp(_grid.nx()-1, j);
297     double u0 = std::max(0.0, _u[index_right]);
298     double coef = _tau*u0/_hx;
299     mat.set_value(index_right, index_right, 1.0 + _tau/_time_step + coef);
300     mat.set_value(index_right, index_right_prev, -coef);
301     _rhs_u[index_right] = u0 + _tau/_time_step * _u_old[index_right];
302   }
```

- При переходе на слудующий шаг по времени в функции происходит вычисление текущего значения температуры, и присваивание значений $\check{u}$, $\check{v}$.

```
147 double ObstacleNonstat2DSimpleWorker::to_next_time_step(){
148   _u_old = _u;
149   _v_old = _v;
150   return set_uvp(_u, _v, _p);
151 }
```

  Вызов `set_uvp` здесь осуществляется для пересборки актуальных матриц.

18.4 Задание для самостоятельной работы

18.4.1 Расчитать коэффициент подъёмной силы

Добавить расчёт коэффициентов подъёмной силы C_y в код для моделирования нестационарного обтекания (18.3.4). В п.(18.3.3.3) представлен используемый алгоритм расчёта коэффицента сопротивления. Отталкиваясь от этого алгоритма необходимо

- в класс `Coefficients` добавить поля `Cpy,Cfy,Cy`,
- добавить расчёт `dvdn` на вертикальных стенках препятствия по формулам (18.30),
- добавить расчёт `pny` на горизонтальных стенках препятствия по формулам (18.27), (18.28),
- аггрегировать эти значения в `sum_cpy,sum_cfy`, из которых вычислить коэффициенты подъёмной силы,
- сами коэффициенты добавить в файл `c.txt`.

18.4.2 Расчёт течения

Провести расчёт течения при параметрах $\mathrm{Re} = 100$, $\varepsilon = 10^{-1}$, $\triangle t = 0.1$, $t_{end} = 200$, $n = 10$. (для ускорения расчётов не забыть переключиться на релизную сборку).

- Показать динамику изменения расчётных полей: скалярных p, $|\mathbf{v}|$ и вектора скорости $\mathbf{v}$,
- Нарисовать графики изменения коэффициентов сопротивления C_x, C_x^p, C_x^f и подъёмной силы C_y, C_y^p, C_y^f во времени.

19 Лекция 19 (7.12)

19.1 Схема PISO

19.1.1 Итерации PISO

Рассмотрим модификацию схемы SIMPLE (п. 17.1.2.1), использующую несколько поправок скорости и давления. В качестве исходных уравнений используем полудискретизованную схему (17.4) – (17.6). По аналогии с (17.8) запишем искомое решение в виде пробной функции и корректоров

$$\begin{aligned} \hat{u} &= u^* + u' + u'' + u''' + \dots, \\ \hat{v} &= v^* + v' + v'' + v''' + \dots, \\ \hat{p} &= p + p' + p'' + p''' + \dots. \end{aligned} \tag{19.1}$$

Дополнительно положим, что все поправки скорости выше первой бездивергентны:

$$\nabla \cdot \mathbf{u}'' = 0, \quad \nabla \cdot \mathbf{u}''' = 0, \quad \dots \tag{19.2}$$

Тогда из требования бездивирегности финальной скорости $\hat{u}$ получим

$$\nabla \cdot (\mathbf{u}^* + \mathbf{u}' + \mathbf{u}'' + \dots) = 0 \quad \Rightarrow \quad \nabla \cdot \mathbf{u}' = -\nabla \cdot \mathbf{u}^* \tag{19.3}$$

Для удобство записи введём обозначения для линеаризованных операторов, действующих на функции скорости

$$\begin{aligned} S^u(\hat{u}) &= \hat{u} + \tau \frac{\partial u\hat{u}}{\partial x} + \tau \frac{\partial v\hat{u}}{\partial y} - \frac{\tau}{\mathrm{Re}} \left(\frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \right), \\ S^v(\hat{v}) &= \hat{v} + \tau \frac{\partial u\hat{v}}{\partial x} + \tau \frac{\partial v\hat{v}}{\partial y} - \frac{\tau}{\mathrm{Re}} \left(\frac{\partial^2 \hat{v}}{\partial x^2} + \frac{\partial^2 \hat{v}}{\partial y^2} \right). \end{aligned} \tag{19.4}$$

Разложим эти операторы как

$$S^u(u) = A^u(u) + H^u(u),$$

где A^u – оператор, который в дискретизованной форме имеет диагоноальную структуру, а H^u – оператор, имеющий в дискретизованном виде нулевую диагональ.

Далее подставим разложения (19.1) в определяющие уравнение (17.4), (17.5). Декомпозицию по-

лученного уравнения проведём следующим образом (на примере уравнения для u):

$$A^u(u^*) + H^u(u^*) = -\tau\frac{\partial p}{\partial x} + u \tag{19.5}$$

$$+ \; A^u(u') = -\tau\frac{\partial p'}{\partial x} \tag{19.6}$$

$$+ \; A^u(u'') + H^u(u') = -\tau\frac{\partial p''}{\partial x} \tag{19.7}$$

$$+ A^u(u''') + H^u(u'') = -\tau\frac{\partial p'''}{\partial x} \tag{19.8}$$

$$\dots$$

$$= \; A^u(\hat{u}) + H^u(\hat{u}) = -\tau\frac{\partial \hat{p}}{\partial x} + u$$

Уравнение (19.5) аналогично уравнению (17.9) метода SIMPLE. Оно используется для определения пробной скорости u^*. Уравнение для первой поправки скорости (19.6) (аналог (17.11)) совместно с уравнением неразрывности (19.3) используется для формулировки задачи Пуассона для p' (17.14):

$$-\nabla \cdot \overline{\mathbf{d}} \nabla p' = -\frac{1}{\tau}\nabla \cdot \mathbf{u}^*. \tag{19.9}$$

Здесь диагональный тензор $\overline{\mathbf{d}}$ равен

$$\overline{\mathbf{d}} = \begin{pmatrix} (A^u)^{-1} & 0 \\ 0 & (A^v)^{-1} \end{pmatrix}$$

Уравнения (19.7), (19.8) и последующие используются для формулировки задачи для p', p'' и т.д. Рассмотрим его получение на примере (19.7). Во-первых, воспользуемя диагональностью (то есть легкообратимостью) оператора A^u и выразим u'':

$$u'' = -\tau(A^u)^{-1}\frac{\partial p''}{\partial x} - (A^u)^{-1}H^u u'.$$

Аналогичное уравнение можно вывести и для вертикальной компоненты скорости:

$$v'' = -\tau(A^v)^{-1}\frac{\partial p''}{\partial y} - (A^v)^{-1}H^v v'.$$

Отсюда можно записать градиент от $\mathbf{u}''$:

$$\nabla \cdot \mathbf{u}'' = -\tau\nabla \cdot \overline{\mathbf{d}}\nabla p'' - \nabla \cdot \overline{\mathbf{d}}\,\overline{\mathbf{H}}\mathbf{u}', \qquad \overline{\mathbf{H}} = \begin{pmatrix} H^u & 0 \\ 0 & H^v \end{pmatrix}.$$

Этот градиент согласно (19.2) равен нулю. Похожих рассуждений будем придерживаться для вывода уравнений для p''' и далее. Итого, уравнения для второй и последующих поправок далвения давления

примут вид

$$-\nabla \cdot \overline{\mathbf{d}} \nabla p'' = \frac{1}{\tau} \nabla \cdot \overline{\mathbf{d}}\, \overline{\mathbf{H}} \mathbf{u}', \tag{19.10}$$
$$-\nabla \cdot \overline{\mathbf{d}} \nabla p''' = \frac{1}{\tau} \nabla \cdot \overline{\mathbf{d}}\, \overline{\mathbf{H}} \mathbf{u}'',$$
$$\dots$$

Последовательность шагов при решении задачи на временном слое примет вид

1. Даны значение $\mathbf{u}$, $\mathbf{v}$, p с прошлого слоя,
2. Собираем сеточные операторы $\overline{\mathbf{A}}$, $\overline{\mathbf{H}}$ как диагональную и внедиагональную часть линеаризованного оператора $\overline{\mathbf{S}}$ (19.4), и обратный к $\overline{\mathbf{A}}$ оператор $\overline{\mathbf{d}}$,
3. Из уравнения (19.5) и его аналога для вертикальной компоненты скорости находим u^*, v^*,
4. Из уравнения (19.9) находим первую поправку для давления p',
5. Из уравнения (19.6) и его аналога для вертикальной компоненты скорости находим u', v',
6. Из уравнения (19.10) находим второую поправку для давления p'',
7. Из уравнения (19.7) и его аналога для вертикальной компоненты скорости находим u'', v'',
8. Повторяем шаги 6, 7 для каждой поправки скорости и давления выше первой,
9. Записываем окончательное решение на слое используя (19.1).

Первые 5 шагов представленного алгоритма полностью соответствуют ранее рассмотренному методу SIMPLE. Шаги 6, 7 – есть дополнительные уточняющие PISO итерации. В отличии от SIMPLE-итераций, эти итерации не требуют пересборки операторов и дополнительной инициализации решателей систем линейных уравнений, поэтому они более эффективны с точки зрения скорости расчёта.

Отметим, что в представленной постановке (в отличии от схемы SIMPLE из п. 17.1.2.1) мы не использовали релаксацию, поэтому эту схему можно использовать для решения нестационарный задачи (18.1), трактуя τ как шаг по реальному времени.

19.1.2 Граничные условия

TODO

19.1.3 Схема с внешними итарциями PIMPLE

TODO

19.2 Конвективный теплообмен

19.2.1 Уравнение теплопроводности

Дополним нестационарную систему уравнений течения (18.1) уравнением теплообмена

$$\rho c_p \left(\frac{\partial T}{\partial t} + u \frac{\partial T}{\partial x} + v \frac{\partial T}{\partial y} \right) = \lambda \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right).$$

Здесь T – температура течения, К; ρ – плотность жидкости, кг/м^3; c_p – теплоёмкость, Дж/кг/К; λ – теплопроводность, Вт/м/К.

В безразмерном виде это уравнение примет вид

$$\frac{\partial T}{\partial t} + u \frac{\partial T}{\partial x} + v \frac{\partial T}{\partial y} = \frac{1}{\mathrm{Pe}} \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right). \tag{19.11}$$

где безразмерная температура T вычислена через размерную T^{dim} как

$$T = \frac{T^{dim} - T^0}{\triangle T},$$

а число Пекле Pe есть

$$\mathrm{Pe} = \frac{UL}{a}, \quad a = \frac{\lambda}{\rho c_p}.$$

19.2.2 Дискретизация по времени

Пользуясь обозначениями из п. 18.1.1 запишем неявную дискретизацию по времени уравнения (19.11) в виде

$$\frac{\hat{T} - \check{T}}{\triangle t} + \hat{u} \frac{\partial \hat{T}}{\partial x} + \hat{v} \frac{\partial \hat{T}}{\partial y} = \frac{1}{\mathrm{Pe}} \left(\frac{\partial^2 \hat{T}}{\partial x^2} + \frac{\partial^2 \hat{T}}{\partial y^2} \right). \tag{19.12}$$

Полученное уравнение не содержит значений u, v с текущего итерационного слоя, и значит может быть решено один раз в конце шага по времени, когда сходимость уже достигнута.

19.2.3 Аппроксимация на разнесённой сетке

Пространственную аппроксимацию уравнения (19.12) будем проводить на разнесённой сетке в центральных (“чёрных”) узлах сетки (рис. 12). При этом конвективную производную будем приближать

с помощью симметричной разности. Полученная конечная разность для узла $i+\frac{1}{2}, j+\frac{1}{2}$ примет вид

$$\begin{aligned}\frac{1}{\triangle t}\hat{T}_{i+\frac{1}{2},j+\frac{1}{2}} &+ \hat{u}_{i+\frac{1}{2},j+\frac{1}{2}}\frac{\hat{T}_{i+\frac{3}{2},j+\frac{1}{2}}-\hat{T}_{i-\frac{1}{2},j+\frac{1}{2}}}{2h_x} \\ &+ \hat{v}_{i+\frac{1}{2},j+\frac{1}{2}}\frac{\hat{T}_{i+\frac{1}{2},j+\frac{3}{2}}-\hat{T}_{i+\frac{1}{2},j-\frac{1}{2}}}{2h_y} \\ &+ \frac{1}{\mathrm{Pe}}\frac{-\hat{T}_{i+\frac{3}{2},j+\frac{1}{2}}+2\hat{T}_{i+\frac{1}{2},j+\frac{1}{2}}-\hat{T}_{i-\frac{1}{2},j+\frac{1}{2}}}{h_x^2} \\ &+ \frac{1}{\mathrm{Pe}}\frac{-\hat{T}_{i+\frac{1}{2},j+\frac{3}{2}}+2\hat{T}_{i+\frac{1}{2},j+\frac{1}{2}}-\hat{T}_{i+\frac{1}{2},j=\frac{1}{2}}}{h_x^2} \\ &= \frac{1}{\triangle t}\check{T}_{i+\frac{1}{2},j+\frac{1}{2}}.\end{aligned} \tag{19.13}$$

Значения компонент скорости в центрах ячеек вычисляются с помощью ближайщей полусуммы

$$\begin{aligned}\hat{u}_{i+\frac{1}{2},j+\frac{1}{2}} &\approx \frac{\hat{u}_{i,j+\frac{1}{2}}+\hat{u}_{i+1,j+\frac{1}{2}}}{2}, \\ \hat{v}_{i+\frac{1}{2},j+\frac{1}{2}} &\approx \frac{\hat{v}_{i+\frac{1}{2},j}+\hat{v}_{i+\frac{1}{2},j+1}}{2}.\end{aligned}$$

19.2.4 Граничные условия

Учёт граничных условий производится за счёт вычисления значений в фиктивных узлах около границ.

Пусть требуется учесть условие на левой стенке ($i = 0$). Тогда соответствующий фиктивный узел будет иметь индекс $-\frac{1}{2}, j$. Ниже приведём его выражения для трёх типов граничных условий.

19.2.4.1 Условия первого рода

Пусть

$$T|_{left} = T^{\Gamma} \tag{19.14}$$

Тогда

$$\frac{T_{-\frac{1}{2},j+\frac{1}{2}}+T_{\frac{1}{2},j+\frac{1}{2}}}{2} = T^{\Gamma}.$$

Отсюда

$$T_{-\frac{1}{2},j+\frac{1}{2}} = -T_{\frac{1}{2},j+\frac{1}{2}} + 2T^{\Gamma}.$$

Таким образом, если в матрицу A^T левой части выражения (19.13) в фиктивную колонку $k\left[-\frac{1}{2}, j+\frac{1}{2}\right]$ требуется добавить какое-то значение a, это равносильно добавлению этого выражнеия с обратным знаком в диагональ и удвоенного выражения, умноженного на граничное значение, в правую часть b^T:

$$\begin{aligned}&k_0 = k\left[\tfrac{1}{2}, j+\tfrac{1}{2}\right], \quad k_1 = k\left[-\tfrac{1}{2}, j+\tfrac{1}{2}\right], \\ &A^T_{k_0,k_1}\mathrel{+}=a \quad \Rightarrow \quad A^T_{k_0,k_0}\mathrel{-}=a, \quad b^T_{k_0}\mathrel{-}=2aT^{\Gamma}.\end{aligned} \tag{19.15}$$

19.2.4.2 Условия второго рода

Если на левой границе задано условие второго рода

$$\left.\frac{\partial T}{\partial n}\right|_{left} = -\left.\frac{\partial T}{\partial x}\right|_{left} = q \tag{19.16}$$

То вычисление фиктивного узла производится из конечной разности вида

$$\frac{T_{-\frac{1}{2},j+\frac{1}{2}} - T_{\frac{1}{2},j+\frac{1}{2}}}{h_x} = q.$$

Отсюда

$$T_{-\frac{1}{2},j+\frac{1}{2}} = T_{\frac{1}{2},j+\frac{1}{2}} + h_x q$$

Тогда

$$A^T_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^T_{k_0,k_0} \mathrel{+}= a, \quad b^T_{k_0} \mathrel{-}= h_x q \tag{19.17}$$

19.2.4.3 Условия третьего рода

Пусть на левой границе задано условие второго рода

$$\left.\frac{\partial T}{\partial n}\right|_{left} = -\left.\frac{\partial T}{\partial x}\right|_{left} = \alpha T + \beta \tag{19.18}$$

Расписывая производную и вычисляя значение температуры на стенке через полусумму, получим

$$\frac{T_{-\frac{1}{2},j+\frac{1}{2}} - T_{\frac{1}{2},j+\frac{1}{2}}}{h_x} = \alpha \frac{T_{-\frac{1}{2},j+\frac{1}{2}} + T_{\frac{1}{2},j+\frac{1}{2}}}{2} + \beta.$$

Отсюда выразим значение в фиктивном узле

$$T_{-\frac{1}{2},j+\frac{1}{2}} = \frac{2+\alpha h_x}{2-\alpha h_x} T_{\frac{1}{2},j+\frac{1}{2}} + \frac{2\beta h_x}{2-\alpha h_x}$$

Тогда

$$A^T_{k_0,k_1} \mathrel{+}= a \quad \Rightarrow \quad A^T_{k_0,k_0} \mathrel{+}= \frac{2+\alpha h_x}{2-\alpha h_x} a, \quad b^T_{k_0} \mathrel{-}= \frac{2\beta h_x}{2-\alpha h_x} a. \tag{19.19}$$

19.2.4.4 Универсальность условий третьего рода

Условие третьего рода (19.18) можно использовать для моделирования условий первого и второго рода. Так, условия второго рода (19.16) получаются, если положить $\alpha = 0$, $\beta = q$. А условия первого (19.14), – если $\alpha = \varepsilon^{-1}, \beta = -\varepsilon^{-1} T^\Gamma$, где ε – малое положительное число.

Если подставить эти выражения в формулу (19.19), то можно убедится, что они дадут выражения (19.17) и (19.15) (в пределе при $\varepsilon \to 0$) соответственно.

19.2.5 Коэффициент теплообмена

На границах, где заданы условия первого рода (19.14) можно вычислить тепловой поток, тем самым определив, сколько тепловой энергии требуется для поддержания этой постоянной температуры.

Безразмерный интегральный коэффициент теплообмена (интегральное число Нуссельта) определяется как

$$\mathrm{Nu} = \int\limits_{\gamma} \frac{\partial T}{\partial n}\, ds. \tag{19.20}$$

Для получения размерной мощности из этого безразмерного коэффициента (измеряемой в Ваттах), необходимо умножить его на $\lambda \triangle T L$.

Вычисление интегрального числа Нуссельта из определения (19.20) происходит по той же схеме, что и вычисление коэффициентов сил (18.24). При этом нормальная производная на границе $\partial T/\partial n$ вычисляется в виде

$$(x,y) \in \gamma_i : \quad \frac{\partial T}{\partial n} \approx \frac{T^{\Gamma} - T_k}{h/2}, \tag{19.21}$$

где γ_i – отрезок границы, k – индекс ячейки, прилегающей к этому отрезку, h – шаг сетки, поперёк границы (h_x для вертикальных границ и h_y – для горизонтальных).

19.3 Задание для самостоятельной работы

В тесте `[heat-transfer-piso]` файла `heat_transfer_piso_test.cpp` представлено решение нестационарной задачи обтекания нагреваемого тела, расчитанного по схеме PISO (п. 19.1.1). На основе этого теста нужно решить задачу обтекания двух тел: одно расположено в прямоугольнике

$$\gamma_1 : \begin{cases} 2.0 \le x \le 2.5, \\ -0.7 \le y \le 0.3, \end{cases}$$

второе –

$$\gamma_2 : \begin{cases} 4 \le x \le 4.5, \\ -0.3 \le y \le 0.7. \end{cases}$$

Использовать условия для температуры:

$$\begin{aligned} (x,y) \in \Gamma_{in} : &\quad T = 0, \\ (x,y) \in \gamma_1 : &\quad T = 0.5, \\ (x,y) \in \gamma_2 : &\quad T = 1.0. \end{aligned}$$

Область расчёта $[0,15] \times [-2,2]$:

```
RegularGrid2D grid(0, 15, -2, 2, 15*n_unit, 4*n_unit);
```

Остальные граничные условия использовать те же, что и в рассмотренной в п. 18.3.4 задаче. Параметры задачи: $\mathrm{Re} = 100$, $\mathrm{Pe} = 100$, $\triangle t = 0.05$, $t_{end} = 200$, $n = 10$.

Необходимо:

- Нарисовать анимированную картину течения: скорость, давление и температуру

- Представить графики изменения коэффициентов сопротивления и теплообмена во времени для обоих обтекаемых тел
- Увеличить количество PISO итераций (например, до `n_piso=3`) и сравнить полученные коэффицинеты сопротивления с случаем одной итерации.

Задание сетки с неактивными ячейками Обтекаемые препятствия следует задавать при определении сетки. В рассмотренном примере из предыдущего пункта это делалось в строке

```
802  grid.deactivate_cells({2, -0.5}, {3, 0.5});
```

В настоящей задачи нужно эту функцию вызвать два раза, указав там по очереди обе нужные области.

Задание граничных условий на температуру Поскольку в задаче граничные условия на обтекаемых телах отличаются по своему значению, то следует модифицировать алгоритм их задания. Граничные условия на температуру задаются в функции `compute_temperature` в строке

```
602         // internal boundary: T = 1
603         mat.add_value(row_index, row_index, -value);
604         rhs[row_index] -= 2*value;
```

Согласно форме (19.15) изменения в левой части не зависит от величины граничной температуры, а в провой – пропорционально ей. Таким образом, для первого тела (где $T^\Gamma = 0.5$) добавка в правую часть будет иметь вид

```
rhs[row_index] -= 2*value*0.5,
```

а для второго – останется такой же, как и раньше.

При этом нужно уметь отличать грани, принадлежащие первому телу от граней, принадлежащих второму. Для этого в классе `HeatTransferPisoWorker` можно объявить функцию, которая определяет ближайшую к ячейке границу. Саму функцию можно реализовать просто используя координаты центра ячейки `_grid.cell_center(icell)`. Например:

```
// => 1 если ячейка icell близка к первому обтекаемому телу и 2 - если ко второму
int HeatTransferPisoWorker::gamma_closest_to_cell(size_t icell){
  double x = _grid.cell_center(icell).x();
  if (x < 3.25) {  // центр между первым и вторым телами
    return 1;
  } else {
    return 2;
  }
}
```

Тогда можно написать

```
double t_gamma = (gamma_closest_to_cell(row_index) == 1) ? 0.5 : 1.0;
rhs[row_index] -= 2*t_gamme*value;
```

Здесь запись `double a = (cond) ? 0.5 : 1.0;` есть сокращение от

```
double a;
if (cond){
  a = 0.5;
} else {
  a = 1.0
}
```

Вычисление коэффициентов Во-первых следует модифицировать структуру, хранящую коэффициенты, сделав там отдельные записи для каждого тела:

```
struct Coefficients{
  double cpx1, cpx2;
  double cfx1, cfx2;
  double cx1, cx2;
  double nu1, nu2;
};
```

Во-вторых, в функции сохранения этих коэффициентов в файл в функции `HeatTransferPisoWorker::save_current_fields`

```
cx_writer << time << " ";
cx_writer << coefs.cx1 << " " << coefs.nu1 << " ";
cx_writer << coefs.cx2 << " " << coefs.nu2 << std::endl;
```

Соответственно можно поправить легенду в функции `initialize_saver()`.

Сами коэффициенты следует вычислять в функции `coefficients()`. Там нужно завести аггрегаторы на оба тела:

```
double sum_cpx1 = 0;
double sum_cfx1 = 0;
double sum_nu1  = 0;
double sum_cpx2 = 0;
double sum_cfx2 = 0;
double sum_nu2  = 0;
```

И далее заполнять их в зависимости от близости ячейки. Так же следует учесть значение граничной температуры при вычислении $\partial T/\partial n$ по формуле (19.21)

Например, для вертикальных граней после определения ячейки `left_cell`:

```
int gamma_i = gamma_closest_to_cell(left_cell);
double t_gamma = (gamma_i == 1) ? 0.5 : 1.0;
```

далее учесть при вычислении `dtdn` (два раза)

```
dtdn = (t_gamma - _t[left_cell]) / (_hx/2.0);
```

а также при выборе агрегатора

```
if (gamma_i == 1){
  sum_cpx1 += pnx * _hy;
  sum_nu1 += dtdn * _hy;
} else {
  sum_cpx2 += pnx * _hy;
  sum_nu2 += dtdn * _hy;
}
```

Аналогичную процедуру следует проделать и для горизонтальных граней.

В конце нужно правильным образом заполнить все поля переменной `coef`.

```
  coefs.cpx1 = 2.0*sum_cpx1;
  coefs.cpx2 = 2.0*sum_cpx2;
  ...
```

20 Лекция 20 (14.12)

20.1 Конечнообъёмное решение нестационарной задачи об обтекании кругового цилиндра

TODO

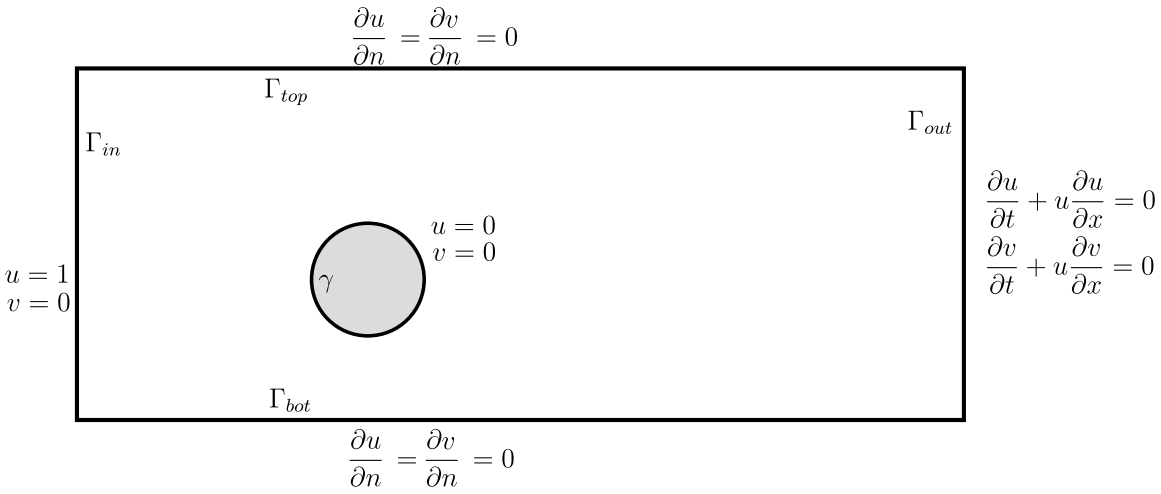


Рис. 17: К постановке задачи об обтекании цилиндра

$$\left(1 + \frac{\tau}{\triangle t}\right) u^* + \tau \nabla \cdot (\mathbf{u} u^*) - \frac{\tau}{\mathrm{Re}} \nabla^2 u^* = u + \frac{\tau}{\triangle t} \check{u} - \tau \frac{\partial p}{\partial x}. \tag{20.1}$$

20.2 Задание для самостоятельной работы

В тесте `[cylinder2-fvm-pimple]` из файла `cylinder_fvm_pimple_test.cpp` приведено решение задачи об обтекании кругового цилиндра на неструктурированной конечнообъёмной сетке. Нужно

1. Визуализировать решение нестационарной задачи до момента $t = 100$,
2. Продемонстрировать эффект от поправки Rhie–Chow (см. аргумент `rhie_chow` в функции `compute_un_face`) на полях скорости и давления,
3. Дополнить задачу уравнением для температуры вида (19.11). Использовать граничные условия(пользуясь обозначениями с рис. 17):

$$\begin{aligned} \Gamma_{in}: &\quad T = 0, \\ \Gamma_{top}, \Gamma_{bot}: &\quad \frac{\partial T}{\partial n} = 0, \\ \Gamma_{out}: &\quad \frac{\partial T}{\partial t} + u\frac{\partial T}{\partial x} = 0, \\ \gamma: &\quad T = 1. \end{aligned}$$

4. Добавить процедуру вычисления интегрального числа Нуссельта Nu на временно̀м слое.
5. Провести расчёты для $\mathrm{Re} = 100$ и трёх чисел Пекле $\mathrm{Pe} = 30, 100, 1000$ до момента времени $t_{end} = 60$. Расчёты проводить в релизной сборке.

6. Сравнить полученные картины течения и графики $\mathrm{Nu}(t)$.

20.2.1 Уравнение для температуры

После дискретизации по времени уравнения (19.11) с шагом $\triangle t$ оно примет вид

$$\hat{T} + \triangle t \nabla \cdot (\mathbf{u}\hat{T}) - \frac{\triangle t}{\mathrm{Pe}} \nabla^2 \hat{T} = \check{T}, \tag{20.2}$$

где $\hat{T}$ – значение температуры на следующем временно́м слое. Как видим, левая часть уравнения для температуры отличается от левой части уравнения переноса для скорости (20.1) только множителями перед слагаемыми. Типы граничных условий для этих уравнений также идентичны. Значит матрицу левой части для уравнения температуры можно собирать по той же процедуре, что и левую часть для u^*. В тестовом рабочем классе эта процедура имеет сигнатуру

```
CsrMatrix CylinderFvmSimpleWorker::assemble_uv_lhs(double coef_u, double coef_conv,
↪  double coef_diff) const;
```

где коэффициенты `coef_u`, `coef_conf`,
`coef_diff` – скалярные коэффициенты перед слагаемыми свободным, конвективным и диффузным слагаемыми левой части. Для сборки уравнения для u^* эти коэффициенты равны

$$c_u = 1 + \frac{\tau}{\triangle t}, \quad c_{conv} = \tau, \quad c_{diff} = \frac{\tau}{\mathrm{Re}}.$$

Тогда для аппроксимации левой части уравнения для температуры нужно вызывать эту функцию с коэффициентами

$$c_u = 1, \quad c_{conv} = \triangle t, \quad c_{diff} = \frac{\triangle t}{\mathrm{Pe}}.$$

Правая часть для внутренних точек коллокации будет аппроксимирована по стандартной конечно-объёмной процедуре:

$$b_i^T = \int\limits_{E_i} \check{T}\, d\mathbf{x} = |E_i| \check{T}_i, \quad i = \overline{0, N_{cells}}.$$

Граничные условия для температуры содержат только одно неоднородное условие: $T = 1$ на γ. Поэтому для точек коллокации на границе γ в правую часть следует поставить единицу

$$b_i^T = 1, \quad i \in \gamma.$$

20.2.2 Коэффициент теплоотдачи

Из определния (19.20) интегральное число Нуссельта расписывается как

$$\mathrm{Nu} = \int\limits_{\gamma} \frac{\partial T}{\partial n}\, ds = \sum_{s \in \gamma} \int\limits_{\gamma_s} \frac{\partial T}{\partial n}\, ds = \sum_{s \in \gamma} \left(\frac{\partial T}{\partial n} \right)_s |\gamma_s|, \tag{20.3}$$

где s – индексы всех граней, принадлежащих границе s, а $\mathbf{n}$ – нормаль к границе, внешняя к области расчёта.

20.2.3 Порядок реализации

1. В рабочий класс необходимо добавить два новых поля: параметр Pe (`_Pe`) и массив T (`_temperature`). Параметр Pe необходимо пробросить в конструктор по аналогии с Re, а массив T инициализировать нулями внутри конструктора (длина массива равна количеству точек коллокации, вычисляемой через метод `vec_size`).

2. Для вычисления поля температуры добавить в рабочий класс новый метод

   ```
   std::vector<double> CylinderFvmSimpleWorker::compute_temperature() const;
   ```

   который будет использовать текущую температуру `_temperature` в качестве значения $\check{T}$ и возвращать новую температуру, посчитанную по формуле (20.2). Этот метод будет иметь следующую реализацию

   ```
   // 1. === Левая часть
   CsrMatrix mat_temp = assemble_uv_lhs(1, _time_step, _time_step/_Pe);
   // 2. === Правая часть
   std::vector<double> rhs_temp(vec_size(), 0);
   // 2.1 Внутренние точки коллокации
   for (size_t icell=0; icell<_grid.n_cells(); ++icell){
     rhs_temp[icell] = _grid.cell_volume(icell) * _temperature[icell];
   }
   // 2.2 Граничные условия на цилиндре
   for (size_t icolloc: _boundary_info.cyl){
     rhs_temp[icolloc] = 1;
   }
   // 2.3 Граничные условия на выходе
   for (size_t icolloc: _boundary_info.output){
     rhs_temp[icolloc] = _temperature[icolloc];
   }
   // 3. === Решение СЛАУ
   // 3.1 Массив с ответом.
   //     Инициализируем текущей температурой в качестве первого приближения
   std::vector<double> new_temperature(_temperature)
   // 3.2 Вызываем решатель СЛАУ
   AmgcMatrixSolver::solve_slae(mat_temp, rhs_temp, new_temperature);
   // 4. === Возвращаем ответ
   return new_temperature;
   ```

3. Вызывать этот метод следует в процедуре `to_next_time_step`, которая вызывается после окончания итераций на временно̀м слое. Возвращаемый процедурой массив нужно присвоить полю `_temperature`, тем самым обновив значение для температуры в классе.

4. В процедуре `save_current_fields` добавить строчку

```
VtkUtils::add_cell_data(_temperature, "temperature", filepath, _grid.n_cells());
```

для сохранения температуры в файл vtk.

5. Добавить публичный метод `double CylinderFvmSimpleWorker::compute_nu() const`, который расчитывает интегральный коэффициент теплоотдачи по текущему полю `_temperature`. В реализации нужно запрограммировать формулу (20.3). Необходимые методы для вычислений:

   - Все индексы точек коллокации, лежащие на цилиндре

     ```
     std::vector<size_t> cyl_collocs = _boundary_info.cyl;
     ```

   - Индекс грани по индексу граничной точки коллокации `icolloc`

     ```
     size_t iface = _collocations.face_index(icolloc);
     ```

   - Производная температуры по нормали к грани с индексом `iface`

     ```
     double dtdn = _dfdn_computer.compute(iface, _temperature);
     ```

     Поскольку в формуле (20.3) используется не нормаль к грани, а внешняя нормаль к области, надо проверить, совпадают ли эти нормали. И если не совпадают (противонаправлены), то взять `dtdn` с обратным знаком:

     ```
     if (_grid.tab_face_cell(iface)[0] == INVALID_INDEX){
       dtdn *= -1.0;
     }
     ```

   - Для вычисления площади грани с индексом `iface`

     ```
     double area = _grid.face_area(iface);
     ```

6. Вызывать этот метод нужно в цикле по времени в функции верхнего уровня после обновления температурного поля:

```
493     worker.to_next_time_step();
```

Например, для печати значения Nu в консоль следует написать

```
std::cout << "time=" << time << " Nu=" << worker.compute_nu() << std::endl;
```

Можно осуществлять этот вызыв не на каждой временно̀й итерации, а с некоторым заданным шагом.

21 Лекция 21 (21.12)

22 Лекция 22 (11.01)

23 Лекция 23 (25.01)

23.1 Вычисление матрицы Якоби через базисные функции

TODO

23.2 Применение квадратурных формул для вычисления элементных интегралов

TODO

23.2.1 Диагонализация матрицы масс

TODO

23.3 Матричный алгоритм построения конечных элементов высокого порядка

TODO

23.4 Неполные конечные элементы

TODO

23.5 Задание для самостоятельной работы

Провести сравнительный анализ на порядок аппроксимации решения двумерного уравнения Пуассона с граничными условиями первого рода следующих конечноэлементных схем

1. Линейные треугольные элементы (рис. 25а)
2. Квадратричные треугольные элементы (рис. 25б)
3. Кубические треугольные элементы (рис. 25в)
4. Неполные 9-узловые кубические треугольные элементы (рис. 25г). В качестве базисных функций использовать полиномы, составленные из 9 слагаемых вида

$$P(\xi,\eta) \in \operatorname{span}\left\{1,\xi,\eta,\cancel{\xi\eta},\xi^2,\eta^2,\xi^2\eta,\xi\eta^2,\xi^3,\eta^3\right\}$$

(из 10 слагаемых, используемых для построения кубического базиса в треугольнике, из соображений симметрии было убрано слагаемое $\xi\eta$).

Построить кривые сходимости для всех четырёх случаев на едином графике. Для кубических треугольных элементов найти оптимальную квадратурную функцию.

Задание в целом аналогично заданию 13.4.

Принципы программирования конечноэлементного решателя описаны в п. 13.3. В настоящем задании была добавлена концепция численного интегрирования конечного элемента, которая заключена в реализации `NumericElementIntegrals` интерфейса `IElementIntegrals`.

Ниже рассмотрено создание кубического треугольного элемента, где для вычисления элементых интегралов используются гауссовые квадратуры, точные для полиномов 4-ой степени (`quadrature_triangle_gauss4`).

```
413     auto geom = std::make_shared<TriangleLinearGeometry>(p0, p1, p2);
414     auto basis = std::make_shared<TriangleCubicBasis>();
415     auto integrals =
        ↪  std::make_shared<NumericElementIntegrals>(quadrature_triangle_gauss4(), geom,
        ↪  basis);
416
417     elements.push_back(FemElement{geom, basis, integrals});
```

Для понижения точности до третьей, следует использовать функцию `quadrature_triangle_gauss3` и т.д.

Все необходимые тесты находятся в файле `poisson_fem_solve_test.cpp` . Для **линейных треугольных** элементов использовать тест `[poisson2-fem-tri]` .

Для **квадратичных треугольных** элементов – `[poisson2-fem-quadratic]` .

Для **кубических треугольных** элементов – `[poisson2-fem-cubic]` .

Для **неполных кубических треугольных** элементов систему локальных базисных функций нужно вычислить самостоятельно используя алгоритм из п.A.3.1.3. На основе полученных соотношений нужно создать класс

```
class TriangleCubicBasis_NoXiEta: public IElementBasis{
public:
  size_t size() const override;
  std::vector<Point> parametric_reference_points() const override;
  std::vector<BasisType> basis_types() const override;
  std::vector<double> value(Point xi) const override;
  std::vector<Vector> grad(Point xi) const override;
};
```

и реализовать все необходимые функции:

- `size` – общее количество базисных функций. Здесь будет девять,
- `parametric_reference_points` – параметрические координаты девяти узловых точек (соблюдая порядок локальной индексации),
- `basis_types` – типы базисных функций. Здесь все базисы узловые (`BasisType::Nodal`),
- `value` – значение девяти базисных функций в заданной параметрической точке. Здесь нужно подставить полученные при вычислении базисы,

- `grad` – градиенты вычисленных базисных функций. Для заполнения нужно подсчитать аналитические производные по ξ и η вычислиенных базисов.

Реализовать этот класс можно взяв в качестве основы уже реализованный класс `TriangleCubicBasis` из файла `cfd24/fem/elem2d/traingle_cubic.hpp`. Реализовывать следует в том же файле. После того, как базис будет реализован, его нужно использовать в тесте `poisson2-fem-cubic`:

```
auto basis = std::make_shared<TriangleCubicBasis_NoXiEta>();
```

Следует убрать последний базис из таблицы связности

```
tab_elem_basis.push_back({
  bas0, bas1, bas2,
  bas3, bas4, bas5, bas6, bas7, bas8
});
```

и сократить общее количество базисных функций

```
size_t n_bases = grid.n_points() + 2*grid.n_faces();
```

24 Лекция 24 (22.03)

24.1 Стабилизация методом характеристик

Введём обозначение

$$u^\theta = \theta\hat{u} + (1-\theta)\,u.$$

Полудискретизованное уравнение конвекции-диффузии со стабилизирущим слагаемым примет вид

$$\frac{\hat{u}-u}{\tau} = -\mathbf{v}\cdot\nabla u + \varepsilon\nabla^2 u^\theta + \frac{\tau}{2}\nabla\cdot\left((\mathbf{v}\cdot\nabla u)\,\mathbf{v}\right) - (1-\theta)\varepsilon\mathbf{v}\cdot\nabla\left(\nabla^2 u\right). \tag{24.1}$$

24.1.1 Конечноэлементная процедура

Далее по стандартной процедуре взвешенных невязок домножим уравнение (24.1) на систему пробных функций ϕ_i и проинтегрируем по области расчёта с применением формулы интегрирования по частям (A.10) к трём последним слагаемым правой части:

$$\begin{aligned}
\int_\Omega \frac{\hat{u}-u}{\tau}\phi_i\,d\mathbf{x} = &-\int_\Omega (\mathbf{v}\cdot\nabla u)\,\phi_i\,d\mathbf{x} \\
&+\varepsilon\left(-\int_\Omega \nabla u^\theta\cdot\nabla\phi_i\,d\mathbf{x} + \int_{\partial\Omega}\frac{\partial u^\theta}{\partial n}\phi_i\,ds\right) \\
&+\frac{\tau}{2}\left(-\int_\Omega (\mathbf{v}\cdot\nabla u)\,(\mathbf{v}\cdot\nabla\phi_i)\;d\mathbf{x} + \int_{\partial\Omega}(\mathbf{v}\cdot\nabla u)\,v_n\phi_i\,ds\right) \\
&+\varepsilon(1-\theta)\left(\int_\Omega \nabla^2 u\nabla\cdot(\phi_i\mathbf{v})\;d\mathbf{x} - \int_{\partial\Omega} v_n\phi_i\nabla^2 u\,ds\right)
\end{aligned}$$

Согласно подходу Бубнова-Галёркина разложим искомую функцию на систему базисных функций, равную системе пробных функций

$$u(\mathbf{x}) = \sum_i u_i\phi_i(\mathbf{x})$$

Получим матричное выражение для перехода на следующий временной слой

$$\begin{aligned}
&\mathrm{L}\hat{u} = \mathrm{R}u \\
&\mathrm{L} = \mathrm{M} + \tau\varepsilon\theta\left(\mathrm{S} - \mathrm{B}^{\mathrm{S}}\right) \\
&\mathrm{R} = \mathrm{M} - \tau\mathrm{K} - \tau(1-\theta)\varepsilon\left(\mathrm{S} - \mathrm{B}^{\mathrm{S}}\right) + \tfrac{\tau^2}{2}\left(-\mathrm{K}^{\mathrm{s}} + \mathrm{B}^{\mathrm{Ks}}\right) + \varepsilon\tau(1-\theta)\left(\mathrm{G}_2 - \mathrm{B}^{\mathrm{G}_2}\right)
\end{aligned} \tag{24.2}$$

где введена новая матрица G_2:

$$\mathrm{G}_2 = \int_\Omega \nabla^2\phi_j\nabla\cdot(\mathbf{v}\phi_i)\;d\mathbf{x},$$

которая обращается в ноль при использовании линейных конечных элементов.

24.2 Задание для самостоятельной работы

В тестовом примере `[convdiff-fem-cg]` из файла `convdiff-fem-test.cpp` производится численного решение той же одномерной задачи, которая рассматривалась ранее в п. 16.3. Решение производится методом конечных элементов со стабилизацией методом характеристик. Задача полудискретизуется по схеме Кранка-Николсон ($\theta = 1/2$) с шагом по времени, вычисленным через число Куранта $\mathrm{C} = 0.5$.

Необходимо:

1. С помощью анимированных графиков сравнить полученное численное решение одномерной задачи (`[convdiff-fem-cg]`) с точным решением, а также с решением, полученнным с помощью SUPG-стабилизации;

2. Сделать расчёты одномерной задачи с различными шагами по времени τ. Продемострировать отличия в полученных решениях на анимированных графиках. Нарисовать зависимость нормы отлонения численного решения от точного на финальный момент времени

$$n_2 = ||u - u^e||_2$$

от шага по времени τ.

3. Написать аналогичный тест для двумерного случая (решение в единичном квадрате) и неконстантного поля скорости: $\mathbf{v} = (-y + 0.5, x - 0.5)$. Точное решение в этом случае будет иметь вид

$$\begin{aligned}
&\bar{x} = 0.5 + (x_0 - 0.5)\cos(t) - (y_0 - 0.5)\sin(t) && \text{– текущее положение пика}\\
&\bar{y} = 0.5 + (x_0 - 0.5)\sin(t) + (y_0 - 0.5)\cos(t) &&\\
&r^2 = (x - \bar{x})^2 + (y - \bar{y})^2 && \text{– расстояние от пика}\\
&u^e = \frac{1}{4\pi\varepsilon(t + t_0)} \exp\left(-\frac{r^2}{4\varepsilon(t + t_0)}\right) && \text{– решение}
\end{aligned}$$

Использовать следующие параметры:

- начальное положение пика $x_0 = 0.8, y_0 = 0.5$,
- сдвиг по времени $t_0 = 0.3$,
- коэффициент диффузии $\varepsilon = 10^{-3}$,
- временной интервал $t \in [0, 2\pi]$.

Анимировать решение, полученное на структурированной и скошенной сетках с количеством ячеек $N \approx 5000$.

4. Для структурированной сетки нарисовать зависимость n_2 на момент $t = 2\pi$ в зависимости от шага сетки h для фиксированного $\mathrm{C} = 0.5$ (в качестве характерной скорости при вычислении числа Куранта использовать единицу).

5. Нарисовать зависимость n_2 от шага по времени τ для фиксированной двумерной сетки $N \approx 5000$. Использовать структурированную и скошенную сетки.

25 Лекция 25 (29.03)

25.1 CBS схема решения уравнений Навье-Стокса

CBS (Characteristic Based Split) метод основан на конечнообъемной аппроксимации определяющих уравнений. При решении уравнения используется стабилизация методом характеристик. Вследствии стабилизации метод позволяет использовать одинаковую конечноэлелментную аппароксимацию для давления и компонент скорости.

25.1.1 Постановка задачи для слабосжимаемой жидкости

Рассмотрим определяющую безразмерную систему уравнений, описывающую течение слабосжимаемой жидкости:

$$\frac{\partial U}{\partial t} + \nabla \cdot (\mathbf{u}U) = -\frac{\partial p}{\partial x} + \frac{1}{\mathrm{Re}}\nabla^2 u \tag{25.1}$$

$$\frac{\partial V}{\partial t} + \nabla \cdot (\mathbf{u}V) = -\frac{\partial p}{\partial y} + \frac{1}{\mathrm{Re}}\nabla^2 v \tag{25.2}$$

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{U} = 0 \tag{25.3}$$

$$\frac{\partial \rho}{\partial t} = \frac{1}{c^2}\frac{\partial p}{\partial t}. \tag{25.4}$$

Здесь введены следующие обозначения: ρ – плотность жидкости, $U = \rho u$, $V = \rho v$ – компоненты импульса течения, c – скорость распространения возмущений (скорость звука).

25.1.2 Конечноэлементная аппроксимация

Для дискретизации по времени будем использовать явную по скорости и θ по давлению двухслойную схему. При записи слабой постановки для соотношений (25.1), (25.2) используем стабилизацию методом характеристик (см. п. 24.1). Тогда получим

$$\begin{aligned}\int\limits_\Omega \frac{\hat{U}-U}{\tau}\phi_i\,d\mathbf{x} = &-\int\limits_\Omega \nabla\cdot(\mathbf{u}U)\phi_i\,d\mathbf{x} - \frac{\tau}{2}\int\limits_\Omega \nabla\cdot(\mathbf{u}U)\nabla\cdot(\mathbf{u}\phi_i)\,d\mathbf{x}\\ &\left(-\int\limits_\Omega \frac{\partial p}{\partial x}\phi_i\,d\mathbf{x} + \frac{\tau}{2}\int\limits_\Omega \frac{\partial p}{\partial x}\nabla\cdot(\mathbf{u}\phi_i)\,d\mathbf{x}\right)^{\theta_2}\\ &-\frac{1}{\mathrm{Re}}\int\limits_\Omega \nabla u\cdot\nabla\phi_i\,d\mathbf{x} + \frac{1}{\mathrm{Re}}\int\limits_{\partial\Omega}\frac{\partial u}{\partial n}\phi_i\,ds + \frac{\tau}{2\mathrm{Re}}\int\limits_\Omega \nabla u\cdot\nabla(\mathbf{u}\cdot\nabla\phi_i)\,d\mathbf{x}\end{aligned}$$

Здесь используется следующая запись для θ-схемы

$$(F)^\theta = \theta\hat{F} + (1-\theta)F.$$

В правой части полученного соотнешнии все слагаемые с множителем $\tau/2$ получены в результате стабилизации. При этом последний из них, как отмечалось ранее, тождественен нулю при использовании линейных лагранжевых конечных элементов. Поэтому его в дальншейшем будем опускать.

Разобъём полученное соотношение на два: первое из которых не содержит давления:

$$\begin{aligned}\frac{1}{\tau}\int\limits_\Omega \delta U^*\phi_i\,d\mathbf{x} = &-\int\limits_\Omega \nabla\cdot(\mathbf{u}U)\phi_i\,d\mathbf{x} - \frac{\tau}{2}\int\limits_\Omega \nabla\cdot(\mathbf{u}U)\nabla\cdot(\mathbf{u}\phi_i)\,d\mathbf{x}\\ &-\frac{1}{\mathrm{Re}}\int\limits_\Omega \nabla u\cdot\nabla\phi_i\,d\mathbf{x} + \frac{1}{\mathrm{Re}}\int\limits_{\partial\Omega}\frac{\partial u}{\partial n}\phi_i\,ds,\end{aligned} \tag{25.5}$$

$$\frac{1}{\tau}\int\limits_\Omega \delta U^{**}\phi_i\,d\mathbf{x} = \left(-\int\limits_\Omega \frac{\partial p}{\partial x}\phi_i\,d\mathbf{x} - \frac{\tau}{2}\int\limits_\Omega \frac{\partial p}{\partial x}\nabla\cdot(\mathbf{u}\phi_i)\,d\mathbf{x}\right)^{\theta_2} \tag{25.6}$$

где введены две поправки импульса:

$$\begin{aligned}\delta U^* &= \tilde{U} - U,\\ \delta U^{**} &= \hat{U} - \tilde{U}\end{aligned}$$

так, что $\hat{U} = U + \delta U^* + \delta U^{**}$.

Соотношение (25.3), (25.4) так же распишем по двухслойной θ схеме:

$$\begin{aligned}\frac{1}{c^2\tau}\int\limits_\Omega(\hat{p}-p)\phi_i\,d\mathbf{x} &= -\left(\int\limits_\Omega \nabla\cdot\mathbf{U}\phi_i\,d\mathbf{x}\right)^{\theta_1}\\ &= -\int\limits_\Omega \nabla\cdot\mathbf{U}\phi_i\,d\mathbf{x} - \theta_1\int\limits_\Omega \nabla\cdot\boldsymbol{\delta}\mathbf{U}^*\phi_i\,d\mathbf{x} - \theta_1\int\limits_\Omega \nabla\cdot\boldsymbol{\delta}\mathbf{U}^{**}\phi_i\,d\mathbf{x}\end{aligned} \tag{25.7}$$

25.1.3 Порядок расчёта на временном слое

Для замены последнего слагаемого в правой части (25.7) воспользуемся соотношением **??** (и аналогичным соотнешением для V^* компоненты импульса), отбрасывая слагаемые при τ^2. Используя интегрирование по частям получим уравнение для давления

$$\begin{aligned}\frac{1}{c^2\tau}\int\limits_\Omega(\hat{p}-p)\phi_i\,d\mathbf{x} + \theta_1\theta_2\tau\int\limits_\Omega\nabla\hat{p}\cdot\nabla\phi_i\,d\mathbf{x} = &-\int\limits_\Omega\nabla\cdot\mathbf{U}\phi_i\,d\mathbf{x}\\ &+\theta_1\int\limits_\Omega\boldsymbol{\delta}\mathbf{U}^*\cdot\nabla\phi_i\,d\mathbf{x}\\ &-\theta_1(1-\theta_2)\tau\int\limits_\Omega\nabla p\cdot\nabla\phi_i\,d\mathbf{x}.\end{aligned} \tag{25.8}$$

Здесь при выводе использовались граничные соотношения $\partial p/\partial n = 0$ и $\boldsymbol{\delta}\mathbf{U}^*\cdot\mathbf{n} = 0$. Тогда порядок расчёта на временном слое примет вид:

1. Определяем первую поправку импульса $\boldsymbol{\delta}\mathbf{U}^*$ из уравнения (25.5);
2. Определяем давления $\hat{p}$ из соотношения (25.8);

3. Определяем вторую поправку импульса $\boldsymbol{\delta}\mathbf{U}^{**}$ из уравнения (25.6).

4. Вычисляем $\hat{\mathbf{U}}$. Удовлетворяем граничные условия по скорости. В случае слабосжимаемой жидкости $c >> 1$ значение плотности будет $\rho \approx 1$, поэтому можно положить $\hat{\mathbf{u}} = \hat{\mathbf{U}}$.

25.1.4 Полунеявная схема

Уравнения с (25.5), (25.6) с первого и третьего шагов алгоритма могут быть решены явно в случае, если их матрицы левой части будет диагональной. Этого можно добиться если использовать линейные конечные элементы и провести процедуру "концентрации масс", в результате которой матрица масс примет диагональный вид

$$m_i = \sum_{j=0}^{N} \int_{\Omega} \phi_j \phi_i \, d\mathbf{x}.$$

Тогда единственным источником неявности остаётся параметр θ_2 из уравнения (25.7). Будем считать схему полунеявной, если $\theta_2 \in [1/2\ , 1]$. В этом случае на каждом слое будет необходимо решить одну СЛАУ на втором шаге алгоритма.

Условие устойчивости для неявной схемы запишем как

$$\tau < \min\left(\frac{h}{|\mathbf{u}|}, \frac{h^2 \mathrm{Re}}{2}\right) \tag{25.9}$$

Здесь h – характерый линейный размер элемента.

25.1.5 Явная схема

Явной будем называть схему, у которой $\theta_2 = 0$. Условием устойчивости для явной схемы сформулируем в виде

$$\tau < \frac{h}{|\mathbf{u}| + c} \tag{25.10}$$

25.1.6 Стационарное течение

В случае, если рассматривается стационарное течение, то постановка (25.1) – (25.4) используется для получения решения методом установления. То есть решение продолжается до тех пор, пока искомые функции $\mathbf{U}, p$ не перестанут меняться. Параметр τ становится шагом по фиктивному времени, т.е. простым итерационным параметром, не имеющим специального физического смысла. Это снимает многие ограничения на его выбор. В частности, можно вместо постоянного шага вычислять собсвенный τ для каждого узла сетки (т.е. для каждой строки системы линейных уравнений). В этом случае в формулах (25.5), (25.6), (25.8) и подобных вместо τ будут присутствовать τ_i, которые в свою очередь будут находится локально. Например из (25.9) получим

$$\tau_i < \min\left(\frac{h_i}{|\mathbf{u}|_i}, \frac{h_i^2 \mathrm{Re}}{2}\right),$$

где h_i – шаг сетки, ассоциированный с i-ым узлом сетки.

25.1.7 Течения несжимаемой жидкости

Постановка задачи (25.1) – (25.4) описывает течение несжимаемой жидкости в случае, если $c \to \infty$. В этом случае полунеявная схема с условием устойчивости (25.9) остаётся применимой. Явную схему в чистом виде применять нельзя из-за вырождения левой части уравнения (25.8) при $\theta_2 = 0$.

25.1.7.1 Искусственная сжимаемость

Часто для расчёта течения стационарного несжимаемого течения используют постановку с наличием слабой искусственной сжимаемости, характеризуемой параметром β – аналогом скорости распространения возмущений c в "сжимаемой" постановке. Такая искуственная сжимаемоть способствут устойчивости расчётной схемы (действительно, наличие $c > 0$ увеличивает диагональные значения матрицы левой части системы (25.8) и тем самым улучшает свойства этой матрицы).

В качестве параметра β рекомендуется использовать соотношение

$$\beta = \max\left(\beta^*, |\mathbf{u}|, \frac{1}{\mathrm{Re}\, h}\right),$$

где β^* – заданное минимальное значение искуственной сжимаемости. По аналогии с τ, в случае стационарных задач параметр β можно выбирать отдельно для каждого узла.

25.1.7.2 Нестационарные задачи

Для расчёта несжимаего течения с использованием искуственной сжимаемости требуется, чтобы решение сошлось по критерию

$$\frac{\partial p}{\partial t} \approx 0.$$

То есть требуется решать задачу методом установления. Если при этом задача является нестационарной, то задача дискретизуется с двойным временем: физическим временем с шагом $\triangle t$ для продвижения вперёд и фиктивным временем с шагом τ для сходимости внутри временного слоя. Построение схем таким методом подробно разбирался в п. 18.1.

25.2 Задание для самостоятельной работы

Тесты `[cavity2-fem-im-cbs]`, `[cavity2-fem-ex-cbs]` из файла `cavity_fem_cbs_test.cpp` реализует полунеявную (п. 25.1.4, $\theta_1 = 1, \theta_2 = 1$) и чисто явную (п. 25.1.5, $\theta_1 = 1, \theta_2 = 0$) CBS схемы расчёта стационарной задачи о течении несжимаемой жидкости в каверне.

В процессе вычисления идёт расчет (и печать в консоль) изменения искомых полей течения по фиктивному времени. Эти изменеия характеризуются среднеквадратичными отклонениями соответствующих производных

$$\left|\frac{\partial u}{\partial t}\right|_2, \left|\frac{\partial v}{\partial t}\right|_2, \left|\frac{\partial p}{\partial t}\right|_2.$$

Как только все три поля перестают меняться (их производная по времени становится меньше заданного `eps`), расчёт останавливается. Задача решается на регулярной сетке 20×20.

Шаг по фиктивному времени τ вычисляется по соотношениям (25.9), (25.10), помноженным на заданный коэффициент `tau_coef`.

Необходимо:

- Для анализа сходимости численного решения необходимо уменьшить `eps` до 10^{-6}.
- Нарисовать полученные поля решения в динамике сходимости на подробной сетке. Использовать изолинии для отрисовки давления и вектора для отрисовки скорости.
- Нарисовать график сходимости выводимых производных по времени (с логарифмической осью ординат) от индекса итерации. Использовать регулярную сетке 50×50 с различными шагами по времени `tau_coef`. Сравнить чисто явную и полунеявную схему.
- Проделать аналогичный тест на неструктурированной сетке `tetragrid` с примерно тем же количеством ячеек.
- При программировании формул (25.5), (25.6) в методах `compute_delta_u_1,2` заменить диагональную (lumped) матрицу масс на полноценную (тем самым моделируя случай нелинейных элементов, когда использование диагональной матрицы масс невозможно). Для этого нужно перенести непостоянный множитель τ вправо, а слева оставить постоянную (не зависящую от итераций) матрицу. Для этой матрицы на этапе инициализации собрать решатель и вычислять с его помощью $\delta U^{*,**}$.

  Выяснить, насколько использование полных матриц масс увеличивает время расчёта.

26 Лекция 26 (05.04)

26.1 Конечнообъёмная coupled схема решения уравнений Навье-Стокса

26.1.1 Задание для самостоятельной работы

В тесте
`[cavity-fvm-coupled]` из файла `[cavity_fvm_coupled_test.cpp]` реализована Coupled схема решения задачи о стационарном течении в каверне с применением МС ограничителя. У программы есть единственный параметр сходимости – шаг по фиктивному времени τ. Необходимо (предварительно установив $\varepsilon = 10^{-6}$)

- Нарисовать раздельно графики сходимости для

$$\max\left|\frac{\partial u}{\partial t}\right|, \quad \max\left|\frac{\partial v}{\partial t}\right|, \quad \max\left|\frac{\partial p}{\partial t}\right|$$

  .

- Исследовать схему на оптимальный τ с точки зрения скорости сходимости с изменением шага по пространству
- Провести аналогичный анализ для изменений числа Рейнольдса (до 1000).
- Проверить полученные оптимумы на неструктурированной pebi сетке.
- Модифицировать алгоритм, отнеся конвективные слагаемые в уравнениях моментов полностью на прошлый временной слой. Проверить, как при этом изменится скорость сходимости.

Программа написана неоптимально. Как с точки зрения производительности, так и с точки зрения организации кода. Необходимо провести рефакторинг программы. Обратить внимание на

- Повторяющийся код в методах `u_equation`, `v_equation`,
- Повторяющийся алгоритм при вычислении поправки Rhie-Chow в методах `continuity_equation`, `compute_un_rhie_chow`,
- Некоторые части блочной матрицы не зависят от временного слоя и могли бы быть собраны один раз

При рефакторинге необходимо следить за неизменностью результатов тестовых проверок.

27 Лекция 27 (12.04)

27.1 Моделирование граничных условий на стенке методом конечных объёмов с использованием пристеночной функции

Рассмотрим задачу вязкого несжимаего течения около стенки. Удельная сила трения на стенке запишется в виде

$$\tau_w = -\nu \left.\frac{\partial u}{\partial n}\right|_\gamma, \tag{27.1}$$

u здесь – касательная к стенке компонента вектора скорости.

Здесь и далее используются безразмерные величины: $\rho = 1$, $\nu = \mathrm{Re}^{-1}$. Плотность в выкладках будет опускаться. Характерная скорость в пограничном слое примет вид

$$u_\tau = \sqrt{|\tau_w|}. \tag{27.2}$$

Тогда запишем нормированные скорость и расстоние поперёк пограничного слоя:

$$u^+ = \frac{u - u_\gamma}{u_\tau}, \quad y^+ = \frac{y u_\tau}{\nu} \tag{27.3}$$

При аппроксимации формулы (27.4) производная справа будет расписана в виде конечной разности. Пусть поведение эпюры скорости с удалением от стенки известно. То есть известна функция $u^+ = f(y^+)$. Подберём такую модифицированную вязкость ν_w, при использовании которой совместно с конечноразностной аппроксимацией нормальной производной, заданное распределение эпюры скорости f сохраниться. То есть

$$\tau_w = \nu_w \frac{u_p - u_\gamma}{h}, \tag{27.4}$$

где u_p – значение касательной скорости в узле сетки, ближайшем к стенке, u_γ – значение касательной скорости на стенке (ноль при условии прилипания), h - нормальное расстояние от точки, в которой задана u_p до стенки.

Комбинируя формулы (27.2) – (27.4), отнесенные к точке, в которой $y = h$, $u = u_p$, выразим модифицированную вязкость

$$\nu_w = \nu \frac{h^+}{f(h^+)}. \tag{27.5}$$

Отметим, что формула (27.5) может быть вычислена только итерационно. Действительно, для вычисления ν_w нужно знать нормированное расстояние h^+, коэффициент нормировки которого зависит от u_τ через (27.3). Скороть u_τ зависит от трения τ_w, которое может быть вычислена только из (27.4) с использованием ν_w, u_p с предыдущей итерации или временного слоя. В качестве начального приближения можно взять $\nu_w = \nu$.

27.2 Задание для самостоятельной работы

В тесте
`[cavity-fvm]` из файла `[cavity_fvm_test.cpp]` реализована SIMPLE схема решения задачи о стационарном течении в каверне методом конечных объёмов на регулярной сетке.

Необходимо дополнить программу алгоритмом коррекции коэффициента диффузии, учитывающий логарифмическое поведение эпюры скорости при удалении от стенки. Для этого использовать соотношение (27.5) а в качестве $f(h^+)$ использовать функцию с автоматической подстройкой:

$$f(y^+) = \begin{cases} y^+, & h^+ < 11.25, \\ \frac{1}{\kappa}\log(Ey^+), & h^+ > 11.25. \end{cases}$$

$$\kappa = 0.4187, \qquad E = 9.793.$$

После реализации нужно

- Настроить параметры счёта для достижения сходимости (уменьшить `eps`)
- Сравнить полученные поля скорости и давления в случае использования логарифмической пристеночной функции со случаем без неё (или тривиальной $f(y^+) = y^+$).
- Так же сравнить значения суммарного трения:

$$\tau = \int\limits_{\delta\Omega} \nu \frac{\partial u}{\partial n}\, ds = \int\limits_{\delta\Omega} \nu_w \frac{u_\gamma - u_p}{h}\, ds.$$

- Провести аналогичный тест с использованием неструктурированных pebi сеток.

27.2.1 Рекомендации

В алгоритм на итерационном слое (в самое его начало) необходимо добавить ещё один пункт: вычисление коэффициента диффузии на гранях конечных объёмов. Для внутренних граней он будет равен Re^{-1}, а для граничных нужно применить формулу (27.5). Далее найденные значения ν нужно использовать вместо Re^{-1} при сборке оператора Лапласа в методе `assemble_u_lhs`.

Некоторые методы, которые могут быть полезны:

- Связность грань-ячейка:
  `IGrid::tab_face_cell`. Возвращает пару ячеек, первая из которых лежит против направления нормали. В случае, если с одной из сторон нет ячейки, на соответствующем месте будет стоять `INVALID_INDEX`. Это можно использовать для определения граничных граней.
- Чистый список граничных граней можно получить из `IGrid::boundary_faces`.
- Для расчёта нормальной к грани производной методом конечных объёмов по принципу

$$\left.\frac{\partial f}{\partial n}\right|_{ij} = \frac{f_i - f_j}{h_{ij}}$$

  можно использовать метод `FvmFacesDn::compute`.

28 Лекция 28 (19.04)

28.1 Стабилизация методом наименьших квадратов

Для уравнения общего вида

$$Ax = b$$

слабая постановка со стабилизацией методом наименьших квадратов имеет вид

$$\int\limits_{\Omega} (Ax - b)\phi \, d\mathbf{x} + \sum_k \gamma_k \int\limits_{E_k} (Ax - b)A\phi \, d\mathbf{x} = 0,$$

где первое слагаемое - следствие аппроксимации Галёркина, а второе – стабилизирующее.

Рассмотрим нестационарное уравнение конвекции диффузии:

$$\frac{\partial u}{\partial t} + \mathbf{v} \cdot \nabla u - \varepsilon \nabla^2 u = 0$$

Для него слабая постановка примет вид

$$\int\limits_{\Omega} \left(\frac{\partial u}{\partial t} + \mathbf{v} \cdot \nabla u - \varepsilon \nabla^2 u \right) \phi \, d\mathbf{x} + \sum_k \gamma_k \int\limits_{E_k} \left(\frac{\partial u}{\partial t} + \mathbf{v} \cdot \nabla u - \varepsilon \nabla^2 u \right) \left(\frac{\partial \phi}{\partial t} + \mathbf{v} \cdot \nabla \phi - \varepsilon \nabla^2 \phi \right) d\mathbf{x} = 0.$$

Поскольку пробная функция ϕ не зависит от времени, то постановка упростится до

$$\int\limits_{\Omega} \left(\frac{\partial u}{\partial t} + \mathbf{v} \cdot \nabla u - \varepsilon \nabla^2 u \right) \phi \, d\mathbf{x} + \sum_k \gamma_k \int\limits_{E_k} \left(\frac{\partial u}{\partial t} + \mathbf{v} \cdot \nabla u - \varepsilon \nabla^2 u \right) \left(\mathbf{v} \cdot \nabla \phi - \varepsilon \nabla^2 \phi \right) d\mathbf{x} = 0.$$

Более того, для линейных конечных элементов внутри элемента справедливо $\nabla^2 \phi = 0$ и стабилизация методом наименьших квадратов сводится к стабилизации методом SUPG.

Далее используем двухслойную схему по времени со степенью неявности θ. Нестационарное слагаемое примет вид

$$\frac{\partial u}{\partial t} = \frac{\hat{u} - u}{\tau},$$

а все вхождения u распишутся в виде

$$u = \theta \hat{u} + (1 - \theta)u.$$

Окончательно систему линейных уравнений на временном слое получим заменив ϕ на ϕ_i и расписав u (и также $\hat{u}$) по базису ϕ:

$$u = \sum_j u_j \phi_j.$$

Параметр γ_k будем определять из следующей таблицы

	P_1	P_n
$\theta = 0$	$\left(\frac{2\|\mathbf{v}\|}{h}\right)^{-1}$	$\left(\frac{2\|\mathbf{v}\|}{h} + \frac{4\varepsilon}{h^2}\right)^{-1}$
$\theta = 1$	$\left(\frac{1}{\tau} + \frac{2\|\mathbf{v}\|}{h}\right)^{-1}$	$\left(\frac{1}{\tau} + \frac{2\|\mathbf{v}\|}{h} + \frac{4\varepsilon}{h^2}\right)^{-1}$
$\theta = 0.5$	$\left(\frac{2}{\tau} + \frac{2\|\mathbf{v}\|}{h}\right)^{-1}$	$\left(\frac{2}{\tau} + \frac{2\|\mathbf{v}\|}{h} + \frac{4\varepsilon}{h^2}\right)^{-1}$

здесь $P_{1/n}$ обозначают элементы первой/высокой степени.

28.2 Задание для самостоятельной работы

В тестовом примере `[convdiff-fem-gsl]` из файла `convdiff_fem_test.cpp` производится численного решение одномерного нестационарного уравнения конвекции-диффузии

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} - \varepsilon\frac{\partial^2 u}{\partial x^2} = 0$$

в области $x \in [0, 4]$ с точным решением вида

$$u^e(x,t) = \frac{1}{\sqrt{4\pi\varepsilon(t+t_0)}} \exp\left(-\frac{(x-vt)^2}{4\varepsilon(t+t_0)}\right)$$

Точное решение используется для формулировки начальных ($t = 0$) и граничных ($x = 0, 4$) условий первого рода. Результат расчёта сохраняется в файл `convdiff-gsl.vtk.series`. Тестовый пример в целом аналогичен рассмотреному ранее в п. 16.3. Численная схема зависит от трёх параметров: степени базисных функций `npower`, степени неявности θ и дополнительного множителя перед γ_k `gamma_mult`. По окончании расчёта на момент времени $t = 2$ программа печатает полученную норму отклонения численного решения от точного.

1. Проиллюстрировать решение, полученное с помощью полунеявной ($\theta = 0.5$) и неявной ($\theta = 1$) схем с уменьшением сетки,

2. Получить зависимость n от множителя `gamma_mult` для неявной и полунеявной схем для двух разных шагов сетки

3. Сравнить поведение норм решения при использовании линейных, квадратичных и кубических элементов (`npower=1,2,3`) при сгущении сетки для неявной и полунеявных схем

4. Для реализации схем высокого порядка точности необходимо уметь считать оператор Лапласа в физическом пространстве от базисной функции $\nabla^2\phi$. Соответствующий множители `laplace_i, laplace_j` появляются в коде при вычислении подинтегральных выражения для стабилизирующего интеграла в `local_gsl`. Несмотря на то, что набор базисных функций в элементе нам известен, вычисление оператора Лапласа от этих функций представляет некоторую сложность, так как функции заданы в параметрическом, а дифференцирование необходимо проводить в физическом пространстве. Этот оператор считается в файле `fem_element.cpp` в процедуре

`IElementIntegrals::OperandArg::laplace(i)` где i – индекс базисной функции, для которой считается оператор Лапласа.

Для вычисления градиента в физической плоскости мы записывали формулу (A.33). Эта формула реализована в процедуре
`IElementIntegrals::OperandArg::grad_phi`. Сейчас оператор лапласа считается с помощью дифференцирования градиента методом конечной разности. Необходимо вывести точную формулу для определения оператора Лапласа в преобразованных координатах $\nabla^2_{\mathbf{xx}} f$. Для этого нужно дважды применить формулу дифференцирования (A.33). Далее эту формулу необходимо упростить для случая линейного геометрического преобразования сегментов, где матрица Якоби (1×1) постоянна. То есть получить зависимость

$$\frac{\partial^2 f}{\partial x^2} = F\left(J_{11}, \frac{\partial^2 f}{\partial \xi^2}\right).$$

Полученную зависимоть вставить в метод
`laplace` вместо имеющейся конечной разности. Получить J_{11} из этого метода можно `geom->jacobi(xi).j11`, а вторую производную функции ϕ_i через `basis->upper_hessian(xi)[i][0]`.

29 Задание 1: $k-\epsilon$ модель турбулентности

Постановка задачи для определения турбулентной вязкости ν_T имеет вид

$$\frac{\partial k}{\partial t} + \nabla \cdot (\mathbf{u}k) - \nabla \cdot \left(\nu + \frac{\nu_T}{\sigma_k}\right) \nabla k = G - \epsilon \tag{29.1}$$

$$\frac{\partial \epsilon}{\partial t} + \nabla \cdot (\mathbf{u}\epsilon) - \nabla \cdot \left(\nu + \frac{\nu_T}{\sigma_\epsilon}\right) \nabla \epsilon = C_1 G \frac{\epsilon}{k} - C_2 \frac{\epsilon^2}{k} \tag{29.2}$$

$$G = 2\nu_T \mathrm{dev}\mathbf{E} : \nabla \mathbf{u} = \nu_T \sum_{i,j} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right) \frac{\partial u_i}{\partial x_j} \tag{29.3}$$

$$\nu = \frac{1}{\mathrm{Re}}, \qquad \nu_T = C_\mu \frac{k^2}{\epsilon}, \qquad \nu_{eff} = \nu + \nu_T. \tag{29.4}$$

$$C_\mu = 0.09, \quad \sigma_k = 1.0, \quad \sigma_\epsilon = 1.3, \quad C_1 = 1.44, \quad C_2 = 1.92$$

Граничные условия:

$$x \in \partial\Omega : \frac{\partial k}{\partial n} = 0 \tag{29.5}$$

Для приграничных ячеек

$$i \in bnd : \epsilon_i = \begin{cases} \dfrac{C_\mu^{3/4} k^{3/2}}{\kappa y}, & y^+ > 11.25, \\ \dfrac{2\nu k}{y^2}, & y^+ < 11.25. \end{cases} \tag{29.6}$$

где $\kappa = 0.41$, y – расстояние от границы, а y^+:

$$y^+ = \frac{y u_\tau}{\nu}, \quad u_\tau = \sqrt{C_\mu^{1/2} k},$$

Решать систему в таком виде нельзя из-за проблем с делением на ноль при вычислении правой части уравнения для диссипации ϵ и вычислении турбулентной вязкости ν_T. Поэтому переформулируем задачу, вводя переменную $\gamma = \epsilon/k$:

$$\frac{\partial k}{\partial t} + \nabla \cdot (\mathbf{u}k) - \nabla \cdot \left(\nu + \frac{\nu_T}{\sigma_k}\right) \nabla k + \gamma k = G,$$
$$\frac{\partial \epsilon}{\partial t} + \nabla \cdot (\mathbf{u}\epsilon) - \nabla \cdot \left(\nu + \frac{\nu_T}{\sigma_\epsilon}\right) \nabla \epsilon + C_2 \gamma \epsilon = C_1 G \gamma.$$

Для вычисления турбулентной вязкости введём минимум как долю δ_ν от кинематической вязкости: $\min(\nu_T) = \delta_\nu \nu$, а также текущую длину смешения $l^* = \min(1, C_\mu k^{3/2} \epsilon^{-1})$. Тогда турбулентную вязкость будем определять как

$$\nu_T = \max(\delta_\nu \nu, l^* \sqrt{k}) \tag{29.7}$$

а переменную γ для избегания деления на ноль в виде:

$$\gamma = C_\mu \frac{k}{\nu_T}.$$

29.0.1 Задание для самостоятельной работы

В тесте `[cavity-fvm-rans-ke]` из файла `[cavity_fvm_rans_ke_test.cpp]` реализована SIMPLE схема решения задачи о стационарном течении в каверне по турбулентной модели $k - \epsilon$. При для некоторых методов (помеченных комментарием `TODO`) стоит фиктивная реализация. Необходимо

- Реализовать все эти методы;
- Отрисовать итоговые поля, характеризующие скорость течения и его турбулентные характеристики;
- Провести анализ как константы модели влияют на уровень турбулентности в каверне. Уровень турбулентности считать как

$$S = \int_\Omega k \, d\mathbf{x}.$$

Рекомендации:

- Значения вязкости на внутренних гранях считать как полусумму вязкостей в соседних ячейках;
- Для вычисления тензора скоростей деформации $\mathbf{E}$ необходимо вычислить $\frac{\partial u_i}{\partial x_j}$. Для этого нужно воспользоваться методом `grad_computer_->compute` для обеих компонент скорости (примеры работы с этой функцией можно найти в других методах).

30 Задание 2: $k-\omega$ модель турбулентности

Введём параметр удельной диссипации кинетической энергии

$$\omega = \frac{\epsilon}{C_\mu k}. \tag{30.1}$$

Замыкающую модель турбулентности в терминах $k-\omega$ запишем согласно (Wilcox, 1988):

$$\frac{\partial k}{\partial t} + \nabla\cdot(\mathbf{u}k) - \nabla\cdot\left(\nu + \frac{\nu_T}{\sigma_k}\right)\nabla k = G - C_\mu \omega k, \tag{30.2}$$

$$\frac{\partial \omega}{\partial t} + \nabla\cdot(\mathbf{u}\omega) - \nabla\cdot\left(\nu + \frac{\nu_T}{\sigma_\omega}\right)\nabla \omega = \frac{\gamma G}{\nu_T} - \beta\omega^2 \tag{30.3}$$

$$G = 2\nu_T \mathrm{dev}\mathbf{E} : \nabla\mathbf{u} = \nu_T \sum_{i,j}\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right)\frac{\partial u_i}{\partial x_j} \tag{30.4}$$

$$\nu = \frac{1}{\mathrm{Re}}, \qquad \nu_T = \frac{k}{\omega}, \qquad \nu_{eff} = \nu + \nu_T. \tag{30.5}$$

$$C_\mu = 0.09, \quad \sigma_k = 2.0, \quad \sigma_\omega = 2.0, \quad \beta = \frac{3}{40}, \quad \gamma = \frac{5}{9}$$

Граничные условия:

$$x \in \partial\Omega : \frac{\partial k}{\partial n} = 0 \tag{30.6}$$

Для приграничных ячеек

$$i \in bnd : \omega_i = \begin{cases} \dfrac{C_\mu^{-1/4} k^{1/2}}{\kappa y}, & y^+ > 11.25, \\ \dfrac{6\nu}{\beta y^2}, & y^+ < 11.25. \end{cases} \tag{30.7}$$

где $\kappa = 0.41$, y – расстояние от границы, а y^+:

$$y^+ = \frac{y u_\tau}{\nu}, \quad u_\tau = \sqrt{C_\mu^{1/2} k},$$

По аналогии с программной реализацией модели $k-\epsilon$ диссипативные слагаемые (слагаемые с отрицательным знаком) из правых частей уравнений (30.2) следует перенести в левую часть. Линеаризация этого слагаемого осуществляется за счёт использования множителя ω с предыдущей итерации. Для избегания деления на ноль вихревую вязкость будем вычислять по формуле (29.7), где длина смешения выразиться как $l^* = \min(1, k^{1/2}\omega^{-1})$.

30.0.1 Задание для самостоятельной работы

Провести ту же работу, что и при анализе модели $k-\omega$. В тесте `[cavity-fvm-rans-ko]` из файла `[cavity_fvm_rans_ko_test.cpp]` реализована SIMPLE схема решения задачи о стационарном течении в каверне по турбулентной модели $k-\omega$. При для некоторых

методов (помеченных комментарием `TODO`) стоит фиктивная реализация. Необходимо

- Реализовать все эти методы;
- Отрисовать итоговые поля, характеризующие скорость течения и его турбулентные характеристики;
- Провести анализ как константы модели влияют на уровень турбулентности в каверне. Уровень турбулентности считать как
$$S = \int\limits_{\Omega} k \, d\mathbf{x}.$$
- Кроме того следует сравнить решения, полученные для этой задачи с решением для $k - \epsilon$ модели. Энергию k сравнивать напрямую, а диссипацию – путём приведения к единому виду (ϵ или ω) по формуле (30.1).

31 Задание 3: Задача о естественной конвекции. Метод дополнений Шура. МКЭ

В тесте `[heat-fem-simple]` из файла `heat_fem_simple_test.cpp` реализована конечноэлементная схема решения нестационарной задачи о естественной конвекции в закрытом объёмы с цилиндрическим нагревателем. Используются MINI-элементы: давление аппроксимируется на линейных треугольниках, скорости – на треугольниках с bubble-точкой. Решения СЛАУ осуществляется методом простой итерации с предобуславливателем Шура.

Безразмерная определяющая система уравнений приведена ниже:

$$\begin{aligned}
&\frac{\partial u}{\partial t} + \mathbf{u}\cdot\nabla u = \frac{1}{\mathrm{Re}}\nabla^2 u - \frac{\partial p}{\partial x},\\
&\frac{\partial v}{\partial t} + \mathbf{u}\cdot\nabla v = \frac{1}{\mathrm{Re}}\nabla^2 v - \frac{\partial p}{\partial y} + T,\\
&\nabla\cdot\mathbf{u} = 0,\\
&\frac{\partial T}{\partial t} + \mathbf{u}\cdot\nabla T = \frac{1}{\mathrm{Pe}}\nabla^2 T
\end{aligned} \tag{31.1}$$

Характерная скорость определена как $u_0 = \sqrt{gL(T_1 - T_0)\beta}$, где β – коэффициент сжимаемости, g – ускорение свободного падения, L – характерный масштаб длины, T_0, T_1 – максимальная и минимальная температуры.

Необходимо:

1. Провести расчёт до установления. Нарисовать динамику полученных полей скорости и температур.

2. Подсчитать коэффициент теплоотдачи: локальный (на разные моменты времени)

$$\mathrm{Nu}(\alpha) = \frac{1}{\mathrm{Pe}}\frac{\partial \mathrm{T}}{\partial \mathrm{n}}(\alpha),$$

   зависящий от угловой координаты α точки на нагревателе, и интегральный

$$\overline{\mathrm{Nu}}(t) = \frac{1}{\mathrm{Pe}}\int_\gamma \frac{\partial T}{\partial n}(t)\,ds$$

   .

3. Оптимизировать процедуры
   `use_A`, `use_preconditioner` используя операции с блоками и отказавшись от сборки глобальных матрицы. Замерить время работы оптимизированных процедур

4. В методе `step` вместо простого итерационного процесса использовать BiCGStab (п. A.5.3.1).

Рекомендации к программированию

- п. 2 Для подсчёта нормальной производной по температуре нобходимо пройти циклом по всем ячейкам, лежащим около границы нагревателя. Далее найти температуру в центре этой ячейки

T_c (как среднее от температур в узлах этой ячейки). Мы знаем, что на границе нагревателя $T(r = 0.5) = 1.0$. Отсюда нормальная производная на текущей грани выпишется как

$$\left.\frac{\partial T}{\partial n}\right|_{\gamma} = \frac{1.0 - T_c}{h_c - 0.5},$$

Здесь h_c – расстояние от центра ячейки до центра нагревателя в точке $(0, 0)$. Для нахождения интегрального коэффициента теплоотдачи это слагаемое нужно умножить на площадь грани и добавить в сумму. Для локального – найти угловую координату по центральной точке грани и приписать значение ей. Некоторые методы, необходимые для вычисления:

- `grid_.boundary_faces()` – список индексов всех граничных граней,
- `grid_.face_center(iface)` – координтата центра грани,
- `grid_.tab_face_cell(iface)` – две ячейки, инцидентные грани. Если грань граничная, одна из них равна `INVALID_INDEX`
- `grid_.cell_center(icell)` – координтата центра ячейки,
- `grid_.tab_cell_point(icell)` – список узлов, инцидентных ячеке. Необходим для вычисления T_c из массива `temperature_`.

- п. 3 В методе `use_A` вычисляется выражение

$$r = \begin{pmatrix} A & B^T \\ B & Z \end{pmatrix} \begin{pmatrix} u \\ p \end{pmatrix}$$

Где Z – это нулевая матрица с модификациями, связанными с граничными условиями первого рода на давление (в одной точке). В неоптимально написанном методе сначала из блоков собирается глобальная матрица `mat`, а потом проводится умножение. Можно обойтись без сборки, если написать

$$r_u = \mathrm{A}u - \mathrm{B}^T p,$$
$$r_p = \mathrm{B}u - \mathrm{Z}p.$$

Учесть, что длины векторов равны:

```
// вектора rhs_u, r_u, u, y_u
// Две компоненты скорости, каждая из которых задана
// на трёх узлах треугольников и bubble-точке
size_t nu = 2 * grid_.n_cells() + 2 * grid_.n_points();
// вектора rhs_p, r_p, p, y_p
// Заданы только в узлах треугольных ячеек
size_t np = grid_.n_points();
```

Для получения подмассива из сплошного вектора можно воспользоваться арифметикой указателей

```
const double* u = uvp_.data() + 0;  // без сдвига от начала uvp_
const double* p = uvp_.data() + nu; // сдвиг на nu элементов

// Умножение матрицы на вектор
std::vector<double> Au = A_->mult_vec_p(u);
```

- п. 3 В методе `use_preconditioner` решается система уравнений

$$\begin{pmatrix} A & 0 \\ B & -S \end{pmatrix} \begin{pmatrix} I & HB^T \\ 0 & I \end{pmatrix} x = r$$

относительно неизвестного x.

Сначала матрица левой части собирается в глобальную, потом вызывается решатель, для нахождения неизвестного вектора x в левой части. Полученная матрица имеет проблему седловой точки (нули на диагоналях), поэтому её нельзя решить общими итерационными методами. Для её решения исполльзуется точный решатель `UmfpackSolver`. Предлагается решать эту систему с следующем порядке

$$\begin{pmatrix} A & 0 \\ B & -S \end{pmatrix} \begin{pmatrix} y_u \\ y_p \end{pmatrix} = \begin{pmatrix} r_u \\ r_p \end{pmatrix}$$

откуда

$$\begin{aligned} y_u &= A^{-1} r_u, \\ y_p &= S^{-1}(B y_u - r_p). \end{aligned}$$

Далее

$$\begin{pmatrix} I & HB^T \\ 0 & I \end{pmatrix} \begin{pmatrix} x_u \\ x_p \end{pmatrix} = \begin{pmatrix} y_u \\ y_p \end{pmatrix}$$

откуда

$$\begin{aligned} x_p &= y_p, \\ x_u &= y_u - HB^T x_p \end{aligned}$$

Для вычисления y воспользоваться матричным решателем. Сначала точным (`UmfpackSolver`), потом, если всё работает, можно перейти на итерационный (`AmgcMatrixSolver`) и сравнить время работы.

- п. 4 Метод вычисления итерационной поправки к решению определяется реализацией интерфейса `IIterator`. В настоящий момент в классе `StationaryIterator` реализован метод простой итерации. Необходимо по аналогии сделать реализацию на основе метода сопряжённых градиентов (п. A.5.3.1):

```
class BiCGStab: public IIterator { ... };
```

Шаги инициализации следуем программировать в методе `init`, а подсчёт поправки – в методе `compute_correction`. При использовании этого итератора нужно отключить релаксации:

$\alpha_u = \alpha_p = 1.0$, так как в BiCGStab уже встроен внутренний механизм релаксации. Для максимальной эффективности итераций имеет смысл добавить внутренние итерации (просто вызывать `Worker::step` несколько раз подряд) и проверить оптимальное количество внутренних итераций.

На шагах 3 инициализации, а также 4 и 7 – основных итераций процесса BiCGStab требуется обратить предобуславливатель. Это нужно делать написанным методом `use_preconditioner`. Поскольку один и тот же предобуславливатель обращается минимум три раза (при одной внутренней итерации BiCGStab), следует разделить инициализацию матричного решателя (делать один раз) и непосредственно решени (делать три раза)

```
// инициализация
UmfpackSolver slv_A; // Или AmgcMatrixSolver
slv_A.set_matrix(A_);
// решения
std::vector<double> r0_prime;
slv_A.solve(r0, r0_prime);
```

Для сравнения скорости сходимости с методом простой итерации имеет смысл уменьшить ε и увеличить шаг по времени. Например

```
double eps = 1e-5;
double dt = 1e-0;
```

32 Задание 4: Параллельные методы решения задач гидродинамики

32.0.1 Распараллеливание циклов и явных схем

Рассмотрим явную схему против потока для одномерного уравнения переноса:

```
void explicit_step(double tau, double h, double V,
                   double bc_left, std::vector<double>& u){

  std::vector<double> unew(u.size());
  unew[0] = bc_left;
  for (size_t i=1; i<u.size(); ++i){
    unew[i] = u[i] - tau * V / h * (u[i] - u[i-1]);
  }
  std::swap(unew, u);
}
```

Параллельная версия этого цикла с использованием библиотеки **OpenMP** будет иметь вид

```
void explicit_step(double tau, double h, double V,
                   double bc_left, std::vector<double>& u){

  std::vector<double> unew(u.size());
  unew[0] = bc_left;
  #pragma omp parallel for
  for (size_t i=1; i<u.size(); ++i){
    unew[i] = u[i] - tau * V / h * (u[i] - u[i-1]);
  }
  std::swap(unew, u);
}
```

Та же схема с использованием системных потоков `std::thread`:

```
void explicit_step(double tau, double h, double V,
                   double bc_left, std::vector<double>& u) {

  constexpr size_t num_threads = 4;  // 4 потока
  std::vector<double> unew(u.size());
  unew[0] = bc_left;

  // тело цикла
  auto worker = [&](size_t start, size_t end) {
    for (size_t i = start; i < end; ++i) {
      unew[i] = u[i] - tau * V / h * (u[i] - u[i-1]);
```

```
    }
  };

  // Разделяем работу между потоками: первая четверть узлов идёт в первый поток,
  // вторая -- во второй и т.д.
  size_t chunk_size = (u.size() - 1) / num_threads;
  std::vector<std::thread> threads;
  for (size_t t = 0; t < num_threads; ++t) {
    size_t start = t * chunk_size + 1;
    size_t end = (t == num_threads - 1) ? u.size()
                                        : (t + 1) * chunk_size + 1;
    threads.emplace_back(worker, start, end);
  }

  // Дожидаемся завершения всех потоков
  for (auto& thread : threads) {
    thread.join();
  }

  std::swap(unew, u);
}
```

32.0.2 Параллельное решение независимых подзадач

Псевдокод решения нескольких дифференциальных уравнений для трёхмерного segragated метода: уравнения для компонент скорости могут быть решены независимо друг от друга в разных потоках. Используем для их решения асинхронное программирование

```
std::vector<double> compute_u_star(const std::vector<double>& uold,
                                   const std::vector<double>& vold,
                                   const std::vector<double>& wold){
  //...
}
std::vector<double> compute_v_star(...
std::vector<double> compute_w_star(...

std::vector<double> compute_p_prime(const std::vector<double>& u_star,
                                    const std::vector<double>& v_star,
                                    const std::vector<double>& w_star){
  //...
}

void simple_step(){
```

```
  std::future<std::vector<double>> u_star = std::async(
      std::launch::async, compute_u_star, u_old, v_old, w_old);
  std::future<std::vector<double>> v_star = std::async(
      std::launch::async, compute_v_star, u_old, v_old, w_old);
  std::future<std::vector<double>> w_star = std::async(
      std::launch::async, compute_w_star, u_old, v_old, w_old);

  std::vector<double> p_prime = compute_p_prime(
      u_star.get(), v_star.get(), w_star.get());
  //...
};//
```

32.0.3 Domain Decomposition Method. Метод Шварца

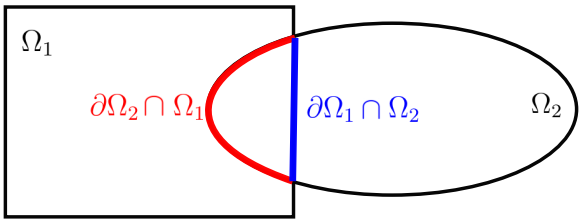


Рис. 18: Область решения Ω, разбитая на два перекрывающихся поддомена Ω_1, Ω_2

Рассмотрим задачу Пуассона с граничными условиями первого рода в области Ω:

$$\begin{cases} -\nabla^2 u = f, & x \in \Omega, \\ u = b.c., & x \in \partial\Omega. \end{cases} \tag{32.1}$$

Разобъём область Ω на несколько перекрывающихся доменов Ω_i (рис. 18). Будем решать только локальные задачи в доменах, а общее решение будем собирать как комбинацию локальных. Постановка задачи на итерации Шварца $n+1$ для i-го домена имеет вид

$$\begin{cases} -\nabla^2 u_i^{n+1} = f, & x \in \Omega_i, \\ u_i^{n+1} = b.c., & x \in \partial\Omega \cap \partial\Omega_i, \\ u_i^{n+1} = u^n, & x \in \partial\Omega_i \cap \Omega_j. \end{cases} \tag{32.2}$$

где функция глобального решения на итерации определяется как

$$u^n = \sum_i E_i \left(\mathcal{X}_i \, u_i^n \right). \tag{32.3}$$

Здесь E_i – функционал, расширяющий функцию, заданную в домене Ω_i на область Ω, дополняя эту функцию нулями. $\mathcal{X}_i$ – ограничивающий весовой коэффициент, обеспечивающий корректное поведе-

ние глобальной функции в зонах перекрытия.

$$\mathcal{X}_i = \begin{cases} 1, & \Omega_i \setminus \cup\, \Omega_j, \\ 0, & \cup \Omega_j \setminus \Omega_i, \\ [0,1], & \Omega_i \cap \Omega_j \end{cases} \qquad \sum_i \mathcal{X}_i = 1. \tag{32.4}$$

В зависимости от того, когда происходит вычисление глобальной функции, алгоритм Шварца можно использовать как в последовательном, так и в параллельном виде. Так, если обновлять глобальное решение u^n (32.3) после решения для каждого поддомена Ω_i (32.2), то получим последовательный (классический) алгоитм Шварца. Если же обновлять глобальное решение один раз на итерации после расчёта всех поддоменов, то получим параллельный алгоритм. Можно провести аналогию с итерационными алгоритмами решения систем линейных уравнений: последовательный алгоритм соответствует методу Зейделя, а параллельный – методу Якоби.

Задача (32.2) может быть переформулирована в теримнах поправки и невязки (с учётом того, что начальное решение u^0 удовлетворяет граничным условиям на $\partial\Omega$):

$$\begin{cases} -\nabla^2 \delta_i^{n+1} = r^n, & x \in \Omega_i, \\ \delta_i^{n+1} = 0, & x \in \partial\Omega_i \end{cases} \tag{32.5}$$

После вычисления поправки глобальное решение на новой итерации считается как

$$\delta^{n+1} = \sum_i E_i \left(\mathcal{X}_i \, \delta_i^{n+1} \right),$$
$$u^{n+1} = u^n + \delta^{n+1}$$

32.0.3.1 Построение сетки для конечнообъёмного DDM-метода

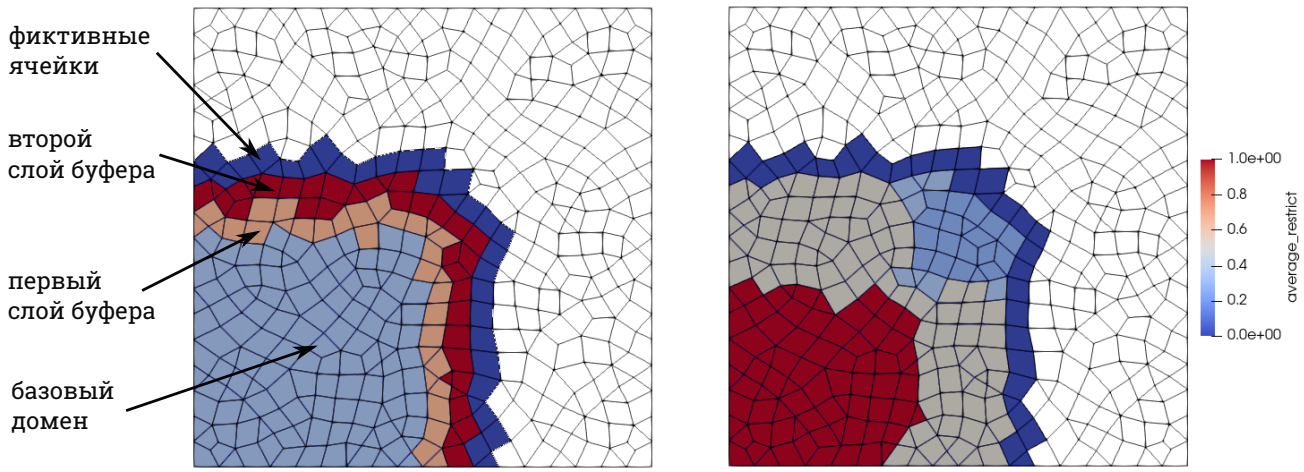


Рис. 19: Сетка в левом нижний домене при разбиении квадрата по схеме 2×2. Слева – буферные слои, справа – усреднённая весовая функция $\mathcal{X}$

1. На первом этапе вся расчётная область покрывается единой конечнообъёной сеткой.

2. Для каждого домена определяется набор ячеек из общей сетки, принадлежащих этому домену. На рис. 19 эта зона обозначена как «базовый домен». Таким образом строится набор непересекрывающихся сеток.

3. Строится буферная зона (зона перекрытия). От каждой базовой сетки отмечается ряд внешних соседних ячеек. Так делается несколько раз в зависимости от количества буферных слоёв. На рис. 19 отмечены два буферных слоя.

4. По такому же принципу отмечается ещё один слой фиктивных ячеек (ghost cells). Эти ячейки будут использоваться для постановки условия на междоменной границе (последнее из условий (32.2)).

5. Для каждого домена строится весовая функция $\mathcal{X}$, удовлетворяющая условию (32.4). В простейшем случае можно использовать среднее арифметическое: $\mathcal{X}_i = 1/N_d$ - где N_d – количество доменов, содерижащих текущую точку области. На рис. 19 справа представлена такая функция. В зависимости от ячейки её значения варьируются между $1, 1/2, 1/3, 1/4, 0$. Отметим, что в слое фиктивных ячеек весовая функция равна нулю, так как этот формальный слой используется только для удобного задания условий Дирихле при сборке СЛАУ методом конечных объёмов.

32.0.3.2 MPI реализация конечнообъёмного метода Шварца

Ниже представлена MPI реализаиция метода Шварца в параллельной постановке

```
98 int main(int argc, char** argv){
99   int mpi_rank, mpi_size;
100
101   // Инициализация MPI
102   MPI_Init(&argc, &argv);
103   // получить ранг и общее количество рангов
104   MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
105   MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
106
107   // построить сетку в области
108   Grid grid = build_grid();
109
110   // глобальные вектора
111   std::vector<double> rhs = build_rhs(grid); // includes global bc
112   std::vector<double> u = std::vector<double>(grid.n_cells(), 0);
113
114   // построить локальную сетку в домене
115   SubGrid local_grid = grid.subgrid(mpi_rank);
116
```

```
// построить локальную матрицу левой части
Matrix local_mat = build_lhs(local_grid); // includes global and local bc
MatrixSolver local_solver = build_local_solver(local_mat);
// локальные вектора
std::vector<double> local_rhs = local_grid.project(rhs);
std::vector<double> local_u = local_grid.project(u);

for (size_t it=0; it < 1'000; ++it){
  // установка граничных условий в ghost-узлах
  for (size_t local_ghost: local_grid.ghost_cells()){
    size_t global_ghost = local_grid.to_global(local_ghost);
    local_rhs[local_ghost] = u[global_ghost];
  }

  // невязка и условие выхода
  double local_residual = local_mat.residual(local_u, local_rhs);
  double global_residual;
  MPI_Allreduce(
    &local_residual,  // Отправляемый буфер
    &global_residual, // Принимающий буфер
    1,                // Количество элементов
    MPI_DOUBLE,       // Тип данных
    MPI_MAX,          // Операция
    MPI_COMM_WORLD);
  if (global_residual < 1e-6) {
    MPI_Barrier(MPI_COMM_WORLD); // Синхронизация перед выходом
    break;
  }

  // Решение локальной системы
  local_u = local_solver.solve(local_rhs);

  // Расширяем локальный вектор до глобального с помощью оператора продолжения
  std::vector<double> local_u_extended = local_grid.extend_with_xi(local_u);

  // Записать итог
  MPI_Allreduce(
    local_u_extended.data(),  // Отправляемый буфер
    u.data(),                 // Принимающий буфер
    u.size(),                 // Количество элементов
    MPI_DOUBLE,               // Тип данных
    MPI_SUM,                  // Операция
```

```
159       MPI_COMM_WORLD);
160
161     // информация
162     if (mpi_rank == 0){
163       std::cout << it << " res= " << global_residual << std::endl;
164       grid.save_data(u, "data" + std::to_string(it));
165     }
166   }
167
168   MPI_Finalize();
169 }
```

32.0.4 Задание для самостоятельной работы

В файле `"poisson_ddm_test.cpp"` реализован тест `"[poisson2-fvm-ddm]"`, в котором решено уравнение Пуассона в единичном квадрате на неструктурированной сетке. Используется метод Шварца в последовательной постановке.

В программе после чтения сетки тестовая задача сначала решается на одном домене. Для сравнения с результатами DDM-метода. Файл с сеткой ищется либо в директории, в которой находится программа, либо в корневой для репозитория папке `test_data`.

Далее строятся поддомены на основе регулярного разбиения квадрата на основе алгоритма, описанного в 32.0.3.1. В строке

```
352   GridPartition gridpart = GridPartition::build_uniform<2>(grid, {1, 2}, 2, 1);
```

задаётся разбиение области. Аргументы этого вызова соответствуют

- `grid` – исходная сетка,
- `{1, 2}` – регулярное разбиение домена. В данном случае по оси x используется один участок, по оси y – два. То есть построено два поддомена примерно соответсвующих областям $y \in [0, 0.5]$ и $y \in [0.5, 1.0]$.
- `2` – количество буферных слоёв ячеек. Для первого домена буферная зона начинается за $y > 0.5$.
- `1` – количество слоёв с фиктивными ячейками (ghost cells).

Далее строится средняя арифметическая весовая функция для каждого поддомена `Xi` и строится аппроксимация задач (32.2) в каждом домене. Итогом сборки являются матрицы `local_mat`, правые части `local_rhs` и решатели СЛАУ `local_solver`.

Далее происходит инициализация сохранения итерационного решения в vtk `writer`. Программа печатает в консоль невязку после каждой из итераций и создаёт файл `schwarz_ddm_poisson.vtk.series` для Paraview, в котором можно посмотреть как само решение на итерационном слое (поле solution) так и отличие от решения, полученного на едином домене (поле error).

Потом начинаются итерации Шварца. Для каждого домена обновляются граничные условия на местах состыковки доменов (условия в фиктивных ячейках – аналог последнего из условий (32.2))

и решается СЛАУ. Поскольку представлена последовательная реализация алгоритма, то после каждого решения в домена происходит обновление глобального решения по формуле (32.3).

Завершают итеарцию вывод информации и сохранением решения.

32.0.4.1 Необходимо

1. Проиллюстрировать сходимость последовательного метода Шварца с помощью визуализации решения и ошибки. Использовать подробную сетку (например, tetragrid с 10 тысяч ячеек). Использовать разбиение по доменам 3×3.

2. Изменить программу для реализации параллельного метода Шварца. Аналогично проиллюстрировать его работу.

3. Построить сравнительные графики уменьшения невязки с продвижением по итерациям для разных величин буферных зоны. Использовать последовательный и параллельный методы.

4. Реализовать параллельную версию алгоритма Шварца на MPI. Замерить время работы отдельных составных частей внутри итерационного цикла: построение правой части, вычисление невязки, решение локальной СЛАУ, обновление глобального решения. Вывести результаты для нескольких разбиений: например 1×2, 2×2, 4×4, 8×8. Интересует как общее время по всем итерациям, так и время, нормированное на одну итерацию (то есть общее время, делённое на количество итераций).

32.0.4.2 Рекомендации к программированию

Для изменения алгоритма с последовательного на параллельный достаточно вынести блок , отвечающий за обновление глобального решения (`//update global solution`), из цикла по доменам (`idom`).

Тестовая MPI программа (слинкованная с библиотекой `cfd25`) лежит в папке `src/mpi_test` . Чтобы активировать её сборку нужно запустить cmake с флагом (если из папки `build`)

```
cmake .. -DCMAKE_BUILD_TYPE=Release -DMPI_TESTS=ON
```

Для сборки MPI программы необходимо установить в систему реализацию MPI (например OpenMPI, MSMPI, MPICH и др.)

После этого процедура сборки будет вместе с иходной программой `cfd25_test` также будет собирать программу `mpi_cfd25_test` . В этой программе реализован простой тест (`simple_mpi_test`) на сложение чисел, лежащих в разных потоках с помощью редукции. Если система настроена правильно, то эта программа должна запускаться и не давать ошибок. Например, запуск на 4 потоках

```
mpirun -H localhost:4 mpi_cfd25_test
```

В этой же программе в файле `ddm_poisson_test` есть функция `schwarz_ddm_poisson_test` которая читает расчётную сетку. Программу расчёта нужно дописывать в эту функцию. При переводе последовательной программы в параллельную необходимо избавиться

от всех циклов по доменам (`idom`). Теперь индексу домена будет соответствовать индекс ранга (`mpi_rank`). Также все переменные, заданные в домене (именованные по шаблону `local_*`) должны перестать быть векторами. Например, `local_mat` в оригинальной программе – это массив из матриц (`vector<CsrMatrix>`). В MPI программе это будет просто матрица (`CsrMatrix`) для текущего ранга. Полную проблему (`full_worker`) в MPI версии делать не обязательно. Соответственно и сравнения с полной и DDM результатов тоже не нужно (расчёт поля `error`).

Замер времени расчёта нужно проводить только в одном ранге (нулевом).

Количество потоков, с которыми запускается программа должна соответсвовать количеству доменов. Например, если вы используете разбиение по дооменам 2×2, то программу нужно запускать на 4-х потоках.

A Формулы и обозначения

A.1 Векторы

A.1.1 Обозначение

Геометрические вектора обозначаются жирным шрифтом $\mathbf{v}$. Скалярные координаты вектора – через нижний индекс с обозначением оси координат: (v_x, v_y, v_z). Если вектор $\mathbf{u}$ – вектор скорости, то его декартовые координаты имеют специальное обозначение $\mathbf{u} = (u, v, w)$. Единичные вектора, соответствующие осям координат, обозначаются знаком $\hat{\cdot}$: $\hat{\mathbf{x}}$, $\hat{\mathbf{y}}$, $\hat{\mathbf{z}}$. Координатные векторы обозначаются по символу первой оси. Например, $\mathbf{x} = (x, y, z)$ или $\boldsymbol{\xi} = (\xi, \eta, \zeta)$.

Операции в векторами имеют следующее обозначение (расписывая в декартовых координатах):

- Умножение на скалярную функцию

$$f\mathbf{u} = (fu_x)\hat{\mathbf{x}} + (fu_y)\hat{\mathbf{y}} + (fu_z)\hat{\mathbf{z}}; \tag{A.1}$$

- Скалярное произведение

$$\mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y + u_z v_z; \tag{A.2}$$

- Векторное произведение

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \hat{\mathbf{x}} & \hat{\mathbf{y}} & \hat{\mathbf{z}} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = (u_y v_z - u_z v_y)\,\hat{\mathbf{x}} - (u_x v_z - u_z v_x)\,\hat{\mathbf{y}} + (u_x v_y - u_y v_x)\,\hat{\mathbf{z}}. \tag{A.3}$$

В двумерном случае можно считать, что $u_z = v_z = 0$. Тогда результатом векторного произведения согласно (A.3) будет вектор, направленный перпендикулярно плоскости xy:

$$\mathbf{u} \times \mathbf{v} = (u_x v_y - u_y v_x)\hat{\mathbf{z}}.$$

При работе с двумерными задачами, где ось $\mathbf{z}$ отсутствует, обычно результатом векторного произведения считают скаляр

$$2D: \ \mathbf{u} \times \mathbf{v} = u_x v_y - u_y v_x. \tag{A.4}$$

Геометрический смысл этого скаляра: площадь параллелограмма, построенного на векторах $\mathbf{u}$ и $\mathbf{v}$.

A.1.2 Набла–нотация

Символ ∇ – есть псевдовектор, который выражает покоординатные производные. Для декартовой системы координат (x, y, z) он запишется в виде

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right).$$

В радиальной (r, ϕ, z):

$$\nabla = \left(\frac{\partial}{\partial r}, \frac{1}{r}\frac{\partial}{\partial \phi}, \frac{\partial}{\partial z} \right).$$

В цилиндрической (r, θ, ϕ):

$$\nabla = \left(\frac{\partial}{\partial r}, \frac{1}{r}\frac{\partial}{\partial \theta}, \frac{1}{r\sin\theta}\frac{\partial}{\partial \phi} \right).$$

Удобство записи дифференциальных выражений с использованием ∇ заключается в независимости записи от вида системы координат. Но если требуется обозначить производную по конкретной координате, то, по аналогии с обычными векторами, это делается через нижний индекс:

$$\nabla_n f = \frac{\partial f}{\partial n}.$$

Для этого символа справедливы все векторные операции, описанные ранее. Так, применение ∇ к скалярной функции аналогично умножению вектора на скаляр (A.1) (здесь и далее приводятся покоординатные выражения для декартовой системы):

$$\nabla f = (\nabla_x f,\ \nabla_y f,\ \nabla_z f) = \frac{\partial f}{\partial x}\hat{\mathbf{x}} + \frac{\partial f}{\partial y}\hat{\mathbf{y}} + \frac{\partial f}{\partial z}\hat{\mathbf{z}}. \tag{A.5}$$

Результатом этой операции является вектор.

Скалярное умножение ∇ на вектор $\mathbf{v}$ по аналогии с (A.2) – есть дивергенция:

$$\nabla \cdot \mathbf{v} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} + \frac{\partial v_z}{\partial z} \tag{A.6}$$

результат которой – скалярная функция.

Двойное применение ∇ к скалярной функции – это оператор Лапласа:

$$\nabla \cdot \nabla f = \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2} \tag{A.7}$$

Ротор – аналог векторного умножнения (A.3):

$$\nabla \times \mathbf{v} = \begin{vmatrix} \hat{\mathbf{x}} & \hat{\mathbf{y}} & \hat{\mathbf{z}} \\ \nabla_x & \nabla_y & \nabla_z \\ v_x & v_y & v_z \end{vmatrix} = \left(\frac{\partial v_z}{\partial y} - \frac{\partial v_y}{\partial z} \right) \hat{\mathbf{x}} - \left(\frac{\partial v_z}{\partial x} - \frac{\partial v_x}{\partial z} \right) \hat{\mathbf{y}} + \left(\frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y} \right) \hat{\mathbf{z}}. \tag{A.8}$$

A.2 Интегрирование

A.2.1 Формула Гаусса–Остроградского

Формула Гаусса–Остроградского, связывающая интегрирование по объёму E с интегрированием по границе этого объёма Г, для векторного поля $\mathbf{v}$ имеет вид

$$\int\limits_E \nabla \cdot \mathbf{v}\, d\mathbf{x} = \int\limits_\Gamma v_n\, ds, \tag{A.9}$$

где $\mathbf{n}$ – внешняя по отношению к области E нормаль. Смысл этой формулы можно проиллюстрировать на одномерном примере. Пусть одномерное векторное поле $v_x = f(x)$ на отрезке $E = [a, b]$ задано функцией, представленной на рис. 20. Разобъем область на $N = 3$ равномерных подобласти

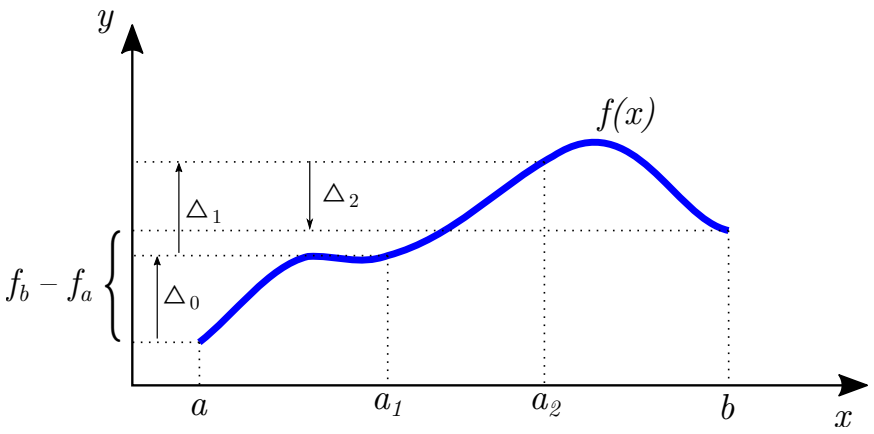


Рис. 20: Формула Гаусса-Остроградского в одномерном случае

длины h. Тогда расписывая интеграл как сумму, а производную через конечную разность, получим

$$\int\limits_E \frac{\partial f}{\partial x}\, dx \approx \sum_{i=0}^{2} h \left(\frac{\partial f}{\partial x}\right)_{i+\frac{1}{2}} \approx \sum_{i=0}^{2} (f_{i+1} - f_i) = \triangle_0 + \triangle_1 + \triangle_2 = f_b - f_a.$$

Очевидно что, при устремлении $N \to \infty$ правая часть предыдущего выражения не изменится. То есть, сумма всех изменений функции в области есть изменение функции по её границам:

$$\int\limits_a^b \frac{\partial f}{\partial x}\, dx = f(b) - f(a).$$

А формула (A.9) – есть многомерное обобщение этого выражения.

A.2.2 Интегрирование по частям

Подставив в (A.9) $\mathbf{v} = f\mathbf{u}$, где f – некоторая скалярная функция, и расписав дивергенцию в виде

$$\nabla \cdot (f\mathbf{u}) = f\nabla\mathbf{u} + \mathbf{u} \cdot \nabla f$$

получим формулу интегрирования по частям

$$\int\limits_E \mathbf{u} \cdot \nabla f \, d\mathbf{x} = \int\limits_\Gamma f u_n \, ds - \int\limits_E f \nabla \cdot \mathbf{u} \, d\mathbf{x} \tag{A.10}$$

Распишем некоторые частные случаи для формулы (A.10). Для $\mathbf{u} = (n_x, 0, 0)$ получим

$$\int\limits_E \frac{\partial f}{\partial x} \, d\mathbf{x} = \int\limits_\Gamma f \cos(\widehat{\mathbf{n}, \mathbf{x}}) \, ds \tag{A.11}$$

При $\mathbf{u} = \nabla g$

$$\int\limits_E f \left(\nabla^2 g\right) \, d\mathbf{x} = \int\limits_\Gamma f \frac{\partial g}{\partial n} \, ds - \int\limits_E \nabla f \cdot \nabla g \, d\mathbf{x} \tag{A.12}$$

При $f = 1$ и $\mathbf{u} = \nabla g$

$$\int\limits_E \nabla^2 g \, d\mathbf{x} = \int\limits_\Gamma \frac{\partial g}{\partial n} \, ds \tag{A.13}$$

A.2.3 Численное интегрирование в заданной области

Квадратурная формула

$$\int\limits_E f(\mathbf{x}) \, d\mathbf{x} = \sum_{i=0}^{N-1} w_i f(\mathbf{x}_i) \tag{A.14}$$

Она определяется заданием узлов интегрирования $\mathbf{x}_i$ и соответствующих весов w_i.

A.3 Интерполяционные полиномы

A.3.1 Многочлен Лагранжа

A.3.1.1 Узловые базисные функции

Рассмотрим функцию $f(\xi)$, заданную в области D. Внутри этой области зададим N узловых точек $\xi_i, i = \overline{0, N-1}$. Приближение функции f будем искать в виде

$$f(\xi) \approx \sum_{i=0}^{N-1} f_i \phi_i(\xi), \tag{A.15}$$

где $f_i = f(\xi_i)$, ϕ_i – узловая базисная функция. Потребуем, чтобы это выражение выполнялось точно для всех заданных узлов интерполяции $\xi = \xi_i$. Тогда, исходя из определения (A.15), запишем условие на узловую базисную функцию

$$\phi_i(\xi_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases} \tag{A.16}$$

Дополнительно потребуем, чтобы формула (A.15) была точной для постоянных функций

$$f(\xi) = \text{const} \quad \Rightarrow \quad f_i = \text{const}.$$

Тогда для любого ξ должно выполняться условие

$$\sum_{i=0}^{N-1} \phi_i(\xi) = 1, \qquad \xi \in D. \tag{A.17}$$

Задача построения интерполяционной функции состоит в конкретном определении узловых базисов $\phi_i(\xi)$ по заданному набору узловых точек ξ_i и значениям функции в них f_i. Будем искать базисы в виде многочленов вида

$$\phi_i(\xi) = \sum_a A_i^{(a)} \xi^a = A_i^{(0)} + A_i^{(1)} \xi + A_i^{(2)} \xi^2 + \ldots, \qquad i = \overline{0, N-1}. \tag{A.18}$$

Определять коэффициенты $A_i^{(a)}$ будем из условий (A.16), которое даёт N линейных уравнений относительно неизвестных $A_i^{(a)}$ для каждого $i = \overline{0, N-1}$. Таким образом, в выражениях (A.18) должно быть ровно N слагаемых. Будем использовать последовательный набор степеней: $a = \overline{0, N-1}$. Выпишем систему линейных уравнений для 0-ой базисной функции

$$\begin{aligned}
&\phi_0(\xi_0) = A_0^{(0)} + A_0^{(1)} \xi_0 + A_0^{(2)} \xi_0^2 + A_0^{(3)} \xi_0^3 + \ldots = 1, \\
&\phi_0(\xi_1) = A_0^{(0)} + A_0^{(1)} \xi_1 + A_0^{(2)} \xi_1^2 + A_0^{(3)} \xi_1^3 + \ldots = 0, \\
&\phi_0(\xi_2) = A_0^{(0)} + A_0^{(1)} \xi_2 + A_0^{(2)} \xi_2^2 + A_0^{(3)} \xi_2^3 + \ldots = 0, \\
&\ldots
\end{aligned}$$

или в матричном виде

$$\begin{pmatrix} 1 & \xi_0 & \xi_0^2 & \xi_0^3 & \dots \\ 1 & \xi_1 & \xi_1^2 & \xi_1^3 & \dots \\ 1 & \xi_2 & \xi_2^2 & \xi_2^3 & \dots \\ 1 & \xi_3 & \xi_3^2 & \xi_3^3 & \dots \\ \dots & & & & \end{pmatrix} \begin{pmatrix} A_0^{(0)} \\ A_0^{(1)} \\ A_0^{(2)} \\ A_0^{(3)} \\ \vdots \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \vdots \end{pmatrix}$$

Записывая аналогичные выражения для остальных базисных функций, получим систему матричных уравнений вида $CA = E$:

$$\begin{pmatrix} 1 & \xi_0 & \xi_0^2 & \xi_0^3 & \dots \\ 1 & \xi_1 & \xi_1^2 & \xi_1^3 & \dots \\ 1 & \xi_2 & \xi_2^2 & \xi_2^3 & \dots \\ 1 & \xi_3 & \xi_3^2 & \xi_3^3 & \dots \\ \dots & & & & \end{pmatrix} \begin{pmatrix} A_0^{(0)} & A_1^{(0)} & A_2^{(0)} & A_3^{(0)} & \dots \\ A_0^{(1)} & A_1^{(1)} & A_2^{(1)} & A_3^{(1)} & \\ A_0^{(2)} & A_1^{(2)} & A_2^{(2)} & A_3^{(2)} & \\ A_0^{(3)} & A_1^{(3)} & A_2^{(3)} & A_3^{(3)} & \\ \vdots & & & & \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & \dots \\ 0 & 1 & 0 & 0 & \\ 0 & 0 & 1 & 0 & \\ 0 & 0 & 0 & 1 & \\ \vdots & & & & \end{pmatrix}$$

Отсюда матрица неизвестных коэффициентов A определится как

$$A = C^{-1} = \begin{pmatrix} 1 & \xi_0 & \xi_0^2 & \xi_0^3 & \dots \\ 1 & \xi_1 & \xi_1^2 & \xi_1^3 & \dots \\ 1 & \xi_2 & \xi_2^2 & \xi_2^3 & \dots \\ 1 & \xi_3 & \xi_3^2 & \xi_3^3 & \dots \\ \dots & & & & \end{pmatrix}^{-1}. \tag{A.19}$$

Подставляя полином (A.18) в условие согласованности (A.17), получим требование

$$\sum_{i=0}^{N-1} A_i^{(a)} = \begin{cases} 1, & a = 0, \\ 0, & a = \overline{1, N-1}. \end{cases}$$

То есть сумма всех свободных членов в интерполяционных полиномах должна быть равна единице, а сумма коэффициентов при остальных степенях – нулю. Можно показать, что это свойство выполняется для любой матрицы $A = C^{-1}$, в случае, если первый столбец матрицы C состоит из единиц. То есть условие согласованности требует наличие свободного члена с интерполяционном полиноме.

A.3.1.2 Интерполяция в параметрическом отрезке

Будем рассматривать область интерполяции $D = [-1, 1]$. В качестве первых двух узлов интерполяции возьмем границы области: $\xi_0 = -1$, $\xi_1 = 1$.

Линейный базис Будем искать интерполяционный базис в виде

$$\phi_i(\xi) = A_i^{(0)} + A_i^{(1)}\xi.$$

на основе двух условий:

$$\phi_i(-1) = A_i^{(0)} - A_i^{(1)} = \delta_{0i}, \quad \phi_i(1) = A_i^{(0)} + A_i^{(1)}\delta_{1i}.$$

Составим матрицу C, записав эти условия в матричном виде

$$C = \left(\begin{array}{c|cc} & A^{(0)} & A^{(1)} \\ \hline \phi(-1) & 1 & -1 \\ \phi(1) & 1 & 1 \end{array}\right)$$

и, согласно (A.19), найдём матрицу коэффициентов

$$A = \begin{pmatrix} A_0^{(0)} & A_1^{(0)} \\ A_0^{(1)} & A_1^{(1)} \end{pmatrix} = C^{-1} = \left(\begin{array}{c|cc} & \phi_0 & \phi_1 \\ \hline 1 & \frac{1}{2} & \frac{1}{2} \\ \xi & -\frac{1}{2} & \frac{1}{2} \end{array}\right).$$

Отсюда узловые базисные функции примут вид (рис. 21)

$$\begin{aligned} \phi_0(\xi) &= \frac{1-\xi}{2}, \\ \phi_1(\xi) &= \frac{1+\xi}{2}. \end{aligned} \tag{A.20}$$

Окончательно интерполяционная функция из определения (A.15) примет вид

$$f(\xi) \approx \frac{1-\xi}{2} f(-1) + \frac{1+\xi}{2} f(1).$$

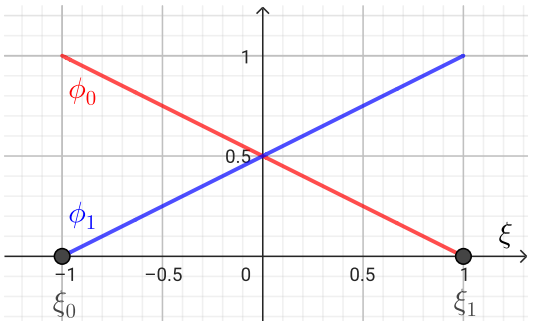


Рис. 21: Линейный базис в параметрическом отрезке

Квадратичный базис Будем искать интерполяционный базис в виде

$$\phi_i(\xi) = A_i^{(0)} + A_i^{(1)}\xi + A_i^{(2)}\xi^2.$$

По сравнению с линейным случаем, в форму базиса добавился ещё один неизвестый коэффициент $A_i^{(2)}$, поэтому в набор условий (A.16) требуется ещё одно уравнение (ещё одна узловая точка). Поме-

стим её в центр параметрического сегмента $\xi_2 = 0$. Далее будем действовать по аналогии с линейным случаем:

$$C = \left(\begin{array}{c|ccc} & A^{(0)} & A^{(1)} & A^{(2)} \\ \hline \phi(-1) & 1 & -1 & 1 \\ \phi(1) & 1 & 1 & 1 \\ \phi(0) & 1 & 0 & 0 \end{array}\right) \quad \Rightarrow \quad A = C^{-1} = \left(\begin{array}{c|ccc} & \phi_0 & \phi_1 & \phi_2 \\ \hline 1 & 0 & 0 & 1 \\ \xi & -\frac{1}{2} & \frac{1}{2} & 0 \\ \xi^2 & \frac{1}{2} & \frac{1}{2} & -1 \end{array}\right).$$

Узловые базисные функции для квадратичной интерполяции примут вид (рис. 22)

$$\begin{aligned} \phi_0(\xi) &= \frac{\xi^2 - \xi}{2}, \\ \phi_1(\xi) &= \frac{\xi^2 + \xi}{2}, \\ \phi_2(\xi) &= 1 - \xi^2. \end{aligned} \tag{A.21}$$

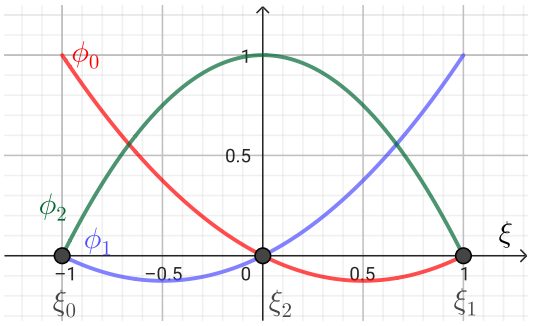


Рис. 22: Квадратичный базис в параметрическом отрезке

Кубический базис Интерполяционный базис будет иметь вид

$$\phi_i(\xi) = A_i^{(0)} + A_i^{(1)}\xi + A_i^{(2)}\xi^2 + A_i^{(3)}\xi^3.$$

Для нахождения четырёх коэффициентов нам понадобится четыре узла интерполяции. Две из них – это границы параметрического отрезка. Остальные две разместим так, чтобы разбить отрезок на равные интервалы: $\xi_2 = -\frac{1}{3}$, $\xi_3 = \frac{1}{3}$. Далее вычислим матрицу коэффициентов:

$$C = \left(\begin{array}{c|cccc} & A^{(0)} & A^{(1)} & A^{(2)} & A^{(3)} \\ \hline \phi(-1) & 1 & -1 & 1 & -1 \\ \phi(1) & 1 & 1 & 1 & 1 \\ \phi(-\frac{1}{3}) & 1 & -\frac{1}{3} & \frac{1}{9} & -\frac{1}{27} \\ \phi(\frac{1}{3}) & 1 & \frac{1}{3} & \frac{1}{9} & \frac{1}{27} \end{array}\right) \quad \Rightarrow \quad A = C^{-1} = \frac{1}{16}\left(\begin{array}{c|cccc} & \phi_0 & \phi_1 & \phi_2 & \phi_3 \\ \hline 1 & -1 & -1 & 9 & 9 \\ \xi & 1 & -1 & -27 & 27 \\ \xi^2 & 9 & 9 & -9 & -9 \\ \xi^3 & -9 & 9 & 27 & -27 \end{array}\right)$$

Узловые базисные функции для квадратичной интерполяции примут вид (рис. 23)

$$\begin{aligned}
\phi_0(\xi) &= \frac{1}{16}\left(-1+\xi+9\xi^2-9\xi^3\right), \\
\phi_1(\xi) &= \frac{1}{16}\left(-1-\xi+9\xi^2+9\xi^3\right), \\
\phi_2(\xi) &= \frac{1}{16}\left(9-27\xi-9\xi^2+27\xi^3\right), \\
\phi_3(\xi) &= \frac{1}{16}\left(9+27\xi-9\xi^2-27\xi^3\right),
\end{aligned} \tag{A.22}$$

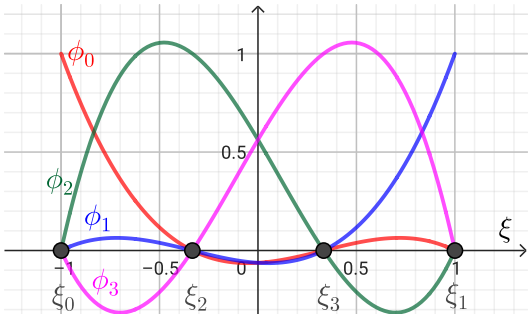


Рис. 23: Кубический базис в параметрическом отрезке

На рис. 24 представлено сравнение результатов аппроксимации функции $f(x) = -x + \sin(2x+1)$ линейным, квадратичным и кубическим базисом. Видно, что все интерполяционные приближения точно попадают в функцию в своих узлах интерполяции, а между узлами происходит аппроксимация полиномом соответствующей степени.

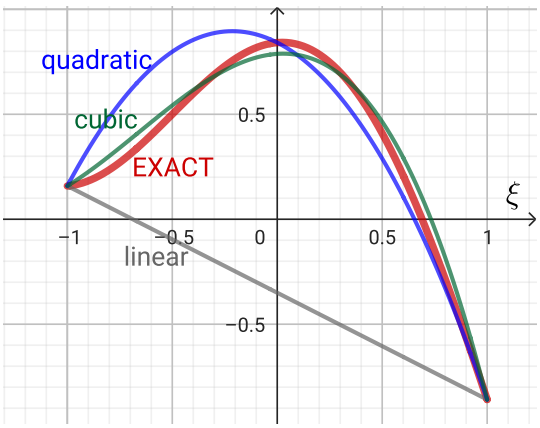


Рис. 24: Результат интерполяции

A.3.1.3 Интерполяция в параметрическом треугольнике

Теперь рассмотрим двумерное обобщение формулы

Линейный базис

$$\phi_i(\xi,\eta) = A_i^{(00)} + A_i^{(10)}\xi + A_i^{(01)}\eta.$$

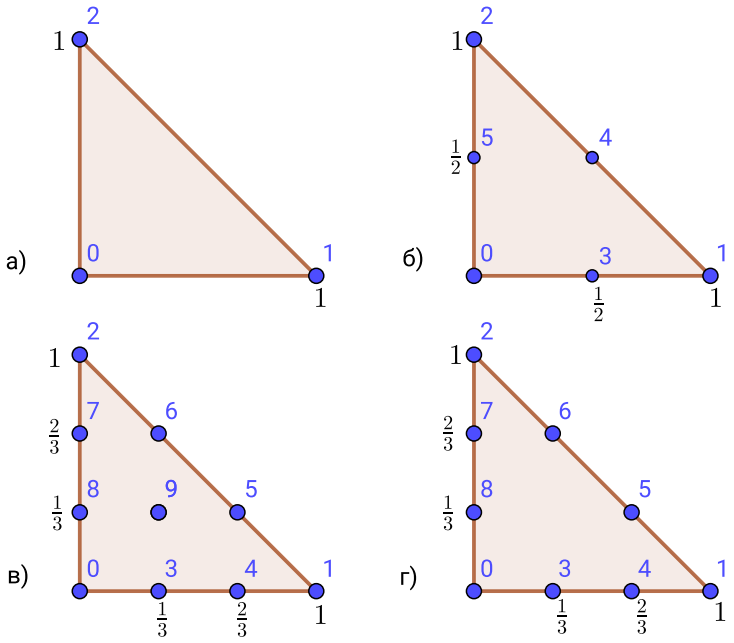


Рис. 25: Расположение узловых точек в параметрическом треугольнике. а) линейный базис, б) квадратичный базис, в) кубический базис, г) неполный кубический базис

$$C = \left(\begin{array}{c|ccc} & A^{(00)} & A^{(10)} & A^{(01)} \\ \hline \phi(0,0) & 1 & 0 & 0 \\ \phi(1,0) & 1 & 1 & 0 \\ \phi(0,1) & 1 & 0 & 1 \end{array}\right) \quad \Rightarrow \quad A = C^{-1} = \left(\begin{array}{c|ccc} & \phi_0 & \phi_1 & \phi_2 \\ \hline 1 & 1 & 0 & 0 \\ \xi & -1 & 1 & 0 \\ \eta & -1 & 0 & 1 \end{array}\right)$$

$$\begin{aligned} \phi_0(\xi,\eta) &= 1-\xi-\eta, \\ \phi_1(\xi,\eta) &= \xi, \\ \phi_2(\xi,\eta) &= \eta, \end{aligned} \tag{A.23}$$

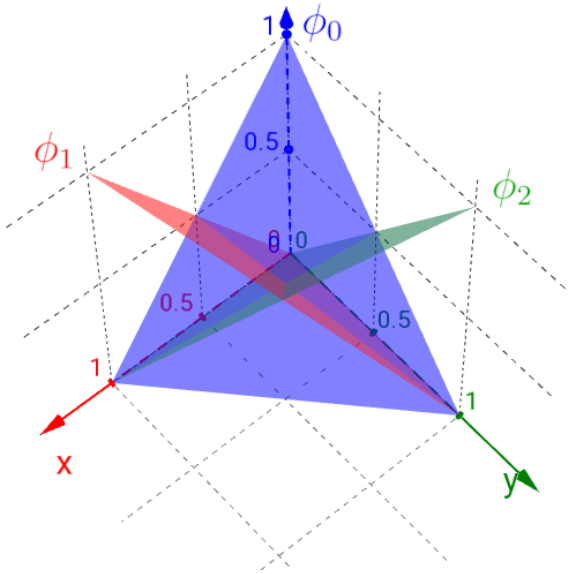


Рис. 26: Линейный базис в параметрическом треугольнике

Квадратичный базис

$$\phi_i(\xi,\eta) = A_i^{(00)} + A_i^{(10)}\xi + A_i^{(01)}\eta + A_i^{(11)}\xi\eta + A_i^{(20)}\xi^2 + A_i^{(02)}\eta^2.$$

$$C = \left(\begin{array}{c|cccccc} & A^{(00)} & A^{(10)} & A^{(01)} & A^{(11)} & A^{(20)} & A^{(02)} \\ \hline \phi(0,0) & 1 & 0 & 0 & 0 & 0 & 0 \\ \phi(1,0) & 1 & 1 & 0 & 0 & 1 & 0 \\ \phi(0,1) & 1 & 0 & 1 & 0 & 0 & 1 \\ \phi(\frac{1}{2},0) & 1 & \frac{1}{2} & 0 & 0 & \frac{1}{4} & 0 \\ \phi(\frac{1}{2},\frac{1}{2}) & 1 & \frac{1}{2} & \frac{1}{2} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \phi(0,\frac{1}{2}) & 1 & 0 & \frac{1}{2} & 0 & 0 & \frac{1}{4} \end{array}\right) \quad \Rightarrow \quad A = \left(\begin{array}{c|cccccc} & \phi_0 & \phi_1 & \phi_2 & \phi_3 & \phi_4 & \phi_5 \\ \hline 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ \xi & -3 & -1 & 0 & 4 & 0 & 0 \\ \eta & -3 & 0 & -1 & 0 & 0 & 4 \\ \xi\eta & 4 & 0 & 0 & -4 & 4 & -4 \\ \xi^2 & 2 & 2 & 0 & -4 & 0 & 0 \\ \eta^2 & 2 & 0 & 2 & 0 & 0 & -4 \end{array}\right)$$

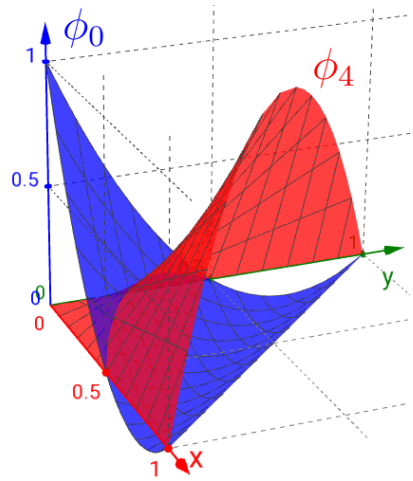


Рис. 27: Квадратичные функции ϕ_0, ϕ_4 в параметрическом треугольнике

Кубический базис TODO

Неполный кубический базис TODO

А.3.1.4 Интерполяция в параметрическом квадрате

Билинейный базис

$$\phi_i = A_i^{00} + A_i^{10}\xi + A_i^{01}\eta + A_i^{11}\xi\eta.$$

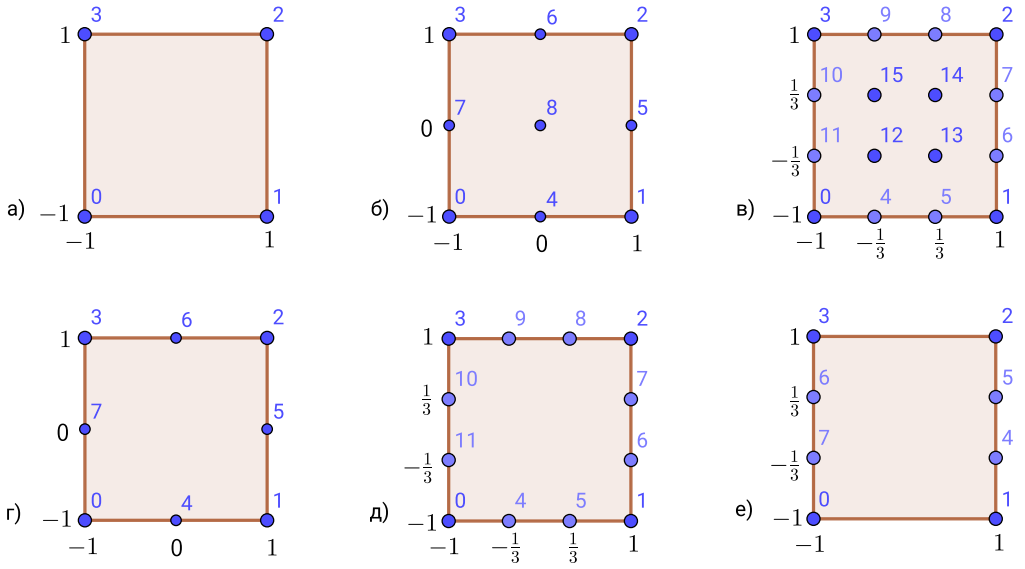


Рис. 28: Расположение узловых точек в параметрическом квадратре

$$C = \left(\begin{array}{c|cccc} & A^{(00)} & A^{(10)} & A^{(01)} & A^{(11)} \\ \hline \phi(-1,-1) & 1 & -1 & -1 & 1 \\ \phi(1,-1) & 1 & 1 & -1 & -1 \\ \phi(1,1) & 1 & 1 & 1 & 1 \\ \phi(-1,1) & 1 & -1 & 1 & -1 \end{array}\right) \quad \Rightarrow \quad A = C^{-1} = \frac{1}{4}\left(\begin{array}{c|cccc} & \phi_0 & \phi_1 & \phi_2 & \phi_3 \\ \hline 1 & 1 & 1 & 1 & 1 \\ \xi & -1 & 1 & 1 & -1 \\ \eta & -1 & -1 & 1 & 1 \\ \xi\eta & 1 & -1 & 1 & -1 \end{array}\right)$$

$$\begin{aligned} \phi_0(\xi,\eta) &= \frac{1-\xi-\eta+\xi\eta}{4} \\ \phi_1(\xi,\eta) &= \frac{1+\xi-\eta-\xi\eta}{4} \\ \phi_2(\xi,\eta) &= \frac{1+\xi+\eta+\xi\eta}{4} \\ \phi_3(\xi,\eta) &= \frac{1-\xi+\eta-\xi\eta}{4} \end{aligned} \tag{A.24}$$

Определение двумерных базисов через комбинацию одномерных Обратим внимание, что в искомые билинейные базисные функции линейны в каждом из направлений ξ, η, если брать их по отдельности. Значит можно представить эти функции как комбинацию одномерных линейных базисов (A.20) в каждом из направлений. Узлы двумерного параметрического квадрата можно выразить через узлы линейного базиса в параметрическом одномерном сегменте, рассмотренном в п. A.3.1.2:

$$\boldsymbol{\xi}_0 = \left(\xi_0^{1D}, \xi_0^{1D}\right), \quad \boldsymbol{\xi}_1 = \left(\xi_1^{1D}, \xi_0^{1D}\right), \quad \boldsymbol{\xi}_2 = \left(\xi_1^{1D}, \xi_1^{1D}\right), \quad \boldsymbol{\xi}_3 = \left(\xi_0^{1D}, \xi_1^{1D}\right).$$

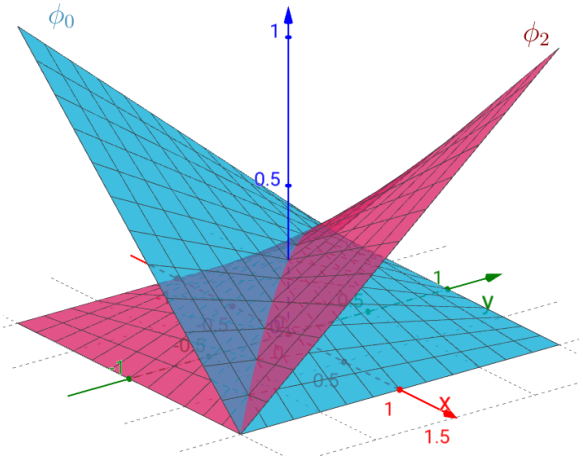


Рис. 29: Билинейные функции ϕ_0, ϕ_2 в параметрическом квадрате

Значит и соответствующие базисные функции можно выразить через линейный одномерный базис ϕ^{1D} из соотношений (A.20):

$$\phi_0(\xi,\eta) = \phi_0^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{1-\xi}{2}\frac{1-\eta}{2},$$
$$\phi_1(\xi,\eta) = \phi_1^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{1+\xi}{2}\frac{1-\eta}{2},$$
$$\phi_2(\xi,\eta) = \phi_1^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{1+\xi}{2}\frac{1+\eta}{2},$$
$$\phi_3(\xi,\eta) = \phi_0^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{1-\xi}{2}\frac{1+\eta}{2}.$$

Раскрыв скобки можно убедится, что мы получили тот же билинейный базис, что и ранее (A.24).

Биквадратичный базис Применим этот метод для вычисления биквадратичного базиса, определённого в точках на рис. 28б. В качестве основе возьмём квадратичный одномерный базис ϕ_i^{1D} из (A.21).

$$\begin{aligned}
&\phi_0(\xi,\eta) = \phi_0^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{\xi^2-\xi}{2}\frac{\eta^2-\eta}{2}, \qquad \phi_1(\xi,\eta) = \phi_1^{1D}(\xi)\phi_0^{1D}(\eta) = \frac{\xi^2+\xi}{2}\frac{\eta^2-\eta}{2},\\
&\phi_2(\xi,\eta) = \phi_1^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{\xi^2+\xi}{2}\frac{\eta^2+\eta}{2}, \qquad \phi_3(\xi,\eta) = \phi_0^{1D}(\xi)\phi_1^{1D}(\eta) = \frac{\xi^2-\xi}{2}\frac{\eta^2+\eta}{2},\\
&\phi_4(\xi,\eta) = \phi_2^{1D}(\xi)\phi_0^{1D}(\eta) = (1-\xi^2)\frac{\eta^2-\eta}{2}, \qquad \phi_5(\xi,\eta) = \phi_1^{1D}(\xi)\phi_2^{1D}(\eta) = \frac{\xi^2+\xi}{2}(1-\eta^2),\\
&\phi_6(\xi,\eta) = \phi_2^{1D}(\xi)\phi_1^{1D}(\eta) = (1-\xi^2)\frac{\eta^2+\eta}{2}, \qquad \phi_7(\xi,\eta) = \phi_0^{1D}(\xi)\phi_2^{1D}(\eta) = \frac{\xi^2-\xi}{2}(1-\eta^2),\\
&\phi_8(\xi,\eta) = \phi_2^{1D}(\xi)\phi_2^{1D}(\eta) = (1-\xi^2)(1-\eta^2).
\end{aligned} \tag{A.25}$$

Бикубический базис

Неполный биквадратичный базис

Неполный бикубический базис

A.4 Геометрические алгоритмы

A.4.1 Линейная интерполяция

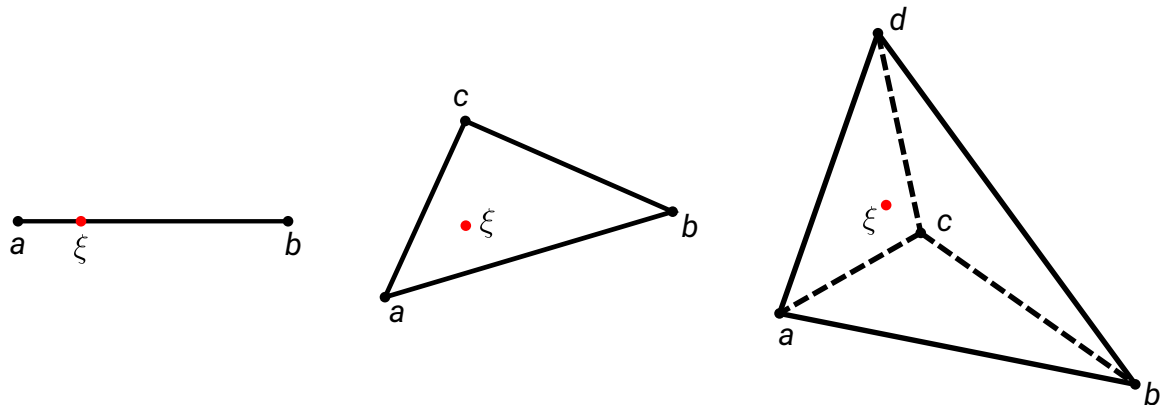


Рис. 30: Порядок нумерации точек одномерного, двумерного и трёхмерного симплекса при линейной интерполяции

Пусть функция u задана в узлах симплекса, имеющего нумерацию согласно рис. 30. Необходимо найти значение этой функции в точке $\boldsymbol{\xi}$ (эта точка вообще говоря не обязана лежать внутри симлекса).

Интерполяция в одномерном, двумерном и трёхмерном виде запишется как

$$u(\xi) = \frac{|\triangle_{\xi a}| u(b) + |\triangle_{b\xi}| u(a)}{|\triangle_{ba}|} \tag{A.26}$$

$$u(\xi) = \frac{|\triangle_{ab\xi}| u(c) + |\triangle_{bc\xi}| u(a) + |\triangle_{ca\xi}| u(b)}{|\triangle_{abc}|} \tag{A.27}$$

$$u(\xi) = \frac{|\triangle_{abc\xi}| u(d) + |\triangle_{cbd\xi}| u(a) + |\triangle_{cda\xi}| u(b) + |\triangle_{adb\xi}| u(c)}{|\triangle_{abcd}|}, \tag{A.28}$$

где $|\triangle|$ – знаковый объём симплекса, вычисляемый как

$$|\triangle_{ab}| = b - a,$$

$$|\triangle_{abc}| = \left(\frac{(\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a})}{2} \right)_z,$$

$$|\triangle_{abcd}| = \frac{(\mathbf{b}-\mathbf{a}) \cdot ((\mathbf{c}-\mathbf{a}) \times (\mathbf{d}-\mathbf{a}))}{6}.$$

A.4.2 Преобразование координат

Рассмотрим преобразование из двумерной параметрической системы координат $\boldsymbol{\xi}$ в физическую систему $\mathbf{x}$. Такое преобразование полностью определяется покоординатными функциями $\mathbf{x}(\boldsymbol{\xi})$. Далее получим соотношения, связывающие операции дифференцирования и интегрирования в физической и параметрической областях.

A.4.2.1 Матрица Якоби

Будем рассматривать двумерное преобразование $(\xi, \eta) \to (x, y)$. Линеаризуем это преобразование (разложим в ряд Фурье до линейного слагаемого)

$$x(\xi_0 + d\xi, \eta_0 + d\eta) \approx x_0 + \left.\frac{\partial x}{\partial \xi}\right|_{\xi_0,\eta_0} d\xi + \left.\frac{\partial x}{\partial \eta}\right|_{\xi_0,\eta_0} d\eta,$$
$$y(\xi_0 + d\xi, \eta_0 + d\eta) \approx y_0 + \left.\frac{\partial y}{\partial \xi}\right|_{\xi_0,\eta_0} d\xi + \left.\frac{\partial y}{\partial \eta}\right|_{\xi_0,\eta_0} d\eta,$$

где $x_0 = x(\xi_0, \eta_0)$, $y_0 = y(\xi_0, \eta_0)$. Переписывая это выражение в векторном виде, получим

$$\mathbf{x}(\boldsymbol{\xi}_0 + \mathbf{d\xi}) - \mathbf{x}_0 = J(\boldsymbol{\xi}_0)\, \mathbf{d\xi}. \tag{A.29}$$

Матрица J (зависящая от точки приложения в параметрической плоскости) называется матрицей Якоби:

$$J = \begin{pmatrix} J_{11} & J_{12} \\ J_{21} & J_{22} \end{pmatrix} = \begin{pmatrix} \dfrac{\partial x}{\partial \xi} & \dfrac{\partial x}{\partial \eta} \\ \dfrac{\partial y}{\partial \xi} & \dfrac{\partial y}{\partial \eta} \end{pmatrix} \tag{A.30}$$

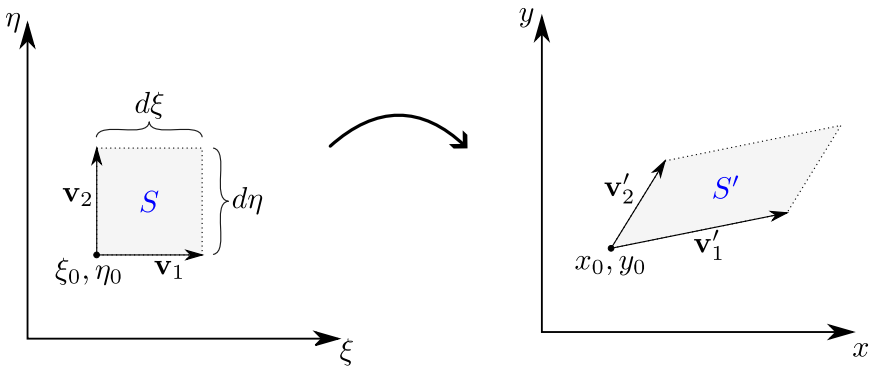


Рис. 31: Преобразование элементарного объёма

Якобиан Определитель матрицы Якоби (якобиан), взятый в конкретной точке параметрической плоскости $\boldsymbol{\xi}_0$, показывает, во сколько раз увеличился элементарный объём около этой точки в результате преобразования. Действительно, рассмотрим два перпендикулярных элементарных вектора в параметрической системе координат: $\mathbf{v}_1 = (d\xi, 0)$ и $\mathbf{v}_2 = (0, d\eta)$ отложенных от точки $\boldsymbol{\xi}_0$ (см. рис. 31). В результате преобразования по формуле (A.29) получим следующие преобразования концевых точек и векторов:

$$\begin{aligned}
&(\xi_0, \eta_0) \to (x_0, y_0), \\
&(\xi_0 + d\xi, \eta_0) \to (x_0 + J_{11} d\xi, y_0 + J_{21} d\xi) \quad \Rightarrow \quad \mathbf{v}_1 \to \mathbf{v}'_1 = (J_{11} d\xi, J_{21} d\xi), \\
&(\xi_0, \eta_0 + d\eta) \to (x_0 + J_{12} d\eta, y_0 + J_{22} d\eta) \quad \Rightarrow \quad \mathbf{v}_2 \to \mathbf{v}'_2 = (J_{12} d\eta, J_{22} d\eta).
\end{aligned}$$

Элементарный объём равен площади параллелограмма, построенного на элементарных векторах. В параметрической плоскости согласно (A.4) получим

$$|S| = \mathbf{v}_1 \times \mathbf{v}_2 = d\xi d\eta,$$

и аналогично для физической плоскости:

$$|S'| = \mathbf{v}'_1 \times \mathbf{v}'_2 = (J_{11}J_{22} - J_{12}J_{21})d\xi d\eta = |J|d\xi d\eta$$

Сравнивая два последних соотношения приходим к выводу, что элементарный объём в результате преобразования увеличился в $|J|$ раз. Тогда можно записать

$$dx\,dy = |J|\,d\xi\,d\eta \tag{A.31}$$

Многомерным обобщением этой формулы будет

$$d\mathbf{x} = |J|\,d\boldsymbol{\xi} \tag{A.32}$$

A.4.2.2 Дифференцирование в параметрической плоскости

Пусть задана некоторая функция $f(x, y)$. Распишем её производную по параметрическим координатам:

$$\frac{\partial f}{\partial \xi} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial \xi} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial \xi},$$
$$\frac{\partial f}{\partial \eta} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial \eta} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial \eta}.$$

Вспоминая определение (A.30), запишем

$$\begin{pmatrix} \dfrac{\partial f}{\partial \xi} \\ \dfrac{\partial f}{\partial \eta} \end{pmatrix} = J^T \begin{pmatrix} \dfrac{\partial f}{\partial x} \\ \dfrac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} J_{11} & J_{21} \\ J_{12} & J_{22} \end{pmatrix} \begin{pmatrix} \dfrac{\partial f}{\partial \xi} \\ \dfrac{\partial f}{\partial \eta} \end{pmatrix}$$

Обратная зависимость примет вид

$$\begin{pmatrix} \dfrac{\partial f}{\partial x} \\ \dfrac{\partial f}{\partial y} \end{pmatrix} = \left(J^T\right)^{-1} \begin{pmatrix} \dfrac{\partial f}{\partial \xi} \\ \dfrac{\partial f}{\partial \eta} \end{pmatrix} = \frac{1}{|J|} \begin{pmatrix} J_{22} & -J_{21} \\ -J_{12} & J_{11} \end{pmatrix} \begin{pmatrix} \dfrac{\partial f}{\partial \xi} \\ \dfrac{\partial f}{\partial \eta} \end{pmatrix}$$

В многомерном виде запишем

$$\nabla_{\mathbf{x}} f = \left(J^T\right)^{-1} \nabla_{\boldsymbol{\xi}} f. \tag{A.33}$$

А.4.2.3 Интегрирование в параметрической плоскости

Пусть в физической области $\mathbf{x}$ задана область D_x. Интеграл функции $f(\mathbf{x})$ по этой области можно расписать, используя замену (А.32)

$$\int\limits_{D_x} f(\mathbf{x})\, d\mathbf{x} = \int\limits_{D_\xi} f(\boldsymbol{\xi})\, |J(\boldsymbol{\xi})| d\boldsymbol{\xi}, \tag{А.34}$$

где $f(\boldsymbol{\xi}) = f(\mathbf{x}(\boldsymbol{\xi}))$, а D_ξ – образ области D_x в параметрической плоскости.

А.4.2.4 Двумерное линейное преобразование. Параметрический треугольник

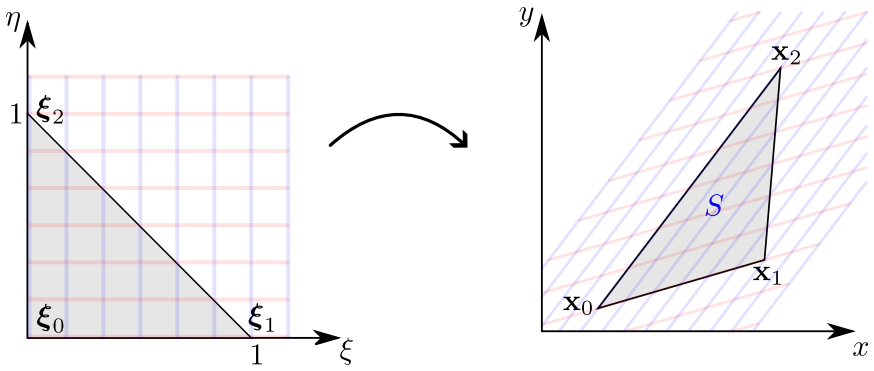


Рис. 32: Преобразование из параметрического треугольника

Рассмотрим двумерное преобразование, при котором определяющие функции являются линейными. То есть представимыми в виде

$$x(\xi, \eta) = A_x \xi + B_x \eta + C_x,$$
$$y(\xi, \eta) = A_y \xi + B_y \eta + C_y.$$

Для определения шести констант, определяющих это преобразование, достаточно выбрать три любые (не лежащие на одной прямой) точки: $(\xi_i, \eta_i) \to (x_i, y_i)$ для $i = 0, 1, 2$. В результате получим систему из шести линейных уравнений (три точки по две координаты), из которой находятся константы $A_{x,y}, B_{x,y}, C_{x,y}$. Пусть три точки в параметрической плоскости образуют единичный прямоугольный треугольник (рис. 32):

$$\xi_0, \eta_0 = (0, 0), \quad \xi_1, \eta_1 = (1, 0), \quad \xi_2, \eta_2 = (0, 1).$$

Тогда система линейных уравнений примет вид

$$x_0 = C_x, \quad y_0 = C_y,$$
$$x_1 = A_x + C_x, \quad y_1 = A_y + C_y,$$
$$y_2 = B_x + C_x, \quad y_2 = B_y + C_y.$$

Определив коэффициенты преобразования их этой системы, окончательно запишем преобразование

$$x(\xi, \eta) = (x_1 - x_0)\xi + (x_2 - x_0)\eta + x_0,$$
$$y(\xi, \eta) = (y_1 - y_0)\xi + (y_2 - y_0)\eta + y_0. \tag{А.35}$$

Матрица Якоби этого преобразования (A.30) не будет зависеть от параметрических координат ξ, η:

$$J = \begin{pmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{pmatrix}. \tag{A.36}$$

Якобиан преобразования будет равен удвоенной площади треугольника S, составленного из определяющих точек в физической плоскости:

$$|J| = (x_1 - x_0)(y_2 - y_0) - (y_1 - y_0)(x_2 - x_0) = (\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0) = 2|S|. \tag{A.37}$$

Распишем интеграл по треугольнику S по формуле (A.34). Вследствии линейности преобразования якобиан постоянен и, поэтому, его можно вынести его из-под интеграла:

$$\int\limits_S f(x,y)\,dxdy = |J| \int\limits_0^1 \int\limits_0^{1-\xi} f(\xi,\eta) d\eta d\xi. \tag{A.38}$$

A.4.2.5 Двумерное билинейное преобразование. Параметрический квадрат

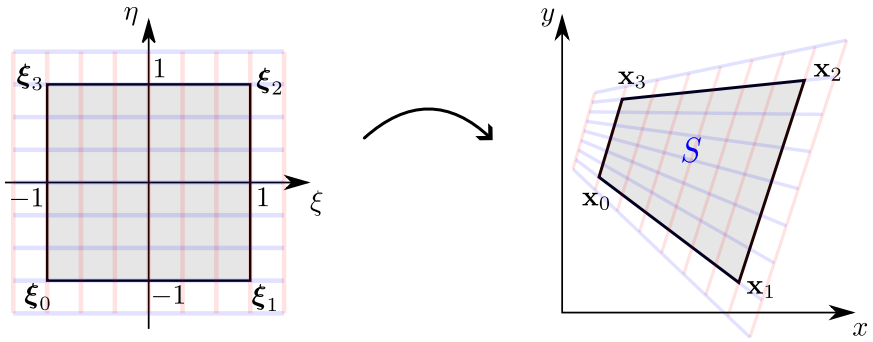


Рис. 33: Преобразование из параметрического квадрата

A.4.2.6 Трёхмерное линейное преобразование. Параметрический тетраэдр

TODO

A.4.3 Свойства многоугольника

A.4.3.1 Площадь многоугольника

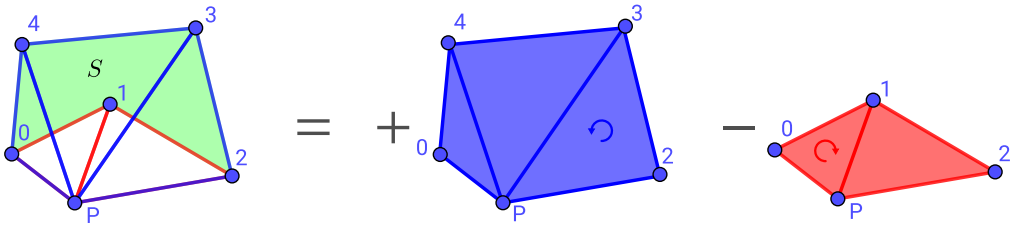


Рис. 34: Площадь произвольного многоугольника

Рассмотрим произвольный несамопересекающийся N-угольник S, заданный координатами своих узлов $\mathbf{x}_i$, $i = \overline{0, N-1}$, пронумерованных последовательно против часовой стрелки (рис. 34). Далее введём произвольную точку $\mathbf{p}$ и от этой точки будем строить ориентированные треугольники до граней многоугольника:

$$\triangle_i^p = (\mathbf{p}, \mathbf{x}_i, \mathbf{x}_{i+1}), \quad i = \overline{0, N-1},$$

(для корректности записи будем считать, что $\mathbf{x}_N = \mathbf{x}_0$). Тогда площадь исходного многоугольника S будет равна сумме знаковых площадей треугольников $\triangle_i^p$:

$$|S| = \sum_{i=0}^{N-1} |\triangle_i^p|, \qquad |\triangle_i^p| = \frac{(\mathbf{x}_i - \mathbf{p}) \times (\mathbf{x}_{i+1} - \mathbf{p})}{2}.$$

Знак площади ориентированного треугольника зависит от направления закрутки его узлов: она положительна для закрутки против часовой стрелки и отрицательна, если узлы пронумерованы по часовой стрелке. В частности, на рисунке 34 видно, что треугольники, отмеченные красным: $P01, P12$, будут иметь отрицательную площадь, а синие треугольники $P23$, $P34$, $P40$ – положительную. Сумма этих площадей с учётом знака даст искомую площадь многоугольника.

Для сокращения вычислений воспользуемся произвольностью положения $\mathbf{p}$ и совместим её с точкой $\mathbf{x}_0$. Тогда треугольники $\triangle_0^p$, $\triangle_{N-1}^p$ выродятся (будут иметь нулевую площадь). Обозначим такую последовательную триангуляцию как

$$\triangle_i = (\mathbf{x}_0, \mathbf{x}_i, \mathbf{x}_{i+1}), \qquad i = \overline{1, N-2}. \tag{A.39}$$

Знаковая площадь ориентированного треугольника будет равна

$$|\triangle_i| = \frac{(\mathbf{x}_i - \mathbf{x}_0) \times (\mathbf{x}_{i+1} - \mathbf{x}_0)}{2}. \tag{A.40}$$

Тогда окончательно формула определения площади примет вид

$$|S| = \sum_{i=1}^{N-2} |\triangle_i|. \tag{A.41}$$

Плоский полигон в пространтве Если плоский полигон S расположен в трёхмерном пространстве, то правая часть формулы (A.40) согласно определению векторного произведения в трёхмерном пространстве (A.3) – есть вектор. Чтобы получить скалярную площадь, нужно спроецировать этот вектор на единичную нормаль к плоскости многоугольника:

$$\mathbf{n} = \frac{\mathbf{k}}{|\mathbf{k}|}, \qquad \mathbf{k} = (\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0).$$

Эта формула записана из предположения, что узел $\mathbf{x}_2$ не лежит на одной прямой с узлами $\mathbf{x}_0$, $\mathbf{x}_1$. Иначе вместо $\mathbf{x}_2$ нужно выбрать любой другой узел, удовлетворяющий этому условию. Тогда площадь ориентированного треугольника, построенного в трёхмерном пространстве запишется через смешанное произведение:

$$|\triangle_i| = \frac{((\mathbf{x}_i - \mathbf{x}_0) \times (\mathbf{x}_{i+1} - \mathbf{x}_0)) \cdot \mathbf{n}}{2}. \tag{A.42}$$

Формула для определения площади полигона (A.41) будет по прежнему верна. При этом итоговый знак величины S будет положительным, если закрутка полигона положительная (против часовой стрелки) при взгляде со стороны вычисленной нормали $\mathbf{n}$.

A.4.3.2 Интеграл по многоугольнику

Рассмотрим интеграл функции $f(x, y)$ по N-угольнику S, заданному последовательными координатами своих узлов $\mathbf{x}_i$. Введём последовательную триангуляцию согласно (A.39). Тогда интеграл по многоугольнику можно расписать как сумму интегралов по ориентированным треугольникам:

$$\int\limits_S f(x,y)\,dxdy = \sum_{i=1}^{N-2} \int\limits_{\triangle_i} f(x,y)\,dxdy. \tag{A.43}$$

Далее для вычисления интегралов в правой части воспользуемся преобразованием к параметрическому треугольнику (п. A.4.2.4). Следуя формуле интегрирования (A.38), распишем интеграл по i-ому треугольнику:

$$\int\limits_{\triangle_i} f(x,y)\,dxdy = |J_i| \int\limits_0^1 \int\limits_0^{1-\xi} f_i(\xi,\eta)\,d\eta d\xi,$$

где якобиан $|J_i|$ согласно (A.37) есть удвоенная площадь ориентированного треугольника $\triangle_i$ (положительная при закрутке против часовой стрелке и отрицательная иначе):

$$|J_i| = 2|\triangle_i| = (\mathbf{x}_i - \mathbf{x}_0) \times (\mathbf{x}_{i+1} - \mathbf{x}_0),$$

а функция $f_i(\xi, \eta)$ есть функция от преобразованных согласно (A.35) переменных:

$$f_i(\xi,\eta) = f\left((\mathbf{x}_i - \mathbf{x}_0)\,\xi + (\mathbf{x}_{i+1} - \mathbf{x}_0)\,\eta + \mathbf{x}_0\right).$$

Окончательно запишем

$$\int\limits_S f(x,y)\,dxdy = 2\sum_{i=1}^{N-2} |\triangle_i| \int\limits_0^1 \int\limits_0^{1-\xi} f_i(\xi,\eta)\,d\eta d\xi. \tag{A.44}$$

Отметим, что эта формула работает и в том случае, когда полигон расположен в трёхмерном пространстве (знаковую площадь при этом следует вычислять по (A.42)).

A.4.3.3 Центр масс многоугольника

По определению, координаты центра масс $\mathbf{c}$ области S равны среднеинтегральным значениям координатных функций. То есть

$$c_x = \frac{1}{|S|}\int\limits_S x\,dxdy, \quad c_y = \frac{1}{|S|}\int\limits_S y\,dxdy.$$

Далее распишем интеграл в правой части через последовательную триангуляцию согласно (A.43) с учётом линейного преобразования (A.35):

$$\begin{aligned}\int\limits_S x\,dxdy &= \sum_{i=1}^{N-2}\int\limits_{\triangle_i} x\,dxdy \\ &= \sum_{i=1}^{N-2}|J_i|\int\limits_0^1\int\limits_0^{1-\xi}((x_i-x_0)\xi+(x_{i+1}-x_0)\eta+x_0)\,d\eta d\xi \\ &= \sum_{i=1}^{N-2}\frac{|J_i|}{2}\frac{x_0+x_i+x_{i+1}}{3} \\ &= \sum_{i=1}^{N-2}|\triangle_i|\frac{x_0+x_i+x_{i+1}}{3}.\end{aligned}$$

Итого, с учётом (A.41), координаты центра масс примут вид

$$\mathbf{c} = \frac{\sum\limits_{i=1}^{N-2}\frac{\mathbf{x}_0+\mathbf{x}_i+\mathbf{x}_{i+1}}{3}|\triangle_i|}{\sum\limits_{i=1}^{N-2}|\triangle_i|}.$$

Если полигон расположен в двумерном пространстве xy, то знаковая площадь треугольников вычисляется по формуле (A.40). В случае трёхмерного пространтва должна использоваться формула (A.42).

A.4.4 Свойства многогранника

A.4.4.1 Объём многогранника

TODO

A.4.4.2 Интеграл по многограннику

TODO

A.4.4.3 Центр масс многогранника

TODO

A.4.5 Поиск многоугольника, содержащего заданную точку

TODO

A.5 Методы решения систем уравнений

Будем рассматривать систему

$$\mathrm{A}u = f. \tag{A.45}$$

A.5.1 Простые итерационные алгоритмы

A.5.1.1 Метод Якоби

TODO

A.5.1.2 Метод Зейделя

TODO

A.5.1.3 Метод последовательных верхних релаксаций SOR

TODO

A.5.2 Метод коррекции поправки

Пусть u^n – некоторое приближённое решение системы (A.45) на итерационном слое. Введём невязки как

$$r^n = f - (\mathrm{A}u)^n \tag{A.46}$$

и поправку для перехода на следующий итерационный слой:

$$\triangle u^n = u^{n+1} - u^n.$$

Обобщённый процесс с предобуславливателем B для исходной системы можно записать в виде

$$\mathrm{B}\triangle u^n = r^n \tag{A.47}$$

Этот процесс является согласованным: если в качестве u^n взять точное решение системы (A.45), то $r^n = 0$, и следовательно, $\triangle u^n = 0$.

Алгоритм решения исходной системы будет иметь следующий вид:

1. задать начальное приближение u^n при $n = 0$
2. вычислить невязку r^n из (A.46)
3. если норма невязки $||r^n|| < \epsilon$, то завершить итерации
4. решив систему (A.47) получить поправку $\triangle u^n = \mathrm{B}^{-1} r^n$.
5. найти следующее приближение $u^{n+1} = u^n + \triangle u^n$
6. увеличить счётчик $n = n + 1$ и перейти на шаг 2.

Конкретный итерационный метод целиком определяется выбором предобуславливаетеля. В качестве предобуславливателя обычно выбирают матрицу, похожую на A, но при этом легкообратимую. Некоторые примеры:

- если выбрать константу $B = 1/\gamma$, то получим простой итерационный процесс Ричардсона с шагом γ.
- диагональная часть исходного оператора $B = \{a_{ii}\}$ приводит к методу Якоби п. A.5.1.1
- нижняя треугольная часть исходной матрицы $B = \{a_{ij}\}, j \leq i$ даёт метод Зейделя п. A.5.1.2.

Если в качестве предобуславливателя взять изходную матрицу A, то итерационный процесс сойдётся за одну итерацию. Действительно из (A.46), (A.47):

$$u^{n+1} = u^n + A^{-1}(f - Au^n) = A^{-1}f$$

Это метод требует прямого обращения только предобуславливаетеля B, но не исходного оператора A, поэтому он может быть применён в том числе и для решения нелинейных систем.

Для обращения самого предобуславливателя в свою очередь так же может быть использован итерационный алгоритм. Этот итерационный процесс будет являться внутриннем, поэтому не будет требовать глубокой сходимости. Зачастую для обращения предобуславливателя достаточно сделать 2-3 поверхностные итерации одним из простых методов из п. A.5.1.

A.5.3 Методы подпространств Крылова

A.5.3.1 Итерационный процесс BiCGStab с предобуславливателем

Инициализация

1. есть начальное приближение u^0
2. вычисляем невязку
$$r_0 = f - \mathrm{A}u_0$$
3. предобуславливаем невязку
$$r'_0 = \mathrm{M}^{-1} r_0$$
4. запоминаем начальную невязку
$$\tilde{r} = r'_0$$
5. инициализируем
$$p'_0 = v'_0 = 0, \quad \rho_0 = \omega_0 = \alpha_0 = 1.0$$

Далее на k-ой итерации

1. проецируем текущую невязку на начальную
$$\rho_k = \tilde{r}^T \cdot r'_{k-1}$$

2. коэффициент β:

$$\beta_k = \frac{\rho_k}{\rho_{k-1}} \frac{\alpha_{k-1}}{\omega_{k-1}}$$

3. Обновляем направление

$$p'_k = r'_{k-1} + \beta_k \left(p'_{k-1} - \omega_{k-1} v'_{k-1} \right)$$

4.

$$v'_k = \mathrm{M}^{-1} \mathrm{A} p'_k$$

5. коэффициент α

$$\alpha_k = \frac{\rho_k}{\tilde{r}^T \cdot v'_k}$$

6. Промежуточный вектор s:

$$s'_k = r'_{k-1} - \alpha_k v'_k$$

7. Промежуточный вектор t

$$t'_k = \mathrm{M}^{-1} \mathrm{A} s'_k$$

8. множитель ω

$$\omega_k = \frac{(t'_k)^T \cdot s'_k}{(t'_k)^T \cdot t'_k}.$$

9. обновляем невязку и решение

$$\begin{aligned} u_k &= u_{k-1} + \alpha_k p'_k + \omega_k s'_k, \\ r'_k &= s'_k - \omega_k t'_k \end{aligned} \tag{A.48}$$

B Работа с инфраструктурой проекта CFDCourse

B.1 Сборка и запуск

B.1.1 Сборка проекта CFDCourse

Описанная ниже процедура собирает проект в отладочной конфигурации. Для проведения необходимых модификаций для сборки релизной версии смотри B.1.3.

B.1.1.1 Подготовка

1. Для сборки проекта необходимо установить `git` и `cmake>=3.0`

   В Windows необходимо скачать и установить диструбутивы:

   - https://github.com/git-for-windows/git/releases/download/v2.43.0.windows.1/Git-2.43.0-64-bit.exe
   - https://github.com/Kitware/CMake/releases/download/v3.28.3/cmake-3.28.3-windows-x86_64.msi

   При установке cmake проследите, что бы путь к `cmake.exe` сохранился в системных путях. Msi установщик спросит об этом в диалоге.

   В **линуксе** используйте менеджеры пакетов, предоставляемые вашим дистрибутивом. Также проследите чтобы были доступны компиллятор `g++` и отладчик `gdb`.

2. Создайте папку в системе для репозиториев. Например `D:/git_repos/`

3. Возьмите необходимые заголовочные библиотеки boost из https://disk.yandex.ru/d/GwTZUvfAqPsZI и распакуйте архив в папку для репозиториев (D:/git_repos/boost). Проследите, чтобы внутри папки boost сразу шли папки с кодом (`accumulators`, `algorithm`, ...) и заголовочные файлы (`align.hpp`, `aligned_storage.hpp`, ...) без дополнительных уровней вложения.

4. Откройте терминал (git bash в Windows).

5. С помощью команды cd в терминале перейдите в папку для репозиториев

   ```
   > cd D:/git_repos
   ```

6. Клонируйте репозиторий

   ```
   > git clone https://github.com/kalininei/CFDCourse25
   ```

   В директории (`D:/git_repos` в примере) появится папка `CFDCourse25`, которая является корневой папкой проекта

B.1.1.2 VisualStudio

1. Создайте папку build в корне проекта CFDCourse25

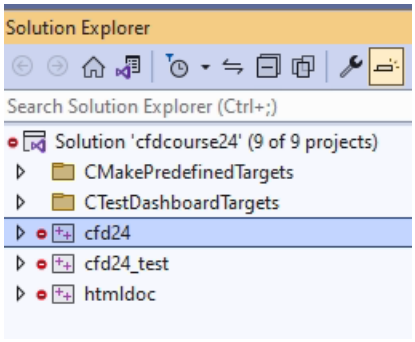
2. Скопируйте скрипт winbuild64.bat в папку build. Далее вносить изменения только в скопированном файле.

3. Скрипт написан для версии `Visual Studio 2019`. Если используется другая версия, измените в скрипте значение переменной `CMGenerator` на соответствующие вашей версии. Значения для разных версий Visual Studio написаны ниже

```
SET CMGenerator="Visual Studio 17 2022"
SET CMGenerator="Visual Studio 16 2019"
SET CMGenerator="Visual Studio 15 2017"
SET CMGenerator="Visual Studio 14 2015"
```

4. Запустите скрипт `winbuild64.bat` из папки `build`. Нужен доступ к интернету. В процессе будет скачано около 200Мб пакетов, поэтому первый запуск может занять время

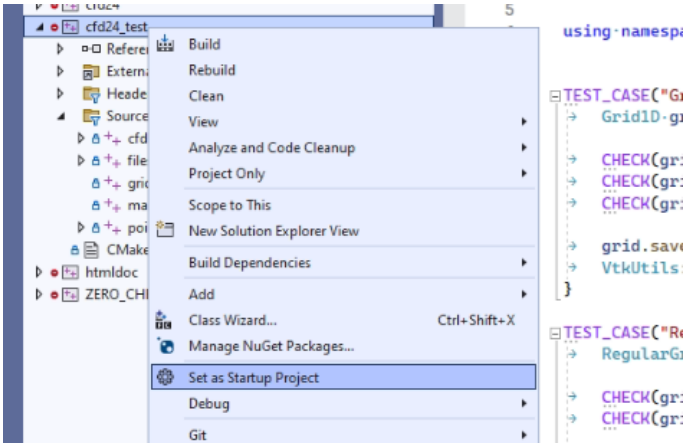
5. После сборки в папке `build` появится проект `VisualStudio` `cfdcourse25.sln`. Его нужно открыть в `VisualStudio`. Дерево решения должно иметь следующий вид:



Проекты:

- `cfd25` – расчётная библиотека
- `cfd25_test` – модульные тесты для расчётных функций

6. Проект `cfd25_test` необходимо назначить запускаемым проектом. Для этого нажать правой кнопкой мыши по проекту и в выпадающем меню выбрать соответствующий пункт. После этого заголовок проекта должен стать жирным.



7. Скомпиллировать решение. Несколько способов:

- `Ctrl+Shift+B`,

- `Build->Build Solution` в основном меню,
- `Build Solution` в меню решения в дереве решения,
- `Build` в меню проекта `cfd25_test`.

8. Запустить тесты (проект `cfd25_test`) нажав `F5` (или кнопку отладки в меню). После отработки должно высветиться сообщение об успешном прохождении всех тестов.

9. Бинарные файлы будут скомпиллированы в папку `CFDCourse25/build/bin/Debug`. В случае работы через отладчик выходная директория, куда будут скидываться все файлы (в частности, vtk), должна быть `CFDCourse25/build/src/test/`.

B.1.1.3 VSCode

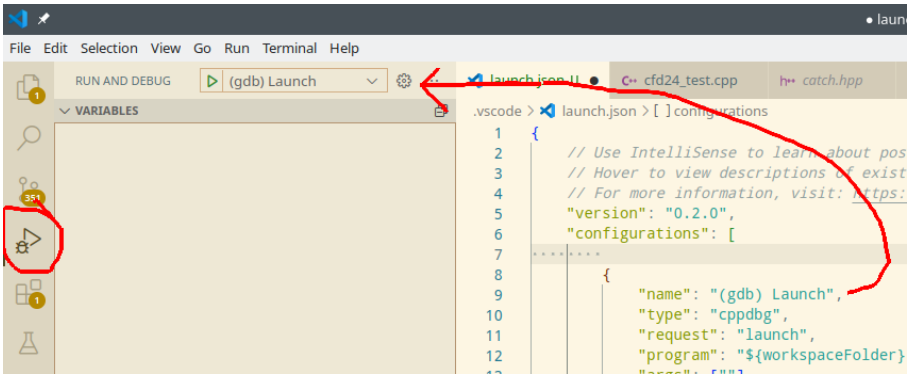
1. Открыть корневую папку проека через `File->Open Folder`

2. Установить предлагаемые расширения cmake, c++

3. Для настройки отладки создайте конфигурацию launch.json следующего вида



- Для этого перейдите в меню `Run and Dubug` (`Ctrl+Shift+D`), нажмите `create launch.json`, выберите пункт `Node.js`.
- После этого в корневой папке появится файл `.vscode/launch.json`.
- Откройте этот файл в `vscode`, нажмите `Add configuration`, `(gdb) Launch` или `(Windows) Launch` в зависимости от ОС.
- Далее напишите имя программы как показано на картинке.
- Используйте поле args для установки аргументов запуска.
- Выберите созданную конфигурацию для запуска отладчика по `F5`



На скриншотах представлены настройки в случае работы в линуксе. Для работы под виндоус

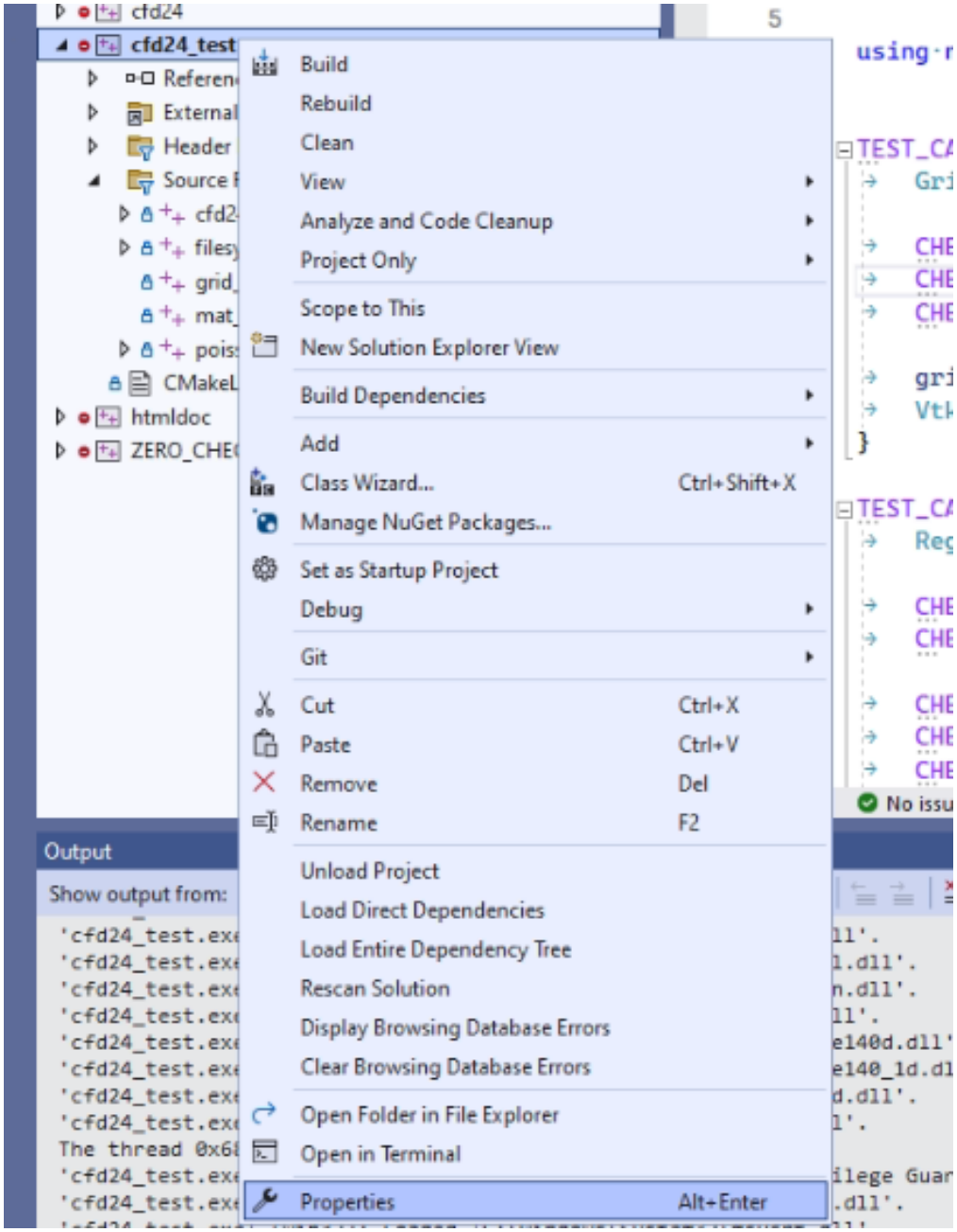
```
"name" : "(Windows) Launch",
"program": "${workspaceFolder}/build/bin/Debug/cfd25_test.exe"
```

B.1.2 Запуск конкретного теста

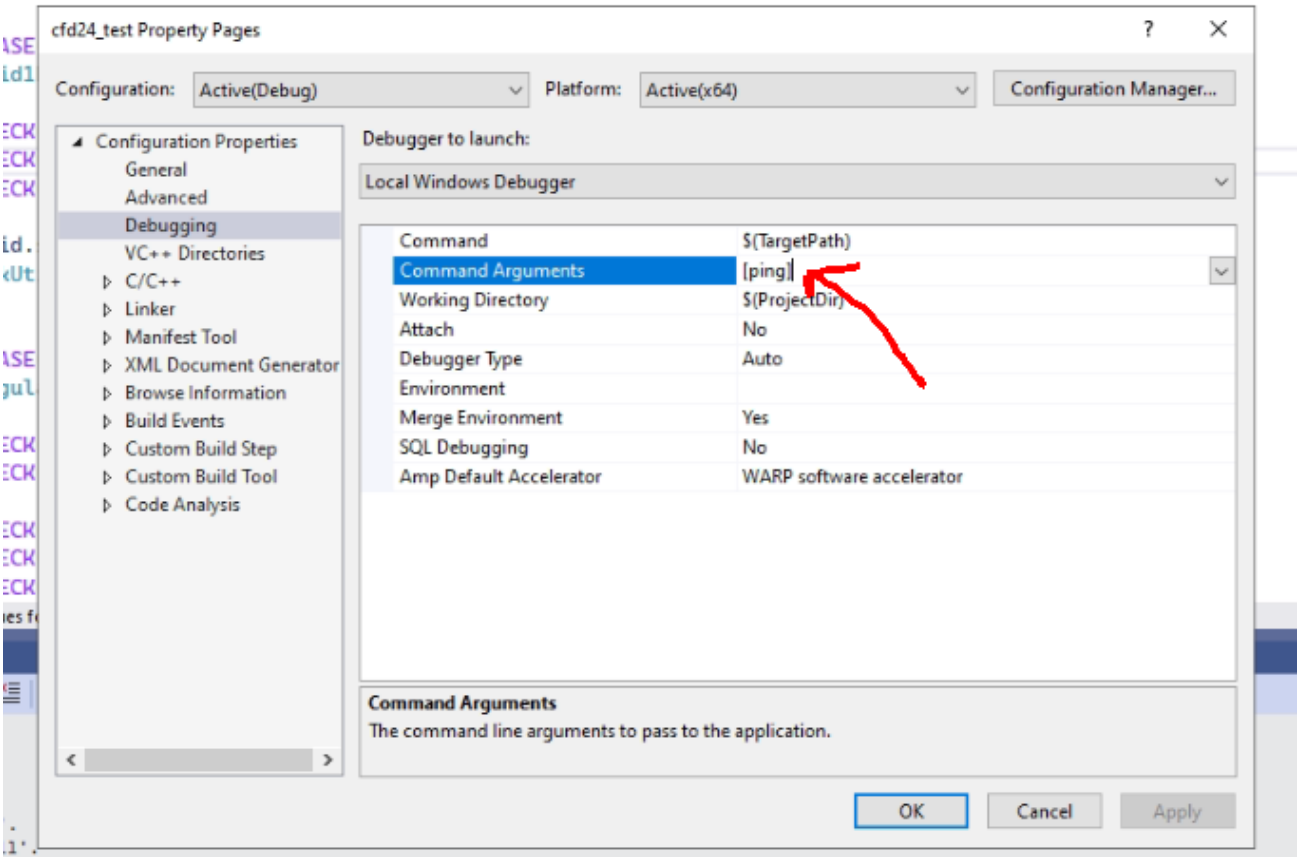
По умолчанию программа `cfd25_test` прогоняет все объявленные в проекте тесты. Иногда может возникнуть необходимость запустить только конкретный тест в целях отладки или проверки. Для этого нужно передать программе аргумент с тегом для этого теста.

Тег для теста – это второй аргумент в макросе `TEST_CASE`, записанный в квадратных скобках. Добавлять нужно вместе со скобками. Например, `[ping]`.

Чтобы добавить аргумент в `VisualStudio`, необходимо в контекстном меню проекта `cfd25_test` выбрать опции отладки



и там в поле Аргументы прописать нужный тэг.



В `VSCode` аргументы нужно добавлять в файле `.vscode/launch.json` в поле args в кавычках (см. картинку B.1.1.3 с настройками launch.json).

B.1.3 Сборка релизной версии

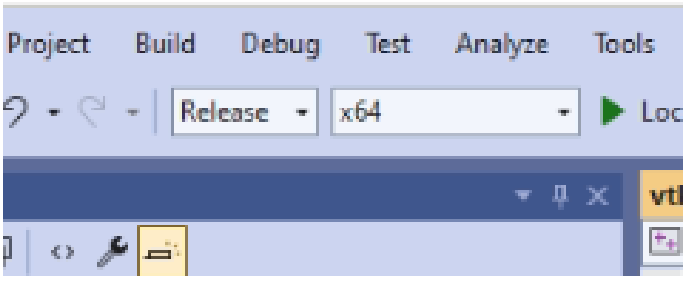
Релизная сборка программ даёт многократное увеличение производительности, но при этом отладка приложений в таком режиме невозможна.

Visual Studio

1. Создать папку `build-release` рядом с папкой `build`.
2. Скопировать в неё файл `winbuild64.bat` из папки `build`.
3. В скопированном файле произвести замену `Debug` на `Release`

```
-DCMAKE_BUILD_TYPE=Release ..
```

4. Запустить `winbuild64.bat` из новой папки
5. Открыть `build-release/cfdcourse25.sln` в `Visual Studio`
6. В проекте студии установить релизную сборку



7. Это новое решение, не связанное настройками с `debug`-версией. Поэтому нужно заново настроить запускускаемым проектом `cfd_test` и, если нужно, настроить аргументы отладки.

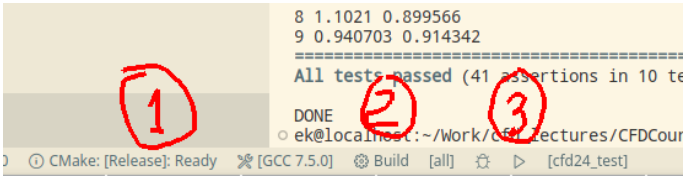
8. Бинарные файлы будут скомпиллированы в папку `CFDCourse25/build_release/bin/Release`. В случае работы через отладчик выходная директория – `CFDCourse25/build_release/src/test/`.

VSCode

1. Выбрать релизную сборку в `build variant`

2. Нажать `Build`

3. Нажать `Launch`



B.2 Git

B.2.1 Основные команды

Все команды выполнять в терминале (`git bash` для виндоус), находясь в корневой папке проета CFDCourse24.

- Для **смены директории** использовать команду `cd`. Например, находясь в папке `A` перейти в папку `A/B/C`

```
> cd B/C
```

- **Подняться** на директорию выше

```
> cd ..
```

- **Просмотр статуса** текущего репозитория: текущую ветку, все изменённые файлы и т.п.

```
> git status
```

- **Сохранить и скоммитить** изменения в текущую ветку

```
> git add .
> git commit -m "message"
```

"message" – произвольная информация о текущем коммите, которая будет приписана к этому коммиту

- **Переключиться на ветку** main

```
> git checkout main
```

работает только в том случае, если все файлы скоммичены и статус ветки 'Up to date'

- **Создать новую ветку** ответвлённую от последнего коммита текущей ветки и переключиться на неё

```
> git checkout -b new-branch-name
```

new-branch-name – имя новой ветки. Пробелы не допускаются

Эта комманда работает даже если есть нескоммиченные изменения. Если необходимо скоммитить изменеия в новую ветку, сразу за этой командой нужно вызвать

```
> git add .
> git commit -m "message"
```

- **Сбросить** все нескоммиченные изменения. Вернуть файлы в состояние последнего коммита

```
> git reset --hard
```

Все изменения будут утеряны

- **Получить последние изменения** из удалённого хранилища с обновлением текущей ветки

```
> git pull
```

Работает только если статус текущей ветки 'Up to date'.
Если требуется получить изменения, но не обновлять локальную ветку:

```
> git fetch
```

Обновленная ветка будет доступна по имени origin/имя ветки.

- **Просмотр истории** коммитов в текущей ветке (последний коммит будет наверху)

```
> git log
```

- **Просмотр доступных веток** в текущем репозитории

```
> git branch
```

- **Просмотр** актуального состояние дерева репозитория в gui режиме

```
> git gui
```

Далее в меню
`Repository->Visualize all branch history`. В этом же окне можно посмотреть изменения файлов по сравнению с последним коммитом.

Альтернативно, при работе в виндоус можно установить программу GitExtensions и работать в ней.

B.2.2 Порядок работы с репозиторием CFDCourse

Основная ветка проекта –
`main`. После каждой лекции в эту ветку будет отправлен коммит с сообщением
`after-lect{index}`. Этот коммит будет содержать краткое содержание лекции, задание по итогам лекции и необходимые для этого задания изменения кода.

Таким образом, **после лекции**, после того, как изменение
`after-lect` придёт на сервер, необходимо выполнить следующие команды (находясь в ветке `main`)

```
> git reset --hard  # очистить локальную копию от изменений,
                    # сделанных на лекции (если они не представляют ценности)
> git pull          # получить изменения
```

Перед началом лекции, если была сделана какая то работа по заданиям,

```
> git checkout -b work-lect{index}    # создать локальную ветку, содержащую задание
> git add .
> git commit -m "{свой комментарий}"  # скоммитить свои изменения в эту ветку
> git checkout main                   # вернуться на ветку main
> git pull                            # получить изменения
```

Даже если задание выполнено не до конца, вы в любой момент можете переключиться на ветку с заданием и его доделать

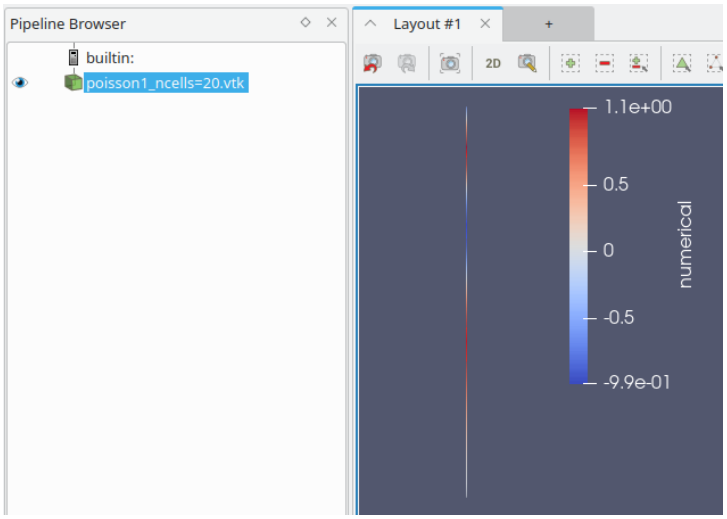
```
> git checkout work-lect{index}
```

Если ничего не было сделано (или все изменения не представляют ценности), можно повторить алгоритм “после лекции”.

B.3 Paraview

B.3.1 Данные на одномерных сетках

Заданные на сетке данные паравью показывает цветом. Поэтому при загрузке одномерных сеток можно видеть картинку типа

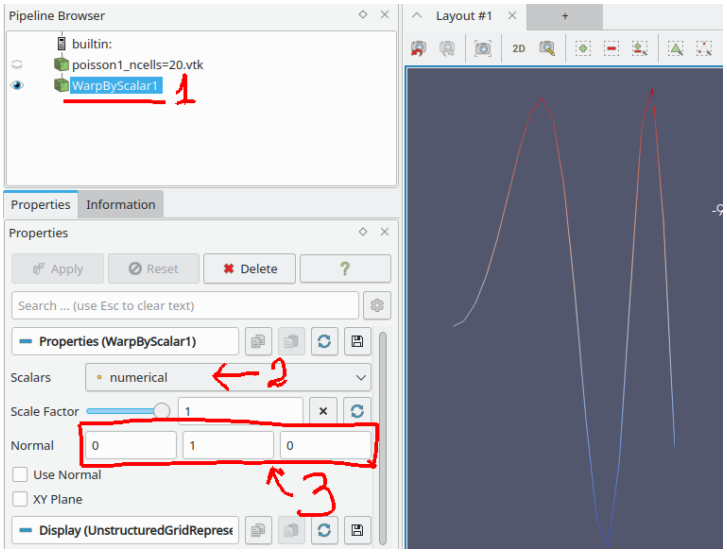


Развернуть изображение в плоскость xy



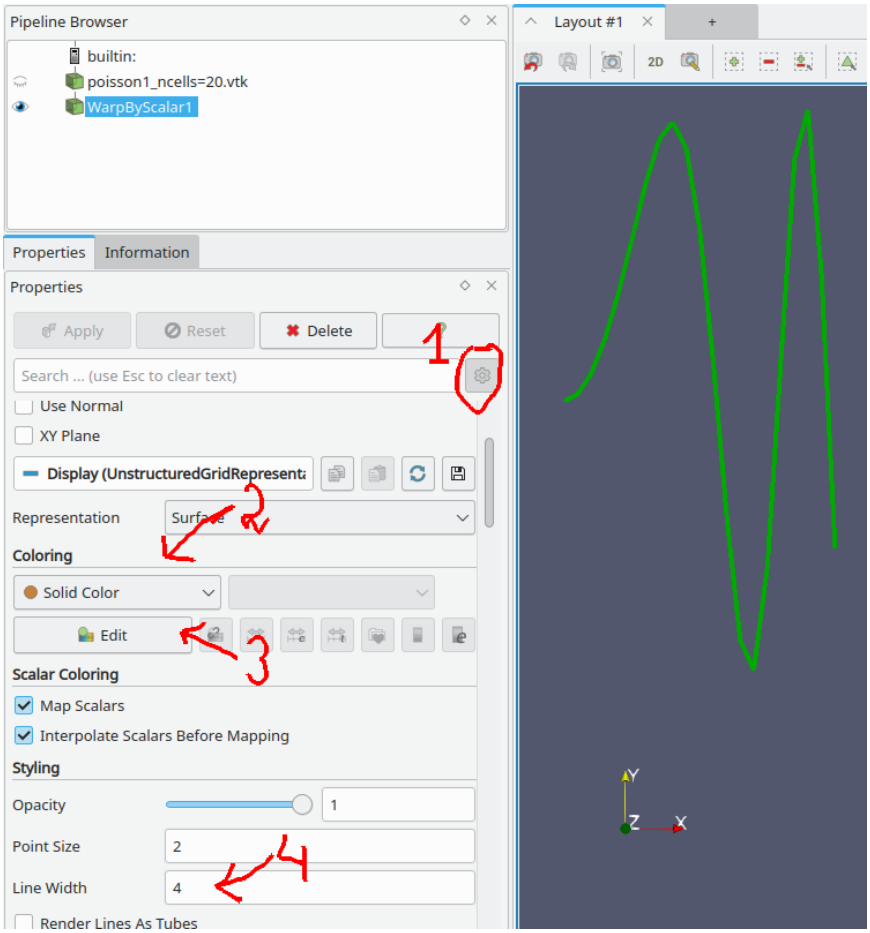
Отобразить данные в виде y-координаты Для того, что бы данные отображались в качестве значения по оси ординат, к загруженному файлу необходимо

1. применить фильтр `WarpByScalar` (В меню `Filters->Alphabetical->Warp By Scalar`)
2. в меню настройки фильтра указать поле данных, для отображения (numerical в примере ниже)
3. И настроить нормаль, вдоль которой будут проецироваться данные (в нашем случае ось y)



Цвет и толщина линии

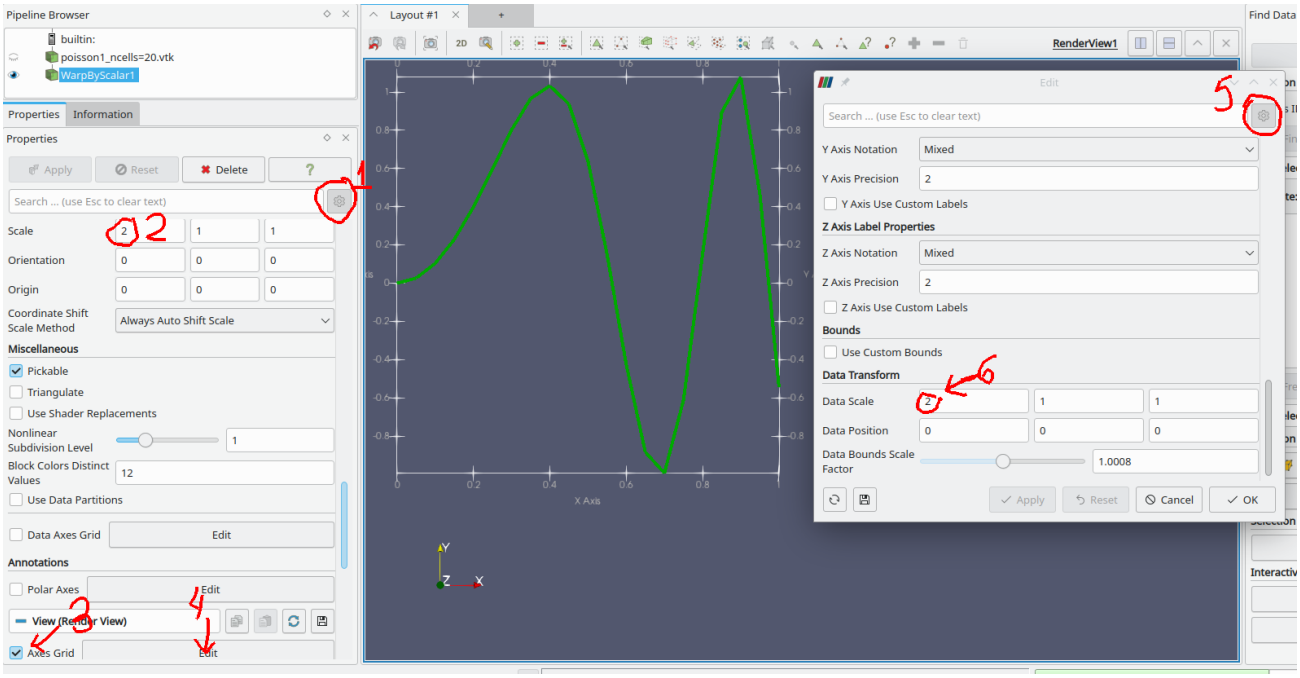
1. Включить подробные опции фильтра
2. Сменить стиль на `Solid Color`
3. В меню `Edit` выбрать желаемый цвет
4. В строке `Line Width` указать толщину линии



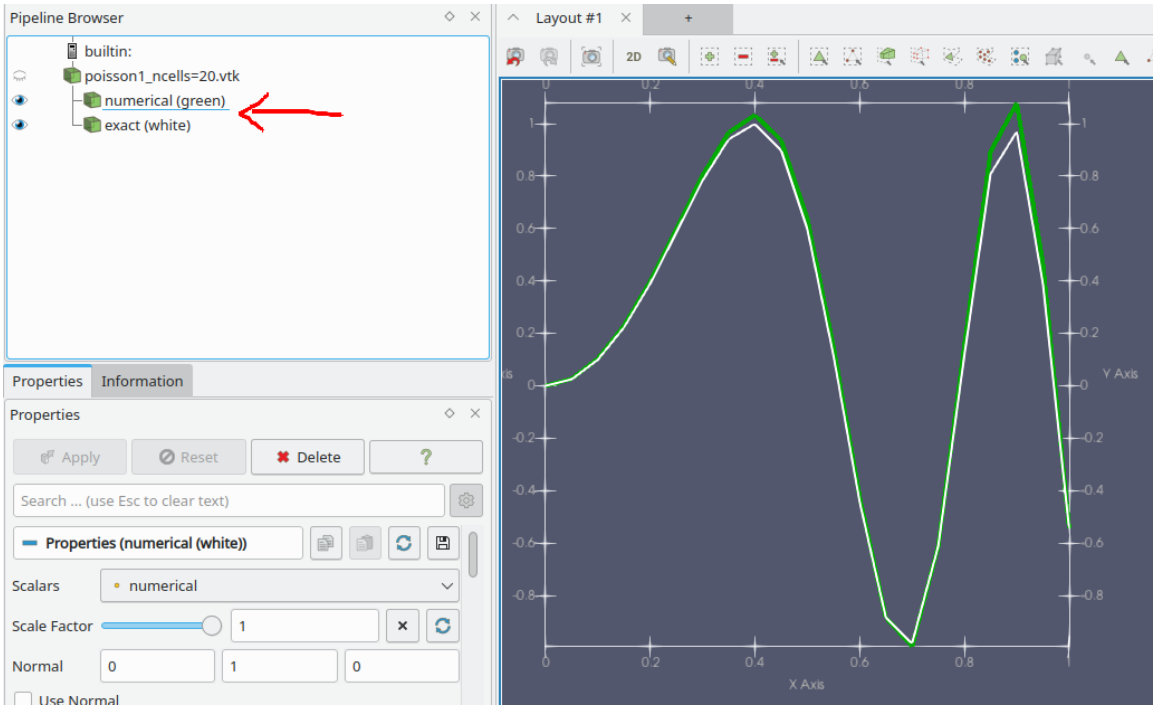
Настрока масштабов и отображение осей координат

1. Отметье подробные настройки фильтра
2. В поле `Transforming/Scale` Установите желаемые масштабы (в нашем случае растянуть в два раза по оси x)
3. Установите галку на отображение осей
4. откройте меню натройки осей
5. В нём включите подробные настроки
6. И также поставьте растяжение осей

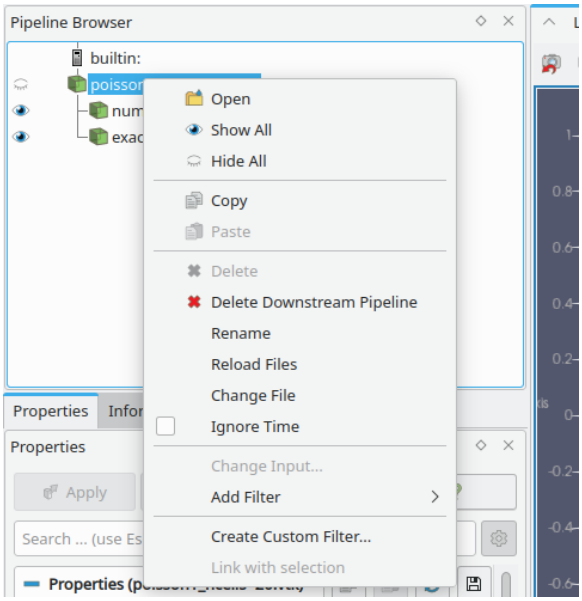
В случае, если масштабировать график не нужно, достаточно выполнить шаг 3.



Построение графиков для нескольких данных Если требуется нарисовать рядом несколько графиков для разных данных из одного файла, примените фильтр `Warp By Scalar` для этого файла ещё раз, изменив поле `Scalars` в настройке фильтра. Для наглядности измените имя узла в Pipeline Browser на осмысленные



Обновление данных при изменении исходного файла В случае, если исходный файл был изменён, нужно в контекстном меню узла соответствующего файла выбрать `Reload Files` (или нажать F5). Если те же самые фильтры нужно применить для просмотра другого файла нужно в этом меню нажать `Change File`.



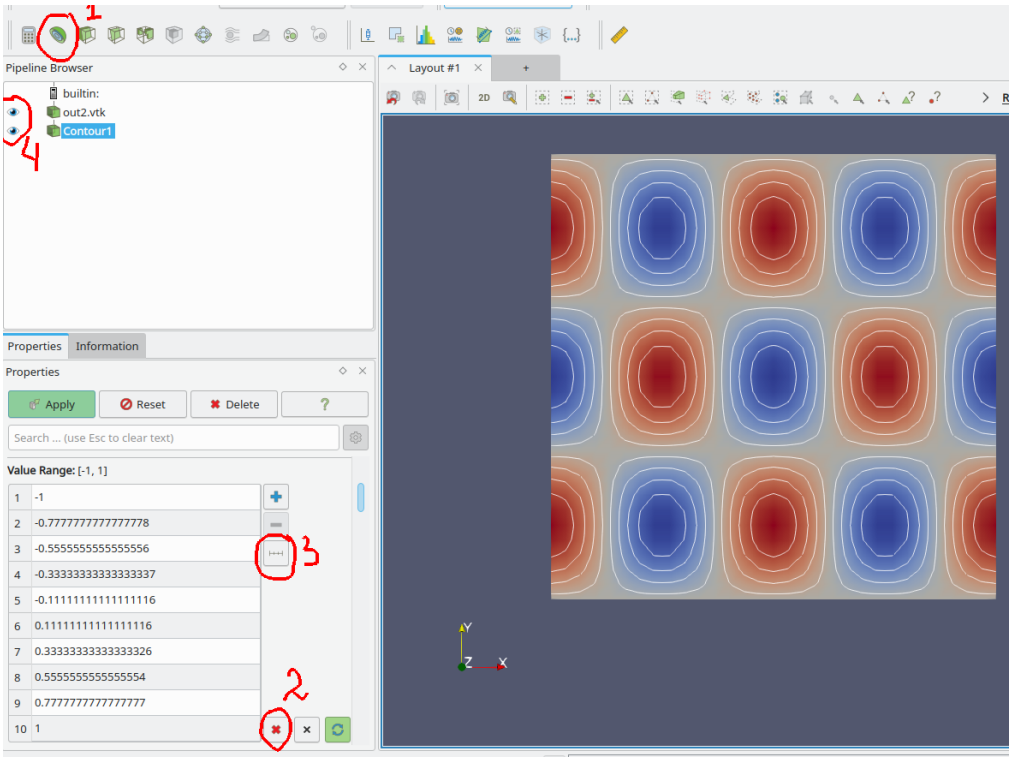
B.3.2 Изолинии для двумерного поля

1. Нажмите иконку `Contour` (или `Filters/Contour`) В настройках фильтра Contour by выберитее данные, по которым нужно строить изолинии.

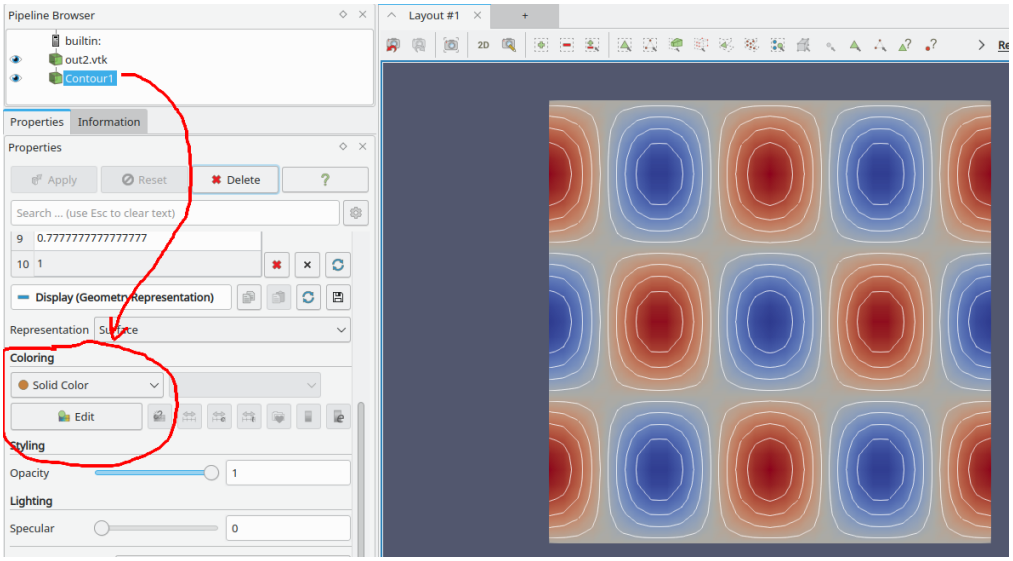
2. В настройках фильтра удалите все существующие записи о значениях для изолиний

3. Добавьте равномерные значения. В появившемся меню установите необходимое количество изолиний и их диапазон.

4. Если необходимо, включите одновременное отображения цветного поля и изолиний.



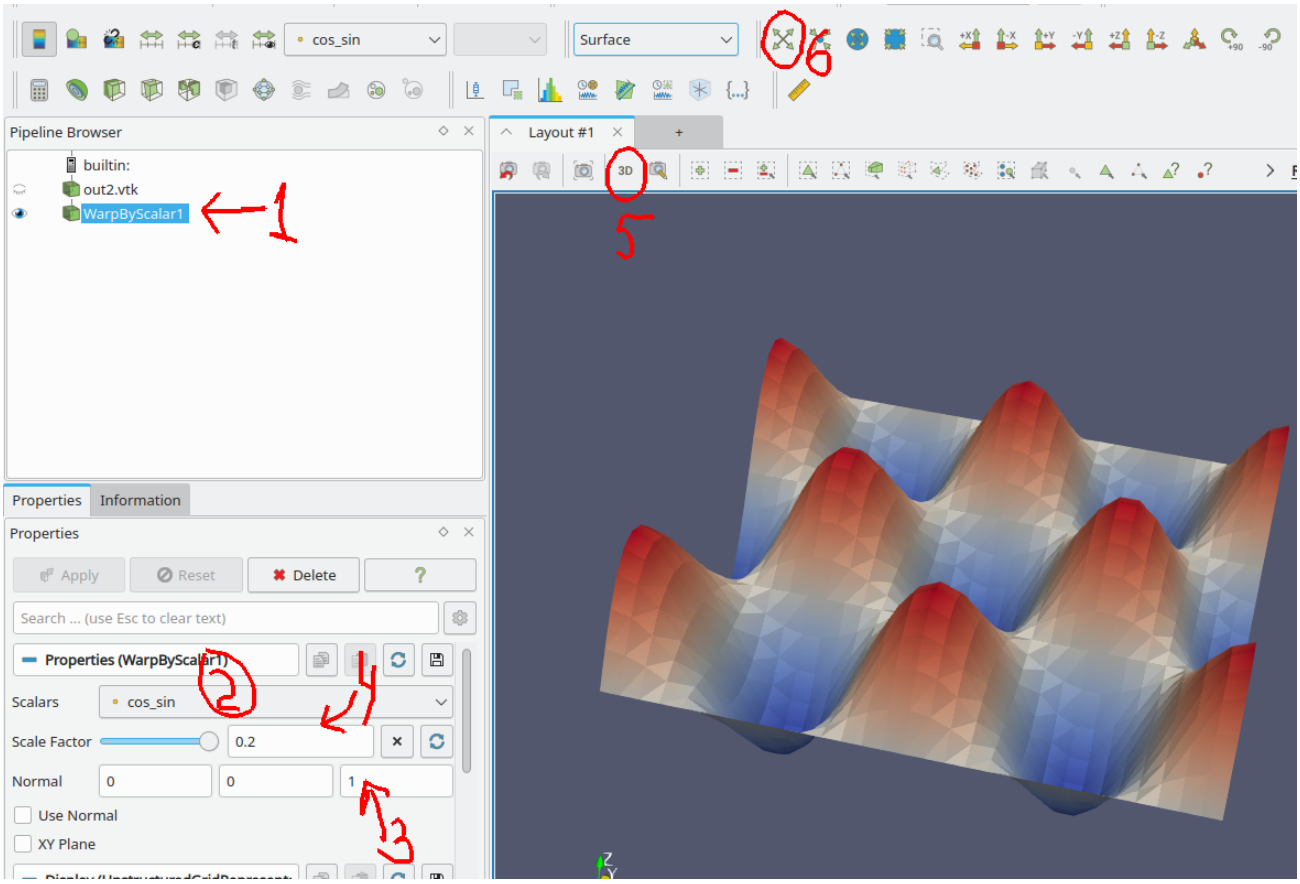
Задание цвета и толщины изолинии В случае, если нужно сделать изолинии одного цвета, установите поле `Coloring/Solid color` в настройках фильтра. Там же в меню `Edit` можно выбрать цвет. Для установления толщины линии включите подробные настройки и найдите там опцию `Styling/Line Width`.



B.3.3 Данные на двумерных сетках в виде поверхности

По аналогии с одномерным графиком (п. B.3.1), двумерные поля так же можно отобразить, проектируя данные на геометрическую координату для получения объёмного графика. Для этого

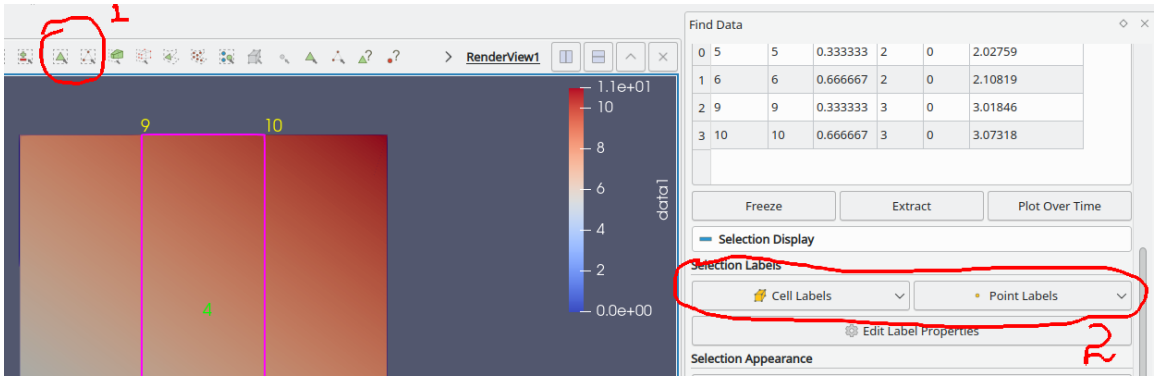
1. Включите фильтр `Filters/Warp By Scalar`
2. В настройках фильтра установите данные, которые будут проектироваться на координату z
3. Установите нормаль для проецирования (ось z)
4. Если нужно, выберите масштабирования для этой координаты
5. После нажатия `Apply` включите трёхмерное отображение
6. Если данные не видно, обновите экран.



B.3.4 Числовых значения в точках и ячейках

Иногда в процессе отладки или анализа результатов расчёта требуется знать точное значение поля в заданном узле или ячейке сетки. Для этого

1. Включить режим выделения точек или ячеек (иконка (1 на рисунке) или горячие клавиши `s`, `d`). Выделить мышкой интересующую область
2. В окне `Find data` (или `Selection Inspector` для старых версий Paraview) отметить поле, которое должно отображаться в центрах ячеек и в точках (2 на рисунке). Если такого окна нет, включить его из основного меню `View`.



B.3.5 Векторные поля

Открыть файл vtk или vtk.series, который содержит векторное поле. Далее

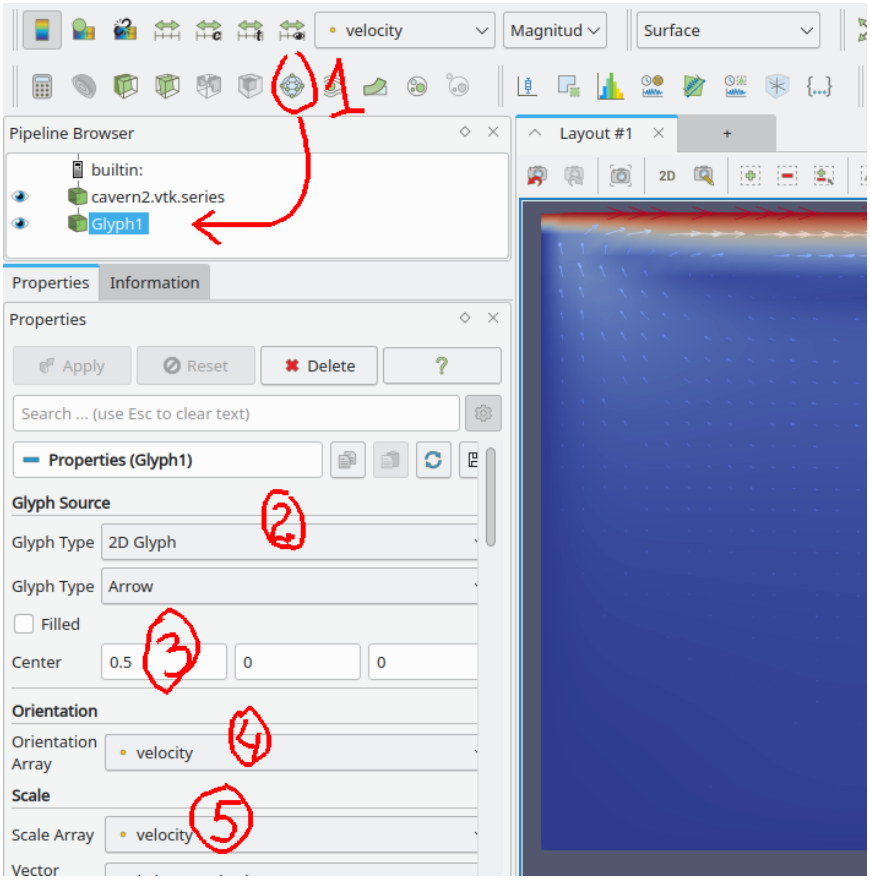
1. Создать фильтр `Glyph`

2. Задать двумерный тип стрелки

3. Сместить центр стрелки, чтобы она исходила из точки, к которой приписана

4. Отметить необходимое векторное поле в качестве ориентации

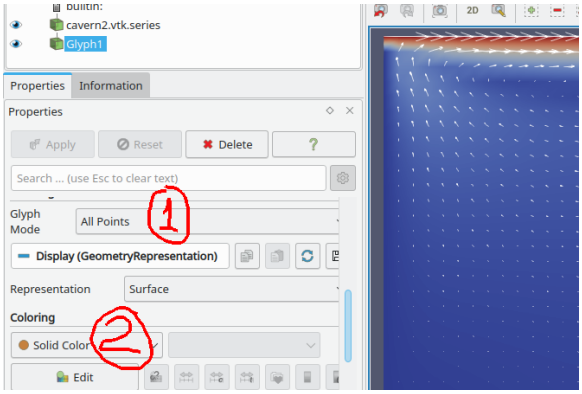
5. Отметить необходимое векторное поле для масштабирования Нажать `Apply`.



Настройка отображения стрелок

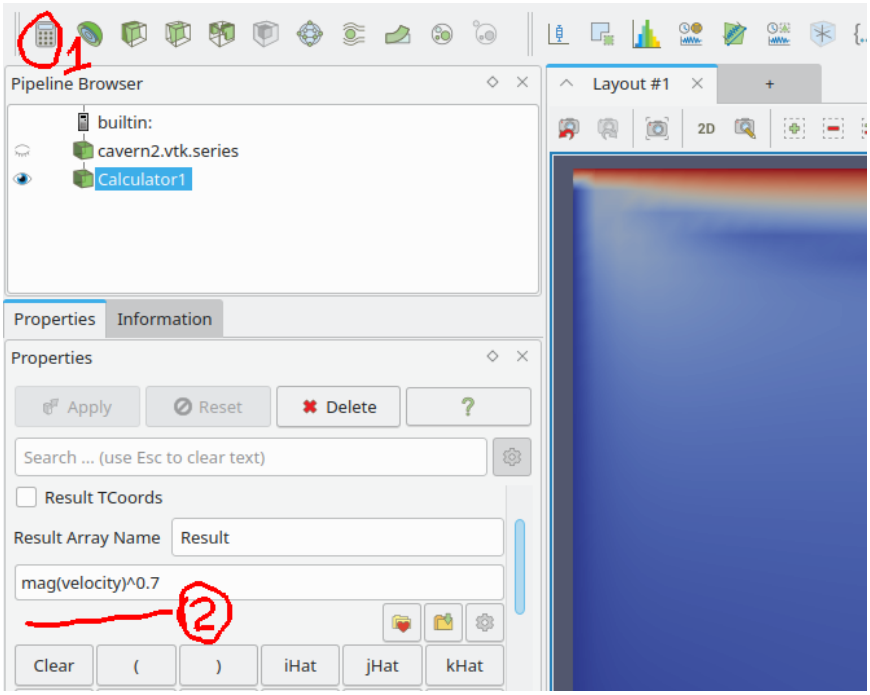
1. Выбрать необходимый `Glyph-mode`. Если сетка небольшая, то можно `All Points`.

2. Установить белый цвет для стрелок. Нажать Apply.



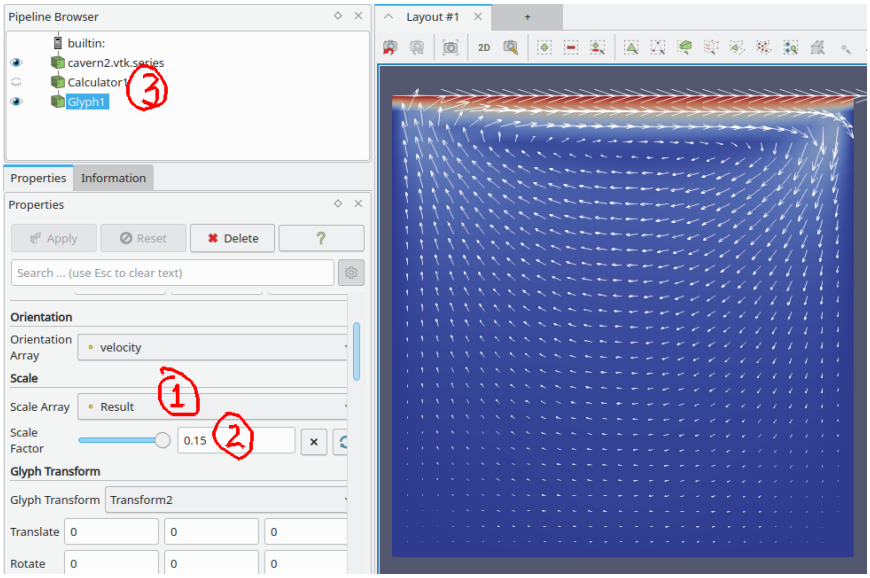
Уменьшения разброса по длине стрелок Если разброс по длинам стрелок слишком велик, его можно подравнять, введя новую функцию $|\mathbf{v}|^\alpha$ – длина вектора в степени меньше единицы (например, $\alpha = 0.7$). Такую функцию можно создать через калькулятор

1. Начиная от загруженного файла создать фильтр `Calculator`
2. Там вбить необходимую формулу



Созданную функцию нужно прокинуть в `Glyph` в качестве коэффициента масштабирования

1. В `Scale Array` фильтра `Glyph` указать уже результат работы `Calculator`-а (`Result` по умолчанию),
2. Подтянуть значение `Scale Factor` до приемлимого
3. Не забыть отключить вспомогательное поле `Calculator` из отображения



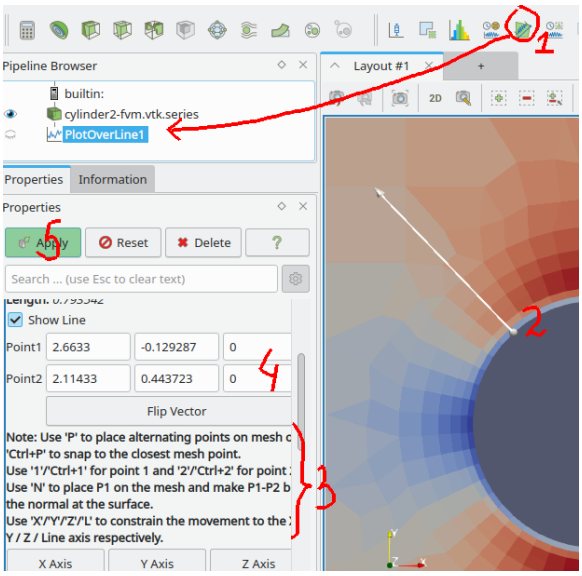
B.3.6 Значение функции вдоль линии

1. Выбрать фильтр `Plot Over Line` иконкой или в меню `Filters`
2. Установить начальную и конечную точку сечения

3. Можно использовать привязку к узлам сетки с помощью горячих клавиш (в подсказках написано)

4. Можно установить координаты руками в соответствующем поле. Для двумерных задач проследить, что координата Z равна нулю

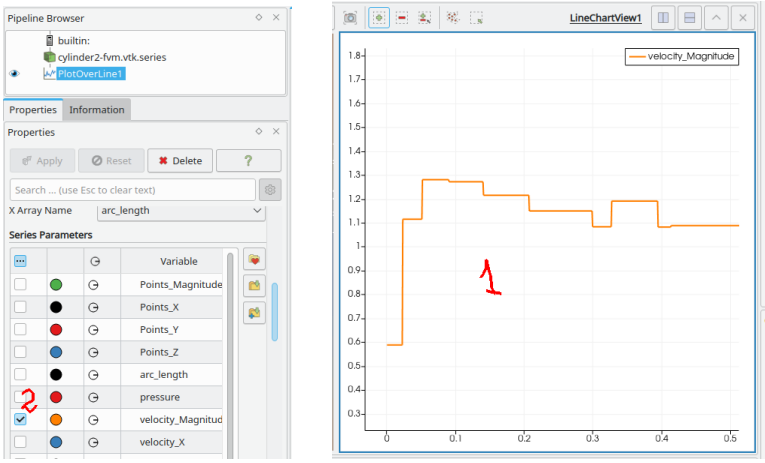
5. Нажать `Apply`



Настройка графика

1. После установок появится дополнительное окно типа `Line Chart View` с нарисованным графиком.

2. Сделав это окно активным в настройках фильтра `PlotOverLine` можно выбрать, какие поля рисовать (`Series Parameters`)

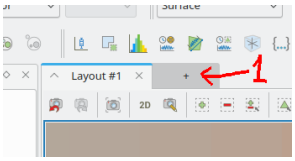


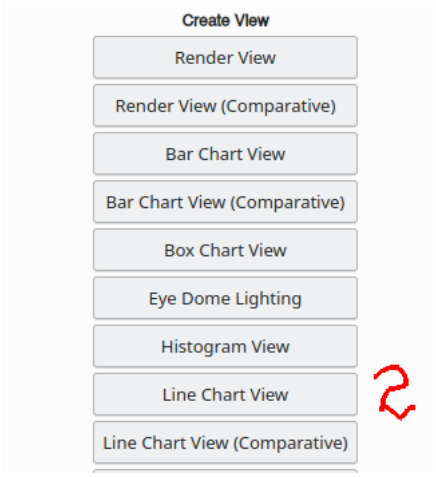
Отрисовка в отдельном окне

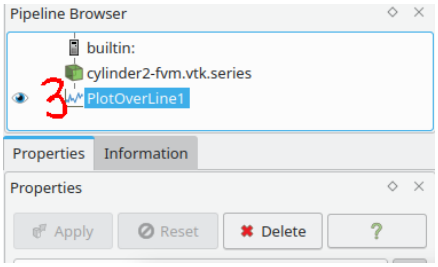
1. Открыть новую вкладку

2. Выбрать `Line Chart View`

3. Выбрать предварительно созданный фильтр с одномерным графиком


Layout #1
1


Create View
Render View
Render View (Comparative)
Bar Chart View
Bar Chart View (Comparative)
Box Chart View
Eye Dome Lighting
Histogram View
Line Chart View
Line Chart View (Comparative)
2


Pipeline Browser
builtin:
cylinder2-fvm.vtk.series
3
PlotOverLine1
Properties
Information
Properties
Apply
Reset
Delete

B.4 Hybmesh

Генератор сеток на основе композитного подхода. Работает на основе python-скрипотов. Полная документация http://kalininei.github.io/HybMesh/index.html

B.4.1 Работа в Windows

Инсталлятор программы следует скачать по ссылке https://github.com/kalininei/HybMesh/releases и установить стандартным образом.

Для запуска скрипта построения `script.py` нужно открыть консоль, перейти в папку с нужным скриптом, оттуда выполнить (при условии, что программа была установлена в папку `C:\Program Files`):

```
> "C:\Program Files\HybMesh\bin\hybmesh.exe" -sx script.py
```

B.4.2 Работа в Linux

Версию для линукса нужно собирать из исходников. Либо, если собрать не получилось, можно строить сетки в Windows и переносить полученные vtk-файлы на рабочую систему.

Перед сборкой в систему необходимо установить dev-версии пакетов `suitesparse` и `libxml2`. Также должны быть доступны компилляторы `gcc-c++` и `gcc-fortan` и `cmake`. Программа работает со скиптами python2. Лучше установить среду anaconda (https://docs.anaconda.com/free/anaconda/install/index.html) И в ней создать окружение с python-2.7:

```
> conda create -n py27 python=2.7   # создать среду с именем py27
> conda activate py27               # активировать среду py27
> pip install decorator             # установить пакет decorator
```

Сначала следует склонировать репозиторий в папку с репозиториями гита:

```
> cd D:/git_repos
> git clone https://github.com/kalininei/HybMesh
```

Поскольку программа не предназначена для запуска из под анаконды, в сборочные скрипты нужно внести некоторые изменения. В корневом сборочном файле `HybMesh/CMakeLists.txt` нужно закомментировать все строки в диапазоне

```
# ========================== Python check
....
# ========================== Windows installer options
```

а в файле `HybMesh/src/CMakeLists.txt` последнюю строку

```
#add_subdirectory(bindings)
```

Далее, находясь в корневой директории репозитория HybMesh, запустить сборку

```
> mkdir build
> cd build
> cmake .. -DCMAKE_BUILD_TYPE=Release
> make -j8
> sudo make install
```

Для запуска скриптов нужно создать скрипт-прокладку

```
import sys
sys.path.append("/path/to/HybMesh/src/py/")  # вставить полный путь к Hybmesh/src/py
execfile(sys.argv[1])
```

и сохранить его в любое место. Например в `path/to/HybMesh/hybmesh.py`.

Для запуска скрипта построения сетки следует перейти в папку, где находится нужный скрипт `script.py`, убедится, что анаконда работает в нужной среде (то есть `conda activate py27` был вызван), и запустить

```
> python /path/to/HybMesh/hybmesh.py script.py
```